

Migrasi

Visual Basic 6 ke Visual Basic .NET

**Erick Kurniawan
Rully Yulian**



PENGANTAR

Sejak Visual Basic 1.0 muncul pada tahun 1991, Visual Basic tumbuh sangat pesat dan menjadi bahasa pemrograman yang paling populer di dunia. Namun popularitas tidak menjamin bahwa Visual Basic mendapat pengakuan dari semua programmer. Ada yang menganggap VB sebagai bahasa sebagai bahasa pemrograman untuk ‘mainan’, dan tidak digunakan untuk membuat aplikasi yang ‘serius’ seperti pada C++ atau Java. VB dianggap tidak memiliki fitur yang digunakan untuk mengembangkan aplikasi bertaraf enterprise. Ada beberapa alasan yang melatarbelakangi salah satunya yaitu VB dianggap tidak mempunyai fitur OOP (Object Oriented Programming) yang lengkap.

Pada tahun 2001 Microsoft memperkenalkan teknologi .NET Framework, dan Visual Basic adalah salah satu bahasa yang disupport oleh .NET Framework. Microsoft memberi nama baru yaitu VB.NET. Dengan dukungan platform .NET, VB.NET menjadi bahasa pemrograman yang modern, *powerfull*, dan mendukung OOP secara lengkap. Sekarang VB.NET dapat disejajarkan dengan bahasa modern lainnya seperti C# atau Java.

Meskipun VB.NET sudah ada selama kurang lebih 9 tahun (sejak .NET 1.0), tapi banyak developer VB6 yang masih belum berpindah untuk menggunakan .NET. Sebagian programmer tidak percaya dengan teknologi yang baru, yang lain mungkin terlalu sibuk dan tidak ada waktu untuk belajar teknologi baru atau takut dengan model baru yang sangat berbeda dengan teknologi sebelumnya.

Sekarang versi VB.NET yang terbaru adalah VB9 (VB 2008). Buku ini akan mengajak anda developer VB6 untuk melihat fitur-fitur terbaru dari VB9 dan kelebihan VB.NET dibandingkan dengan VB6 sehingga dapat dijadikan referensi

jika anda memutuskan untuk migrasi aplikasi anda ke VB.NET. Selain untuk VB6 developer, buku ini juga dapat dijadikan referensi oleh programmer lain yang ingin belajar bahasa pemrograman VB.NET dan mempelajari fitur-fitur baru dari VB9.

Erick Kurniawan



Erick lahir di Jogjakarta pada tanggal 2 Maret 1981. Penulis menyelesaikan pendidikan S1 jurusan Teknik Informatika di Universitas Kristen Duta Wacana pada tahun 2004, kemudian melanjutkan pendidikan S2 di jurusan Ilmu Komputer Universitas Gajah Mada dan selesai pada tahun 2006.

Dari tahun 2004 sampai sekarang penulis bekerja sebagai dosen Prodi Sistem Informasi di Universitas Kristen Duta Wacana Jogjakarta.

Selain mengajar, menulis buku dan menjadi pengembang software, penulis juga aktif sebagai pembicara seminar, kuliah umum dan memberikan training terutama yang berkaitan dengan Teknologi Microsoft.

Saat ini penulis aktif di beberapa kegiatan komunitas diantaranya MUGI (Microsoft User Group) dan INDC (Indonesia .NET Developer Community). Penulis juga tercatat sebagai pengurus komunitas MUGI Jogjakarta.

Pada bulan July 2009 penulis mendapatkan award Microsoft MVP (Most Valuable Professional) untuk bidang keahlian Visual Basic (<https://mvp.support.microsoft.com/profile/erick>).

Untuk menghubungi penulis anda dapat mengirimkan email ke erick.kurniawan@gmail.com

Anda dapat mengunjungi blog penulis di:

<http://mugi.or.id/blogs/erick>
<http://geeks.netindonesia.net/blogs/erickkurniawan>

Rully Yulian MF



Rully lahir di Cianjur pada tanggal 5 Juli 1976. Penulis menyelesaikan pendidikan S1 jurusan Teknik Geofisika di Institut Teknologi Bandung pada tahun 2002.

Pada tahun 2003 penulis memulai pekerjaannya sebagai programmer di sebuah konsultan IT di Bandung. Tidak lama berselang beberapa bulan

kemudian penulis berpindah tempat kerja masih sebagai programmer pada sebuah perusahaan konsultan IT yang berlokasi di kota Bandung. Pekerjaan tetap terakhir penulis yaitu sebagai IT Trainer pada sebuah IT Training Center yang masih berlokasi di kota Bandung.

Dari pertengahan tahun 2008 sampai sekarang penulis bekerja sebagai freelance IT Trainer di beberapa training center, kampus-kampus dan perusahaan-perusahaan khusus untuk materi yang berkaitan dengan teknologi Microsoft terutama materi .NET programming dan Microsoft RDBMS. Selain mengajar, penulis juga menjadi pengembang software untuk project yang sifatnya lepas, aktif sebagai pembicara seminar, dan kuliah umum yang berkaitan dengan teknologi dan sertifikasi Microsoft.

Saat ini penulis aktif di beberapa kegiatan komunitas baik itu online maupun offline di MUGI (Microsoft User Group) dan INDC (Indonesia .NET Developer Community). Penulis juga tercatat sebagai pengurus komunitas MUGI Bandung sebagai wakil ketua.

Sertifikasi yang telah penulis dapatkan yaitu MCAD.NET, MCTS, dan MCPD untuk bidang Microsoft .NET programming.

Sedangkan sertifikasi di bidang IT Trainer yaitu MCT (Microsoft Certified Trainer).

Pada bulan Januari 2009 penulis mendapatkan award Microsoft MVP (Most Valuable Professional) untuk bidang keahlian Visual Basic (<https://mvp.support.microsoft.com/profile/Rully>).

Untuk menghubungi penulis anda dapat mengirimkan email ke rully@yulianmf.com

Anda dapat mengunjungi beberapa blog penulis yang berisi tentang kegiatan penulis dan artikel-artikel yang berhubungan dengan teknologi Microsoft di :

<http://yulianmf.com>

<http://geeks.netindonesia.net/blogs/yulian>

<http://mugi.or.id/blogs/rully>

Daftar Isi

PENGANTAR.....	2
BAB 1_MIGRASI KE .NET FRAMEWORK.....	8
BAB 2_VISUAL STUDIO IDE.....	17
BAB 3_DASAR VB 2008.....	30
BAB 4_WINDOWS FORM.....	76
BAB 5_OBJECT ORIENTED PROGRAMMING.....	101
BAB 6_OBJEK ORIENTED PROGRAMMING BAGIAN 2.....	123
BAB 7_COLLECTION	144
BAB 8_PENANGANAN KESALAHAN.....	151
BAB 9_ADO .NET	165
BAB 10_CRYSTAL REPORTS.....	277
BAB 11_.NET ASSEMBLIES.....	321
BAB 12_FITUR BARU VB9	348
BAB 13_LINQ TO SQL.....	379
BAB 14_LINQ TO XML	404
BAB 15_VB 6.0 TO VB.NET MIGRATION TOOLS HELPER.....	432
BAB 16_SETUP DAN DEPLOYMENT	462

BAB 1

Migrasi ke .NET Framework

Bab ini akan membahas topik tentang komponen-komponen utama yang ada dalam .NET Framework seperti CLR (Common Language Runtime) dan BCL (Base Class Library), bab ini juga membahas kelebihan fitur-fitur yang ada pada VB.NET dibandingkan dengan VB6. Pada bab ini juga akan dibahas beberapa alasan yang dapat menjadi pertimbangan developer VB6 untuk menggunakan VB.NET.

VB.NET <> VB6 + 1

Jika anda developer VB6 maka anda harus tahu terlebih dahulu bahwa VB.NET bukan hanya VB6 yang ditambah beberapa fitur baru. VB.NET adalah bahasa baru yang sudah di redesign dan direstrukturisasi. Bahasa ini sudah dibuat menjadi lebih modern dengan penambahan fitur baru pada object design, data akses, form, dan masih banyak lagi. Bahkan format file juga sudah berubah.

Sayangnya perubahan yang signifikan tersebut membuat VB.NET tidak 100% backward compatible dengan VB6. Project dari VB6 harus di upgrade untuk dapat berjalan di VB.NET. Anda dapat menggunakan upgrade wizard untuk masalah ini, namun untuk aplikasi yang kompleks tetap membutuhkan tambahan modifikasi secara manual. Banyak orang yang berpendapat upgrade dari VB6 ke VB.NET tidak sekedar upgrade melainkan migrasi aplikasi.

Meskipun untuk migrasi membutuhkan kerja ekstra namun VB.NET akan sangat menarik untuk dipelajari oleh developer VB6. Karena dengan VB.NET anda tidak hanya dapat membuat aplikasi berbasis windows saja tapi berbagai

macam platform seperti Web, Mobile, XML Web Service, Class Library bahkan untuk Game dengan XNA. VB.NET juga dapat berintegrasi dengan bahasa lain yang berjalan diatas .NET Framework seperti C# dan C++.

Tiga perbedaan mendasar pada VB6 dan VB.NET yaitu:

- Integrated Development Environment (IDE).
- Perubahan Sintaks dan Object Model dari Class.
- Perubahan Kompilasi kode dan menjalankan program.

VB.NET masih menggunakan Visual Studio sebagai IDE, namun sudah berbeda dengan Visual Studio yang digunakan pada VB6, sekarang Visual Studio hanya mensupport tiga bahasa utama yaitu VB, C#, dan C++.

Perbedaan yang kedua terletak pada perbedaan sintaks, karena VB.NET sudah didesign ulang menjadi lebih modern maka banyak sintaks yang dikurangi dan ditambahkan, misal perintah GoSub sudah dihilangkan tapi banyak keyword baru terutama untuk Object Oriented Programming seperti Inherits, Interface, dll.

Perbedaan yang ketiga terletak pada proses untuk kompilasi dan menjalankan program. VB.NET application akan dikompilasi menjadi kode assembly tidak seperti kebanyakan aplikasi Win32. .NET juga memiliki komponen Garbage Collection yang akan secara otomatis membersihkan object yang anda buat dari memory ketika sudah tidak diperlukan. Jadi anda tidak perlu menghapus object secara manual.

Setelah melihat banyak perbedaan yang ada antara VB6 dan VB.NET, pertanyaanya apakah developer VB6 harus pindah ke VB.NET? dan apa alasannya? Microsoft sendiri membuat VB.NET karena beberapa alasan, diantaranya keterbatasan VB6 untuk memenuhi kebutuhan aplikasi saat ini. Sekarang banyak kebutuhan aplikasi yang berbasis web sedangkan VB6 sangat minim dukungan terhadap pembuatan aplikasi web, anda dapat membuat ActiveX control dengan VB6, namun pengguna harus mendownload ActiveX tersebut sebelum dapat menjalankannya di web. Arsitektur ini tidak bagus karena kita mengharapakan client yang benar-benar

'thin' (hanya cukup browser tanpa perlu install program tambahan). Pengguna menginginkan kode yang dijalankan di server, mereka juga menginginkan jaminan keamanan yang baik, dan scalability dari aplikasi yang dibuat. Itu semua tidak bisa dilakukan oleh VB6.

Berdasarkan beberapa alasan tersebut Microsoft merasa perlu untuk membuat architecture yang lebih baik dan modern. Programmer juga menginginkan fitur-fitur Object Oriented Programming (OOP) seperti inheritance dan polymorphism untuk membuat komponen yang lebih bagus dan menangani masalah versioning control pada komponen atau dikenal dengan istilah 'DLL hell'. Untuk memberikan solusi atas berbagai masalah diatas Microsoft mencoba membuat kesatuan framework untuk pengembangan aplikasi yang diberi nama .NET Framework.

Mengapa VB.NET Tidak Kompatible Dengan VB6?

Mungkin anda bertanya kenapa Microsoft tidak mendukung *backward compatibility* dari VB.NET ke VB6. Kenapa tidak menambahkan fitur di VB6 saja? Kenapa harus didesain ulang? Ada beberapa alasan untuk pertanyaan diatas.

Penambahan Fitur Baru

Beberapa fitur baru yang ditambahkan membutuhkan redesign ulang. Misal penambahan fitur inheritance pada form membutuhkan redesign object model pada form. Penambahan Interface untuk mendukung polymorphism membutuhkan perubahan pada bahasa dan file format. Untuk memperbaiki 'DDL hell' berarti versioning komponen harus di redesign ulang.

Perbaikan Bahasa

VB6 sudah mengalami banyak penambahan fitur seiring dengan kebutuhan developer. Pada beberapa kasus fitur baru tersebut menyebabkan inkonsistensi dan kerumitan. Misalnya untuk keyword New dan Set pada saat object akan dibuat.

Kemudian tipe varian yang dapat menampung tipe apa saja yang dapat mengurangi performa program.

Membuat Lebih Modern

Pada VB.NET tipe Long menjadi 64bit, dan Integer menjadi 32bit. Keyword Type pada VB6 diganti menjadi Structure.

Mengapa Perlu Untuk Migrasi?

Anda mungkin bertanya, Apakah perlu untuk migrasi ke VB.NET? Apakah tidak cukup dengan hanya memodifikasi aplikasi VB6 yang ada sekarang?. Alasan utama untuk migrasi adalah adanya fitur-fitur baru di VB.NET yang akan membantu developer VB untuk membuat aplikasi yang lebih scalable, maintainable, dan powerfull. Beberapa aplikasi baru yang ada di VB.NET adalah?

Fitur Baru

VB.NET mempunyai beberapa fitur baru yang membuat bahasa VB menjadi lebih powerfull sehingga dapat mematahkan mitos bahwa VB hanya bahasa mainan (toy language) bila dibandingkan dengan bahasa lain seperti C++ dan Java. Fitur-fitur tersebut antara lain:

- Dukungan Object Oriented Programming : VB.NET adalah bahasa pemrograman yang full Object Oriented. Jadi VB.NET mendukung fitur-fitur OOP seperti Inheritance, Interface, Method Overloading, Polymorphism yang akan dibahas lebih lanjut pada bab-bab selanjutnya.
- Structure Exception Handling : untuk menggantikan perintah OnError Goto pada VB6, VB.NET menyediakan Try..Catch..Finally error handling. Error handling pada VB.NET ini lebih mudah digunakan karena anda hanya cukup menaruh kode yang akan dicek di dalam Try, dan menyiapkan exception handling nya di dalam

catch. Topik tentang error handling juga akan dibahas pada bab selanjutnya dari buku ini.

- .NET Framework : VB.NET mempunyai koleksi class library yang sangat banyak dan terorganisasi dengan baik sehingga mudah digunakan dan dicari. Dengan menggunakan class library ini programmer tidak perlu membuat kode sendiri dari awal.
- GDI+: GDI+ adalah library Graphic yang digunakan untuk mengembangkan aplikasi windows form.
- Web Services dan Web Form : dengan VB.NET anda dapat membuat aplikasi berbasis web dengan menggunakan Web Form (ASP.NET). anda juga dapat membuat aplikasi web service untuk membuat three tier application.
- Cross-Language Interoperability : karena setiap program yang berjalan di .NET dicompile menjadi assembly maka anda dapat membuat aplikasi dengan bahasa pemrograman yang berbeda yang berjalan diatas platform .NET seperti C# dan C++. Jadi anda juga dapat menggunakan komponen yang dibuat menggunakan C# atau C++ untuk digunakan di VB.
- Multithreading : secara default aplikasi VB.NET adalah single thread tapi anda dapat mengimplementasikan multi thread. Fitur ini sangat berguna jika anda mempunyai aplikasi yang proses komputasinya memakan waktu lama.
- Type Safe Collection : fitur ini mulai ada di .NET 2.0 (VB8 atau VB 2005). Dengan fitur ini anda dapat membuat object collection yang type safe. Penjelasan lebih jauh mengenai collection akan dibahas pada bab selanjutnya.
- LINQ : Fitur ini mulai ada pada .NET 3.5 (VB9 atau VB 2008). LINQ (Language Integrated Query) adalah fitur baru untuk mengquery data yang ditambahkan kedalam bahasa VB dan C# sehingga kedua bahasa tersebut dapat melakukan query ke object, database,

xml, dan sumber data lainnya. Lebih detail mengenai LINQ akan dibahas di bab selanjutnya.

- XML Literal : Fitur ini mulai ada pada VB9, fitur ini adalah fitur special dari VB9 karena bahasa lain seperti C# tidak mendukung fitur ini. XML Literal akan sangat membantu anda dalam bekerja dengan XML, pada VB9 XML menjadi 'first class citizen' yang berarti anda dapat menuliskan XML secara literal seperti anda menuliskan string. Fitur ini juga akan dibahas pada buku ini di bab selanjutnya.

Beberapa Perbedaan Mendasar Dengan VB6

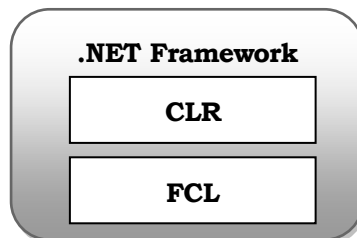
- Pada VB.NET index array dimulai dari 0
- Internet Project dengan Web Classes dan DHTML sudah tidak disupport. Pada VB.NET anda dapat menggunakan Web Form (ASP.NET).
- VB.NET tidak menyediakan default properties, jadi anda tidak dapat menuliskan Text1.Text hanya dengan Text1 seperti di VB6.
- Teknik-teknik lama di VB6 untuk print document, menggambar graphic, membaca textfile sudah tidak dapat digunakan di VB.NET, anda harus siap untuk mempelajari teknik yang baru.
- VB.NET tidak mensupport deterministic finalization, jadi ketika object sudah tidak digunakan maka ia akan tetap di memory untuk sementara sebelum dibersihkan oleh Garbage Collector.
- Cara lama mengakses database di VB6 menggunakan DAO dan RDO tidak disupport secara penuh (tidak dapat digunakan sebagai data binding).
- Walaupun anda menggunakan upgrade wizard untuk migrasi dari VB6 namun pada beberapa bagian anda harus menulis ulang, terutama untuk aplikasi-aplikasi yang kompleks.

- VB.NET tidak mensupport pointer jadi anda tidak dapat menggunakannya lagi (pada VB6 anda dapat menggunakan fungsi StrPtr() dan ObjPtr() untuk mengakses pointer).
- Perintah Goto dan GoSub sudah tidak disupport di VB.NET.
- Cara drawing pada Form juga berubah, jika anda menggunakan custom drawing di VB6 anda harus menulis ulang kembali kodenya.

.NET Framework

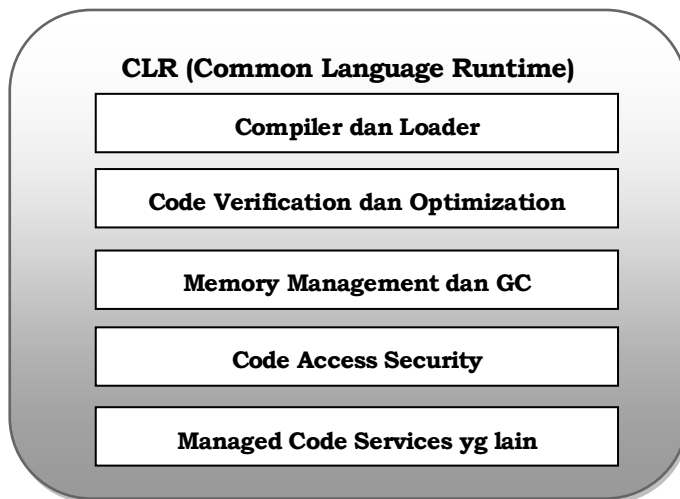
Untuk bekerja di lingkungan .NET Framework maka anda harus mengetahui arsitektur dan komponen apa saja yang ada didalamnya.

.NET Framewok mendukung beberapa bahasa pemrograman, adapun bahasa pemrograman yang disupport secara resmi oleh Microsoft adalah C# (CSharp), VB, dan C++, tetapi sekarang banyak bahasa lain yang juga dikembangkan untuk mensupport platform .NET diantaranya Delphi, Phytion (IronPhytion), dll. Untuk mengembangkan aplikasi berbasis .NET sebenarnya dapat digunakan lebih dari satu bahasa pemrograman (Language Interoperability) misal sebagian program menggunakan C# dan sebagian lagi menggunakan VB, tetapi disarankan untuk memilih hanya satu bahasa pemrograman saja agar aplikasi yang dibuat lebih mudah untuk di-maintain. Bahasa paling banyak digunakan di platform .NET saat ini adalah C# dan VB.



Gambar 1.1 .NET Framework

.NET Framework sebenarnya terdiri dari dua komponen utama yaitu CLR (Common Language Runtime) dan FCL (Framework Class Library).

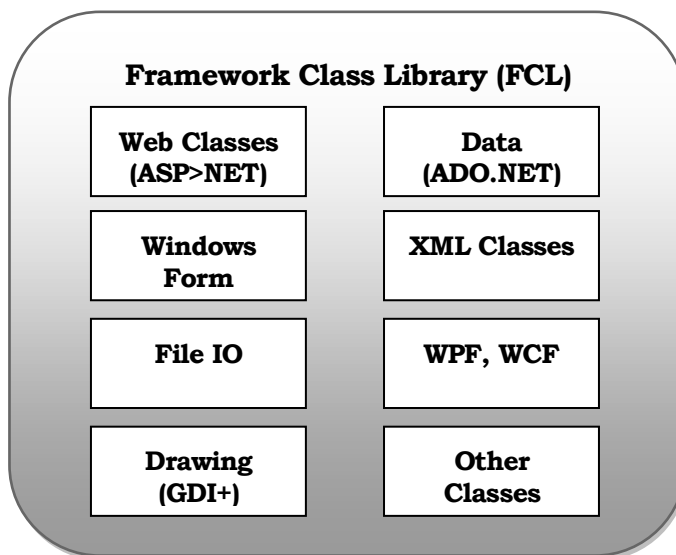


Gambar 1.2 Common Language Runtime

Common Language Runtime (CLR) adalah pondasi utama dari Framework .NET. CLR merupakan komponen yang bertanggung jawab terhadap berbagai macam hal, seperti bertanggung jawab untuk melakukan manajemen memory, melakukan eksekusi kode, melakukan verifikasi terhadap keamanan kode, menentukan hak akses dari kode, melakukan kompilasi kode, dan berbagai layanan system lainnya. Dengan adanya fungsi CLR ini, maka aplikasi berbasis .NET biasa juga disebut dengan *managed code*, sedangkan aplikasi di luar itu biasa disebut dengan *un-managed code*. Dengan adanya CLR maka tugas pengembang program menjadi lebih ringan karena

banyak tugas yang dahulu harus dikerjakan oleh pengembang sudah digantikan secara otomatis oleh komponen CLR ini.

CLR akan melakukan kompilasi kode-kode aplikasi kita menjadi bahasa assembly MSIL (Microsoft Intermediate Language). Proses kompilasi ini sendiri dilakukan oleh komponen yang bernama *Just In Time* (JIT).



Gambar 1.3 Framework Class Library

.NET Framework Class Library atau sering juga disebut **Base Case Library (BCL)** adalah koleksi dari *reusable types* yang sangat banyak dan terintegrasi secara melekat dengan CLR. Kumpulan Class Library ini sangat berguna untuk pengembangan aplikasi karena developer tidak perlu membuat semuanya dari awal karena sudah disediakan oleh .NET, misal class untuk membuat aplikasi berbasis windows, class untuk membuat objek-objek koleksi, class untuk koneksi dengan database (ADO.NET), class untuk mengembangkan aplikasi berbasis web, class WPF (Windows Presentation Foundation), dan masih banyak lagi.

BAB 2

Visual Studio IDE

Untuk mengembangkan aplikasi VB.NET anda sebenarnya tidak harus menggunakan editor Visual Studio IDE (Integrated Development Environmet), bahkan anda juga dapat menggunakan editor sederhana seperti notepad dan mengkompilasi kode tersebut menggunakan kompiler vbc.exe yang sudah ada dalam .NET Framework. Penggunaan IDE bertujuan untuk mempermudah anda dapal mengembangkan aplikasi sehingga lebih cepat dan produktif.

Visual Studio merupakan IDE yang sangat lengkap dan dapat membuat kita menjadi jauh lebih produktif, namun kita juga harus mempelajari secara detail fitur-fitur yang ada di Visual Studio agar dapat memanfaatkannya secara lebih maksimal.

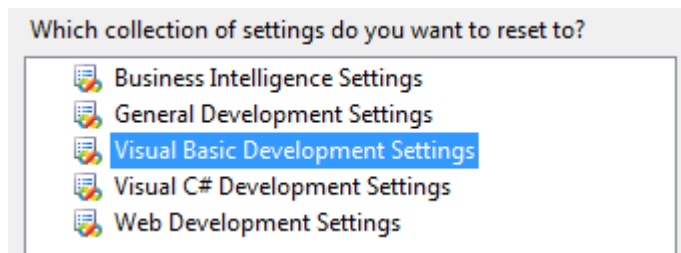
Fitur-fitur baru Visual Studio

Saat ini Visual Studio sudah mencapai versi 2008, Jika dibandingkan dengan Visual Studio lama milik VB6, banyak sekali firut-fitur baru yang ditambahkan, diantaranya adalah:

- Penambahan Fitur Intellisense
- Code Snippets
- Import dan Export Setting (Visual Studio Theme)

Halaman Awal

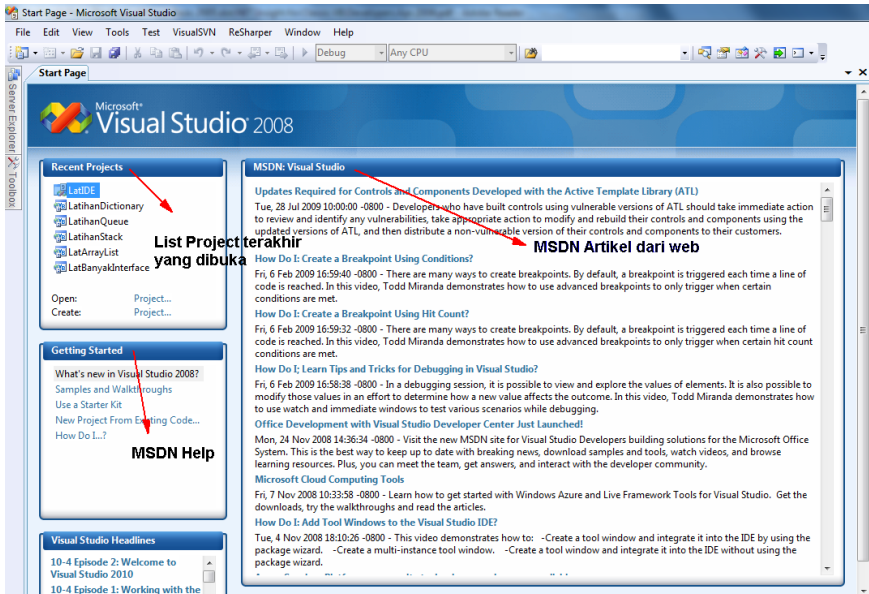
Pertama kali Visual Studio dijalankan, anda diharuskan untuk memilih salah satu dari beberapa setting yang disediakan.



Gambar 2.1 Pilihan Setting Development

Karena anda akan menggunakan Visual Basic maka pilih 'Visual Basic Development Setting'. Setting ini disesuaikan dengan kebiasaan programmer VB6 di Visual Studio Classic.

Tampilan Visual Studio 2008 pertama kali ketika anda menjalankannya.

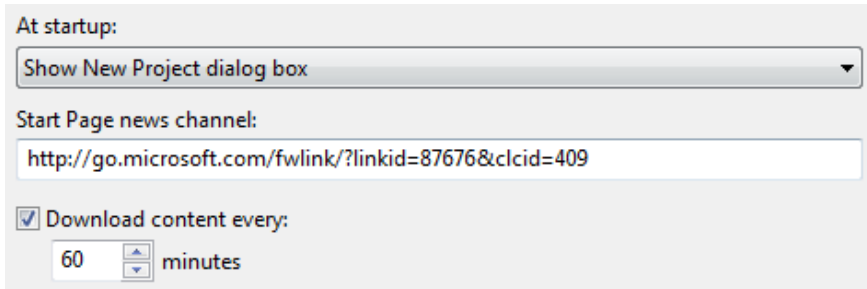


Gambar 2.2 Visual Studio 2008

Bagian yang paling penting dari halaman awal ini adalah Recent Project. Yang berisi list dari aplikasi yang anda gunakan terakhir kali. Jika komputer anda terkoneksi dengan internet maka anda juga dapat mengakses informasi artikel-artikel terbaru dari official website Visual Studio, jadi sebenarnya Visual Studio juga mempunyai integrated browser untuk menampilkan artikel-artikel.

Konfigurasi Halaman Startup

Visual Studio menyediakan fasilitas untuk konfigurasi tampilan halaman startup walaupun sangat terbatas. Caranya klik pada menu tools ► options ► kemudian pilih Environmnt ► bagian statup.



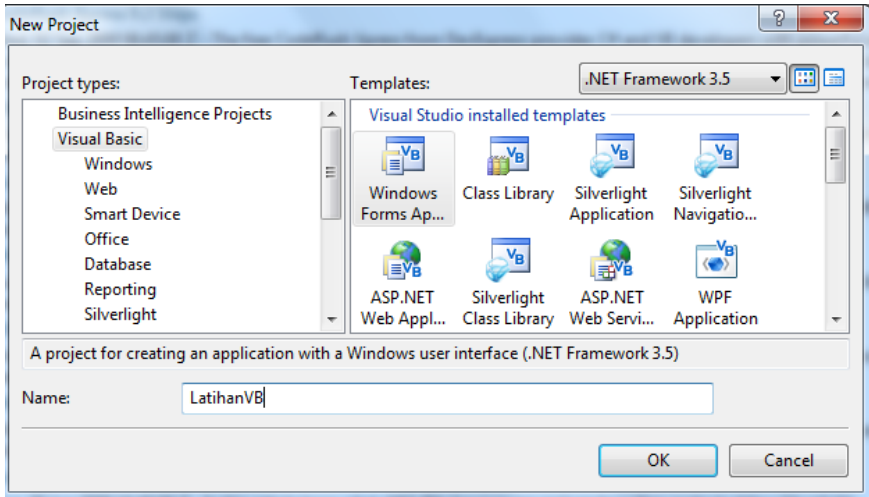
Gambar 2.3 Startup menu

Anda dapat melakukan konfigurasi seting ‘At statup’ yang akan menentukan action yang dilakukan oleh Visual Studio ketika pertama kali dijalankan nilai defaultnya adalah ‘Show statup page’ namun anda juga dapat memilih ‘Show New Project dialog box’ jika ingin ketika Visual Studio dijalankan langsung menampilkan menu ‘New Project’.

Yang kedua anda juga dapat mengganti url dari news channel yang akan ditampilkan. Terakhir anda juga dapat menentukan setiap berapa menit Visual Studio akan mererefresh content untuk mengambil informasi terbaru dari website.

Membuat Project Baru

Untuk membuat project baru di Visual Studio anda dapat melakukan klik pada link ‘Create Project’ pada startup atau klik pada menu ‘File’ ► ‘New Project’, maka akan ditampilkan windows baru berisi daftar pilihan dari project yang dapat anda buat. Pilih ‘Windows Form Application’ ► beri nama projectnya ‘LatihanVB’. File project anda sementara akan disimpan di file temporary C:\Documents and Settings\[UserName]\Local Settings\Application Data\Temporary Projects\[ProjectName]. Ketika anda sudah melakukan save pada project maka file akan dipindahkan dari temporary ke dalam file anda.

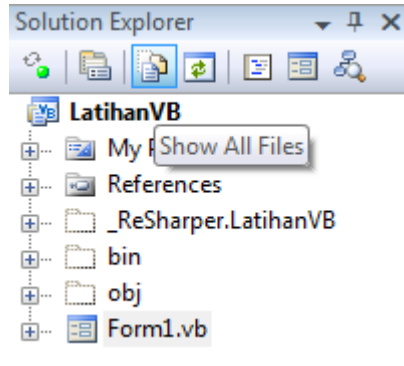


Gambar 2.4 New Project

Solution Explorer

Jika anda sudah membuat project baru maka disebelah kanan atas Visual Studio terdapat jendela Solution Explorer. Solution Explorer adalah pengganti dari Project Explorer pada VB6. Solution Explorer berisi daftar semua file yang kita gunakan untuk membuat aplikasi. Solution Explorer dapat mengandung lebih dari satu project (sama dengan project group di VB6).

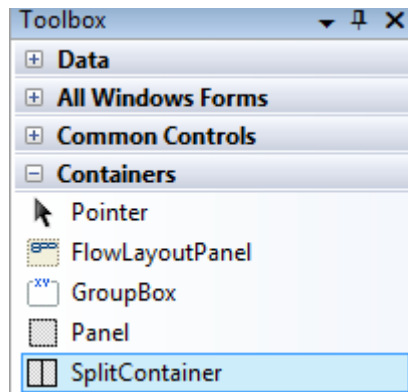
Pada VB tidak semua file dalam solution explorer ditampilkan, ada beberapa yang disembunyikan misalnya file yang menyimpan kode design form dan reference file. Untuk menampilkan semua file yang ada klik pada tab 'Show All Files'.



Gambar 2.5 Solution Explorer

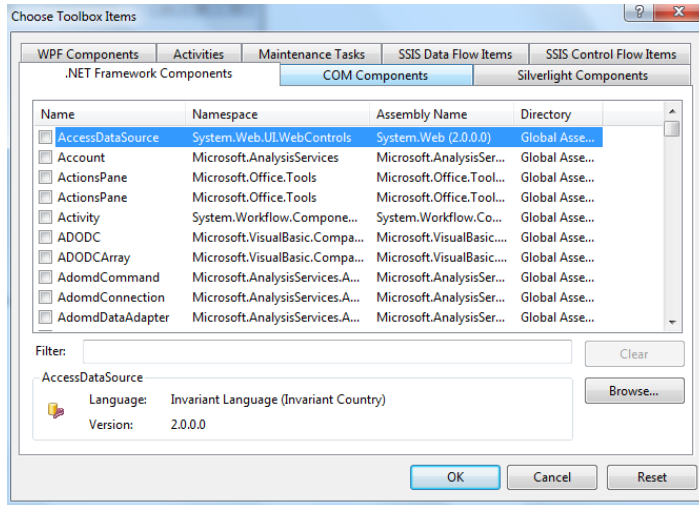
Toolbox

Toolbox berisi control-control yang dapat anda gunakan untuk mendesign antar muka grafis. Pada Visual Studio 2008 pengorganisasian control lebih rapi dan teratur sehingga lebih mudah untuk menemukan control yang anda inginkan.



Gambar 2.6 Toolbox

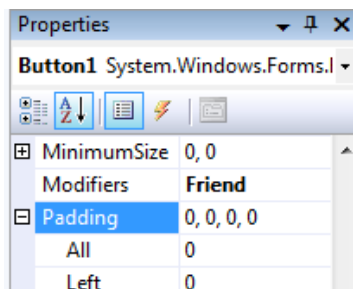
Jika anda menginstall komponen dari vendor pihak ketiga seperti Telerik atau DevExpress maka anda dapat menambahkan komponen tersebut pada toolbox dengan cara klik kanan pada toolbox ► pilih 'Choose Items' ► kemudian pilih komponen yang diinginkan, maka otomatis komponen tersebut akan ditambahkan kedalam toolbox.



Gambar 2.7 Menambahkan Komponen Baru

Properties Window

Tampilan properties windows masih sama dengan yang ada di VB6 dari segi letak dan fungsinya. Properties window digunakan untuk memberi nilai pada properti control yang anda gunakan dalam aplikasi. Tambahan fitur pada properties window terletak pada kemampuan untuk menampilkan kategori tersembunyi menggunakan collapsible panel dengan melakukan klik pada tanda (+) dan minus (-).

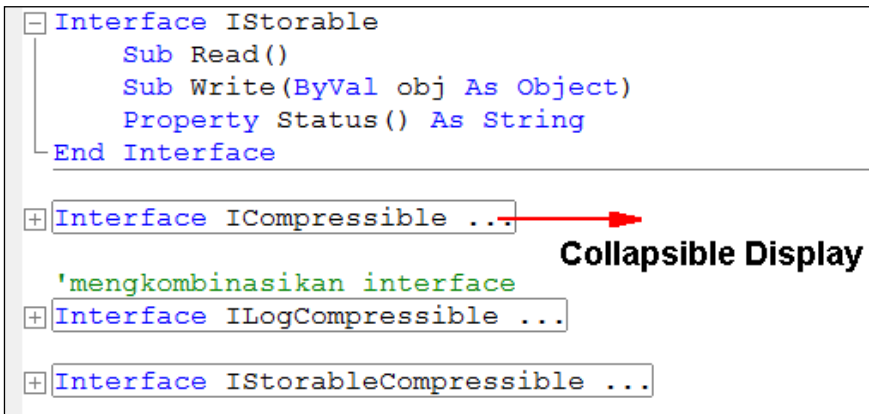


Gambar 2.8 Properties Window

Tampilan Kode

Tampilan kode di VB9 mempunyai banyak fitur baru yang tidak ada di VB6.

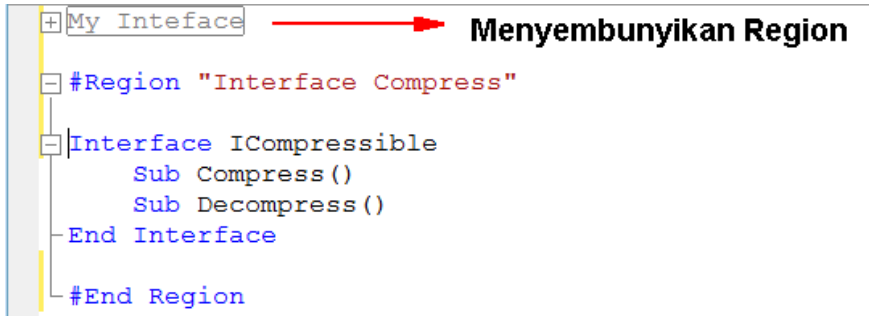
Auto-formatter : anda tidak perlu khawatir tidak rapi dalam menulis kode, karena Visual Studio akan secara otomatis merapikannya untuk anda. Misal Visual Studio mengatur indentasi dari kode yang kita ketikan, merubah keyword menjadi letter case secara otomatis, dll. Fitur yang paling menarik adalah collapsible display sehingga anda dapat menyembunyikan kode yang mempunyai banyak baris agar pembacaan lebih mudah.



Gambar 2.9 Collapsible Display

Anda juga dapat menambahkan keyword #Region untuk membuat region yang digunakan untuk mengelompokkan kode-kode anda sehingga lebih mudah untuk diatur. Region juga dapat di atur collapsible displaynya menjadi hide atau show.

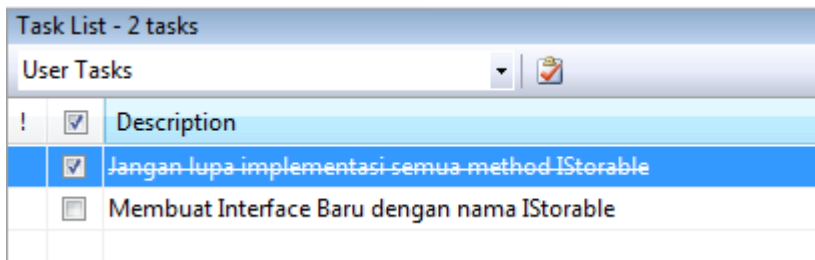
```
#Region "My Interface"
    Interface IStorable
        Sub Read()
        Sub Write(ByVal obj As Object)
        Property Status() As String
    End Interface
#End Region
```



Gambar 2.10 Region

Task List

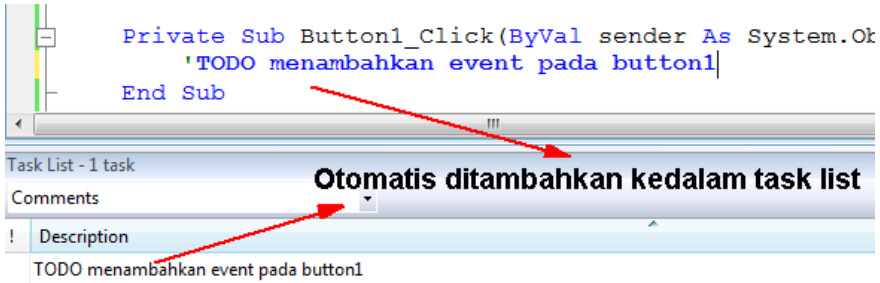
Task List digunakan untuk membantu anda mengatur programming task. Untuk menampilkan task list pilih menu 'View' ► 'Other Window' ► 'Task List'. Untuk menambahkan task baru kedalam task list klik icon 'Create Task'.



Gambar 2.11 Task List

Jika task sudah selesai dikerjakan anda dapat menambahkan check di sebelah kiri task. Fitur yang paling menarik pada Task List adalah anda dapat menambahkan task dengan cara menuliskan task diawali dengan keyword 'TODO' pada kode anda, maka secara otomatis task tersebut akan ditambahkan kedalam task list.

Anda juga dapat menambahkan keyword sendiri selain 'TODO' agar komentar tersebut dapat ditambahkan dalam task list, anda juga dapat memberi prioritas pada task tersebut (low, high, atau normal). Caranya klik pada menu 'Tools' ► 'Options' ► pastikan check 'Show All Setting' terpilih ► kemudian pilih 'Task List'.

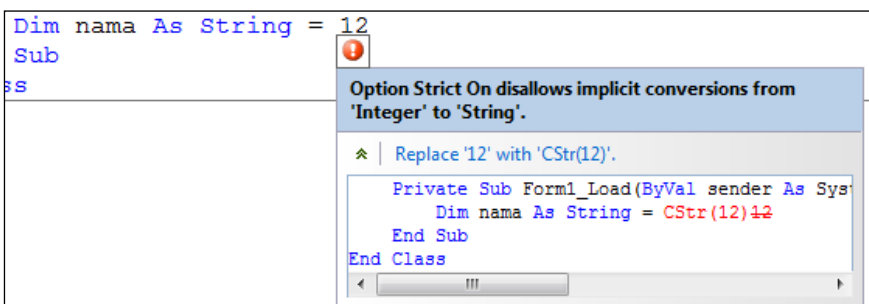


Gambar 2.12 Menambahkan 'TODO'

Penambahan Fitur Intellisense

Dengan fitur intellisense programmer VB menjadi lebih produktif karena tidak perlu mengetikkan keseluruhan program secara lengkap dan dibantu untuk tidak mengetikkan kode yang salah.

Visual Studio juga membantu programmer ketika ditemukan kesalahan pada saat design time dengan memberikan keterangan kesalahan yang terjadi lewat tooltip. Ketika anda mengetikkan perintah-perintah seperti if, for, while maka Visual Studio secara otomatis juga menambahkan kalang penutup end if, end for, end while.

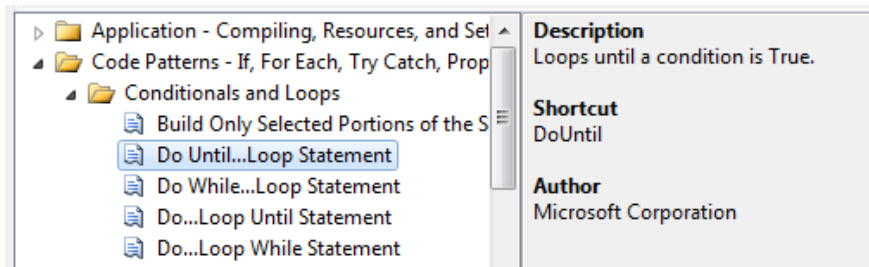


Gambar 2.1 ErrorTooltip

Code Snippet

Secagai programmer anda pasti sering mnuliskan kode yang sama untuk , misal menggunakan statement For Each, membaca file, membuat property pada class, dll. VB9 menyediakan fitur untuk menuliskan rutinitas kode tersebut secara lebih singkat disebut dengan snippet.

Visual Studio sudah menyediakan snippet library yang berisi kode-kode yang sering digunakan. Anda dapat menampilkan library tersebut dengan cara klik 'Tools' ► kemudian pilih 'Code Snippet Manager'. Anda dapat memilih snippet yang ingin digunakan berdasarkan kategori yang ada.



Gambar 2.13 Code Snippet Manager

Setiap snippet mempunyai description yang menjelaskan kegunaan dari kode snippet tersebut dan mempunyai shortcut untuk menggunakannya pada program. Misal jika anda ingin menuliskan perintah Do Until Loop pada program ada cukup menuliskan DoUntil (snippet shortcut) kemudian diikuti dengan menekan tab.

```
' You can change more print options via PrintOptions property
Dim report As New ReportDocument()
report.Load("C:\My Crystal Reports\Report Name.rpt")
report.PrintToPrinter(1, True, 0, 0)
report.Close()
```

Gambar 2.14 Code Snippet

Anda cukup mengganti di bagian kotak yang berwarna biru dengan kondisi yang anda inginkan. Contoh pada gambar diatas adalah contoh snippet yang digunakan untuk print pada Crystal Report.

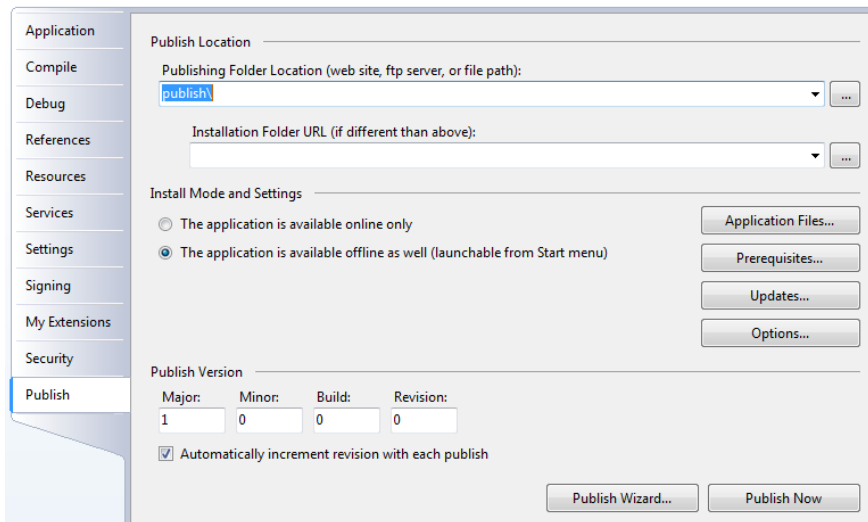
Anda dapat menambahkan snippet baru selain yang disediakan oleh Visual Studio. Caranya anda dapat menekan tombol 'Search Online' pada 'Code Snippet Manager'. Jika anda tertarik anda juga dapat membuat code snippet template sendiri dengan cara mendownload snippet editor di <http://msdn.microsoft.com/vbasic/downloads/tools/snippeteditor>.

Project Properties

Project properties adalah control panel khusus yang disediakan untuk menkonfigurasi project. Untuk masuk ke tampilan Project Properties klik kanan pada project ► pilih 'Properties' maka jendela Project properties akan terbuka. Ada beberapa tab pada project properties yaitu:

- **Application** : anda dapat mengganti nama file berekstensi .exe yang akan dihasilkan oleh aplikasi anda.
- **Compile** : digunakan untuk mengatur kompiler VB, seperti option strict, option explicit, dan option infer.
- **Debug** : berisi command line argument, mengatur startup directory, dan mengatur setting yang berpengaruh pada debbuging session.
- **References** : berisi daftar referensi yang digunakan oleh aplikasi anda.
- **Resources** : berisi daftar resources berupa binary file seperti image, audio yang digunakan pada aplikasi anda.
- **Settings** : digunakan untuk mengatur application setting yang akan tersimpan di file configuration (app.config), misal untuk koneksi ke database.
- **Signing** : untuk memberikan strongly typed name ke project anda, agar dapat ditambahkan di GAC (Global Assembly Cache).

- Security: untuk pengaturan security setting pada click once (tool untuk deployment aplikasi).
- Publish : digunakan untuk mempublikasikan aplikasi anda menggunakan ClickOnce, jadi aplikasi anda dapat diinstall di web atau jaringan lokal.



Gambar 2.15 Project Properties

BAB 3

Dasar VB 2008

Sejauh ini anda telah mempelajari komponen-komponen penyusun .NET Framework dan bagaimana menggunakan Visual Studio sebagai IDE (Integrated Design Environment). Tapi sebelum anda mulai untuk membuat program dengan VB (VB 2008) maka terlebih dahulu kita akan mempelajari beberapa konsep dasar pemrograman VB9. Kita juga akan melihat beberapa perbedaan sintaks yang ada pada VB6 dan VB9 seperti tipe data, assignment, scope variable, penggunaan namespaces, dan common class library.

Kita akan memulai chapter ini dengan pembahasan tentang namespace, common class library dan VB9 file format, topik-topik tersebut adalah aspek yang paling banyak berubah dan harus diketahui oleh VB6 developer jika ingin berpindah ke VB9.

Apa yang baru di .NET ?

Bagi anda developer VB6 yang belum pernah menggunakan .NET, kita akan membahas beberapa fitur yang ada pada .NET.

Common Class Library

Jika kita pernah membuat program dengan Java maka anda pasti pernah menggunakan Java API, yang berisi kumpulan class Library lengkap, C++ juga memiliki library seperti MCF dan ATL. VB6 tidak mempunyai library yang selengkap Java dan C++. Dengan menggunakan .NET developer VB akhirnya memiliki kumpulan class library yang

modern dan lengkap yang dapat digunakan untuk mengembangkan aplikasi bertaraf enterprise.

My Object

Mulai VB8 diperkenalkan object baru dengan nama My Object yang digunakan sebagai shortcut untuk mengakses fitur-fitur yang paling sering digunakan.

Array

Array pada VB9 berbeda dengan VB6, Pada VB9 untuk menyesuaikan dengan .NET CLR (Common Language Runtime) dan alasan konsistensi maka index array harus dimulai pada elemen ke-0. Pada VB9 array juga berupa object bukan stucture.

Perubahan Sintaks

Untuk menyesuaikan dengan CLR maka team VB dari Microsoft memutuskan untuk merevisi hampir beberapa sintaks yang ada, diantaranya new assgiment shortcut, function, dan optional parameter.

Method Overloading

Pada VB9 anda dapat melakukan overloading pada method (sub atau function), jadi anda dapat membuat method dengan nama yang sama hanya jumlah atau tipe parameternya berbeda.

Delegates

Delegates adalah variable yang mereferensi ke function atau subroutine. Anda dapat menggunakan variable ini untuk mengeksekusi subroutine atau function kapan saja anda butuhkan.

LINQ

LINQ kepanjangan dari Language Integrated Query adalah fitur baru pada C# dan VB yang diperkenalkan pada .NET versi 3.5. LINQ mempunyai query expression seperti from, where, select yg biasa ditemui pada bahasa SQL. Dengan menggunakan LINQ, anda dapat melakukan query ke berbagai macam data seperti ke database SQL Server (LINQ to SQL), ke object DataSet (LINQ to DataSet), ke entity (LINQ to Entity Framework), data XML (LINQ to XML) dan masih banyak lagi.

Pengantar Class Library

Salah satu fondasi utama pada .NET Framework adalah BCL (Base Class Library) yang berisi kumpulan dari ribuan class yang dapat anda gunakan pada VB9. Sebagai contoh anda dapat menggunakan class library untuk membangun GUI pada Windows Form, atau membangun aplikasi berbasis web (ASP.NET).

Class Library yang ada pada .NET Framework sangat banyak sehingga perlu pengelompokan terhadap file-file tersebut.

Namespace

Setiap kode yang kita tulis ada didalam objek konseptual yang bernama namespace. Namespaces menjaga .NET kode dari kebingungan akibat nama class yang sama. Misal perusahaan asuransi dengan nama Act menggunakan namespace 'ActInsurance' untuk semua program, didalam 'ActInsurance' juga terdapat namespace dengan nama 'Policy', jadi kita bisa mengakses semua class yang ada didalam namespace 'Policy' dengan memanggil 'ActInsurance.Policy'.

Namespace dapat diibaratkan seperti folder yang biasa kita gunakan untuk mengelompokan file. Namespace juga bersifat hirarki sama seperti folder. Dengan menggunakan namespace

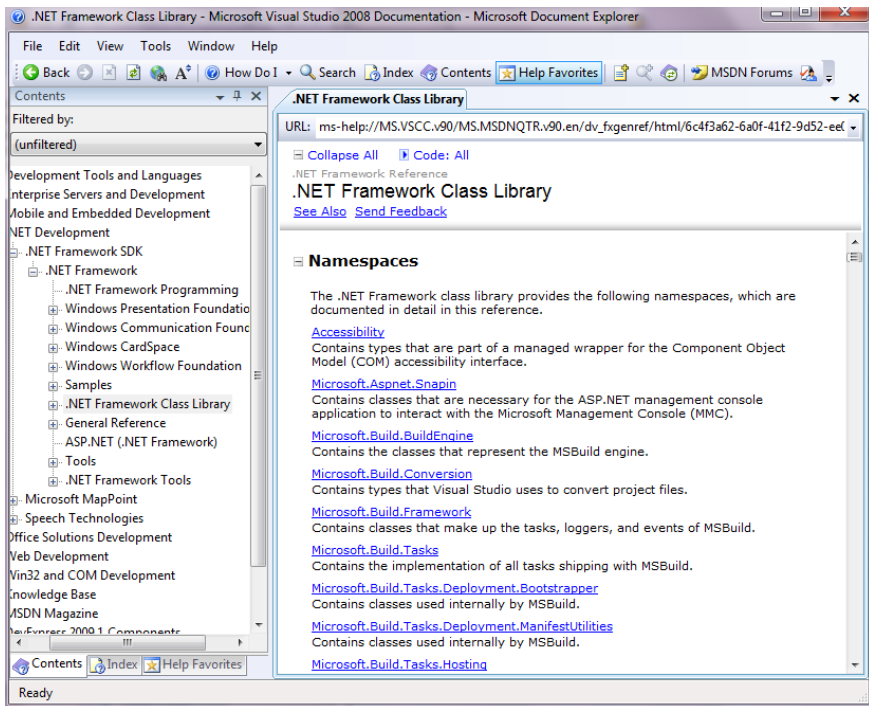
kode yang kita buat dapat dikelompokkan menjadi lebih rapi dan terstruktur.

Namespace 'ActInsurance' penting karena digunakan untuk membedakan dengan program lain, misal untuk perusahaan asuransi Ecme juga memiliki namespace 'EcmeInsurance' dan didalamnya memiliki namespace dengan nama sama yaitu 'Policy' tapi implementasi class-class yang ada di dalamnya berbeda dengan namespace 'Policy' pada namespace 'ActInsurance' yang sudah dibuat sebelumnya. Untuk mengakses 'Policy' pada 'EcmeInsurance' digunakan 'EcmeInsurance.Policy' sedangkan untuk mengakses 'Policy' pada 'ActInsurance' digunakan 'ActInsurance.Policy'.

Dengan menggunakan namespace kita tidak akan bingung untuk mengakses class-class yang kita inginkan karena sudah dikelompokkan dengan terstruktur dan dibedakan dengan nama namespacesnya.

VB6 Note: Jika kita membandingkan namespace dengan Windows API pada VB6, maka dapat dilihat bahwa penggunaan namespace pada .NET lebih rapi, terstruktur dan mempermudah developer. Windows API hanya mendukung penggunaan single namespace dengan ribuan function didalamnya yang akan sangat menyulitkan dalam pemberian nama dan pengelompokan function.

Jika kita melihat BCL (Base Class Library) / FCL (Framework Class Library) yang ada pada .NET, ribuan class library tersebut dikelompokkan ke dalam lebih dari 100 namespace. Misal untuk menggunakan class-class untuk membuat GUI pada aplikasi windows anda dapat menggunakan namespace 'System.Windows.Form'. untuk lebih jelasnya anda dapat membuka Visual Studio 2008 Documentation, arahkan ke .NET Development - .NET Framework SDK - .NET Framework - .NET Framework Class Library.



Gambar 3.1 Class Library Reference

Assemblies

Setelah pada topik sebelumnya kita membahas tentang namespaces untuk mengelompokkan class-class yang kita buat, sekarang kita akan belajar untuk menggunakan namespace.

Untuk menggunakan namespace pada aplikasi yang anda buat, anda butuh untuk mengakses *assembly* yang tepat yang berupa file fisik (.dll atau .exe) yang mengandung kode yang sudah anda compile. Misal jika anda ingin menggunakan namespace 'System.Windows.Form' maka anda harus mengakses file 'System.Windows.Form.dll' (biasanya nama

namespace sama dengan nama assembly namun bisa saja lain walau tidak disarankan).

Types

Setelah membaca tentang Class Library anda mungkin bertanya apa yang ada didalam Class Library, jawabannya adalah Types. Types pada .NET terdiri dari classes, event, structure, dan beberapa fitur baru seperti enumeration dan delegates.

Model ini sangat berbeda dengan VB classic (VB6), VB6 memiliki beberapa object bawaan seperti Printer, Screen, App dan Err, anda juga dapat mengakses COM jika ingin komponen yang lebih lengkap. Lain halnya dengan VB9 yang sudah memiliki ratusan object yang dikelompokan dalam namespace pada .NET Class Library.

Class, Object dan Instance

Mungkin anda pernah bertanya apa sich bedanya class dan object?, ilustrasi ini mungkin bisa menjelaskan perbedaannya. Jika anda ingin membangun rumah Class adalah blueprint / gambar (berisi spesifikasi lengkap dari rumah yang akan dibuat, ukuran, jumlah, dll).

Sedangkan object adalah bentuk fisik / rumah yang dibangun berdasarkan blueprint / class yang sudah ada.



Gambar 3.2 Class vs Object

Sebuah class terdiri dari properties, method, dan events (yang biasa disebut dengan member).

Instance adalah copy unik dari class yang merepresentasikan object. Untuk menggunakan class anda harus membuat instance terlebih dahulu baru dapat mengakses class member, kecuali anda menggunakan shared member, maka anda tidak perlu membuat instance untuk mengakses member yg ada dalam class tersebut.

Properties

Properties menyimpan informasi tentang object yang dibuat. Sebagai contoh pada aplikasi windows object 'TextBox' memiliki property 'Text', yang berisi teks yang akan ditampilkan pada textbox. Jika anda memiliki textbox dengan nama txtNama maka anda dapat menuliskan **strNama = txtNama.Text** untuk mengcopy nilai dari textbox ke variabel strNama.

Method

Method adalah perintah yang membuat object melakukan sesuatu. Sebagai contoh **MessageBox.Show()** menggunakan **Show()** method dari MessageBox object untuk menampilkan pesan, dan **MyDoc.Print()** menggunakan **Print()** method menggunakan **MyDoc** object untuk mengirimkan data ke printer. Penulisan method pada VB9 diikuti dengan tanda kurung ().

Events

Events adalah notifikasi yang dikirimkan oleh object, dimana anda dapat *listen* atau *ignore* notifikasi tersebut. Sebagai contoh object button mengirimkan Click event, dan anda dapat merespond event tersebut dan menjalankan code anda.

Instance Member dan Shared Member

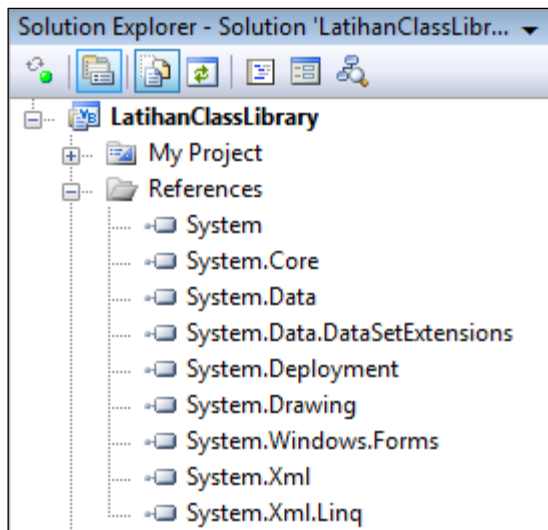
Bagi pemula salah satu aspek yang paling membingungkan pada OOP adalah Instance dan Shared member. Anda dapat menggunakan member pada class dengan cara membuat instance terlebih dahulu, kemudian baru dapat mengakses member yang ada di dalamnya, tapi pada kasus tertentu anda juga dapat langsung mengakses member tersebut tanpa harus membuat instance, caranya adalah menggunakan keyword 'shared' pada member.

Menggunakan Class Library

Sekarang kita akan mencoba membuat aplikasi yang menggunakan class library didalamnya. Ada 2 cara untuk menggunakan class library, pertama pastikan project anda mereferensi class library tersebut, kemudian yang kedua import namespace yang akan digunakan untuk mempermudah dalam penulisan kode.

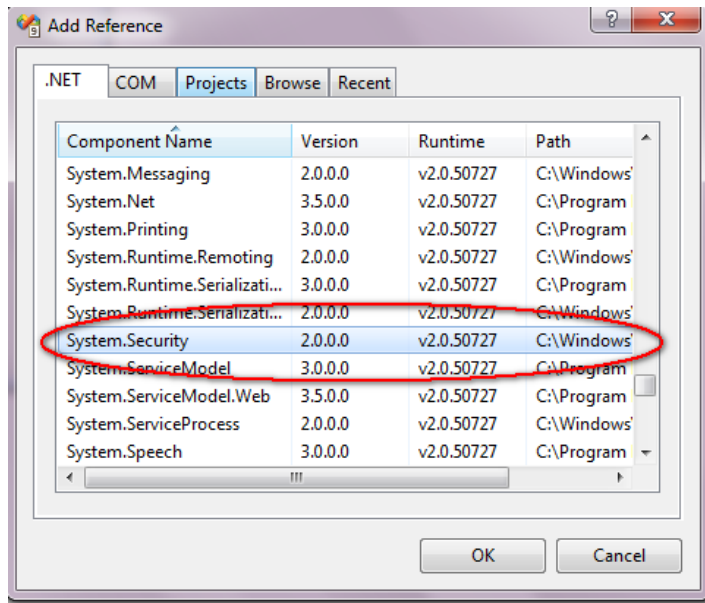
Listing 3.1 Menggunakan Class Library

1. Buka Visual Studio 2008, kemudian buat project Windows App baru dengan nama 'LatihanClassLibrary'.
2. Pada Solution Explorer pilih 'Show All Files', kemudian pilih 'Reference', maka disana akan ada daftar dari class library yang digunakan.



Gambar 3.3 Class Library

3. Misal anda ingin menggunakan class library untuk enkripsi file, maka anda dapat menambahkan referensi dengan cara klik kanan pada references, kemudian pilih 'Add Reference', akan muncul daftar library yang dapat digunakan, kemudian pilih 'System.Security'.



Gambar 3.4 Pilih Referensi Class

- Setelah menambahkan library **System.Security**, langkah berikutnya adalah menggunakannya pada program yang kita buat.
- Cara untuk menggunakan referensi yang telah kita tambahkan pada program adalah dengan menggunakan keyword 'Imports' dan menyebutkan nama namespace dengan lengkap.
- Pada form tambahkan control berikut:

Control	Property	Value
TextBox	Name	txtPassword
TextBox	Name	txtMd5
Button	Name	txtProses
	Text	Proses

- Kemudian tuliskan kode berikut :

```
Imports System.Security.Cryptography
Imports System.Text
```

```
Public Class Form1
```

```

Private Sub btnProses_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs)
Handles btnProses.Click
    Dim md5Obj As New MD5CryptoServiceProvider
    Dim byteToHash() As Byte =
        Encoding.ASCII.GetBytes(txtPassword.Text)
    byteToHash = md5Obj.ComputeHash(byteToHash)

    txtMd5.Text =
        Convert.ToBase64String(byteToHash, 0,
            byteToHash.Length)
End Sub
End Class

```

8. Pada kode diatas kita menggunakan keyword **Imports System.Security.Cryptography** agar dapat menggunakan class MD5 yang ada didalam namespace tersebut. MD5 adalah salah satu algoritma standar untuk menghitung fungsi hash (biasanya digunakan untuk mengenkripsi password).

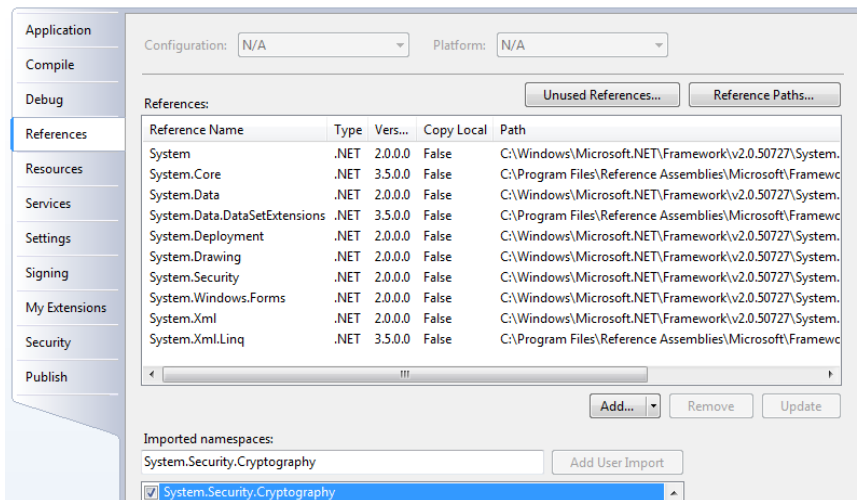
Dapat kita lihat pada contoh diatas bahwa .NET Framework sudah menyediakan koleksi library yang banyak, sehingga anda tidak perlu membuat sendiri dari awal. Koleksi class library pada .NET Framework juga sangat terstruktur sehingga memudahkan kita dalam melakukan pencarian.

Project Wide Import

Dengan menggunakan statement **Imports** maka class library tersebut hanya dapat diakses di file itu sendiri (pada aplikasi diatas filenya Form1.vb), jadi ketika anda membuat file yang lain misal Form2.vb maka anda harus menuliskan kembali statement **Imports** jika hendak menggunakan class MD5 yang ada dalam **System.Configuration**.

Agar anda tidak perlu menambahkan statements Imports di setiap file, maka anda dapat menggunakan Imports untuk

keseluruhan file pada project anda. Caranya klik kanan pada project, kemudian pilih properties, dan pilih references tab.



Gambar 3.5 References Tab

Anda dapat menambahkan **System.Security** kedalam 'References' dan mencontreng **System.Security.Cryptography** pada 'Imported namespace' agar tidak perlu menuliskan **Imports System.Security.Cryptography** di setiap file.

Menggunakan Aliases

Anda juga dapat menggunakan alias pada Imports statement, alias akan sangat berguna jika anda mengimport beberapa namespace yang didalamnya memiliki nama class yang sama.

```
Imports sc = System.Security.Cryptography
Imports wf = System.Windows.Forms
Dim objMd5 As New sc.MD5CryptoServiceProvider
wf.MessageBox.Show("Hello !!")
```

Macam-macam Class Library Namespace

Untuk mempermudah anda mengerti struktur Class Library Namespaces yang sering digunakan pada .NET maka

disini akan dijelaskan beberapa namespace untuk memberikan gambaran singkat kepada anda.

System

Ini adalah core namespaces yang berisi basic data types seperti string, array, events dan exception yang akan anda pelajari di bab selanjutnya.

System.Collection dan System.Collection.Generic

Namespace ini berisi class-class collection. Collection adalah object yang dapat mengandung group dari object.

System.Data

Namespace ini berisi types yang dibutuhkan oleh ADO.NET jika anda ingin bekerja dengan database seperti Sql Server atau Oracle.

System.Drawing

Namespace ini berisi class-class yang digunakan jika anda hendak bekerja dengan font, color, dan menggambar langsung pada form. Fitur-fitur tersebut dibundle menjadi satu koleksi yang dinamakan GDI+.

System.Drawing.Printing

Ini adalah namespace yang berisi class-class untuk mensupport fasilitas print dan print preview.

System.IO

Namespace ini berisi class-class yang digunakan untuk mengakses file termasuk manajemen file, membaca, dan menulis ke file.

System.Net

Namespace ini berisi class-class yang digunakan untuk komunikasi jaringan secara low-level, misal anda ingin mengambil data dari web tanpa menggunakan ASP.NET.

System.Runtime.Serialization

Ini adalah class library yang berisi beberapa namespace yang digunakan untuk serialisasi object.

System.Threading

Namespace ini digunakan jika anda hendak membuat program aplikasi yang multithread.

System.Web

Digunakan jika anda ingin membuat aplikasi berbasis web dengan menggunakan ASP.NET.

System.Windows.Forms

Namespace ini menyediakan semua types yang dibutuhkan jika anda hendak membuat aplikasi berbasis Windows Desktop, termasuk control textbox, button, dll.

System.Xml

Namespace ini digunakan jika anda hendak bekerja dengan dokumen XML.

Microsoft Visual Basic

Ini adalah small namespace yang berisi fungsi-fungsi dari VB6 (VB Classic) yang dapat digunakan di .NET seperti string manipulation function (**left()**, **right()**).

Selain yang disebutkan diatas masih banyak namespace yang lain yang ada di .NET Framework seperti untuk security, untuk mengakses FTP, Regular Expression, Advanced String Manipulation, dll.

Menggunakan My Object

Setelah VB7 direlease (.NET 1.0), Microsoft menyadari bahwa developer VB kesulitan dalam menemukan class-class yang akan digunakan, karena Class Library yang ada di .NET terlalu banyak dan besar. Karena alasan tersebut VB Team dari Microsoft mencoba membuat shortcut untuk mengakses class-class penting yang sering digunakan dengan membuat sebuah object dengan nama My object.

Sayangnya My object hanya menyediakan shortcut untuk mengakses sebagian kecil dari Framework Class Library yang sangat banyak, karena jika terlalu banyak My object akan semakin besar dan kompleks, ini akan menyulitkan programmer untuk mempelajarinya, karena cara pengaksesannya yang tidak standar.

Dengan menggunakan My object anda dapat bekerja dengan lebih cepat, misalnya anda ingin mengetahui informasi dari komputer anda seperti waktu, username, computer name, dan directory, anda dapat menuliskanya seperti ini:

```
MessageBox.Show(My.Computer.Clock.LocalTime)
MessageBox.Show(My.Computer.Name)    MessageBox.Show(
    My.Computer.FileSystem.CurrentDirectory)
MessageBox.Show(My.User.Name)
```

My Core Object

Untuk memudahkan penggunaan My object Visual Studio sudah menyediakan intellisense sehingga pengguna mudah mencari object yang dibutuhkan. My object dibagi menjadi 7 sub bagian pokok yaitu:

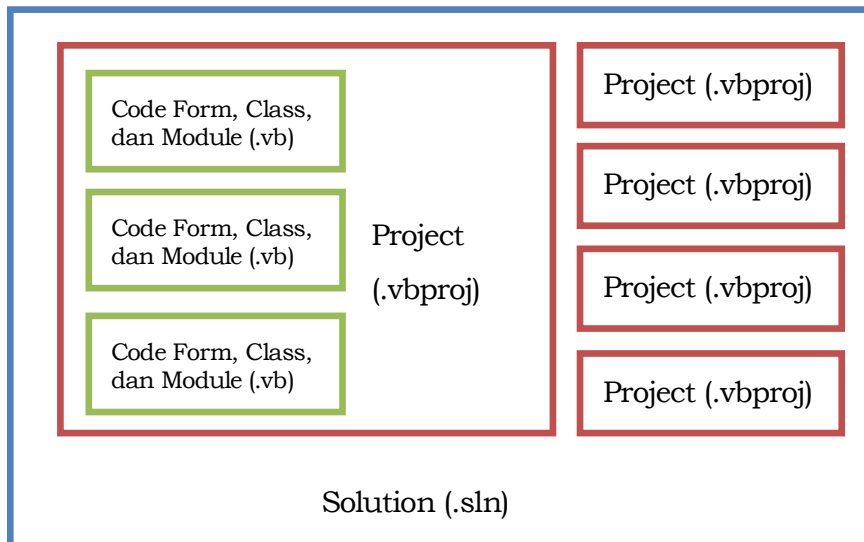
- **My.Computer** : berisi object yang digunakan untuk mengambil informasi dari komputer. Misal anda dapat mengakses clipboard (**My.Computer.Clipboard**), memainkan lagu (**My.Computer.Audio**), mengambil informasi dari registry (**My.Computer.Registry** dan **My.Computer.FileSystem**), dan mengecek network status (**My.Computer.Network**).
- **My.Application** : menyediakan informasi tentang aplikasi yang anda buat termasuk assembly dan version number, dan folder dimana aplikasi anda dijalankan.
- **My.User** : object ini menyediakan informasi tentang user yang sedang login pada computer, anda dapat menggunakan object ini untuk menuliskan security code yang mengecek user account atau group membership.
- **My.Setting** : digunakan untuk mengambil informasi dari application configuration file.
- **My.Resources** : digunakan untuk mengambil resources (binary atau text data yang dimasukkan kedalam project pada saat kompilasi, misal: image, sound).
- **My.Form** : adalah default instance yang digunakan pada setiap form, dapat digunakan untuk memanggil form yang lain yang ada dalam satu project.
- **My.WebServices** : object ini menyediakan default proxy object untuk setiap web services yang digunakan pada aplikasi anda.

Code Files

Pada Visual Basic 6 dan pendahulunya, terdiri dari beberapa file dengan tipe tertentu yang membentuk project, Form file (.frm) yang berisi Graphical layout dan event handling code dari form, class files (.cls) yang berisi single class yang anda buat sendiri, Module files (.mod) yang mengandung variable dan function yang dapat diakses secara global. File-file yang sudah disebutkan diatas dikelompokkan menjadi sebuah group yang deskripsinya ditulis pada Visual Basic project file (.vbproj). Karena jenis file yang bervariasi maka akan relatif sulit untuk memaintain file-file tersebut.

Namun VB9 memiliki model manajemen file yang berbeda dengan VB6, yaitu:

- Ketika anda membuat aplikasi baru maka secara otomatis anda membuat file solution (.sln).
- Satu solution dapat berisi satu atau lebih project file (.vbproj).



Gambar 3.6 Solution File

- Satu project terdiri dari banyak code file (.vb) yang dapat berisi multiple class, module, dan form.

Class dan Module Block

Code File (.vb) dapat berisi multiple class atau module, misal anda dapat membuat class dan module dalam sebuah code file (.vb)

```
Public Class LatihanClass
    'tulis kodenya disini
End Class
```

```
Public Module MyModule
    'tulis kodenya disini
End Module
```

Didalam class atau modul anda dapat menambahkan procedure atau menuliskan code seperti biasa, tapi anda tidak dapat menambahkan procedure atau variable declaration diluar class atau module.

```
Public bil As Double 'Tidak boleh
```

```
Public Class LatihanClass
    Private nim As String 'Boleh
    Public Sub MySub() 'Boleh
        'tulis kodenya disini
    End Sub
End Class
```

```
Public Sub Hello() 'Tidak benar

End Sub
```

```
Public Module MyModule
    Public Sub Hello() 'Boleh
        'tulis kodenya disini
    End Sub
End Module
```

Semua class dan module yang anda buat terdapat dalam root namespace (namanya sama dengan nama project anda),

pada VB9 root namespace ini tidak kelihatan. Jadi jika anda ingin mengakses method **Hello()** didalam module **MyModule** maka anda dapat menuliskannya sebagai berikut **MyProject.MyModule.Hello()**.

Namespace Block

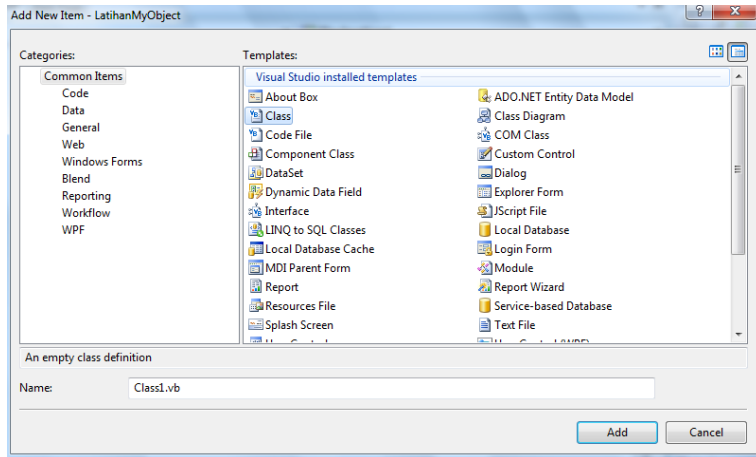
Semua code yang anda buat secara otomatis disimpan dalam root namespace, secara default VB9 akan menggunakan nama dari aplikasi/project anda sebagai root namespace. Untuk melihat nama root namespace klik kanan pada project kemudian pilih properties, kemudian lihat pada textbox Root Namespace, anda dapat mengubah nama namespacesnya jika diinginkan. Anda juga dapat mengubah nama dari assembly (ini akan sama dengan nama file .exe yang akan dihasilkan). Anda dapat menggunakan namespace untuk membantu mengorganisasikan kode pada aplikasi yang besar dan kompleks. Untuk menggunakan namespace tulis program anda didalam kalang namespace.

```
Namespace MyNamespace
    Module MyModule
        Public Sub MySub()
            'tulis kode disini
        End Sub
    End Module
End Namespace
```

Pada contoh diatas and adapt mengakses subrutin **MySub** dengan menuliskan **MyNamespace.MyModule.MySub()**.

Menambahkan Code File

Anda dapat menambahkan code file seperti module, class, form dengan mudah, caranya klik kanan pada project ► add new item ► kemudian pilih code file yang akan ditambahkan kedalam project.



Gambar 3.7 Menambahkan Code File.

Tipe Data

Hampir semua tipe data yang ada di VB6 terlihat sama dengan yang ada di VB9. Misalnya code berikut masih valid.

```
Dim intBil As Integer
Dim strNama As String
Dim tanggal As Date
```

Biarpun terlihat sama tapi ada beberapa perbedaan yang mendasar yaitu semua tipe data pada VB9 terdapat dalam namespace khusus yaitu **System**. Sebagai contoh keyword Date sebenarnya sama dengan **System.DateTime**, dan keyword String sebenarnya merepresentasikan **System.String**, ini yang disebut sebagai CTS (Common Type System).

Value Type vs Reference Type

Tipe Data pada VB9 sebenarnya dibagi menjadi 2 macam yaitu value type dan reference type.

Value Type

Tipe data value type disimpan pada stack memory, tidak bersifat garbage collected. Value type diturunkan dari **System.ValueType** namespace. Jenis data value type memiliki copy data masing-masing, sehingga perubahan nilai pada satu variable tidak berpengaruh pada variable yang lain. Contoh jenis tipe data value type adalah:

- Simple Types :
 - signed : **Sbyte, Short, Integer, Long**
 - unsigned : **Byte, Ushort, UInt, Ulong**
 - unicode characters : **Char**
 - floating point : **Single, Double**
 - high precision: **Decimal**
 - boolean : **Boolean**
- Enum Types
 - user defined type: **Enum..End Enum**
- Struct Types
 - user defined type: **Structure..End Structure**

Contoh penggunaan Value Type pada program:

```
Dim bil1 As Integer = 5
Dim bil2 As Integer = bil1 'bil2 juga bernilai 5
bil2 = 7 'bil2 dirubah nilainya menjadi 7
```

```
'bil1 tetap bernilai 5 dan bil2 bernilai 7
MsgBox("bil1 : " & bil1 & " bil2 : " & bil2)
```

Dari program diatas dapat dilihat bahwa biarpun **bil1=bil2** namun perubahan nilai pada **bil2** tidak menyebabkan **bil1** berubah, ini disebabkan karena value type memiliki copy data pada masing-masing variable.

Category	Bits	Type	Range/Precision
Signed integer	8	Sbyte	-128...127
	16	Short	-32,768...32,767
	32	Integer	-2,147,483,648...2,147,483,647

	64	Long	-9,223,372,036,854,775,808 ...9,223,372,036,854,775,807
Unsigned integer	8	Byte	0...255
	16	Ushort	0...65,535
	32	UInteger	0...4,294,967,295
	64	Ulong	0...18,446,744,073,709,551,615
Floating point	32	Float	1.5×10^{-45} to 3.4×10^{38} , 7-digit precision
	64	Double	5.0×10^{-324} to 1.7×10^{308} , 15-digit precision
Decimal	128	Decimal	1.0×10^{-28} to 7.9×10^{28} , 28-digit precision

Reference Type

Berbeda dengan value type yang dialokasikan di stack memory, reference type dialokasikan di heap memory. Reference type juga bersifat garbage collected sehingga akan otomatis dibersihkan oleh CLR jika object sudah tidak digunakan lagi.

Reference type diturunkan dari **System.Object** namespace. Satu object dapat direferensi oleh lebih dari satu object yang lain, jadi perubahan nilai pada object referensi yang sama akan mempengaruhi nilai pada object yang lain. Reference type mirip dengan tipe pointer pada bahasa C.

Yang termasuk jenis tipe data Reference Type:

- Class Type
 - Base class dari semua class : **Object**
 - Unicode string : **String**
 - User defined type : **Class..End Class**
- Interface Type
 - User defined type : **Interface..End Interface**
- Array Type
 - Single dan multidimensi : **Dim nama(3) As String, Dim matrix(2, 2) As Integer**
- Delegate Type

Listing 3.2 Penggunaan Reference Type

1. Buat project windows application dengan nama 'LatihanTypes'.
2. Tambahkan button control, set property name='btnReference', dan text='Reference Type'.
3. Klik kanan pada project kemudian tambahkan class baru dengan nama 'Mahasiswa.vb'. tulis kode berikut:

```
Public Class Mahasiswa
    Public nim As String
    Public nama As String
    Public ipk As Double
End Class
```

4. Kemudian double click pada tombol 'btnReference'. Tulis kode berikut:

```
Private Sub btnReference_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnReference.Click
    Dim objMhs1 As New Mahasiswa
    'objMhs2 mereferensi object yg sama dengan objMhs1
    Dim objMhs2 As Mahasiswa = objMhs1
    'objMhs3 mereferencsi object yg sama dengan objMhs2
    Dim objMhs3 As Mahasiswa = objMhs2
    objMhs1.nim = "23070007"
    objMhs1.nama = "Erick"
    MessageBox.Show("Mhs1 nim : " & objMhs1.nim & "
        Mhs1 nama : " & objMhs1.nama)
    MessageBox.Show("Mhs2 nim : " & objMhs2.nim & "
        Mhs2 nama : " & objMhs2.nama)
    MessageBox.Show("Mhs3 nim : " & objMhs3.nim & "
        Mhs2 nama : " & objMhs3.nama)
    'merubah field nama pada objMhs2
    objMhs2.nama = "Rully"
    MessageBox.Show("Mhs1 nim : " & objMhs1.nim & "
        Mhs1 nama : " & objMhs1.nama)
    MessageBox.Show("Mhs3 nim : " & objMhs3.nim & "
        Mhs3 nama : " & objMhs3.nama)
End Sub
```

Pada contoh program diatas dapat dilihat bahwa jenis tipe data reference type berbeda dengan value type, karena jika **objMhs2** mereferensi ke **objMhs1**, dan **objMhs3** mereferensi ke **objMhs2**, maka ketika anda merubah field nama pada **objMhs2** maka otomatis fields nama pada **objMhs1** dan **objMhs3** ikut berubah karena mempunyai satu referensi object yang sama.

Common Type System

CTS (Common Type Sistem) adalah bagian dari spesifikasi Common Language Infrastructure (CLI) yang digunakan oleh semua .NET Language (VB, C#, C++).

CTS adalah definisi bagaimana types dideklarasikan, digunakan, dan diatur pada runtime, dan merupakan bagian penting yang mendukung cross-language integration.

Pada VB9 anda dapat menggunakan keyword yang disediakan oleh VB atau menggunakan System namespace.

- **VB String = System.String**
- **VB Integer = System.Int32**
- **VB Long = System.Int64**
- **VB Date = System.DateTime**
- **VB String = C# String**

System Types

Tipe yang ada pada **System** namespace dapat digunakan oleh semua .NET Language (VB, C#, C++), dengan begitu anda dapat membuat program dengan bahasa berbeda dan kemudian mengintegrasikannya tanpa masalah. Jika anda ingin mengupgrade program yang ada di VB6 maka anda harus menyesuaikan dengan CTS yang ada di .NET. Sebagai contoh Integer pada VB9 sama dengan **System.Int32** pada CTS, karena Int32 maka range nilainya dari -2M sd 2M, ini

jauh lebih besar daripada range tipe Integer pada VB6 yang hanya 16bit (-32.000 sd 32.000).

```
Dim intBil1 As Integer '32 bit integer  
Dim intBil2 As System.Int32 '32 bit integer
```

Tipe String pada VB9 juga relatif sama dengan VB6 kecuali sekarang anda tidak lagi dapat menggunakan fixed-length string.

Deklarasi Banyak Variable

VB9 mendukung deklarasi banyak variable sekaligus seperti:

```
Dim intA, intB, intC As Integer
```

Pada VB6 jika anda menuliskan statement diatas maka artinya anda mendeklarasikan **intC** sebagai tipe Integer, dan **intA**, dan **intB** sebagai tipe varian (varian adalah tipe variable default di VB6). VB9 sudah tidak lagi mendukung varian, jika anda ingin mendeklarasikan variable yang dapat diisi dengan nilai yang tipenya berbeda, maka anda dapat menggunakan tipe yang umum yaitu **System.Object**.

Initializer

Initializer adalah salah satu fitur yang mempermudah anda dalam memprogram, anda dapat memberi nilai pada variable yang anda deklarasikan sevara langsung.

```
Dim intBil1 As Integer
```

Anda juga dapat menggunakannya untuk inisialisasi object array.

```
'membuat array sekaligus menambahkan 5 nilai  
Dim numArray() As Integer = {12, 34, 56, 78, 14}
```

Dapat juga digunakan untuk membuat object, dan langsung menginisialisasinya menggunakan konstruktor.

```
Dim myFile As System.IO.FileInfo =  
New System.IO.FileInfo("d:\vbnet.txt")
```

```
Dim myFile2 As New System.IO.FileInfo("d:\vbnet.txt")
```

Anda juga dapat membuat variable didalam statement program yang lain, seperti pada pemanggilan function. Tapi ini dapat membuat kode anda sulit untuk dibaca.

```
AmbilInfoFile(New System.IO.FileInfo("d:\vbnet.txt"))
```

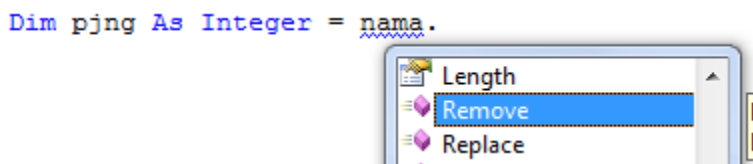
String

Pada .NET tipe String adalah object dari System.String. Karena String merupakan object maka sudah tersedia method-method bawaan untuk manipulasi string. Misalnya anda dapat menggunakan method **Length()** untuk mengetahui panjang String.

```
Dim nama As String = "Erick Kurniawan"  
'anda dapat menggunakan cara lama  
Dim jml As Integer = Len(nama)  
'atau menggunakan cara baru pada .NET  
Dim pjng As Integer = nama.Length
```

Cara pertama yang menggunakan method `len()` tidak disarankan untuk digunakan karena mungkin anda tidak dapat menghapuskan semua built-in function yang jumlahnya ribuan. Kelemahan kedua adalah built-in function ini tidak terorganisasi dengan baik jadi kita tidak tahu function yang digunakan masuk dalam kategori apa.

Cara kedua lebih disarankan karena lebih terorganisir, function **Length()** ada pada class System.String jadi lebih mudah untuk menemukannya. Dengan cara kedua anda juga dibantu oleh intellisense pada Visual Studio sehingga lebih memudahkan.



Gambar 3.8 Bantuan Intellisense

Anda juga dapat menggunakan function yang lain yang ada pada **System.String**. Jika anda ingin melihat detail function apa saja yang dipunyai oleh **System.String** anda dapat membuka MSDN documentation.

```
Dim myName As String = " Erick Kurniawan "  
'huruf besar dan menghilangkan spasi kanan kiri  
MessageBox.Show(myName.ToUpper().Trim())  
  
'memotong string dari karakter ke 1 sebanyak 5  
Dim potong As String = myName.Substring(1, 5)  
MessageBox.Show(potong)  
  
'konversi integer to string  
Dim bil As Integer = 14  
Dim strBil As String = bil.ToString()
```

VB6. Fungsi-fungsi string yang lama pada VB6 masih disupport (backward compatibility), hanya lebih disarankan menggunakan cara Object Oriented yang lebih elegant.

Bekerja Lebih Efisien dengan StringBuilder

Menggunakan String pada VB9 cukup mudah, tapi untuk beberapa kasus tertentu performanya tidak bagus, misalnya anda ingin menambahkan string baru pada string yang sudah ada secara berulang.

```
Dim strTampung As String = ""  
For i = 1 To 10000  
    'menambahkan string  
    strTampung &= i.ToString & " "  
Next
```

Kode diatas akan sangat tidak efisien dan sangat jelek dalam performa karena String pada VB9 bersifat immutable, jadi ketika anda menggunakan operator **&=** untuk menambahkan string baru ke string yang lama sebenarnya

operasinya tidak sekedar ditambah tapi membuat object string baru lagi.

Untuk menangani sifat string yang immutable maka jika anda ingin secara dinamis menambahkan string disarankan menggunakan object `StringBuilder`. `StringBuilder` memiliki method **`Append()`** untuk menambahkan string, cara ini jauh lebih efisien dan lebih cepat dari sisi performa. Untuk menggunakan `StringBuilder` anda harus mengimport namespace **`System.Text`**. Untuk mengkonversi `StringBuilder` menjadi string biasa anda dapat menggunakan method **`ToString()`**.

```
Imports System.Text
Dim sb As New StringBuilder
For i = 1 To 10000
    sb.Append(i.ToString())
Next
Dim strTmp As String = sb.ToString
```

Dates and Times

Tipe data `Date` pada VB9 juga sangat mudah digunakan, karena memiliki banyak function bawaan sehingga memudahkan anda mengambil informasi `Date Time` dan melakukan proses kalkulasi. Anda juga dapat menggunakan `TimeSpan` class untuk menyimpan data interval waktu. Perhatikan contoh dibawah ini.

```
Dim myDate As Date
Dim mySpan As TimeSpan

myDate = Date.Now
mySpan = TimeSpan.FromDays(1)

'menampilkan hari
MessageBox.Show(myDate.Day)

'menambahkan satu hari
myDate = myDate.Add(mySpan)
```

```
MessageBox.Show(myDate.Day)
```

```
'mengurangi satu hari dengan fungsi bawaan  
myDate = myDate.AddDays(-1)  
MessageBox.Show(myDate.Day)
```

Array

Array pada VB9 selalu berawal dari index 0 untuk menyesuaikan dengan class library pada .NET (ini berbeda dengan VB6 yang mempunyai index 1 di beberapa tipe collection tertentu).

Untuk mendeklarasikan array anda harus menuliskan batas atas (upper boundary) dari array tersebut.

```
Dim arrNama(10) As String 'membuat 11 element dari 0 sd 10  
Dim arrBil1(1 To 10) As Integer 'tidak boleh karena lower  
bound harus 0  
Dim arrBil2(0 To 10) As Integer 'boleh
```

Anda dapat juga menginisialisasi array dengan cara berikut:

```
'langsung inisialisasi pada saat deklarasi  
Dim number() As Integer = {12, 45, 67, 33}  
  
Dim number(3) As Integer  
number(0) = 12  
number(1) = 34  
number(2) = 88  
number(3) = 76
```

Array pada .NET ukurannya terbatas (mempunyai limit), jika anda ingin tipe koleksi yang lebih fleksible maka anda dapat menggunakan object oriented type collection.

Array dan IEnumerable

Salah satu tambahan fitur untuk array di VB9 adalah IEnumerable Interface, dengan IEnumerable anda dapat

melakukan iterasi terhadap semua elemen dalam array menggunakan keyword `For..Each`, cara ini lebih mudah dibandingkan anda melakukan perulangan dari index ke 0 sampai dengan batas atas array seperti yang biasa anda lakukan.

Namun dengan menggunakan `For..Each` anda hanya dapat membaca elemen saja (readonly), tidak untuk mengubah datanya.

```
Dim arrNama() As String = {"erick", "rully", "budi",  
"bayu"}  
  
'mengambil semua elemen pada arrNama  
For Each nama In arrNama  
    'memasukan nilai kedalam list  
    lstNama.Items.Add(nama)  
Next
```

Fitur Bawaan Array

Sama seperti `String`, `Array` juga merupakan object dari `System.Array` class. Karena merupakan object maka array juga memiliki method bawaan yang cukup lengkap untuk manipulasi array. Sebagai contoh anda dapat mengetahui informasi batas atas dan batas bawah dari elemen array dengan menggunakan method `GetUpperBound()` dan `GetLowerBound()`.

```
Dim arrNama() As String = {"erick", "rully", "budi",  
"bayu"}  
For i = arrNama.GetLowerBound(0) To  
arrNama.GetUpperBound(0)  
    lstNama.Items.Add(arrNama(i))  
Next
```

Parameter 0 yang diberikan mempunyai arti dari elemen yang pertama (Pada VB9 array dimulai dari index ke 0). Karena batas bawah sudah pasti 0, sebenarnya anda dapat langsung menuliskan 0 daripada menggunakan `GetLowerBound(0)`.

Dua method diatas menggantikan fungsi **LBound()** dan **UBound()** pada VB6 meskipun anda masih dapat menggunakan dua fungsi lama ini, tapi tidak disarankan.

Class Array juga menyediakan *shared* method (akan dibahas di chapter 5 tentang OOP), anda harus menuliskan nama class baru kemudian nama shared methodnya.

```
Dim arrBil() As Integer = {12, 45, 22, 66, 11}
```

```
'akan diurutkan secara ascending
Array.Sort(arrBil)
For Each bil In arrBil
    lstNama.Items.Add(bil)
Next
```

Anda juga dapat melakukan pencarian alamat index array dengan mudah.

```
Dim arrNama() As String = {"erick", "rully", "budi",
                           "bayu"}

'jika ditemukan akan mengembalikan alamat dari elemen
'jika tidak ditemukan akan mengembalikan -1
Dim pos As Integer = Array.IndexOf(arrNama, "budi")
MessageBox.Show(pos)
```

Untuk mengetahui method-method yang ada dalam class Array anda dapat melihat referensi class library.

Array sebagai Reference Type

Adalah sangat penting untuk mengetahui bahwa array pada VB9 adalah reference type, tidak sama dengan tipe data lain seperti Integer, Double yang value type. Jadi anda harus hati-hati dalam memperlakukan object array agar tidak keliru. Contoh perbedaan array dengan tipe data value type dapat dilihat pada contoh dibawah ini.

```
Dim intBil1 As Integer = 12
'karena Integer adalah value type maka 12 akan 'dikopikan
nilainya kedalam intBil2
```

```

Dim intBil2 As Integer = intBil1
'jika intBil2 nilainya diubah menjadi 34
intBil2 = 34
'maka nilai intBil1 tidak berubah tetap 12
MessageBox.Show(intBil2)

'berbeda dengan array yang reference type
Dim arrBil1() As Integer = {12, 34, 56, 12, 44}
'statement ini artinya arrBil1 dan arrBil2 mereferensi 'ke
object yang sama
Dim arrBil2() As Integer = arrBil1
'jika salah satu elemen arrBil2 dirubah maka elemen 'pada
arrBil1 juga ikut berubah
arrBil2(2) = 99
MessageBox.Show(arrBil1(2))

'jika anda tidak ingin mereferensi ke object array 'yang
sama, tapi hendak membuat object baru maka anda 'dapat
menggunakan method clone()
Dim arrBil3() As Integer = {2, 3, 4}
Dim arrBil4() As Integer = arrBil3.Clone()
'jika elemen pada arrBil4 diubah maka sudah tidak
'berpengaruh pada arrBil3 karena beda referensi object
arrBil4(2) = 55
MessageBox.Show(arrBil3(2))

```

Anda dapat menggunakan method **clone()** untuk membuat object baru sekaligus mengkopi datanya jika tidak menghendaki kedua variable tersebut mengacu ke object yang sama.

Perubahan pada Operator

Operator yang ada pada VB9 relatif sama dengan yang ada pada VB6, seperti untuk mengoperasikan number dan string, hanya ada beberapa tambahan fitur shortcut untuk mempermudah penggunaan operator.

Assignment Shorthand

VB9 menyediakan langkah yang lebih mudah untuk bekerja dengan operator arithmetic, cara ini sama dengan yang ada pada bahasa C pada umumnya.

```
Dim intBil As Integer = 12
intBil += 2 'sama dengan intBil = intBil + 2
intBil -= 2
intBil *= 2
intBil /= 2
Dim strNama As String = "Erick "
'sama dengan strNama & "Kurniawan"
strNama &= "Kurniawan"
```

Konversi Antar Tipe Data

VB9 menyediakan option untuk menggunakan automatic conversion (Option Strict), anda dapat mengatur 'Option Strict' menjadi 'On' atau 'Off'. Secara default opsi 'Option Strict' pada VB9 adalah 'Off'. Jika 'On' maka VB compiler akan secara otomatis mengkonversi tipe data (sama dengan di VB6). Namun untuk menghindari kesalahan yang tidak terdeteksi pada waktu *compile time checking* (konversi dari string ke number, atau konversi dari 32bit Integer menjadi 16bit Integer). *Compile time checking* adalah pengecekan kesalahan kode yang dilakukan visual studio pada saat program belum dijalankan (*run time*).

'akan dikonversi secara otomatis dan benar

```
Dim strNum As String = "12"
Dim intNum As Integer = strNum
```

'kode ini salah tapi tidak terdeteksi bila menggunakan
'Option Strict Off

```
Dim strBil As String = "erick"
'menghasilkan error invalid cast exception
```

```
Dim bil As Integer = strBil
Dim bil32 As Integer = 50000
'batas dari Int16 hanya sampai 32000, kode dibawah
'menyebabkan overflow
```

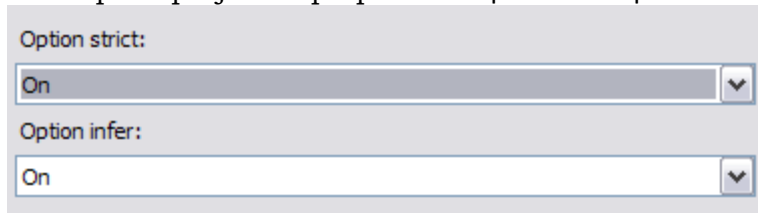
```
Dim bil16 As Short = bil32
```

Maka untuk menghindari error-error yang tidak terdeteksi seperti contoh diatas disarankan untuk mengubah opsi 'Option Strict' menjadi 'On'.

Konversi otomatis yang dilakukan oleh compiler (*late binding*) juga menyebabkan performa program anda menjadi lebih lambat. Namun pada kasus tertentu penggunaan 'Option Strict' menjadi 'Off' amat membantu terutama jika anda membutuhkan *late binding*. Late binding digunakan jika kita tidak tahu persis tipe dari objectnya. Ketika proses *late binding* sebenarnya tipe data dirubah menjadi object terlebih dahulu

```
'Contoh late binding
Dim objData As Object
objData = 34
'menghasilkan System.Int32
MessageBox.Show(objData.GetType().ToString)
objData = "Hello"
'menghasilkan System.String
MessageBox.Show(objData.GetType().ToString)
```

Anda dapat mengubah setting Option Strict dengan cara klik kanan pada project ► properties ► pilih tab compile.



Gambar 3.9 Setting Option Strict

Jika setting Option Strict menjadi 'On' maka anda harus melakukan konversi secara eksplisit. Anda dapat menggunakan method **Ctype()** atau method yang ada pada **System.Convert**.

```
Dim intBil As Integer = 12
'konversi data menggunakan method Ctype()
Dim dblBil As Double = CType(intBil, Double)
'menggunakan System.Convert
Dim dblBil2 As Double = Convert.ToDouble(intBil)
```

```
'menggunakan method bawaan Cdbl, Cint, Cstr
Dim dblBil3 As Double = CDb1(intBil)
Dim strBil As String = CStr(dblBil)
```

Fungsi Matematika

.NET mempunyai class Math yang berisi shared method untuk operasi matematika. Misal untuk menampilkan konstanta pi, menghitung sinus, cosinus, log, dan operasi yang lain.

```
Dim num As Double
num = Math.PI 'konstanta pi 22/7
num = Math.Sqrt(25) 'akar 25
num = Math.Sin(30)
num = Math.Abs(-30) 'fungsi absolut 30
num = Math.Round(25.6745, 2) 'pembulatan 25.67
num = Math.Log(1.44)
```

Random Numbers

Tidak seperti di VB6, VB9 menyediakan class System.Random sehingga mudah untuk menggenerate angka random. Misal contoh dibawah ini akan menggenerate angka random.

```
Dim rnd As New Random
Dim intBil As Integer
'menggenerate nilai random dari 0 sd 5
intBil = rnd.Next(5)

'menggenerate nilai random dari 1 sd 6
intBil = rnd.Next(5) + 1
```

Scope Variable

Ada beberapa aturan yang perlu dipahami tentang scope variable di .NET karena berbeda dengan VB6. Pada VB9 private variable yang dideklarasikan didalam class atau module tidak

dapat diakses oleh semua code diluar class atau module tersebut. Semua variable dalam method (sub atau function) juga tidak dapat diakses oleh kode diluar method tersebut, dan semua variable yang ada didalam loop atau conditional block juga tidak dapat digunakan oleh code diluar loop atau conditional block.

```
Sub ScopeVar()  
    Dim nama As String = "Erick"  
End Sub  
  
Sub Hello()  
    'variable nama di sub ScopeVar() tidak dapat diakses 'di  
    sub Hello()  
        nama = "Budi"  
    End Sub  
  
For i As Integer = 1 To 10  
    lstNama.Items.Add(i.ToString)  
Next  
    'variable i pada loop tidak dapat digunakan di luar 'block  
    loop  
i = 12  
  
Dim cek As Boolean = True  
If cek = True Then  
    Dim bil As Integer = 3  
End If  
  
    'variable bil tidak dapat diakses diluar blok if  
bil = 33
```

Short Circuit Logic

Pada VB6 anda dapat menggunakan dua tipe logical operator yaitu **And** dan **Or**, pada VB9 ditambahkan 2 operator baru yaitu **OrElse** dan **AndAlso**.

Penggunaan **OrElse** dan **AndAlso** sama dengan operator **Or** dan **And** biasa hanya saja operator baru ini mempunyai

kemampuan untuk mengecek hanya sebagian dari satu statement kondisi yang panjang.

```
Dim nama As String = "Erick"  
Dim ipk As Double = 3.4  
If nama = "budi" And ipk > 2.6 Then  
    'tuliskan kode disini  
End If
```

Pada program diatas kondisi pertama akan dicek terlebih dahulu kemudian kondisi kedua juga dicek, padahal untuk operator And jika salah satu kondisi sudah 'false' maka kondisi tersebut tidak akan menghasilkan 'true' jadi seharusnya kondisi kedua tidak perlu dicek lagi (pada operator And jika salah satu kondisi 'false' maka hasilnya juga pasti false). Dengan menggunakan AndAlso maka ketika kondisi pertama sudah menghasilkan 'false' maka kondisi kedua tidak perlu dicek karena kondisi tersebut sudah pasti menghasilkan 'false'.

```
Dim nama As String = "Erick"  
Dim ipk As Double = 3.4  
  
'kondisi kedua tidak dicek karena kondisi pertama 'sudah  
false maka pasti hasil akhirnya false  
If nama = "budi" AndAlso ipk > 2.6 Then  
    'tuliskan kode disini  
End If
```

Demikian juga dengan operator logical Or jika salah satu kondisinya sudah 'true' maka seharusnya kondisi yang lain tidak perlu dicek karena hasil akhirnya pasti menghasilkan 'true'.

```
'kondisi pertama true, kondisi kedua false, logika Or 'jika  
salah satu true maka hasil pasti true jadi 'kondisi kedua  
tidak perlu dicek lagi.  
If nama = "erick" OrElse ipk > 3.6 Then  
    'tuliskan kode disini  
End If
```

Dengan menggunakan short-circuit evaluation ini maka kode menjadi lebih efisien dan akan meningkatkan performa.

Skip Pada Saat Looping

VB9 menambahkan statement **continue** yang dapat anda gunakan pada block loop. Ada tiga macam keyword yaitu **Continue For**, **Continue Do**, dan **Continue While** tergantung looping yang anda gunakan. Ketika continue dijalankan maka program secara otomatis kembali ke loop tanpa membaca kode program dibawah continue (skip).

```
'mencetak 1 2 3 4 5 6 8 9 10
For i = 1 To 10
    If i = 7 Then
        Continue For
    End If
    lstNama.Items.Add(i.ToString)
Next
```

Penambahan Fitur Pada Procedure

Seperti pada VB6 , VB9 juga terdiri dari dua macam procedure yaitu function yang mengembalikan nilai dan subroutine yang tidak mengembalikan nilai. Tetapi ada beberapa penambahan fitur pada VB9 yang selanjutnya akan kita bahas.

Pemanggilan Method

Ada sedikit perubahan pemanggilan method pada VB9 dibandingkan dengan VB6 yaitu sekarang anda harus menggunakan kurung buka tutup untuk memanggil method.

```
'pada VB6
'tidak harus menggunakan kurung buka tutup
MsgBox "Hello World ", vbOKOnly
```

```
'harus menggunakan kurung buka tutup
hasil = MsgBox("Hapus File?", vbYesOrNo, "Konfirmasi")
```

```
'pada VB9 semua harus menggunakan kurung buka tutup
'pada VB9 method MsgBox diganti dengan MessageBox 'Class
MessageBox.Show("Hello VB !", "Info", _
MessageBoxButtons.OK)
```

```
Dim result As System.Windows.Forms.DialogResult =  
MessageBox.Show("Hello VB!", "Info", _  
MessageBoxButtons.OK)
```

VB9 menggunakan kurung buka tutup disetiap method yang dipanggil bahkan ketika method tersebut tidak mempunyai parameter.

```
Form1.Show()
```

ByVal dan ByRef

ByVal adalah default keyword yang baru untuk method parameter. Visual Studio akan menambahkannya secara otomatis ketika anda membuat method parameter.

Jika parameter pada method di deklarasikan ByVal berarti nilai dari parameter tersebut adalah kopian dari nilai parameter method pemanggil. Jadi ketika parameter pada method diubah maka tidak berpengaruh ke parameter method pemanggil.

Lain halnya jika anda menggunakan keyword ByRef, dengan ByRef maka parameter pada method mereferensi ke object yang sama dengan parameter pada method pemanggil, maka jika parameter pada method berubah ini juga berpengaruh ke parameter di method pemanggil.

Listing 3.3 Perebedaan ByRef dan ByVal

1. Buat project windows application baru dengan nama 'MethodByValRef'.
2. Kemudian masukan dua button kedalam form.

Control	Property	Value
Button	Name	btnByVal
	Text	ByVal Parameter
Button	Name	btnByRef
	Text	ByRef Parameter

3. Double click pada **btnByVal** kemudian tulis kode berikut:

```

Function Segitiga(ByVal alas As Double, _
ByVal tinggi As Double) As Double
    alas += 2
    tinggi += 2
    Return 0.5 * alas * tinggi
End Function

Private Sub btnByVal_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btnByVal.Click
    Dim alas As Double = 12
    Dim tinggi As Double = 14
    'ByVal = nilai dari method pemanggil dicopy ke
    'method
    Dim luas As Double = Segitiga(alas, tinggi)
    MessageBox.Show(luas.ToString)
    'tetap bernilai 12 walaupun pada method
    'ditambah 2
    MessageBox.Show(alas.ToString)
    'tetap bernilai 14 walaupun pada method
    'ditambah 2
    MessageBox.Show(tinggi.ToString)
End Sub

```

4. Double click pada **btnByRef** kemudian tulis kode berikut:

```

Function LuasLingkaran(ByRef panjang As Double, ByRef lebar
As Double) As Double
    panjang += 2
    lebar += 2
    Return panjang * lebar
End Function

Private Sub btnByRef_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btnByRef.Click
    Dim panjang As Double = 12
    Dim lebar As Double = 14
    'karena menggunakan ByRef maka parameter pada
    'method pemanggil mempunyai referensi yang
    'sama dengan parameter pada function, jika
    'nilai parameter pada function dirubah maka
    'nilai variabel pada method pemanggil juga

```

```

'berubah
Dim luas As Double = LuasLingkaran(panjang,
    lebar)
MessageBox.Show(luas.ToString)
'menjadi 14 karena ditambah 2 pada function
MessageBox.Show(panjang.ToString)
'menjadi 16 karena ditambah 2 pada function
MessageBox.Show(lebar.ToString)
End Sub

```

Keyword Return

Salah satu fitur yang sangat membantu pada keyword return yang ada di VB9 adalah anda dapat menuliskan:

```

Function Kali(ByVal bil1 As Double, ByVal bil2 As Double)
    As Double
    'sekarang anda dapat menuliskan statement
    'return bil1*bil2 daripada hasil = bil1*bil2
    Return bil1 * bil2
End Function

```

Penggunaan Optiona Parameter

Method tetap dapat menggunakan nilai optional (jumlah parameter bisa tidak pasti) dengan menggunakan parameter array. Sebagai contoh anda dapat membuat fungsi sebagai berikut:

```

Function GetMahasiswa(ByVal nim As String, ByVal ParamArray
    info() As String) As String
    Dim tmpInfo As String = ""
    For Each strInfo In info
        tmpInfo &= strInfo & " "
    Next
    Return "Nim :" & nim & " Info :" & tmpInfo
End Function

```

```

Private Sub btnOptional_Click(ByVal sender As
    System.Object, ByVal e As System.EventArgs) Handles
    btnOptional.Click

```

```

    Dim tampil As String = _
        GetMahasiswa("23092321", "Erick", _
            "Gowongan", "SI")
    MessageBox.Show(tampil)
End Sub

```

Jadi dengan VB9 anda dapat membuat parameter yang jumlah parameternya belum pasti. Ini akan sangat berguna ketika anda bekerja dengan aplikasi office dengan VB.

Default Value

Anda juga dapat membuat method yang parameternya belum tentu digunakan oleh user, dengan menggunakan keyword Optional tapi anda harus memberi default valuenya jika parameter tersebut optional. Jika parameter tidak diisi maka otomatis nilai yang akan digunakan adalah nilai defaultnya.

```

Function GetDosen(ByVal nik As String, _
    Optional ByVal nama As String = "Erick", _
    Optional ByVal alamat As String = "") As String
    Return "Nik :" & nik & " Nama :" & nama & "
        Alamat :" & alamat
End Function

```

```

Private Sub btnDefault_Click(ByVal sender As System.Object,
    ByVal e As System.EventArgs) Handles btnDefault.Click
    Dim tampil As String = ""
    'anda dapat mengisi semua parameter
    tampil = GetDosen("12345", "Erick",
        "Gowongan")
    'atau dapat diisi hanya 2 parameter, yg
    'Optional boleh tidak diisi
    tampil = GetDosen("12345", , "Gowongan")
    'atau hanya 1 parameter saja
    tampil = GetDosen("12345")
End Sub

```

Method Overloading

VB9 mempunyai fitur untuk membuat fungsi dan subrutin dengan lebih fleksible. Pada VB9 anda dapat membuat fungsi atau subrutine dengan nama yang sama tapi jumlah parameter atau tipe datanya harus berbeda biasa disebut method overloading (menggunakan keyword *Overload*).

```
Public Overloads Function Tambah(ByVal bil1 As Integer,
ByVal bil2 As Integer) As Integer
    Return bil1 + bil2
End Function
```

```
Public Overloads Function Tambah(ByVal bil1 As Double,
ByVal bil2 As Double) As Double
    Return bil1 + bil2
End Function
```

```
Public Overloads Function Tambah(ByVal bil1 As Integer,
ByVal bil2 As Integer, ByVal bil3 As Integer) As Integer
    Return bil1 + bil2 + bil3
End Function
```

```
Private Sub btOverload_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btOverload.Click
    Dim dblHasil As Double = Tambah(2.2, 2.2)
    Dim intHasil As Integer = Tambah(2, 2)
    Dim intHasil2 As Integer = Tambah(12, 13, 14)
End Sub
```

Jika anda perhatikan kode diatas terdapat 3 fungsi dengan nama sama yaitu Tambah, jumlah parameternya sama namun tipe data pada parameternya berbeda yaitu Integer dan Double, pada method yang ketiga nama dan tipe fungsi sama tapi jumlah parameternya yang berbeda.

Delegates

VB9 mempunyai fitur menarik yaitu delegates untuk membantu anda berkerja dengan fungsi dan subrutin. Delegates adalah variable khusus yang dapat menyimpan lokasi dari method (mereferensi ke method). Sebelum anda

menggunakan delegate anda harus mendeklarasikannya dengan cara:

```
Public Delegate Function Proses(ByVal strIn As String) As String
```

Statement diatas tidak mendeklarasikan variable tapi mendefinisikan tipe delegates. Fungsi Proses adalah delegate yang dapat digunakan untuk menyimpan lokasi dari method yang signaturenya (parameter, tipe data, dan return valuenya) sama dengan fungsi Proses (mempunyai 1 parameter bertipe string dan mempunyai nilai kembalian berupa string pula). Kemudian anda dapat mendeklarasikan delegate variable sebagai berikut:

```
Dim myDelegate As Proses
```

Variable delegate hanya dapat menyimpan fungsi atau subrutin yang signaturnya sama dengan definisi delegate. Ini adalah kegunaan dari delegate yaitu memastikan bahwa anda mereferensi ke fungsi atau subrutin yang benar.

```
Public Function UbahCapital(ByVal nama As String) As String  
    'fungsi ini dapat disimpan referensinya karena  
    'mempunyai signature yang sama  
End Function
```

```
Public Function UbahLower(ByVal nama As String, ByVal  
panjang As Integer) As String  
    'tidak dapat disimpan referensinya karena mempunyai  
    'signature yang berbeda  
End Function
```

```
Public Sub UbahUpper(ByVal nama As String)  
    'tidak sesuai dengan signature karena bukan  
    'function  
End Sub
```

Jika anda sudah mendeklarasikan variable delegate maka cara untuk mereferensi fungsi atau subrutin adalah dengan menggunakan operator **AddressOf**. **AddressOf** memberitahu pada compiler bahwa variable tersebut mereferensi ke method

bukan untuk menjalankan methodnya. Signature dari method yang akan direferensi dengan signature delegate harus sama.

```
Public Function UbahCapital(ByVal nama As String) As String
    Return nama.ToUpper()
End Function
```

```
Private Sub btnPanggil_Click(ByVal sender As System.Object,
    ByVal e As System.EventArgs) Handles btnPanggil.Click
    Dim myDelegate As Proses
    myDelegate = AddressOf UbahCapital
End Sub
```

Setelah anda mereferensi method UbahCapital maka anda dapat menjalankan method yang direferensi dengan menggunakan variable delegate.

```
Dim strUpper As String = myDelegate("erick kurniawan")
MessageBox.Show(strUpper)
```

Delegates adalah teknik yang sangat berguna karena dengan delegate anda dapat melakukan penambahan layer of indirection, sehingga kode yang anda buat lebih umum dan dapat digunakan kembali. Dibawah ini adalah contoh fungsi yang dapat menerima parameter berupa variable delegate.

```
'method yang menerima parameter berupa variable bertipe
'delegates
Sub ProsesArray(ByVal strArr() As String, ByVal delFungsi
As Proses)
    For i = 0 To strArr.GetUpperBound(0)
        strArr(i) = delFungsi(strArr(i))
    Next
End Sub
'cara pemanggilan methodnya
Dim strArr() As String = {"erick", "rully", "naren",
"wely"}
ProsesArray(strArr, AddressOf UbahCapital)
```

Dengan menggunakan delegate parameter seperti kode diatas maka implementasi fungsi yang kita buat dapat lebih fleksible untuk diubah asalkan signaturennya sama dengan tipe delegate yang dideklarasikan.

BAB 4

Windows Form

Windows Form adalah salah satu aplikasi yang dapat dibangun menggunakan VB9. Banyak aplikasi yang berjalan diatas sistem operasi Windows dibangun dengan teknologi Windows Form seperti aplikasi MS Office, aplikasi multimedia (Windows Media Player), dan bahkan game interaktif.

.NET Framework menyediakan class library khusus untuk membangun aplikasi berbasis Windows Form. Pada VB9 cukup mudah untuk mendesain form yang rich dan interaktif dengan adanya fasilitas resizing, splitting, anchoring, dan docking pada control. VB9 juga mempermudah penggunaan aplikasi MDI (Multiple Document Interface), mempermudah penggunaan control toolbar dan menu.

Perbedaan yang sangat mendasar dengan VB6 (VB Classic) adalah semua control yang ada pada form ditulis dengan menggunakan kode vb. Jadi ketika anda melakukan drag and drop komponen ke form maka secara otomatis Visual Studio akan menggenerate kode yang disimpan dalam sebuah file berekstensi .vb, dan anda juga dapat memodifikasinya secara manual atau bahkan membuat kode untuk menggenerate control secara dinamis pada saat program dijalankan.

Apa yang baru di .NET

.NET Framework menyediakan model yang baru untuk bekerja dengan Form.

Standarisasi Windows Application

Semua bahasa yang didukung .NET menggunakan Windows Form yang sama. Jadi meskipun anda menggunakan

VB atau C# tetap menggunakan class library yang sama untuk membuat aplikasi Windows Form.

Form Merupakan Class

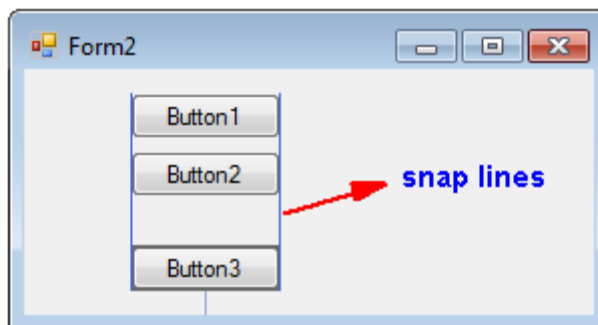
Pada VB6 form mempunyai identitas ganda sebagai object juga sebagai class pada saat yang sama. Pada VB9 form hanya class yang diturunkan dari `System.Windows.Forms.Form`, kelebihanannya anda dapat mengontrol semua property pada form secara mudah karena sudah terdapat didalam class.

Menyediakan Extender Provider

Untuk memudahkan penggunaan control yang dapat digunakan oleh control yang lain. Misalnya anda ingin menggunakan control tooltips pada controls yang lain anda dapat menambahkannya pada komponen tray.

Fitur Snap Line

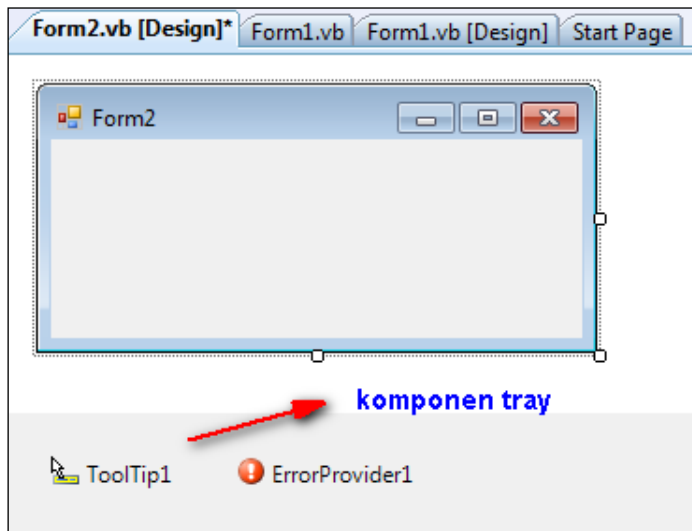
Cara untuk mendesain aplikasi Windows Form pada VB9 sama dengan VB6, anda dapat melakukan drag and drop dari toolbox yang ada, kemudian merubah properti dari control tersebut. Toolbox control yang ada pada VB9 jauh lebih lengkap dibandingkan dengan VB6, pengaturan layout pada form juga jauh lebih mudah karena adanya fitur *snap line*.



Gambar 4.1 fitur *snap line*

Komponen Tray

Pada VB6 ketika anda menggunakan control yang tidak memiliki tampilan secara visual misalnya Timer control, anda akan menaruh komponen tersebut pada form yang sama dengan control lain yang memiliki tampilan visual seperti textbox dan button. VB9 memiliki area khusus untuk menaruh control yang tidak memiliki tampilan visual yaitu didalam Komponen Tray sehingga lebih mudah untuk pengaturannya.

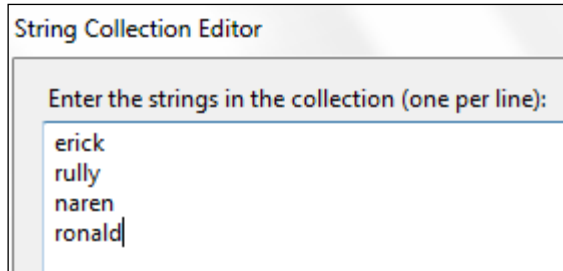


Gambar 4.1 Komponen Tray

Custom Designer

Pada VB6 beberapa control memiliki properti yang kompleks yang tidak dapat diisi dengan menambahkannya di menu properties, sebagai contoh adalah komponen **Tree View** yang sangat kompleks. **Tree View** mempunyai nodes yang tidak dapat diisi pada saat design time, namun anda harus melakukan koding secara manual.

Misal control **ListBox** sekarang mempunyai property Items dan anda dapat memilih tanda (...) disebelah properti tersebut untuk mengisi daftar list yang akan ditampilkan.



Gambar 4.2 Konfigurasi list menggunakan designer

Mengunci Control

Untuk memastikan bahwa layout pada Windows anda tidak berubah-ubah anda dapat menggunakan fasilitas locking. Sama seperti pada VB6 anda dapat mengunci layout dari control anda dengan cara klik kanan pada form kemudian pilih 'Lock Control'.



Gambar 4.3 Lock Control

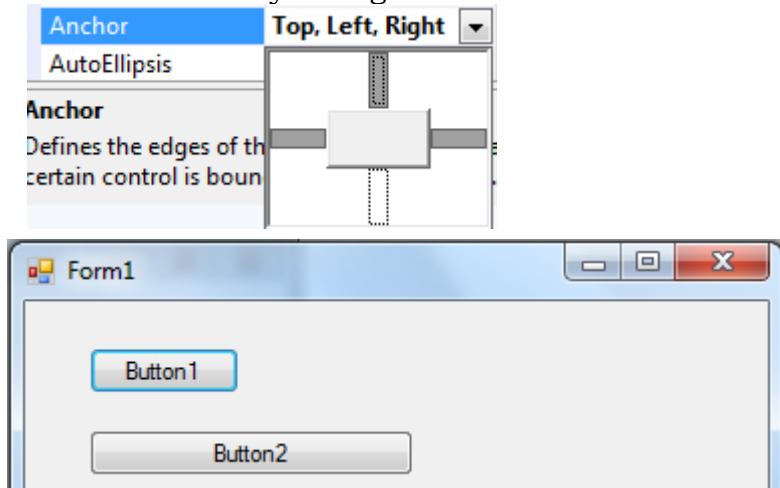
Pada VB9 fitur untuk mengunci kontrol dapat digunakan tidak hanya untuk keseluruhan control yang ada di form (seperti pada VB6), namun anda dapat mengunci posisi per control yang anda inginkan sehingga lebih fleksible. Untuk mengunci posisi per control anda dapat mengganti properti 'lock' pada control menjadi **true**.

Anchoring

Dengan menggunakan anchoring maka control yang anda buat akan menyesuaikan ukuran dengan sendirinya ketika

anda merubah ukuran form, jadi anda tidak perlu menulis kode secara manual untuk resize seperti pada VB Classic.

Anchoring adalah fitur yang sangat membantu anda dalam mendesign form. Anchor berperan seperti jangkar. Secara default control ter-'anchor' di posisi pojok kiri atas dari form, jadi ketika anda merubah ukuran form dengan cara menarik form ke arah kanan, maka control yang anda buat tidak otomatis berubah juga ukurannya. Jika anda menginginkan control tersebut ikut berubah jika ditarik ke kanan maka anda dapat merubah property anchor menjadi **Top, Left, Right**, jadi ketika anda menarik form ke arah kanan control yang sudah anda set property anchornya akan ikut berubah ukurannya mengikuti form.



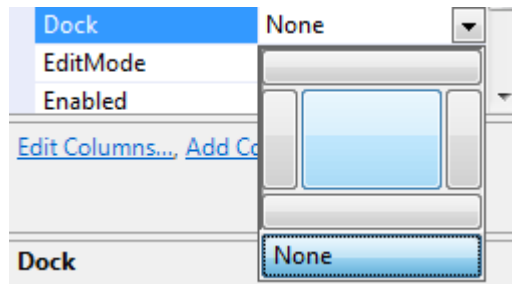
Gambar 4.4 Penggunaan Anchor

Pada contoh diatas Button2 akan berubah juga ukurannya ketika anda menarik form ke kanan.

Docking

Properti docking pada control digunakan jika anda menginginkan control tersebut 'menempel' pada posisi kiri, kanan, atas, bawah, atau seluruh form (fill). Dengan menggunakan docking control juga akan berubah ukurannya

jika anda merubah posisi form. Misal anda ingin control **DataGridView** memenuhi keseluruhan form dan akan ikut berubah posisinya jika anda mengubah ukuran form. Ubah properti Dock menjadi 'fill' (pilih kotak bagian tengah).



Gambar 4.5 Menggunakan Properti Dock

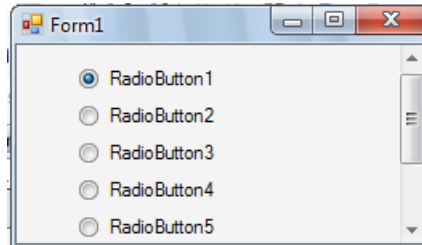
Jika anda ingin mengatur beberapa control menggunakan docking, anda dapat memberi jarak pada kedua control tersebut dengan mengatur properti 'padding'.

Maximum dan Minimum Windows Size

VB9 juga mempunyai properti maximum dan minimum yang digunakan untuk menentukan ukuran minimum dan maximum dari form. Pada VB6 anda harus melakukan koding secara manual untuk membatasi ukuran minimum dan maksimum dari form.

Automatic Scrolling

Automatic scrolling sangat membantu jika anda ingin menampilkan informasi yang banyak. VB9 mendukung scroll otomatis pada beberapa control seperti **Form** dan **Panel**. Anda tinggal mengatur properti **AutoScroll** menjadi **true**.



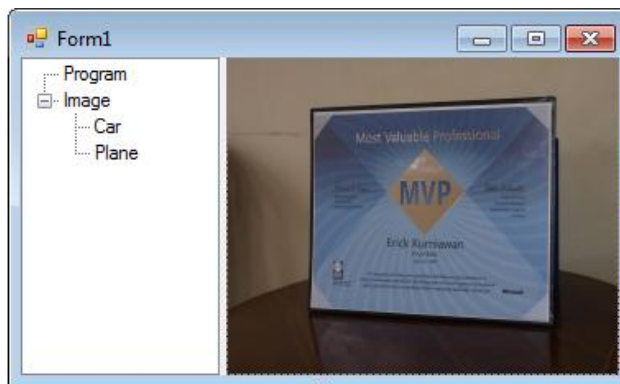
Gambar 4.6 Properti AutoScroll

Spilt Windows

Split Windows adalah model design yang banyak digunakan pada aplikasi windows modern saat ini. Spilt Window juga banyak digunakan sebagai alternatif selain MDI Form karena lebih simple dan efisien.

Untuk membuat split window anda harus menambahkan **SplitContainer** control, secara default split container terbagi menjadi dua bagian, anda dapat menambahkan control di bagian kiri dan kanan. Ketika menambahkan control jangan lupa untuk mengatur properti dock dari control tersebut agar ikut berubah jika form di resize.

SplitContainer juga mempunyai properti **Panel1MinSize** dan **Panel2MinSize** untuk membatasi ukuran minimal panel tersebut dapat di resize. Misal pada contoh dibawah ini menggunakan dua control yaitu **TreeView** di panel kiri dan **PictureBox** di panel kanan.

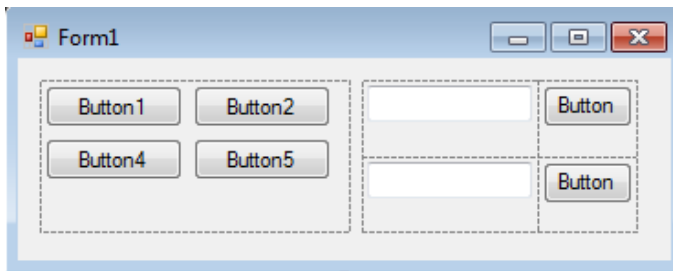


Gambar 4.7 Penggunaan SplitWindows

Container Control

Selain **SplitContainer** ada dua jenis container lagi yang sering digunakan yaitu **Panel** dan **GroupBox**. Panel control adalah dasar dari semua container control yang lain seperti **SplitContainer** yang terdiri dari dua panel di kiri dan kanan.

Panel control juga memiliki fasilitas untuk scrolling, bedanya dengan **GroupBox**, Panel tidak memiliki properti caption. Ada beberapa control baru yang diturunkan dari Panel yaitu **FlowLayoutPanel** dan **TableLayoutPanel**. **FlowLayoutPanel** mirip dengan properti Layout pada CSS ketika anda mendesign web, control akan mengisi panel sampai batas panel tersebut, jika flow left berarti mengisi dari kiri jika sudah sampai batas form maka control tersebut akan mengisi bawahnya. **TableLayoutPanel** mirip seperti Grid jadi anda dapat meletakkan control pada row-row yang anda di **TableLayoutPanel**.



Gambar 4.8 Penggunaan FlowLayoutPanel

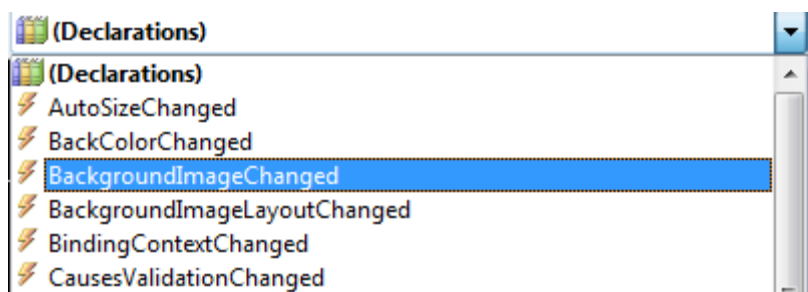
Controls dan Event

Sama seperti pada VB6 setiap control yang ada pada VB9 memiliki properti dan event handler. Untuk bekerja dengan event handler di Visual Studio, klik kanan pada form ► pilih view code ► pada combo box pertama pilih control yang akan diisi event handlernya.



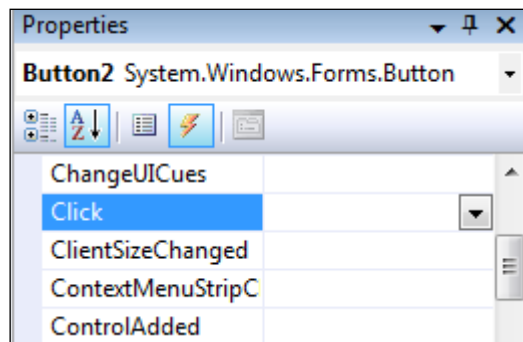
Gambar 4.9 Pilih Control

Kemudian setelah memilih control, pilih event handler pada combobox sebelahnya.



Gambar 4.10 Pilih Event Handler

Anda juga dapat mengisi event handler melalui menu properties pada design form.



Gambar 4.11 Memilih event melalui Properties

Anda juga dapat melakukan double click pada control tersebut untuk menambahkan event handler, secara default jika anda double click pada button maka akan membuat event handler **button_click**, jika **TextBox** maka akan masuk **textbox_TextChanged**. Ini adalah contoh penanganan event handler ketika tombol di klik.

```
Private Sub Button1_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles Button1.Click  
    MessageBox.Show("Hello Word !!")  
End Sub
```

Pada .NET setiap method untuk menangani event handler mempunyai dua parameter, yaitu sender yang mereferensi ke object yang mengirimkan event (pada contoh diatas adalah button), yang kedua adalah object e yang membungkus semua informasi tambahan yang dibutuhkan untuk event tersebut. Untuk event yang berbeda juga menggunakan tipe object yang berbeda juga pada object e. Secara default object e bertipe EventArgs yang tidak berisi informasi (pada button). Lain halnya dengan event mouse move, object e mengirimkan informasi posisi dari pointer mouse.

```
Private Sub Form1_MouseMove(ByVal sender As System.Object,  
ByVal e As System.Windows.Forms.MouseEventArgs) Handles  
MyBase.MouseMove  
    Label1.Text = "Posisi X: " & e.X & " Y: " & e.Y  
End Sub
```

Menangani Lebih dari Satu Event

Salah satu perbedaan dengan VB6, pada VB9 anda dapat membuat satu method event handler yang bekerja untuk beberapa control sekaligus, hal ini akan membantu anda untuk me-reuse (menggunakan kembali) kode yang sudah anda buat.

VB6. Pada VB6 anda dapat menggunakan control array untuk membuat satu event handler untuk banyak control, namun VB9 sudah tidak mensupport penggunaan control array.

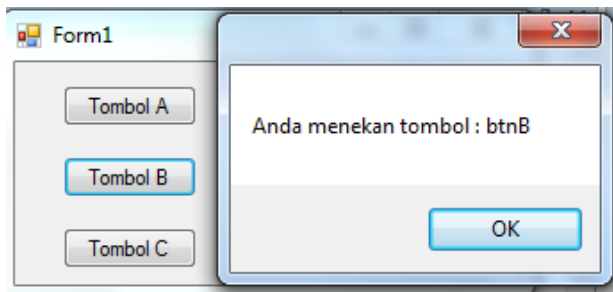
Pada VB9 event handler terkoneksi dengan control menggunakan keyword **Handles** yang ada di akhir penulisan event handler.

Listing 4.1 Menghandle lebih dari satu event

1. Buat windows form application baru, beri nama MultipleHandler.
2. Kemudian masukan 3 button kedalam form, beri nama **btnA**, **btnB**, dan **btnC**.
3. Buat event handler yang akan digunakan untuk ketiga tombol diatas.

```
Public Sub KlikHandle(ByVal sender As Object, ByVal e As  
EventArgs) Handles btnA.Click, btnB.Click, btnC.Click  
    Dim tombol = CType(sender, Control)  
  
    MessageBox.Show("Anda menekan tombol : " & _  
        tombol.Name)  
End Sub
```

4. Ketika anda melakukan klik pada ketiga button tersebut maka **KlikHandle** akan dijalankan.



Gambar 4.12 Menggunakan Multiple Event Handler

Tombol Accept dan Cancel

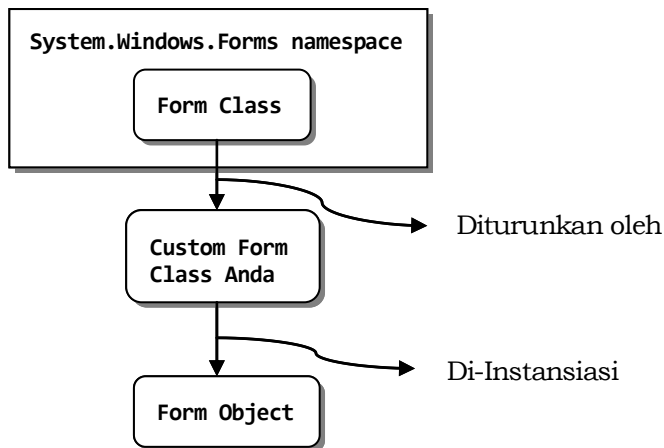
Form mempunyai dua properti yang digunakan untuk membuat tombol khusus yaitu **AcceptButton** dan **CancelButton**, properti ini akan dijalankan secara otomatis ketika user menekan tombol enter (accept) dan tombol esc

(cancel). Misal anda mempunyai tombol dengan nama **btnProses**, anda dapat mengeset properti **AcceptButton** pada form menjadi **btnProses** sehingga ketika user menekan enter maka yang akan dijalankan adalah event click pada **btnProses**. Demikian juga dengan property **CancelButton** anda dapat mengisinya dengan tombol tertentu sehingga jika user menekan esc yang dijalankan adalah tombol tersebut.

VB6. Fitur ini juga ada di VB6 tetapi harus menggunakan Default dan Cancel property pada button control.

Eksplorasi .NET Form

Pada .NET Framework, object form diturunkan dari class Form yang ada di dalam namespace System.Windows.Forms. Hirarki nya dapat dilihat pada gambar dibawah ini.



Gambar 4.13 Hirarki Form

Seperti dapat dilihat pada gambar diatas, Custom Form (class form pada aplikasi anda) diturunkan dari class Form dalam namespace System.Windows.Forms, dan Custom Form

akan diinstansiasi untuk membuat object Form yang tampil pada aplikasi anda.

Dua Cara Untuk Menampilkan Form

Pada VB9 ada dua cara untuk menampilkan form, yaitu:

- Explicit : dengan cara ini anda akan secara eksplisit mendefinisikan class Form dan menginstansiasi object form yang akan digunakan (cara standar pada .NET).
- Implicit : dengan cara shortcut ini object form otomatis dicreate ketika form tersebut dipanggil (seperti cara lama pada VB6).

Cara explicit dapat dilihat pada kode dibawah ini:

```
Dim frm2 As New Form2  
frm2.Show()
```

Pada kode diatas baris pertama anda membuat object **frm2** kemudia baris kedua method **show()** digunakan untuk menampilkan form.

Anda juga dapat menggunakan cara implicit seperti pada VB6, kodenya:

```
Form2.Show()
```

Cara ini menggunakan *default instance* dari Form2, VB compiler akan secara otomatis mengcreate object Form2 ketika form tersebut dipanggil. Cara ini memang membuat programmer menjadi lebih mudah karena tidak perlu secara eksplisit menginstansiasi object, namun cara ini juga memungkinkan terjadinya kesalahan.

```
Dim frm2 As New Form2  
Form2.Show()
```

Pada kode diatas object **frm2** tidak digunakan padahal sudah dibuat karena user langsung mengakses default instance dari **Form2**. Untuk penggunaan implicit form anda disarankan menggunakan object **My** jadi kode menjadi lebih jelas karena anda akan mengetahui form tersebut sudah dibuat secara implicit.

My.Forms.Form2.Show()

Disarankan untuk menggunakan **My** object untuk memanggil default instance (cara implicit) pada aplikasi yang anda buat.

Modal dan Modeless Form

Contoh yang kita buat sebelumnya dengan menggunakan method **show()** akan menampilkan form dengan cara modeless, jadi ketika form tersebut tampil anda masih dapat mengakses form yang lainnya.

Anda juga dapat menampilkan form dengan cara modal, yaitu anda tidak dapat mengakses form lain ketika form modal masih terbuka. Cara untuk membuat modal form pada VB9 adalah dengan menggunakan method **ShowDialog()**.

My.Forms.Form2.ShowDialog()

Method **ShowDialog()** juga akan menghentikan kode anda, kode akan diteruskan dibaca ketika user sudah menutup form modal.

Startup Form dan Shutdown Mode

Kebanyakan aplikasi terdiri dari banyak form, anda dapat menentukan form mana yang terlebih dahulu akan dijalankan dengan cara klik kanan pada project ► Properties ► pilih Application Tab, kemudian pilih form yang akan dijalankan pertama kali pada 'Startup Form'.

Anda juga dapat menentukan kapan aplikasi anda benar-benar diakhiri pada menu 'Shutdown mode', anda dapat memilih 'When Startup Form Close' ini berarti program akan diakhiri ketika anda menutup startup form, atau anda juga dapat memilih 'When Last Form Close' ini akan membuat aplikasi anda tetap berjalan sebelum semua form tertutup, atau anda dapat menggunakan perintah **Application.Exit()** untuk menutup semua form pada aplikasi anda.

Application Events

Pada kasus tertentu anda tidak hanya ingin menampilkan satu form saja ketika aplikasi dijalankan maka anda dapat menambahkan event handler pada aplikasi. Caranya klik tombol 'View Application Events' di pojok kanan bawah Application Tab, ketika anda pertama kali mengklik tombol tersebut maka akan dicreate sebuah file dengan nama ApplicationEvents.vb. Daftar event pada application:

- **Startup** : dijalankan pertama kali pada saat aplikasi dibuat, tapi sebelum form startup dibuat. Jika anda ingin membuat form lain selain startup form ketika aplikasi pertama kali dijalankan anda dapat menaruhnya disini.
- **Shutdown** : dijalankan ketika semua form dalam aplikasi sudah ditutup, sebelum aplikasi berakhir.
- **UnhandledException** : dijalankan ketika aplikasi anda berakhir disebabkan oleh error.
- **StartupNextInstance** : dijalankan ketika aplikasi kedua dijalankan (ada 2 instance aplikasi yang dijalankan). Sebenarnya anda tidak perlu menggunakan event handler ini karena anda dapat memastikan bahwa satu instance aplikasi saja yang berjalan dengan cara check pada pilihan 'Make single instance application' pada Application Tab.
- **NetworkAvailabilityChanged** : dijalankan jika ada koneksi jaringan yang terhubung atau putus. Even ini dijalankan jika aplikasi anda butuh terhubung ke Internet.

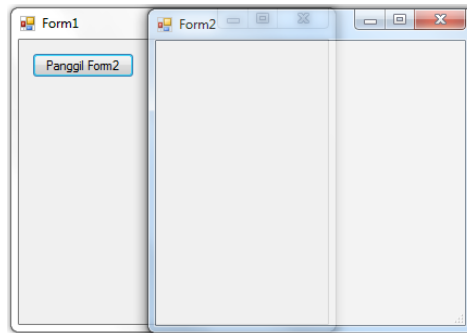
Contoh penggunaan application event untuk menampilkan Splash Screen sebelum form startup dijalankan.

```
Partial Friend Class MyApplication
    Private Sub MyApplication_Startup(ByVal sender As
Object, ByVal e As Microsoft.VisualBasic.
ApplicationServices.StartupEventArgs)
        Handles Me.Startup
            My.Forms.MySplashScreen.Show()
```

```
'kode untuk mmbuat delay waktu  
My.Forms.MySplashScreen.Close()  
End Sub  
End Class
```

Opacity

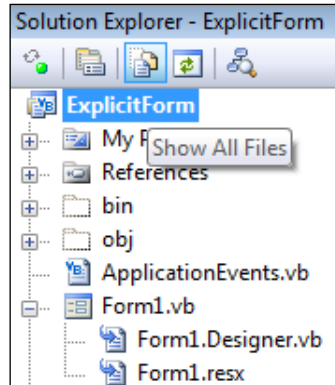
Pada VB9 form juga memiliki property yang cukup unik yaitu Opacity dan TransparencyKey. Anda dapat membuat form menjadi transparan dengan mengatur property opacity, semakin sedikit prosentase opacity maka semakin transparan formnya.



Gambar 4.14 Opacity 80%

Form ‘Behind The Scene’

Pada saat anda menambahkan form kedalam project, sebenarnya Visual Studio akan membuat anda kode yang digunakan untuk menginisialisasi form yang anda buat, demikian juga ketika anda menambahkan control kedalam form tersebut. Misal untuk sebuah form dengan nama Form1 Visual Studio secara otomatis akan membuat dua file yaitu **Form1.vb** dan **Form1.Designer.vb**, namun pada solution explorer tidak semua file ditampilkan, untuk menampilkan keseluruhan file klik tombol ‘Show All Files’ pada solution explorer.



Gambar 4.15 Show All Files pada Solution Explorer

Kode yang digenerate secara otomatis oleh Visual Studio ketika anda membuat form dan menambahkan control didalamnya disimpan pada file Form1.Designer.vb. Jadi kode yang kita buat dan kode yang digenerate terpisah, ini memudahkan kita untuk membaca kode. Jika anda mencermati kode pada file designer.

```

Friend WithEvents btnForm2 As System.Windows.Forms.Button
Private components As System.ComponentModel.IContainer
    <System.Diagnostics.DebuggerStepThrough()> _
    Private Sub InitializeComponent()
        Me.btnForm2 = New System.Windows.Forms.Button
        Me.SuspendLayout()
        '
        ' btnForm2
        '
        Me.btnForm2.Location = _
            New System.Drawing.Point(12, 12)
        Me.btnForm2.Name = "btnForm2"
        Me.btnForm2.Size = New System.Drawing.Size(94, 23)
        Me.btnForm2.TabIndex = 0
        Me.btnForm2.Text = "Panggil Form2"
        Me.btnForm2.UseVisualStyleBackColor = True
        '
        ' Form1
        '
        Me.AutoScaleDimensions = _

```

```
New System.Drawing.SizeF(6.0!, 13.0!)
Me.AutoScaleMode = _
    System.Windows.Forms.AutoScaleMode.Font
Me.ClientSize = New System.Drawing.Size(284, 262)
Me.Controls.Add(Me.btnForm2)
Me.Name = "Form1"
Me.Text = "Form1"
Me.ResumeLayout(False)
```

End Sub

Dari kode diatas kita dapat menyimpulkan bahwa:

- Setiap control didefinisikan sebagai variable di form. Access modifier yang digunakan adalah Friend agar control tersebut dapat diakses dari form lain dalam satu assembly (satu project).
- Semua control pada form terdapat pada method **InitializeComponent()** yang dipanggil secara otomatis ketika form tersebut di load.
- Inisialisasi properti pada setiap form dipisahkan pada blok yang berbeda (ada keterangan nama form dalam bentuk comment).
- Kode **Me.Controls.Add(Me.btnForm2)** digunakan untuk menambahkan control kedalam form.

Binary File

Binary file seperti file image akan disimpan kedalam folder resources ketika anda menggunakan **PictureBox** atau **ImageList**. VB9 akan menyimpan informasi untuk membaca file tersebut didalam file .resx (lain dengan VB6 yang menyimpan informasi resource pada file .frx).

Menambahkan Control Secara Dinamis

Mungkin pada kasus tertentu anda ingin menambahkan form secara dinamis ketika program berjalan, misal untuk

membuat diagram yang otomatis di create ketika aplikasi dijalankan. Pada contoh dibawah ini kita akan membuat label control secara runtime ketika tombol **btnProses** diklik.

```
Private Sub btnProses_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btnProses.Click
    Dim lblHello As New System.Windows.Forms.Label
    lblHello.Location = New System.Drawing.Point(50, 50)
    lblHello.Name = "lblHello"
    lblHello.Size = New System.Drawing.Size(150, 30)
    lblHello.Text = "Hello VB !!"
    Controls.Add(lblHello)
End Sub
```

Pada kode diatas kita menambahkan **lblHello** secara runtime kedalam form dengan menggunakan method **Controls.Add()**. Karena lblHello berapada di dalam subrutin maka tidak dapat diakses dari luar subrutin **btnPoses_Click**, untuk mengakses control **lblHello** dari button lain anda dapat menggunakan cara:

```
Me.Controls("lblHello").Text
```

Menambahkan Event Secara Dinamis

Selain menambahkan control secara runtime, anda juga dapat menambahkan event handler secara runtime. Untuk menambahkan event secara runtime anda tidak dapat menggunakan keyword **WithEvents**. Keyword **WithEvents** hanya mempermudah anda untuk mengetahui bahwa control tersebut mempunyai event (dengan keyword **Handle**), namun jika control tersebut belum ada (belum dicreate) maka anda tidak dapat menggunakannya.

Solusinya adalah anda dapat menggunakan keyword **AddHandler**. Pada contoh dibawah ini akan ditunjukkan bagaimana cara untuk menambahkan event secara runtime.

Listing 4.2 Menambahkan Event Secara Runtime

1. Buat project Windows Form Application dengan nama 'TambahEventRuntime'.

2. Tambahkan method **CreateRandom** yang digunakan untuk menggenerate nilai random dari 1 sd 10.

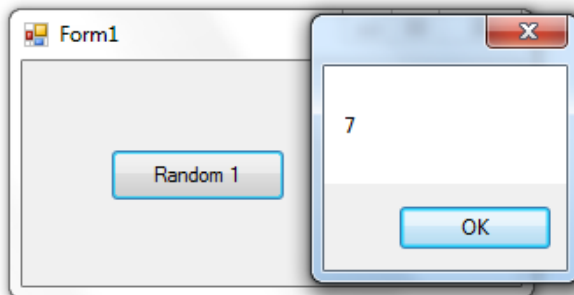
```
Private Sub CreateRandom(ByVal sender As Object, ByVal e As EventArgs)
    Dim rndNumber As New Random
    'generate nilai random dari 1-10
    MessageBox.Show(rndNumber.Next(10) + 1)
End Sub
```

3. Pada event **Form1_Load** tambahkan kode berikut untuk meregister method **CreateRandom** sebagai event pada **btnRandom1**.

```
Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load
    Dim btnRandom1 As New Button
    btnRandom1.Location = New System.Drawing.Point(50, 50)
    btnRandom1.Size = New System.Drawing.Size(100, 30)
    btnRandom1.Name = "btnRandom1"
    btnRandom1.Text = "Random 1"
    Controls.Add(btnRandom1)

    'meregister method menjadi event handler
    AddHandler btnRandom1.Click, AddressOf CreateRandom
End Sub
```

4. Jika program dijalankan maka otomatis akan di create tombol **btnRandom1**, dan ketika di klik maka akan dijalankan method **CreateRandom** yang sudah didaftarkan menjadi event handler.



Gambar 4.16 Menggunakan Dynamic Event Handler

Interaksi Antar Form

Pada VB9 anda dapat melakukan interaksi control antar form dengan mudah menggunakan bantuan object My. Problem yang biasanya ditemui adalah ketika hendak mengambil atau memberi nilai pada control yang letaknya pada form yang lain.

Listing 4.3 Interaksi antar Form

1. Buat Windows Form Application dengan nama 'InteraksiAntarForm'.
2. Pada Form1 tambahkan sebuah ComboBox dengan nama cmbHobi, tambahkan pula sebuah button dengan nama btnForm2, ubah properti Text='Panggil Form2'.
3. Kemudian tambahkan sebuah form dengan nama Form2 isi dengan dua control yaitu textbox dengan nama txtHobi dan sebuah button dengan nama btnTambah, atur properti Text = 'Tambah'.
4. Double click pada btnForm2 di Form1 kemudian tambahkan kode berikut:

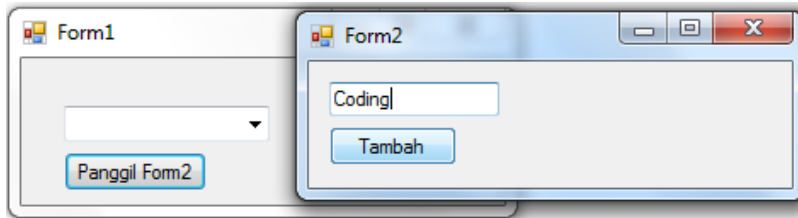
```
Private Sub btnForm2_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnForm2.Click  
    My.Forms.Form2.Show()  
End Sub
```

5. Double click pada btnTambah di Form2 kemudian tambahkan kode berikut:

```
Private Sub btnTambah_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnTambah.Click  
    My.Forms.Form1.cmbHobi.Items.Add(txtHobi.Text)  
End Sub
```

6. Jika anda menambahkan hobi di Form2 maka nilainya akan dimasukkan kedalam combobox yang ada di Form1, jadi interaksi antar control yang berada di form yang berbeda cukup mudah karena setiap form sudah otomatis dibuatkan instancinya (implicit), dan control di

dalam form tersebut access modifiernya Friend jadi bisa diakses dari form lain yang ada dalam satu project.



Gambar 4.17 Interaksi Antar Form

Menggunakan Dialog Windows

Anda dapat mendapatkan response dari user dengan menggunakan dialog windows. anda dapat menampilkan dilaoag windows dengan menggunakan MessageBox.

```
Dim result As DialogResult
result = MessageBox.Show("Apakah benar akan menghapus data ?", "Konfirmasi", MessageBoxButtons.OKCancel)

If result = Windows.Forms.DialogResult.OK Then
    MessageBox.Show("Anda mengklik OK")
Else
    MessageBox.Show("Anda mengklik Cancel")
End If
```

Jawaban dari user akan berupa object DialogResult (bisa OK atau Cancel). Sangat mudah menggunakan MessageBox untuk menampilkan form konfirmasi ke user, namun bagaimana caranya jika anda ingin menggunakan Form biasa untuk mengembalikan result dari user?. Misal selain tombol OK dan Cancel anda juga ingin menampilkan Label atau ListBox yang berisi informasi tertentu, anda tidak bisa menggunakan MessageBox untuk keperluan seperti ini.

Untuk menggunakan form yang dapat mengembalikan result seperti **MessageBox** anda bisa mengatur properti **DialogResult** pada tombol. Misal anda mempunyai form yang berisi **ListBox**, dan dua tombol **btnOk** dan **btnCancel**. Atur

properti **DialogResult** pada **btnOk** menjadi 'OK' dan **btnCancel** menjadi 'Cancel'.

```
Dim result As DialogResult
result = My.Forms.Form2.ShowDialog()

If result = Windows.Forms.DialogResult.OK Then
    MessageBox.Show("Anda menekan tombol OK")
Else
    MessageBox.Show("Anda menekan tombol Cancel")
End If
```

Owned Forms

.NET Framework mempunyai konsep owned forms. Owned form adalah form yang dimiliki oleh form lain, jika form pemilik diminimize maka semua form yang dimilikinya juga akan di minimize. Form yang dimiliki (owned form) juga selalu ditampilkan diatas form pemilik (owner). Biasanya owned form digunakan untuk membuat floating toolbox dan command windows (missal form find dan replace windows di MS Word). Cara untuk mengantur sebuah form menjadi owned form adalah:

```
My.Forms.SearchForm.Owner = Me
My.Forms.ReplaceForm.Owner = Me
```

```
My.Forms.SearchForm.Show()
```

MDI Form

MDI (Multiple Document Interface) program adalah program yang berbasikan Form tunggal yang mempunyai banyak form anak. Penggunaan model MDI ditujukan agar user dapat menggunakan banyak form secara bersamaan (report, grid,list, dll).

Setiap Form dapat menjadi container MDI dengan cara mengatur properti **IsMdiContainer** menjadi **true**. Untuk membuat sebuah form sebagai anggota dari form container

anda dapat mengatur properti **MdiParent** (mirip seperti pengaturan pada owned form).

```
Me.IsMdiContainer = True
```

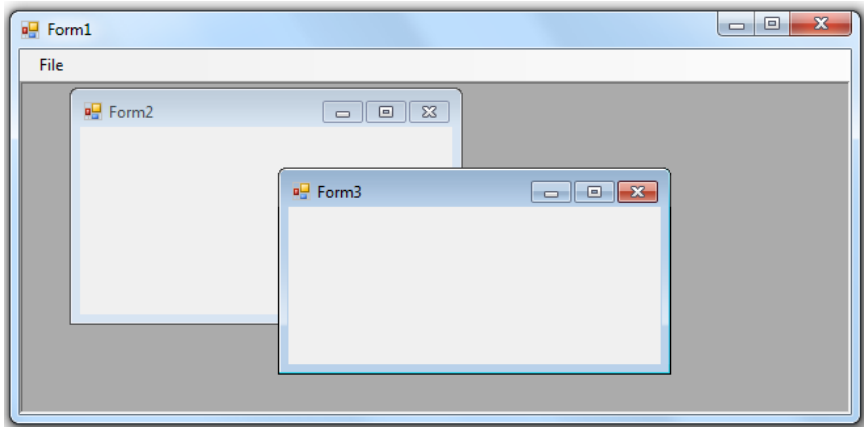
```
My.Forms.Form2.MdiParent = Me
```

```
My.Forms.Form3.MdiParent = Me
```

```
My.Forms.Form2.Show()
```

```
My.Forms.Form3.Show()
```

Kode diatas ditulis di form yang bertindak sebagai form container, pada kode tersebut form yang akan digunakan sebagai child didaftarkan terlebih dahulu dengan mengatur properti **MdiParent**.



Gambar 4.18 Menggunakan Mdi Form

Strips dan Menu

VB9 menyediakan model menu yang amat sangat mudah untuk digunakan, anda tidak perlu melakukan koding secara manual, anda hanya cukup melakukan drag and drop kemudian mengatur properti menu. Menu pada VB9 sering disebut strip control karena semua diturunkan dari class **System.Windows.Forms.ToolStrip**. Menu control yang ada di VB9 adalah:

- **ToolStrip** : untuk meletakkan toolbar control.
- **MenuStrip** : untuk membuat menu windows.
- **ContextMenuStrip** : digunakan untuk menampilkan popup menu jika user melakukan klik kanan pada control.
- **StatusStrip** : digunakan untuk menampilkan status bar. Status bar mirip dengan ToolStrip hanya saja tidak menampilkan tombol melainkan text.
- **ToolStripContainer** : digunakan untuk menaruh beberapa ToolStrip yang dapat di atur letaknya.

System Tray Icons

VB9 menyediakan cara yang mudah untuk menggunakan system tray icon, yang perlu anda lakukan hanya meletakkan **NotifyIcon** control kedalam component try dan menyediakan icon yang akan ditampilkan menggunakan properti icon.



Gambar 4.18 Menggunakan Tray Icon

Provider

Provider adalah tipe komponen yang unik, provider menambahkan properti pada control lain dalam form. Misalnya jika anda menambahkan **ToolTipProvider** ke component try maka secara otomatis properti dengan nama ToolTip akan muncul pada properti control yang lain pada form tersebut (misal pada button, textbox).

Ada beberapa provider yang lain di VB9 yaitu:

- **HelpProvider**: provider ini digunakan untuk menampilkan pesan help atau menampilkan windows help. Anda dapat menggunakan tombol F1 untuk menampilkan help.

- **ErrorProvider:** provider ini digunakan untuk menampilkan error pada control tertentu, misalnya pada TextBox ketika kosong, atau inputan yang dimasukan oleh user tidak sesuai.

Bab 5

Object Oriented Programming

Bahasa Vb pertama kali memperkenalkan Object Oriented Programming (OOP) pada versi 4.0, namun sampai dengan versi 6.0 fitur-fitur OOP tersebut masih belum dirasa lengkap dan belum bisa menjawab kebutuhan developer.

Setelah berjalan pada .NET Framework mulai dari VB7 (.NET 1.1), bahasa VB mengalami revolusi besar dengan bertambahnya banyak sekali fitur OOP yang masuk kedalamnya.

Jika anda ingin melakukan sesuatu di .NET anda pasti membutuhkan object, misal ketika anda bekerja dengan file maka anda harus menggunakan object File atau Directory, untuk mencetak ke printer dapat menggunakan PrintDocument object, dan untuk berinteraksi dengan database membutuhkan DataReader, DataSet object. Jadi dapat disimpulkan bahwa pada VB9 semua adalah object.

VB9 data types seperti array, string, dan bahkan integer adalah object, pada chapter sebelumnya kita bekerja dengan menggunakan Form yang juga object.

Untuk bekerja dengan object secara maksimal menggunakan VB9 anda harus mengerti secara benar konsep object oriented programming.

Pengantar Object Oriented Programming

Bahasa VB terkenal karena kemudahan dalam mengembangkan aplikasi secara terstruktur. Programmer VB biasa menggunakan style event-driven programming yang menyebabkan terjadi duplikasi dan kode tersebar di berbagai tempat. Jika anda ingin membuat kode yang lebih baik, lebih reliable, extensible, dan dapat di reuse dengan mudah, anda perlu mempelajari teknik OOP.

Apa itu OOP?

Secara mudah OOP dapat disimpulkan sebagai suatu teknik atau metode untuk menangani kompleksitas pembuatan program. Jika anda mengikuti aturan-aturan dalam metode tersebut maka program yang anda buat akan lebih mudah untuk dimaintain dan dikembangkan.

Mengapa Perlu OOP?

Aplikasi berbasis windows dan web adalah program yang sangat kompleks yang menyajikan tampilan menarik kepada user dan menawarkan user interface yang kompleks dengan tombol, dropdown, popup menu, toolbox, dan masih banyak komponen yang lain. Dibelakang user interface tersebut masih ada hubungan bussiness proses yang kompleks seperti misal hubungan antara Product, Customer, dan Order yang melibatkan banyak logika dan pengolahan data.

Untuk menangani berbagai kompleksitas yang terjadi maka programmer mengembangkan teknik yang diberi nama “Object Oriented Programming”, dengan tujuan yang sederhana yaitu mengatur kompleksitas dengan memodelkan

aspek esensial yang ada. Jika model yang dikembangkan semakin mendekati problem yang akan dipecahkan maka semakin mudah pula persoalan tersebut dapat dimengerti sehingga program lebih mudah ditulis dan di-maintain.

Types

Jika kita cermati .NET class library terdiri dari beberapa type yaitu:

- **Structure** : adalah kumpulan data types, structure membungkus beberapa variable menjadi satu package. Perbedaan yang mendasar antara structure dan class adalah stucture merupakan value type sedangkan class merupakan reference type.
- **Classes** : class mirip dengan structure tapi lebih advanced, terdiri dari fields (class variable), method (sub dan function), properties, dan event.
- **Delegates** : delegate mendefinisikan signature dari method, dengan menggunakan delegate anda dapat mereferensi variable untuk mereferensi ke method tertentu.
- **Enumeration** : Enumeration adalah kumpulan konstanta. Enumeration memudahkan programmer untuk bekerja dengan banyak pilihan nilai.
- **Interfaces** : interface adalah kontrak berisi properti atau method yang nantinya harus dimiliki oleh class yang mengimplementasikan interface tersebut.

Structure

Sebelum muncul class dalam OOP maka ide pertama adalah membuat structure. Structure digunakan untuk melakukan grouping data (beberapa variable) secara bersama.

Misalnya anda ingin menyimpan informasi tentang data mahasiswa anda dapat membuat beberapa variable seperti

nim, nama, alamat, ipk, dan nohp, namun jika anda membuat menjadi variable yang berdiri sendiri-sendiri maka kode anda akan menjadi semakin kompleks, dan kita tidak bisa tahu bahwa variable-variable tersebut punya hubungan. Jika anda ingin menyimpan data beberapa mahasiswa maka ini akan sangat sulit dilakukan untuk melacak hubungan informasi antar mahasiswa. Untuk menjadikan informasi yang ada menjadi satu group anda dapat menggunakan structure.

Public Structure Mahasiswa

```
Dim nim As String
Dim nama As String
Dim ipk As Double
Dim nohp As String
```

End Structure

Pada VB9 anda dapat menggunakan structure pada file atau module level. Anda dapat menggunakan structure yang anda buat diatas dengan cara:

```
Dim mhs1 As Mahasiswa
mhs1.nim = "23080008"
mhs1.nama = "Erick Kurniawan"
mhs1.ipk = 3.5
mhs1.nohp = "08156881169"
```

Kode diatas mudah untuk dibaca, ketika anda hendak mengubah nilai dari variable anda juga mengetahui mahasiswa mana yang informasinya akan diubah. Anda juga dapat melakukan parsing semua informasi mahasiswa dengan menggunakan structure dari subroutines atau function yang anda buat.

Function GetMhs(ByVal mhs1 As Mahasiswa)

```
'code disini..
```

End Function

Structure memiliki konsep yang sama seperti pada database, di database anda biasa membuat table, dan table terdiri dari field-field yang membentuk sebuah record. Structure dapat diibaratkan seperti record dalam table yang berisi variable (field-field) yang menyusun informasi tertentu.

Class

Bagaimana membuat structure yang anda buat menjadi lebih smart? Misalnya structure mahasiswa tidak mengijinkan panjang dari nim lebih dari 8 karakter, dapat memberi informasi lengkap dari mahasiswa, dan dapat memberi peringatan jika ip sudah dibawah 2.0.

Untuk keperluan diatas anda harus menggunakan Class. Structure dan class memiliki kesamaan yaitu dapat menyimpan informasi, namun class memiliki banyak fitur yang tidak dimiliki oleh structure. Untuk menambahkan class pada project, klik kanan project ► pilih add ► new item ► pilih class ► kemudian beri nama class-nya.

Jika kode structure yang sebelumnya diubah menjadi class maka kodenya:

```
Public Class Mahasiswa
```

```
    'data mahasiswa
    Public nim As String
    Public nama As String
    Public ipk As Double
    Public nohp As String
```

```
    'fitur untuk menampilkan data mahasiswa
```

```
    Public Function GetMhsInfo() As String
        Return "Nim : " & nim & " Nama : " & nama & _
            " Ipk : " & ipk
    End Function
```

```
End Class
```

Jika anda cermati kode diatas mirip dengan kode untuk membuat structure, perbedaannya terletak pada keyword public yang digunakan pada variable. Sebenarnya secara default modifier dari variable yang ada di dalam class adalah private (tidak dapat diakses dari luar class) tapi karena kita menghendaki variable tersebut dapat diakses maka dibuat public. Class Mahasiswa juga mempunyai tambahan method dengan nama **GetMhsInfo()** untuk menampilkan informasi dari mahasiswa.

Instansiasi Object

Untuk menggunakan class Mahasiswa yang telah kita buat sebelumnya, anda harus membuat object dari class tersebut (instansiasi).

```
'beberapa cara untuk instansiasi class  
Dim mhs1 As Mahasiswa = New Mahasiswa()  
Dim mhs2 As New Mahasiswa
```

Pada VB9 anda dapat membuat class dengan cara yang berbeda, yang pertama anda dapat mendelkarsikan variable dengan tipe class terlebih dahulu baru membuat object, dan cara kedua digunakan jika anda ingin mendeklarasikan variable sekaligus membuat object. Keyword **New** wajib digunakan jika anda ingin membuat object baru. Anda tidak perlu menggunakan keyword **set** seperti pada VB6.

Setelah object dibuat anda dapat memberi nilai fields (class variable nya).

```
mhs1.nim = "23080009"  
mhs1.nama = "Erick Kurniawan"  
mhs1.ipk = 3.5  
mhs1.nohp = "08156881169"  
MessageBox.Show(mhs1.GetMhsInfo(), "Keterangan")
```

Untuk release object (variable tidak lagi mereferensi ke object) anda dapat menggunakan keyword **Nothing**.

```
mhs1=Nothing
```

Penggunaan keyword **Nothing** memberi tahu pada compiler VB bahwa object tersebut sudah tidak dibutuhkan lagi. Sebenarnya compiler akan segera membersihkan object tersebut secara otomatis ketika keluar dari kalang program (out of scope), misal diakhir kalang subrutin object tersebut akan dibersihkan, jika anda ingin membersihkan object sebelum out of scope anda dapat menggunakan keyword **Nothing**.

Lebih Dalam Tentang Object

Ketika anda membuat object dari class, anda akan menyadari bahwa object tersebut memiliki ciri yang berbeda dengan variable biasa (sudah anda pelajari di **Bab3** tentang value type dan reference type) ini disebabkan karena class adalah reference type.

Mengkopi Object

Jika anda ingin mengcopy nilai seperti anda biasa lakukan di tipe data biasa (integer, double) maka anda tidak akan mendapatkan hasil yang anda harapkan, misal:

```
Dim mhs1 As New Mahasiswa
```

```
Dim mhs2 As New Mahasiswa
```

```
mhs1.nama = "Erick"
```

```
'mhs2 akan mereferensi ke object yang sama dengan mhs1,
```

```
'object dari mhs2 tidak digunakan
```

```
mhs2 = mhs1
```

Statement terakhir **mhs2 = mhs1** artinya **mhs2** sekarang mereferensi ke object yang sama dengan **mhs1**, bukan nilai dari **mhs1** dicopykan ke **mhs2**. Sedangkan object asli dari **mhs2** sudah tidak direferensi lagi.

```
'akan menghasilkan erick
```

```
MessageBox.Show(mhs2.nama)
```

Biarpun kode diatas 'sepertinya' benar, bahwa **mhs2.nama** menghasilkan 'Erick', namun nilai 'Erick' tidak dicopy ke variable **mhs2**.

```
mhs2.nama = "Rully"
```

```
MessageBox.Show(mhs1.nama)
```

Jika anda mengganti fields pada object **mhs2** maka nilai fields pada **mhs1** juga otomatis berubah, karena **mhs1** dan **mhs2** mereferensi ke object yang sama (tidak hanya mengcopy nilai masing-masing).

Jika anda ingin mengcopy object beserta dengan valuenya, atau ingin membuat object baru yang persis sama beserta valuenya, tidak hanya mereferensi ke object yang sama, maka anda dapat menggunakan method **clone()**. Method ini disupport oleh sebagian class yang ada di .NET contohnya Array.

```
Dim arrNama1() As String = {"erick", "rully", "naren"}  
Dim arrNama2() As String = arrNama1.Clone()
```

```
'jika elemen arrNama2 diganti  
arrNama2(0) = "bejo"
```

```
'tidak berpengaruh pada arrNama1 karena referensi  
'objectnya berbeda  
MessageBox.Show(arrNama1(0))
```

Membandingkan Object

Tipe reference juga punya aturan tersendiri untuk membandingkan object, anda tidak dapat menggunakan tanda '=' untuk operator pembandingan antar object. Untuk membandingkan dua variable mereferensi ke object yang sama atau tidak anda dapat menggunakan keyword **Is** atau **IsNot**.

```
Dim mhs1 As New Mahasiswa  
Dim mhs2 As New Mahasiswa
```

```
mhs1.nama = "Erick"  
mhs2.nama = "Erick"
```

```
'menghasilkan false karena tidak mereferensi ke object yg  
'sama  
MessageBox.Show(mhs1 Is mhs2)
```

```
mhs2 = mhs1  
'menghasilkan true karena mereferensi ke object yang sama  
MessageBox.Show(mhs1 Is mhs2)
```

Biarpun fields pada object **mhs1** dan object **mhs2** sama namun **mhs1** tidak sama dengan **mhs2** karena keduanya mereferensi ke object yang berbeda.

Null Reference Exception

Jika anda sudah mendefinisikan variable bertipe class namun belum menginstansiasi kemudian langsung digunakan maka akan terjadi **NullReferenceException**.

```
Dim mhs1 As Mahasiswa  
'menghasilkan NullReferenceException  
mhs1.nama = "Erick Kurniawan"
```

Partial Class

Seperti yang sudah anda ketahui bahwa anda dapat membuat beberapa class sekaligus di dalam file .vb, dan nama file tidak harus sama dengan nama classnya. VB9 juga memiliki fitur untuk memecah penulisan class pada beberapa file yang berbeda menggunakan method Partial. Partial class akan sangat berguna jika anda bekerja dengan class yang sangat besar.

Misal anda dapat membuat class Mahasiswa yang disimpan dalam file Mahasiswa1.vb.

```
Partial Public Class Mahasiswa  
    Public nim As String  
    Private nama As String  
    Private ipk As Double  
End Class
```

Kemudian anda dapat menambahkan bagian dari class diatas pada file lain misalnya Mahasiswa2.vb

```
Partial Public Class Mahasiswa  
    Public Function GetMhsInfo() As String  
        Return "Nim : " & nim & " Nama : " & nama & _  
            " IpK : " & ipk.ToString  
    End Function  
End Class
```

Biarpun class Mahasiswa terpisah di dua file tapi compiler tetap tahu kedua file tersebut satu kesatuan class.

Class Properties

Seperti kita ketahui class terdiri dari class member yaitu :

- Fields (class variable)
- Properti (untuk mengakses private fields)
- Method (sub dan function)
- Event

Fields pada class Mahasiswa yang kita buat sebelumnya memiliki access modifier Public yang berarti fields tersebut dapat diakses oleh class lain secara bebas. Memang membuat class menjadi Public memudahkan anda untuk bekerja dengan fields namun cara ini juga membahayakan karena kita tidak dapat memastikan bahwa value yang dimasukkan kedalam fields adalah value yang benar (yang kita harapkan). Misal kita mengharapkan fields nim dari class Mahasiswa diisi dengan format yang benar, terdiri dari 8 karakter. Jika anda menggunakan public fields anda tidak dapat memvalidasi apakah inputan fields benar.

Karena alasan diatas maka anda disarankan membuat fields dengan access modifier private kemudian untuk mengakses fields tersebut dari luar class anda dapat membuat properti. Properti digunakan untuk mengenkapsulasi logika program yang kita buat untuk memvalidasi inputan yang masuk ke fields. Property terdiri dari method get dan set, method get digunakan untuk mengambil nilai dari private fields, sedangkan method set digunakan untuk memberi nilai kedalam private fields.

Public Class Mahasiswa

'private fields

Private _nim As String

'untuk mengakses private fields diatas gunakan property

Public Property Nim() As String

Get

```

        Return _nim
    End Get
    Set(ByVal value As String)
        _nim = value
    End Set
End Property
End Class

```

Kemudian setelah membuat property cara untuk mengaksesnya adalah:

```

Dim mhs1 As New Mahasiswa
mhs1.Nim = "23080007"

```

Anda mungkin bertanya jika menuliskan kode seperti diatas apa bedanya jika anda menggunakan Public fields. Bedanya dengan menggunakan property maka anda dapat memastikan bahwa inputan ke fields benar. Anda dapat menambahkan kode validasi (misal: panjang dari nim harus 8 karakter). Untuk menambahkan validasi tambahkan kode berikut pada property.

```

Private _nim As String

Public Property Nim() As String
    Get
        Return _nim
    End Get
    Set(ByVal value As String)
        If value.Length <> 8 Then
            MessageBox.Show("Panjang nim harus 8 karakter")
        Else
            _nim = value
        End If
    End Set
End Property

```

Anda dapat menambahkan kode untuk validasi di dalam method set pada property, sehingga sebelum nilai masuk dalam private fields terlebih dahulu harus dipastikan bahwa nilai yang diinputkan valid.

ReadOnly Property

ReadOnly property digunakan jika anda hanya mengijinkan private field hanya dibaca saja, tidak untuk diberi nilai. Misal untuk field Ipk tidak dapat diganti nilainya, hanya dapat diambil nilainya.

```
Private _ipk As Double
Public ReadOnly Property Ipk() As Double
    Get
        Return _ipk
    End Get
End Property
```

Constructor

Constructor pada VB9 adalah method dengan nama **New** yang digunakan untuk menginisialisasi (memberi nilai) awal fields pada saat class tersebut di instansiasi.

```
Public Class Mahasiswa
    Private nim As String
    Private nama As String
    Private ipk As Double

    'constructor yang digunakan untuk inisialisasi awal
    Public Sub New()
        nim = "23080007"
        nama = "Erick Kurniawan"
    End Sub

    Public Function GetInfo() As String
        Return "Nim : " & nim & " Nama : " & nama
    End Function
End Class
```

Ketika anda membuat object baru dari class Mahasiswa maka field nim dan nama akan otomatis diinisialisasi dengan nilai yang anda definisikan di constructor.

```
'field otomatis diberi nilai oleh constructor
Dim mhs1 As New Mahasiswa
```

```
MessageBox.Show(mhs1.GetInfo())
```

Constructor Berparameter

Constructor juga dapat memiliki parameter, contoh constructor berparameter dapat dilihat pada kode dibawah ini:

```
Public Sub New(ByVal nim As String, ByVal nama As String,
ByVal ipk As Double)
    Me.nim = nim
    Me.nama = nama
    Me.ipk = ipk
End Sub
```

Jika anda menggunakan constructor berparameter maka cara membuat instan class (object):

```
Dim mhs2 As New Mahasiswa("23080007", "Erick Kurniawan", _
                           3.5)
MessageBox.Show(mhs1.GetInfo())
```

Overloading Constructor

VB9 juga mendukung penggunaan Multiple Constructor, anda dapat membuat lebih dari satu constructor (overloading constructor). Untuk membuat lebih dari satu constructor syaratnya adalah parameter dari constructor tipe datanya harus berbeda atau jumlah parameternya berbeda.

```
'contoh overloading constructor
Public Sub New()
    nim = "23080007"
    nama = "Erick Kurniawan"
End Sub

Public Sub New(ByVal nim As String)
    Me.nim = nim
End Sub
```

```
Public Sub New(ByVal ipk As Double)
    Me.nama = nama
End Sub

Public Sub New(ByVal nim As String, _
ByVal nama As String, ByVal ipk As Double)
    Me.nim = nim
    Me.nama = nama
    Me.ipk = ipk
End Sub
```

Default Constructor

Jika anda tidak membuat constructor maka class secara implicit secara otomatis sudah membuat constructor kosong.

```
'default constructor
Public Sub New()
    'inisialisasi field
End Sub
```

Destructor

Pada .NET anda tidak perlu lagi membuat destructor ketika object tersebut sudah tidak lagi dibutuhkan. Object yang sudah tidak digunakan atau out of scope akan secara otomatis dibersihkan oleh Garbage Collection. Garbage Collection adalah salah satu komponen yang ada di CLR (Common Language Runtime) .NET yang bertugas untuk membersihkan object yang sudah tidak digunakan.

Akhir Object VB6

Pada VB6 jika anda sudah tidak membutuhkan object maka anda dapat menggunakan keyword set dan diberi nilai Nothing. Karena object dapat direferensi oleh beberapa variable, maka semua variable harus di set Nothing sampai

variable terakhir yang mereferensi object tersebut. Jika sudah tidak ada yg mereferensi maka otomatis object tersebut akan dihapus dari memory. Metode ini dinamakan *deterministic finalization* karena anda harus tahu kapan object tersebut dihapus.

Akhir Object VB9

Pada VB9 programmer menjadi lebih mudah karena tidak perlu menghapus object secara manual karena ketika object tersebut sudah tidak diperlukan atau out of scope secara otomatis Garbage Collection akan membersihkannya. Metode ini disebut *non deterministic finalization*. Kelemahannya anda tidak tahu pasti kapan object tersebut dihapus karena tergantung pada Garbage Collection. Anda sebenarnya dapat memanggil GC secara langsung dengan perintah.

GC.Collect()

Namun cara ini tidak disarankan karena akan memperlambat program, karena GC akan menscan semua memory dan memeriksa object yang sudah tidak digunakan.

Cara yang disarankan digunakan untuk membuat program anda menjadi lebih efektif adalah dengan membuat sendiri 'destruktor' pada class yang anda buat yaitu menggunakan method **Dispose()**. Umumnya class tidak memerlukan **Dispose()** method, tapi untuk kasus tertentu ada juga yang membutuhkan **Dispose()**. Misalnya class yang digunakan untuk koneksi dengan database atau file yang harus melepaskan resource secepat mungkin (koneksi dengan database harus ditutup secepat mungkin jika anda sudah tidak memerlukannya karena resource nya dibagi banyak user). Untuk menggunakan method **Dispose()** anda harus mengimplementasikan Interface **IDisposable**.

Events

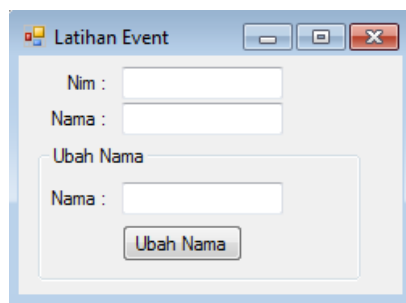
Salah satu bagian dari class adalah event. Event adalah notifikasi yang dikirimkan sebuah object ke object yang lain. Anda dapat mendeklarasikan event di class dengan keyword event.

```
Public Class Mahasiswa
    Private nim As String
    Private ipk As String
    'deklarasi event
    Public Event UbahData()
End Class
```

Event UbahData() pada kod diatas akan digunakan untuk memberi notifikasi ketika data mahasiswa diubah. Anda dapat memanggil event yang sudah dideklarasikan menggunakan keyword **RaiseEvent**.

Listing 5.1 Penggunaan Events

1. Buat windows application dengan nama “LatihanEvent”
2. Design tampilan windows sebagai berikut:



Gambar 5.1

3. Tambahkan class Mahasiswa, tambahkan kode berikut:

```
Public Class Mahasiswa
    Public Event UbahData()

    Private _nim As String
    Public Property Nim() As String
```

```

        Get
            Return _nim
        End Get
        Set(ByVal value As String)
            _nim = value
        End Set
    End Property

    Private _nama As String
    Public Property Nama() As String
        Get
            Return _nama
        End Get
        Set(ByVal value As String)
            _nama = value
            RaiseEvent UbahData()
        End Set
    End Property

    Public Sub New(ByVal nim As String, _
        ByVal nama As String)
        Me.Nim = nim
        Me.Nama = nama
    End Sub
End Class

```

4. Kemudian pada form tambahkan kode berikut:

```

Public Class Form1
    Private WithEvents mhs As New Mahasiswa("23080007",
        "Erick Kurniawan")

    Private Sub RefreshData() Handles mhs.UbahData
        txtNim.Text = mhs.Nim
        txtNama.Text = mhs.Nama
    End Sub

    Private Sub Form1_Load(ByVal sender As System.Object,
        ByVal e As System.EventArgs) Handles MyBase.Load
        RefreshData()
    End Sub

    Private Sub btnUbahNama_Click(ByVal sender As

```

```

System.Object, ByVal e As System.EventArgs) Handles
    btnUbahNama.Click
    mhs.Nama = txtUbahNama.Text
End Sub
End Class

```

Pada class Mahasiswa kita sudah mendeklarasikan event dengan nama UbahData() dan event tersebut dipanggil ketika data pada field nama diubah.

Kemudian pada form kita membuat object mhs dengan keyword WithEvents yang berarti object tersebut menggunakan event. Kita juga mendeklarasikan method dengan nama RefreshData() dengan menggunakan keyword handle, ini berarti method tersebut akan dipanggil ketika event UbahData() dijalankan (dengan keyword RaiseEvent) pada saat field nama pada object mhs diubah. Method RefreshData() akan mengupdate tampilan textbox nim dan nama pada form.

Enumeration

Enumeration adalah sebuah type yang terdiri dari kumpulan konstanta. Enumeration dibuat untuk menyelesaikan beberapa persoalan, misal ada kasus anda membuat method untuk pengecekan jurusan yang diambil oleh mahasiswa.

```

Function CekJurusan(ByVal jurusan As String) As String
    Dim kode As String = ""
    If jurusan = "TI" Then
        'tulis kode disini
    ElseIf jurusan = "SI" Then
        'tulis kode disini
    ElseIf jurusan = "Manajemen" Then
        'tulis kode disini
    End If
    Return kode
End Function

```

Beberapa masalah yang akan ditemui dari kode yang kita buat diatas.

- Pertama kita tidak dapat memastikan bahwa nilai dari parameter yang diinputkan benar (sesuai dengan yang diharapkan)
- Pengguna method akan bingung karena tidak tahu pasti data apa yang harus dimasukan kedalam parameter.
- Kode lebih sulit untuk dimengerti.
- Cara penulisan kode tidak standar karena mungkin ada programmer lain yang membuat method lain untuk menyelesaikan masalah yang sama.
- Tidak ada bantuan intellisense pada Visual Studio untuk mengisi nilainya sehingga kesalahan sangat mungkin terjadi.

Dalam kasus seperti diatas penggunaan Enumeration lebih disarankan. Dengan menggunakan enumeration maka kode diatas dapat diubah menjadi.

```
Public Enum TipeJurusan
    TI
    SI
    Manajemen
    Akuntansi
End Enum

Function CekJurusan(ByVal jurusan As TipeJurusan) As String
    Dim kode As String = ""
    If jurusan = TipeJurusan.TI Then
        kode = "22"
    ElseIf jurusan = TipeJurusan.SI Then
        kode = "21"
    ElseIf jurusan = TipeJurusan.Manajemen Then
        kode = "11"
    ElseIf jurusan = TipeJurusan.Akuntansi Then
        kode = "12"
    End If
    Return kode
End Function
```

Dengan enumeration masalah-masalah diatas dapat diatasi dan penggunaan method akan menjadi lebih mudah karena nilai parameternya sudah pasti.

```
Dim kode As String = CekJurusan(TipeJurusan.SI)
```

Shared Member

Pada Bab3 kita telah membahas bahwa class dapat memiliki shared member yaitu member yang dapat diakses tanpa anda harus membuat instan class dari class tersebut.

Shared Method

Jika anda mendeklarasikan method dengan keyword shared, maka anda akan dapat mengakses method tersebut langsung dari class tanpa harus membuat instan class terlebih dahulu. Ini sama dengan class bawaan dari .NET yang sering anda gunakan yaitu class Math, class MessageBox, dan class Console.

```
Dim hasil = Math.Sqrt(81)
```

Penggunaan keyword shared berarti shared method akan di share ke semua instance dari class tersebut dan selalu dapat diakses.

```
Public Class Mahasiswa
    Public Shared Function CekLulus(ByVal nilai As Double)
        As Boolean
            Return If(nilai >= 70, True, False)
        End Function
End Class
```

```
'mengakses shared method pada class Mahasiswa
Dim cek = Mahasiswa.CekLulus(85)
MessageBox.Show(cek)
```

Jika anda membuat shared method, anda tidak boleh menggunakan instance member (yang tidak menggunakan keyword shared) pada method yang anda buat.

```
Public Shared Function CekLulus() As Boolean
```

```
'error karena tidak dapat mengakses ipk
Return If(ipk >= 70, True, False)
End Function
```

Shared Properties

Sama seperti ketika anda membuat shared method, anda juga dapat membuat shared variable dan properties dengan menambahkan keyword shared.

```
Public Class Mahasiswa
    Private Shared _count As Integer
    Public Shared Property Count() As Integer
        Get
            Return _count
        End Get
        Set(ByVal value As Integer)
            _count = value
        End Set
    End Property

    Public Sub New()
        _count += 1
    End Sub
End Class
```

Pada kode diatas dapat dilihat bahwa setiap anda membuat object baru maka fields _count akan ditambah satu, anda dapat mengakses properties Count untuk mengetahui jumlah object Mahasiswa yang di create.

```
Dim mhs1 As New Mahasiswa
Dim mhs2 As New Mahasiswa
```

```
'menghasilkan 2 karena ada 2 object yang dibuat
MessageBox.Show(Mahasiswa.Count.ToString)
```

Menggunakan Module

Pada VB, class bukanlah satu-satunya tempat untuk menuliskan kode, anda dapat juga menuliskan kode anda di

dalam module. Modul biasanya digunakan untuk membuat helper method (method yang sering digunakan di beberapa tempat pada aplikasi anda). Bagi kebanyakan programmer VB module cara untuk melakukan reuse pada kode.

Fungsi modul hampir mirip dengan shared method yang baru saja kita pelajari. Module adalah cara yang paling simple untuk sharing function pada aplikasi.

```
Module MyModule
```

```
    Function Tambah(ByVal bil1 As Integer, ByVal bil2 As Integer) As Integer
```

```
        Return bil1 + bil2
```

```
    End Function
```

```
    Function Kurang(ByVal bil1 As Integer, ByVal bil2 As Integer) As Integer
```

```
        Return bil1 - bil2
```

```
    End Function
```

```
End Module
```

```
'cara untuk memanggil module
```

```
Dim hasil As Integer = MyModule.Tambah(12, 23)
```

BAB 6

Objek Oriented Programming bagian 2

Pada bab sebelumnya kita telah mempelajari tentang apa itu class, properties, constructor, method, events dan membuat object dari class. Pada bab ini akan dipelajari beberapa topik Object Oriented Programming lanjutan seperti inheritance, interface, dan collection.

Prinsip-prinsip OOP

Untuk berubah dari paradigma traditional menjadi Object Oriented Programming tidak mudah karena banyak perbedaan yang signifikan. OOP mempunyai prinsip dan aturan yang harus diikuti untuk dapat membangun program yang reusable, efisien, dan dapat dimanage dengan mudah. Beberapa prinsip OOP adalah:

Enkapsulasi Data dengan Class

Enkapsulasi atau penyembunyian informasi adalah konsep yang selama ini sering anda dengar. Pada OOP informasi dibungkus dalam sebuah Class sehingga orang yang menggunakan class tersebut tidak perlu tahu detail implementasi dari class, hanya cukup mengetahui method dan property apa saja yang ada dalam class tersebut beserta kegunaannya.

Karena terenkapsulasi dalam class, setiap komponen dalam program dapat diubah tanpa harus 'merusak' kode yang lama.

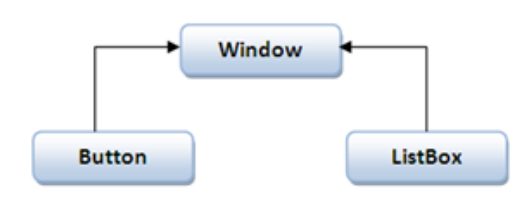
Spesialisasi dan Generalisasi

Class dan instannya (objek) tidak berdiri sendiri, seperti objek manusia yang merupakan makhluk sosial tidak hanya berdiri sendiri, objek akan saling berelasi dan berinteraksi, demikian pula pada OOP.

Salah satu hubungan antar objek yang paling penting yang terjadi di dunia nyata adalah spesialisasi yang bisa didefinisikan sebagai *is-a relationship*, misal anjing adalah mamalia, jika anjing termasuk anggota mamalia maka dia memiliki ciri-ciri special khas yaitu menyusui, punya bulu. Selain sifat-sifat pada mamalia anjing juga memiliki ciri yang spesifik khas anjing yang tidak dipunyai anggota mamalia lainnya seperti kucing.

Relasi spesialisasi dan generalisasi terjadi pada contoh diatas, anjing dan kucing secara general merupakan anggota mamalia tetapi juga terspesialisasi secara spesifik sebagai anjing dan kucing yang mempunyai ciri khas yang berbeda. Relasi tersebut dapat digambarkan secara hirarki.

Misal gambar dibawah akan menunjukkan hirarki antara class window, button, dan listbox



Gambar 6.1 Generalisasi dan Specialisasi

Gambar diatas menunjukkan relasi antara window, button, dan listbox. Button dan listbox sama-sama memiliki karakteristik yang dimiliki oleh window, tetapi button dan listbox juga memiliki ciri-ciri spesifik khas mereka sendiri.

Inheritance

Inheritance merupakan fitur OOP baru yang tidak ada di VB6. Dengan menggunakan Inheritance anda dapat membuat

class baru (derived class) dari class yang sudah ada (base class). Derived class akan mempunyai behaviour (field, method, properties) dari base class dan ditambah dengan behaviour miliknya sendiri. Inheritance juga merupakan penerapan konsep spesialisasi pada OOP.

Kegunaan dari Inheritance yang terutama adalah untuk code reuse (penggunaan kode kembali). Misal pada .NET anda sudah mempunyai class button, namun anda tidak puas karena tidak ada properti shape yang dapat digunakan untuk merubah bentuk button menjadi menarik, maka anda dapat membuat class baru yang diturunkan dari class button yang sudah ada, kemudian ditambahkan dengan properti shape. Jadi anda tidak perlu membuat class button lagi dari awal karena kita sudah mempunyai class button yang lama yang dapat di reuse.

Jika anda menggunakan aplikasi windows maka form yang anda buat pasti diturunkan dari class **System.Windows.Forms.Form**. Anda juga harus menambahkan referensi class library yang diperlukan.

Penerapan Inheritance di VB menggunakan keyword **Inherits**. Contoh penggunaan Inheritance secara sederhana ditunjukkan pada contoh dibawah ini.

Pertama kita membuat sebuah class dengan nama MyControl yang akan kita jadikan sebagai base class.

```
Public Class MyControl
    Private left As Integer
    Private top As Integer

    'Constructor
    Public Sub New(ByVal left As Integer, ByVal top As Integer)
        Me.left = left
        Me.top = top
    End Sub

    Public Function GambarMyControl() As String
        Return String.Format("Menggambar MyControl pada posisi ({0},{1})",left, top)
    End Function
```

End Class

Class diatas mempunyai dua buah private fields, sebuah constructor dan sebuah function. Sekarang kita akan membuat derived class dari class tersebut. Untuk menurunkan class gunakan keyword **Inherits**.

Class MyListBox**Inherits MyControl****Private content As String****Public Sub New(ByVal left As Integer, ByVal top As Integer, ByVal content As String)****'menjalankan constructor milik base class****MyBase.New(left, top)****Me.content = content****End Sub****Public Function GambarMyListBox() As String****'menjalankan method milik base class****Return MyBase.GambarMyControl() & " dan isi content
: " & content****End Function****End Class**

Class MyListBox diturunkan dari class MyControl jadi class MyListBox juga memiliki behaviour (field, properties, method) dari base class-nya. Hanya saja field left dan top tidak dapat diakses karena access modifiernya private jadi hanya dapat diakses dari class itu sendiri, tidak bisa diakses dari class trurunannya.

Class MyListBox juga harus memiliki constructor karena MyControl yang menjadi base class-nya hanya memiliki sebuah constructor berparameter (tidak punya constructor kosong). Jadi constructor di MyListBox juga harus memanggil constructor di MyControl untuk menginisialisasi field left dan top. Untuk memanggil base constructor (constructor dari MyControl) digunakan keyword **MyBase.New()**. **MyBase** digunakan untuk mengakses method atau properties dari base

class, misal untuk menggunakan method `GambarMyControl()` penulisannya `MyBase.GambarMyControl()`.

Tambahkan sebuah class lagi dengan nama `Button` yang juga diturunkan dari `MyControl`.

```
Class MyButton  
    Inherits MyControl  
  
    Private warna As String  
  
    Public Sub New(ByVal left As Integer, ByVal top As Integer, ByVal warna As String)  
        MyBase.New(left, top)  
        Me.warna = warna  
    End Sub  
  
    Public Function GambarMyButton() As String  
        Return MyBase.GambarMyControl & " dan warnanya : "  
        & warna  
    End Function  
End Class
```

Kemudian terakhir buat object dari class-class yang sudah anda buat.

```
Dim objControl As New MyControl(12, 12)  
MessageBox.Show(objControl.GambarMyControl())  
  
Dim objListBox As New MyListBox(24, 24, "ListBox Content")  
MessageBox.Show(objListBox.GambarMyListBox())  
  
Dim objButton As New MyButton(88, 88, "merah")  
MessageBox.Show(objButton.GambarMyButton())
```

Protected Modifier

Pada kode diatas kita tidak dapat mengakses field `left` dan `top` dari class `MyControl` dari class turunannya karena access modifiernya `private`, jika anda ingin agar kedua field tersebut dapat diakses dari class turunannya namun juga tidak dapat diakses dari instan classnya maka anda dapat menggunakan access modifier `protected` pada field `left` dan `top`. Sebagai

contoh coba ubah access modifier dari field top dan left pada class MyControl dengan keyword Protected.

```
Public Class MyControl
    Protected left As Integer
    Protected top As Integer
    -----
End Class
```

Kemudian pada class MyListBox ubah kode dari method GambarListBox().

```
Class MyListBox
    Inherits MyControl
    -----
    Public Function GambarMyListBox() As String
        Return String.Format("Menggambar ListBox pada
            posisi({0},{1}) dan isi content: {2}", left, top,
            content)
    End Function
End Class
```

Seperti anda lihat pada kode diatas, kita bisa mengakses field left dan top dari class MyControl secara langsung pada class MyListBox. Ini dapat terjadi karena access modifier field di case class adalah protected.

Polymoprhism

Ada dua aspek yang paling penting pada inheritance yang pertama adalah code reuse, sebagai contoh ketika anda membuat class ListBox maka anda dapat menggunakan logic yang sudah dibuat pada class Window

Aspek kedua yang tidak kalah penting dari inheritance adalah polymorhism, poly berarti banyak dan morph berarti bentuk, jadi polymorphism dapat berarti dapat menggunakan banyak bentuk tanpa harus mengacu details yang sama.

Dari istilahnya Polymorphism kesannya sulit dan kompleks, tetapi sebenarnya simple, ilustrasinya begini ketika sebuah perusahaan telpon atau provide telpon mengirimkan signal ke anda (dering telpon), mereka tidak tahu telpon apa yang anda gunakan, bisa model lama dengan dering kuno, bisa

HP dengan dering music mp3, TV Phone, PDA Phone dll. yang provider tahu bahwa masing-masing telepon mempunyai 'base type' telepon dan mempunyai method untuk berdering. Ketika provider mengirimkan signal mereka hanya berharap bahwa telepon akan berdering tidak peduli jenis telponnya, nah provider telepon memperlakukan telepon anda secara polymorphism.

Dari gambar 6.1 diatas dapat anda lihat, ListBox is-a Window (karena ListBox turunan dari Window), Button is-a Window, kita berharap bahwa dapat memperlakukan keduanya sebagai Window. Sama juga dengan kucing dan sapi yang diturunkan dari class Mamalia, anda dapat mengatakan bahwa kucing dan sapi sama-sama Mamalia.

Misal anda mempunyai banyak control di collection untuk digambar ke windows, dan mengharapkan ketika memanggil method draw() pada control maka control2 tersebut akan tergambar sesuai dengan tipe objectnya (mis: ListBox, RadioButton, Command, dll).

Anda dapat mengimplementasikan polymorphism dengan dua cara yaitu:

- Membuat method dengan keyword *Overridable* pada base class (class induk). *Overridable* method adalah method yang akan di override di class turunannya.
- Membuat class turunan yang mempunyai method dengan keyword *overrides* yang mengganti isi dari method dengan keyword *virtual* yang ada pada base class-nya.

Method Overriding yaitu method dengan nama yang sama, tipe dan jumlah parameternya juga sama namun implementasinya berbeda, digunakan untuk mengoverride method yang ada pada base class.

Penggunaan prinsip polymorphism dapat dilihat pada contoh program dibawah ini. Pertama buat class dengan nama MyControl yang akan digunakan sebagai base class.

```
Public Class MyControl
    Private _left As Integer
    Public Property Left() As Integer
```

```

        Get
            Return _left
        End Get
        Set(ByVal value As Integer)
            _left = value
        End Set
    End Property

    Private _top As Integer
    Public Property Top() As Integer
        Get
            Return _top
        End Get
        Set(ByVal value As Integer)
            _top = value
        End Set
    End Property

    Public Sub New(ByVal left As Integer, ByVal top As Integer)
        Me.Left = left
        Me.Top = top
    End Sub

    Public Overridable Function DrawControl() As String
        Return String.Format("Menggambar Control pada posisi ({0},{1})", Left, Top)
    End Function
End Class

```

Kemudian buat class turunannya yaitu MyListBox dan MyButton.

```

Class MyListBox
    Inherits MyControl

    Private _content As String
    Public Property Content() As String
        Get
            Return _content
        End Get
        Set(ByVal value As String)

```

```

        _content = value
    End Set
End Property

Public Sub New(ByVal left As Integer, ByVal top As
Integer, ByVal content As String)
    MyBase.New(left, top)
    Me.Content = content
End Sub

Public Overrides Function DrawControl() As String
    Return String.Format("Menggambar ListBox pada
        posisi({0},{1}) dan isi content: {2}", Left, Top,
        Content)
End Function
End Class

Class MyButton
    Inherits MyControl

    Private _warna As String
    Public Property Warna() As String
        Get
            Return _warna
        End Get
        Set(ByVal value As String)
            _warna = value
        End Set
    End Property

    Public Sub New(ByVal left As Integer, ByVal top As
Integer, ByVal warna As String)
        MyBase.New(left, top)
        Me.Warna = warna
    End Sub

    Public Overrides Function DrawControl() As String
        Return String.Format("Menggambar Button pada posisi
            ({0},{1}) dan warnanya {2}", Left, Top, Warna)
    End Function
End Class

```

Pada kode diatas dapat dilihat bahwa method DrawControl() pada class MyControl di override oleh kedua class turunannya yaitu MyListBox dan MyButton. Maka kita dapat menerapkan Polymorphism, dengan cara:

```
Dim objControl As New MyControl(12, 12)
    Dim objListBox As New MyListBox(24, 24, "Content
        ListBox")
    Dim objButton As New MyButton(24, 24, "Merah")

    'polymorphism MyListBox dan MyButton dianggap
    'sebagai MyControl
    Dim arrControl(2) As MyControl
    arrControl(0) = objControl
    arrControl(1) = objListBox
    arrControl(2) = objButton

    For Each ctr In arrControl
        MessageBox.Show(ctr.DrawControl())
    Next
```

Jika anda perhatika kode diatas dapat dilihat adalah arrControl adalah variable bertipe array dari MyControl. Maka seharusnya yang dapat dimasukan kedalam array tersebut adalah object dari class MyControl, namun dalam kasus diatas objListBox dan objButton juga dapat dimasukan kedalam arrControl padahal dua object tersebut bukan dari class MyControl. Kenapa bisa begitu? Ini terjadi karena konsep polymorphism, class MyListBox dan MyButton adalah turunan dari class MyControl maka dapat digeneralisasi bahwa dua class tersebut juga merupakan MyControl.

Kemudian dengan menggunakan overriding method, method DrawControl() yang ada di base class dapat di override di class turunannya sehingga dapat mempunyai implementasi yang berbeda dengan base class.

Casting antar Object

Dengan menerapkan polymorphism anda juga dapat melakukan casting antar object, misalnya anda dapat mengkonversi dari tipe MyListBox menjadi MyControl.

```
Dim objList As New MyListBox(23, 23, "Ini ListBox")

    Dim objControl As MyControl = CType(objList,
        MyControl)
    MessageBox.Show(objControl.DrawControl())

    Dim objButton As New MyButton(24, 24, "Merah")

    Dim objControl2 As MyControl = CType(objButton,
        MyControl)
    MessageBox.Show(objControl2.DrawControl())

    'akan gagal tidak dapat cast dari MyControl ke
    'MyListBox InvalidCastException
    Dim objCtr As New MyControl(33, 33)
    Dim objLst As MyListBox = CType(objCtr, MyListBox)
    objLst.DrawControl()
```

Dengan menggunakan CType() anda dapat mengkonversi tipe yang lebih spesifik ke tipe yang lebih general, tapi tidak sebaliknya. Jika anda mencoba mengkonversi dari tipe MyControl yang lebih general ke tipe MyListBox yang lebih spesifik maka akan menyebabkan terjadi kesalahan InvalidCastException.

Abstract Class (MustInherit)

Setiap tipe Window pasti mempunyai bentuk tampilan yang berbeda misal combobox, listbox, button, dll. Berarti setiap class turunan dari Window harus mempunyai method DrawWindow() nya sendiri yang berbeda satu sama lain. Untuk memastikan bahwa setiap class turunan Control harus mempunyai method DrawWindow() maka anda harus mendefinisikan class tersebut sebagai abstract class dan memiliki method abstract yang belum diimplementasikan.

Abstract method tidak mempunyai implementation, abstract method hanya semacam kontrak yang harus dipenuhi. Semua class yang diturunkan dari class yang mempunyai abstract method harus mengimplementasikan method tersebut.

Class yang didefinisikan sebagai abstract class tidak dapat dibuat instan class / objek, class yang didefinisikan abstract hanya dapat digunakan dengan cara diturunkan.

Pada VB9 anda harus menggunakan keyword MustInherit untuk membuat abstract class. Kemudian untuk method yang harus diimplementasikan di class turunan diberi keyword MustOverride.

```
MustInherit Class MyControl
    Private _left As Integer
    Public Property Left() As Integer
        Get
            Return _left
        End Get
        Set(ByVal value As Integer)
            _left = value
        End Set
    End Property

    Private _top As Integer
    Public Property Top() As Integer
        Get
            Return _top
        End Get
        Set(ByVal value As Integer)
            _top = value
        End Set
    End Property

    Public Sub New(ByVal left As Integer, ByVal top As Integer)
        Me.Left = left
        Me.Top = top
    End Sub

    'method abstract yang belum memiliki implementasi
    Public MustOverride Function DrawControl() As String
End Class
```

Class MyControl diatas mempunyai keyword MustInherit jadi anda tidak dapat menginstansiasi class tersebut secara langsung, class tersebut hanya dapat digunakan dengan cara

diturunkan. Method DrawControl() juga memiliki keyword MustOverride ini berarti method tersebut harus dioverride di class turunannya karena di abstract class method ini belum diimplementasikan.

```
Class MyListBox
    Inherits MyControl

    Private content As String

    Public Sub New(ByVal left As Integer, ByVal top As
Integer, ByVal content As String)
        MyBase.New(left, top)
        Me.content = content
    End Sub

    'method yang harus diimplementasikan dari abstract class
    Public Overrides Function DrawControl() As String
        Return String.Format("Menggambar ListBox pada
            posisi ({0},{1}) dan isi content :{2}", Left,
            Top, content)
    End Function
End Class

Class MyButton
    Inherits MyControl

    Public Sub New(ByVal left As Integer, ByVal top As
Integer)
        MyBase.New(left, top)
    End Sub

    Public Overrides Function DrawControl() As String
        Return String.Format("Menggambar Button pada posisi
            ({0},{1}) dan warnanya :{2}", Left, Top)
    End Function
End Class
```

Class MyListBox dan MyButton diturunkan dari abstract class MyControl. Ini berarti kedua class tersebut harus menyetujui kontrak yang sudah didefinisikan di base class,

yaitu harus mengimplementasikan method yang MustOverride yaitu DrawControl().

Abstrac class juga mendukung konsep polymorphism sama seperti contoh penggunaan Inheritance sebelumnya.

```
Dim objList As New MyListBox(12, 12, "ListBox Content")
Dim objButton As New MyButton(24, 24)
```

```
Dim arrControl(1) As MyControl
arrControl(0) = objList
arrControl(1) = objButton
```

```
For Each ctr In arrControl
    MessageBox.Show(ctr.DrawControl())
Next
```

Karena MyListBox dan MyButton merupakan turunan dari abastract class MyContol maka kedua class tersebut dapat digeneralisasi sebagai class MyControl juga.

Interface

Jika anda ingin mendefinisikan tipe baru tetapi tidak ingin mengimplementasikan isinya secara langsung maka anda dapat menggunakan interfaces.

Pada dasarnya interface adalah sebuah kontrak, ketika anda mendesign interface seolah-olah anda mengatakan “jika anda ingin menggunakan class ini maka anda harus mengimplementasikan method dan property yang sudah disediakan”. Class yang menggunakan interface tersebut setuju dengan kontrak dan mengimplementasikan requirement yang ditentukan.

Dibandingkan dengan abstract class, abstract class hanya bisa digunakan di class paling atas ‘top-hierarchy’ sedangkan interface dapat digunakan di setiap class yang diinginkan (setiap class dapat mengimplementasikan lebih dari satu interface).

Perbedaan Interface dan Abstract Class

Programmer sering merasa bingung dalam membedakan Interface dan Abstract Class. Beberapa perbedaannya adalah:

- Cara penggunaan Abstract Class adalah diturunkan, sedangkan cara penggunaan Interface adalah diimplementasikan.
- Sebuah class hanya dapat diturunkan dari satu abstract class tapi dapat menggunakan lebih dari satu interfaces.
- Nama Interface disarankan menggunakan awalan huruf 'I', misal: IDisposable, IStorable.
- Method dan member variable pada abstract class boleh sudah ada isinya, sedangkan pada interfaces semua belum ada implementasinya.
- Pada Abstract Class semua method atau member variable yang MustOverride harus diimplementasikan di class turunannya.
- Pada Interface semua member variable dan method harus diimplementasikan di class yang menggunakan interface tersebut.
- Access Modifier pada method dan member variable di Interface secara implisit adalah public.

Anda dapat membuat interface pada VB dengan menggunakan keyword **Interface**. Semua method dan property yang dideklarasikan di Interface belum ada implementasinya.

```
Interface IStorable
    Sub Read()
    Sub Write()
    Property Status() As Integer
End Interface
```

Interface hanya dapat digunakan dengan cara diimplementasikan pada class tertentu. Class yang menggunakan interface tersebut harus mengimplementasikan

semua method dan properties yang dideklarasikan pada interface.

```
Class Document
    Implements IStorable

    Private _status As Integer = 1

    Public Sub New(ByVal doc As String)
        MessageBox.Show("Create :" & doc)
    End Sub

    'mengimplementasikan method Read()
    Public Sub Read() Implements IStorable.Read
        MessageBox.Show("Mengimplementasikan Method Read
            Pada Document")
    End Sub

    'mengimplemetasikan properties Status()
    Public Property Status() As Integer Implements
        IStorable.Status
        Get
            Return _status
        End Get
        Set(ByVal value As Integer)
            _status = value
        End Set
    End Property

    'mengimplementasikan method Write
    Public Sub Write() Implements IStorable.Write
        MessageBox.Show("Mengimplementasikan Method Write
            Pada Document")
    End Sub
End Class
```

Method dan properties yang ada di IStorable akan diimplementasikan di class Document.

```
Dim objDoc As New Document("New Document")
    objDoc.Read()
    objDoc.Write()
```

```
        MessageBox.Show("Document Status : " &  
objDoc.Status)
```

Penerapan Polymorphism pada Interface

Anda juga dapat menerapkan Polymorphism pada Interface. Dari contoh yang sudah anda buat, tambahkan class baru dengan nama Class Image.

```
Class Image  
    Implements IStorable  
  
    Private _status As Integer = 0  
  
    Public Sub New(ByVal img As String)  
        MessageBox.Show("Create Image")  
    End Sub  
  
    Public Sub Read() Implements IStorable.Read  
        MessageBox.Show("Mengimplementasikan Method Read  
        Pada Image")  
    End Sub  
  
    Public Property Status() As Integer Implements  
    IStorable.Status  
        Get  
            Return _status  
        End Get  
        Set(ByVal value As Integer)  
            _status = value  
        End Set  
    End Property  
  
    Public Sub Write() Implements IStorable.Write  
        MessageBox.Show("Mengimplementasikan Method Write  
        Pada Image")  
    End Sub  
End Class
```

Kemudian kita akan membuat array bertipe IStorable dan memasukan class Documen dan class Image didalamnya, ini

dapat dilakukan karena class Document dan class Image mengimplementasikan interface IStorable.

```
Dim objDoc As New Document("New Document")
Dim objImage As New Image("New Image")
```

```
Dim arrStore(1) As IStorable
arrStore(0) = objDoc
arrStore(1) = objImage
```

```
For Each store In arrStore
    store.Read()
    store.Write()
Next
```

Mengimplentasikan Banyak Interface

Tidak seperti abstract class atau class yang hanya dapat diturunkan dari satu base class, anda dapat mengimplementasikan beberapa interface sekaligus pada class yang sama. Misal untuk class dokumen diatas selain dapat disimpan (menggunakan IStorable) juga dapat dikompres datanya (menggunakan ICompressible).

```
Interface IStorable
    Sub Read()
    Sub Write(ByVal obj As Object)
    Property Status() As String
End Interface
```

```
Interface ICompressible
    Sub Compress()
    Sub Decompress()
End Interface
```

```
'mengkombinasikan interface
Interface ILogCompressible
    Inherits ICompressible

    Sub LogSavedBytes()
End Interface
```

```
Interface IStorableCompressible
```

```

    Inherits IStorable, ILogCompressible

    Sub LogOriginalSize()
End Interface

Interface IEncryptable
    Sub Encrypt()
    Sub Decrypt()
End Interface

Class Document
    Implements IStorableCompressible, IEncryptable

    Private _status As Integer

    Public Sub New(ByVal s As String)
        MessageBox.Show("Create : " & s)
    End Sub

    Public Sub Compress() Implements ICompressible.Compress
        MessageBox.Show("Mengimplementasikan method
            Compress")
    End Sub

    Public Sub Decompress() Implements
        ICompressible.Decompress
        MessageBox.Show("Mengimplementasikan method
            Decompress")
    End Sub

    Public Sub Decrypt() Implements IEncryptable.Decrypt
        MessageBox.Show("Mengimplementasikan method
            Decrypt")
    End Sub

    Public Sub Encrypt() Implements IEncryptable.Encrypt
        MessageBox.Show("Megimplementasikan method
            Encrypt")
    End Sub

    Public Sub LogSavedBytes() Implements

```

```

ILogCompressible.LogSavedBytes
    MessageBox.Show("Mengimplementasikan method
        LogSavedBytes")
End Sub

Public Sub Read() Implements IStorable.Read
    MessageBox.Show("Mengimplementasikan method Read")
End Sub

Public Property Status() As String Implements
IStorable.Status
    Get
        Return _status
    End Get
    Set(ByVal value As String)
        _status = value
    End Set
End Property

Public Sub Write(ByVal obj As Object) Implements
IStorable.Write
    MessageBox.Show("Mengimplementasikan method Write")
End Sub

Public Sub LogOriginalSize() Implements
IStorableCompressible.LogOriginalSize
    MessageBox.Show("Mengimplementasikan method
        LogOriginalSize")
End Sub
End Class

```

Dapat dilihat pada contoh diatas bahwa sebuah class dapat mengimplementasikan banyak interface.

```

Dim objDoc As New Document("New Document")
objDoc.Compress()
objDoc.Decompress()
objDoc.Decrypt()
objDoc.Encrypt()
objDoc.LogOriginalSize()
objDoc.LogSavedBytes()
objDoc.Read()

```

```
Dim objA As New Object  
objDoc.Write(objA)  
MessageBox.Show(objDoc.Status)
```

BAB 7

Collection

Pada bab ini kita akan membahas salah satu object yang banyak digunakan pada .NET yaitu Collection. Collection mirip dengan array namun lebih fleksible. Array mempunyai kapasitas yang terbatas, anda harus menentukan kapasitas dari array terlebih dahulu sebelum menggunakannya. Anda juga harus mendefinisikan tipe data dari nilai yang akan dimasukan kedalam array. Collection dapat berisi tipe apapun karena akan dikonversi menjadi tipe object. Collection juga memiliki kapasitas yang dinamis jadi anda tidak perlu mendefinisikan ukurannya terlebih dahulu seperti pada Array.

Pada bab ini akan dibahas bermacam-macam object Collection yang ada pada .NET yaitu:

- ArrayList
- List(Of T)
- Queue(Of T)
- Stack(Of T)
- Dictionary(Of K,T)

ArrayList

Tidak seperti array yang punya keterbatasan ukuran, dengan menggunakan ArrayList anda dapat membuat array yang dinamis. ArrayList terdapat pada namespace **System.Collection**, cara penggunaannya:

```
Dim arrNama As New ArrayList
'menambahkan object kedalam ArrayList
arrNama.Add("erick")
arrNama.Add("rully")
arrNama.Add("naren")
```

```
arrNama.Add("ronald")

MessageBox.Show(arrNama(3).ToString)

'menampilkan semua elemen dalam ArrayList
For Each nama In arrNama
    MessageBox.Show(nama.ToString)
Next
```

Dapat anda lihat pada kode diatas bahwa ArrayList tidak membatasi tipe data apa saja yang dapat dimasukan. Jadi ketika anda memasukan data kedalam ArrayList maka tipenya akan dikonversi menjadi object (dikenal dengan istilah boxing), dan ketika anda mengambil data dari ArrayList maka terjadi proses konversi dari object menjadi tipe data yang diinginkan. Misal pada kode diatas ketika for each dijalankan maka akan terjadi proses konversi dari object menjadi string (dikenal dengan istilah unboxing).

Boxing dan unboxing dapat memperlambat performa program yang anda buat. Misal ketika anda memasukan nilai integer kedalam ArrayList maka nilai akan dikonversi menjadi object (boxing), pada saat ini terjadi proses perubahan jenis tipe data dari value type ke reference type. Kemudian ketika kita menampilkan nilai yang ada pada ArrayList akan terjadi perubahan data dari reference type ke value type (unboxing) yaitu dari tipe object ke integer.

Selain proses boxing dan unboxing penggunaan ArrayList juga dapat menyebabkan kesalahan yang fatal dan tidak terdeteksi pada saat design time. Contohnya:

```
Dim mhs As New Mahasiswa

Dim arrNama As New ArrayList
arrNama.Add("erick")
arrNama.Add("rully")
'kesalahan tapi tidak terdeteksi
arrNama.Add(mhs)
arrNama.Add("naren")

For Each nama As String In arrNama
    MessageBox.Show(nama.ToString)
```

Next

Pada kode diatas penambahan elemen bertipe object mhs sebenarnya tidak boleh karena elemen yang lain bertipe string (collection biasanya menyimpan elemen yang tipenya sejenis) tapi karena elemen dari ArrayList semua dikonversi ke object kesalahan tersebut tidak terdeteksi pada saat design time. Namun ketika elemen ditampilkan akan terjadi error karena object mhs gagal dikonversi menjadi string.

Untuk menghindari kesalahan dan masalah boxing dan unboxing maka disarankan menggunakan Collection yang mendukung Strongly Typed Collection.

Generic Collection

Fitur Generic mulai ada di .NET versi 2.0 Dengan menggunakan Generic anda dapat membuat Typed Safe Collection untuk menghindari kesalahan seperti pada contoh ArrayList sebelumnya. Typed Safe Collection juga tidak harus melewati proses boxing dan unboxing seperti pada ArrayList. Pada .NET yang termasuk Strongly Typed Collection adalah List(Of T), Stack(Of T), Queue(Of T), dan Dictionary(Of K,T).

List(Of T)

List(Of T) merupakan object based collection yang diturunkan dari class System.Collection.Generic dan termasuk Typed Safe Collection. Fungsi dari List(Of T) sama dengan ArrayList hanya saja tipe data dari elemen yang disimpan sudah jelas (bukan tipe object seperti pada ArrayList).

```
Dim lstNama As New List(Of String)
lstNama.Add("erick")
lstNama.Add("rully")
lstNama.Add("naren")

MessageBox.Show(lstNama(1))

For Each nama As String In lstNama
    MessageBox.Show(nama.ToString)
```

Next

Dengan menggunakan strongly typed collection List(Of T) maka anda dapat menentukan tipe data yang akan dimasukan kedalam collection. Jika tipe data yang dimasukan tidak tepat maka Visual Studio akan menampilkan kesalahan pada saat design time (jika option strict off).

```
Dim lstNama As New List(Of String)
lstNama.Add("erick")
lstNama.Add("rully")
'akan muncul kesalahan karena tipe yang diinputkan bukan
'string tapi integer
lstNama.Add(12)
```

List(Of T) juga memiliki method-method yang dapat digunakan untuk memanipulasi elemen yang ada dalam collection. Misalnya anda ingin menghapus collection, mengurutkan, mencari, dan masih banyak lagi.

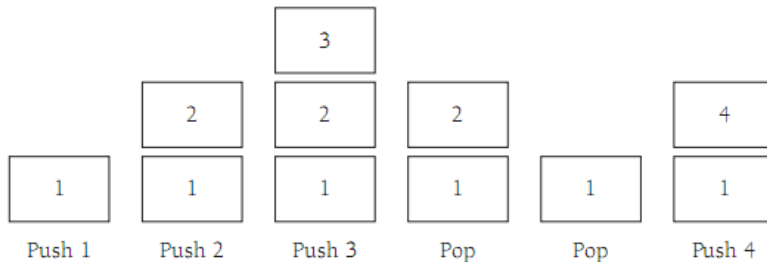
```
'menghapus elemen dengan alamat 2
lstNama.RemoveAt(2)
```

```
'mengurutkan elemen
lstNama.Sort()
```

```
'menghapus semua elemen
lstNama.Clear()
```

Stack(Of T)

Stack adalah Collection yang merepresentasikan struktur data tumpukan secara LIFO (Last In First Out). Stack(Of T) juga diturunkan dari namespace System.Collection.Generic. Stack(Of T) memiliki ukuran kapasitas yang dinamis dan termasuk dalam type safe Collection. Stack(Of T) mempunyai dua method utama yang dapat digunakan yaitu push dan pop. Method push untuk menambahkan elemen kedalam Stack(Of T), sedangkan method pop untuk mengambil elemen dari Stack(Of T).



Gambar 7.1 Representasi Tumpukan

```
Dim stackBil As New Stack(Of Integer)
```

```
stackBil.Push(12)
```

```
stackBil.Push(7)
```

```
stackBil.Push(9)
```

```
stackBil.Push(33)
```

```
'mengambil tumpukan yang paling atas
```

```
Dim bil As Integer = stackBil.Pop
```

```
MessageBox.Show("Bilangan teratas :" & bil)
```

```
'untuk menampilkan semua elemen dalam tumpukan
```

```
For Each b As Integer In stackBil
```

```
    MessageBox.Show(b.ToString)
```

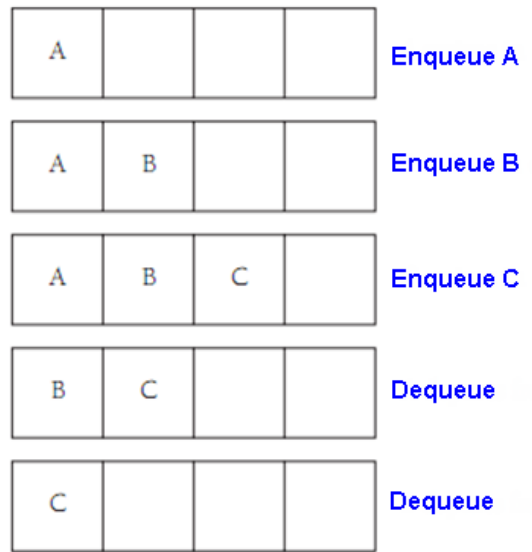
```
Next
```

Method push() digunakan untuk menambahkan elemen ke stack, dan method pop() digunakan untuk mengambil elemen dari stack. Karena stack merepresentasikan struktur data tumpukan (LIFO) maka ketika method pop() dijalankan yang diambil adalah elemen teratas dari stack. Jika anda menampilkan semua elemen menggunakan for each maka data yang ditampilkan juga akan terurut mulai dari elemen yang paling atas.

Queue(Of T)

Queue adalah struktur data yang merepresentasikan struktur data antrian secara FIFO (Fist In First Out). Queue(Of T) juga diturunkan dari namespace System.Collection.Generic. Queue(Of T) memiliki ukuran yang dinamis dan termasuk

dalam type safe Collection. Queue(Of T) mempunyai dua method utama yaitu enqueue() untuk memasukan elemen kedalam queue dan dequeue() untuk mengambil elemen dari dalam queue.



Gambar 7.2 Representasi Antrian

```
Dim queBil As New Queue(Of Integer)
queBil.Enqueue(12)
queBil.Enqueue(22)
queBil.Enqueue(55)
queBil.Enqueue(11)
```

```
Dim bil As Integer = queBil.Dequeue
```

```
For Each b As Integer In queBil
    MessageBox.Show(b.ToString)
Next
```

Method enqueue() digunakan untuk menambahkan elemen kedalam queue, sedangkan method dequeue() digunakan untuk mengambil elemen dari queue. Karena Queue merepresentasikan antrian (FIFO) maka ketika method dequeue() dijalankan yang diambil adalah antrian dengan

urutan yang pertama. Jika anda menampilkan semua elemen pada queue menggunakan for each maka urutan akan ditampilkan dari elemen yang paling awal.

Dictionary(Of T)

Dictionary merupakan object based Collection yang terdapat dalam namespace System.Collection.Generic. Dictionary adalah collection yang memiliki key dan value (mirip dengan hash table). Dictionary(Of T) juga termasuk dalam safe type collection. Dengan menggunakan Dictionary anda dapat membuat collection yang memiliki index bertipe selain integer (mis string, double).

```
Dim dictBio As New Dictionary(Of String, String)
dictBio("name") = "Erick Kurniawan"
dictBio("jobs") = "Lecturer"
dictBio("city") = "Jogjakarta"
```

```
MessageBox.Show(dictBio("jobs"))
```

```
For Each kvp As KeyValuePair(Of String, String) In dictBio
    MessageBox.Show(kvp.Key & " : " & kvp.Value)
Next
```

Dengan menggunakan dictionary anda dapat membuat collection yang index atau kuncinya bertipe string, jadi anda dapat mengambil nilai dari elemen tertentu menggunakan kunci, misal dictBio("name").

Untuk mengambil keseluruhan kunci beserta elemen dalam dictionary kita dapat menggunakan object bertipe KeyValuePair untuk menampung datanya. KeyValuePair terdiri dari dua properti yaitu key yang merepresentasikan kunci dan properti value yang merepresentasikan nilai.

BAB 8

Penanganan Kesalahan

Pada saat anda mengembangkan aplikasi, anda pasti pernah menjumpai kesalahan baik yang terjadi pada saat anda menuliskan kode, atau kesalahan yang disebabkan oleh inputan dari user, karena itu perlu bagi anda untuk dapat menangani kesalahan-kesalahan yang mungkin terjadi pada aplikasi anda. VB9 memiliki struktur exception handling untuk menangani error, ini menggantikan Error Goto statement yang ada pada VB6. Dengan menggunakan exception handling anda dapat memfilter error secara spesifik dan mengembalikan error menggunakan exception object. Anda hanya perlu menaruh kode yang akan dicek kesalahannya di kalang try dan meletakkan exception di kalang catch. Fitur baru pada VB9 untuk penanganan kesalahan diantaranya:

- Structure Exception Handling
- Error Highlighting
- Type Safety
- Debugging Tools yang baru

Memahami Error

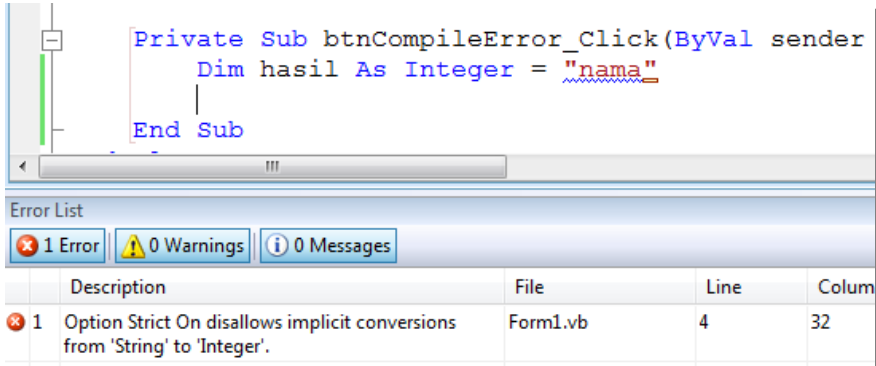
Pada aplikasi yang anda buat kesalahan bisa ditemukan dimana saja. Macam-macam error yang mungkin terjadi ketika anda menulis program adalah:

Sintaks Error

Kesalahan ini adalah yang paling mudah untuk dideteksi, karena terjadi pada saat design time (saat anda menuliskan sintaks program di editor). VB9 editor di Visual Studio menyediakan pengecekan error yang sangat user friendly karena ketika anda salah mengetikan sintaks maka Visual Studio langsung memberi tanda, atau membetulkannya secara otomatis.

Compile Time Error

Compile Error dapat terjadi jika anda mengabaikan sintaks error yang muncul pada saat anda menulis kode di Visual Studio atau ketika anda melakukan kesalahan tertentu seperti mengoperasikan string untuk perhitungan matematika. Ini hanya akan terdeteksi saat anda melakukan kompilasi. Semua error yang ditemukan akan dicatat di Output Window dan Error List.

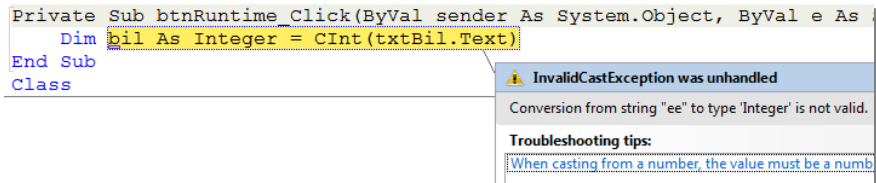


Gambar 8.1 Compile Time Error

Runtime Error

Runtime Error akan terjadi pada saat program dijalankan. Ketika error terjadi maka akan ada peringatan error kepada user dan kemudian program akan dihentikan secara otomatis. Misalnya ada kode untuk membuka file, namun file tersebut tidak ada, atau seharusnya input adalah tipe integer tapi anda

memasukan tipe string. Anda dapat menangani jenis error ini menggunakan exception handling.



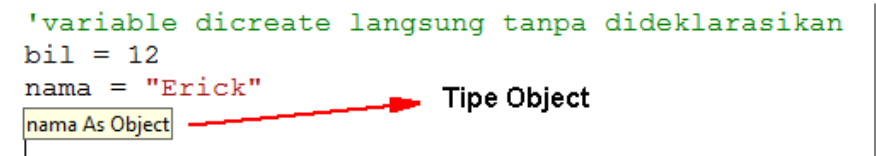
Gambar 8.2 Runtime Error

Logic Error

Logic Error terjadi dikarenakan kesalahan logika pemrograman, ini adalah jenis error yang paling sulit untuk dideteksi. Untuk mendeteksi Logic Error anda dapat melakukan test atau debugging pada kode anda.

Option Explicit, Option Strict, Option Infer

Jika Option Explicit 'Off' maka kompiler VB akan mengijinkan anda untuk tidak mendeklarasikan variable terlebih dahulu sebelum menggunakannya. Walaupun nampaknya membantu tapi dapat menimbulkan kebingungan karena variable tidak dideklarasikan sebelumnya. Kelemahan yang lain adalah tipe variable yang dicreate bertipe object, sehingga terjadi proses boxing dan unboxing ketika nilai value type dimasukan. Maka disarankan untuk mengatur Option Explicit menjadi 'On'. Option Explicit 'On' adalah pengaturan default dari Visual Studio.



Gambar 8.3 Penggunaan Option Explicit Off

Pengaturan Option Strict 'Off' berarti kompiler VB akan melakukan konversi narrowing secara otomatis. Narrowing

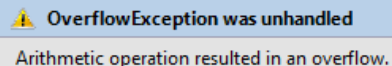
konversi adalah konversi dari tipe data yang lebih besar ke tipe data yang lebih kecil. Misal dari tipe long ke tipe integer. Untuk konversi dari tipe data yang lebih kecil ke tipe data yang lebih besar disebut konversi widening (otomatis dilakukan oleh compiler walau Option Strict 'On').

```
'Option Strict Off
'otomatis narrowing conversion
Dim dblBil As Double = 12.54
Dim intBil As Integer = dblBil
MessageBox.Show(intBil)

'Option Strict On
'narrowing conversion dilakukan eksplisit oleh programmer
Dim dblBil As Double = 14.45
Dim intBil As Integer = CInt(dblBil)
MessageBox.Show(intBil.ToString)
```

Penggunaan Option Strict 'Off' tidak disarankan karena dapat menimbulkan error pada saat runtime. Misal anda memberikan nilai yang lebih besar dari tipe datanya.

```
Dim intBil As Integer = 50000
Dim shBil As Short = intBil
MessageBox.Show(shBil)
```



Gambar 8.4 Opton Strict 'Off'

Kode diatas jika dijalankan akan error karena variable shBil yang bertipe Short hanya mampu menampung nilai maksimal 32000, jika nilai yang dimasukan lebih besar maka akan terjadi error OverFlowException. Jika anda menggunakan Option Strict 'Off' maka kesalah ini tidak akan terdeteksi pada saat design time.

Option Strict 'Off' juga mengijinkan anda untuk melakukan *late binding*. Late binding adalah sebuah cara untuk menentukan jenis tipe data dari variable diakhir setelah variable tersebut diberi nilai. Karena belum tahu tipe datanya maka variable tersebut bertipe Object.

'parameter pada function belum ditentukan tipe datanya

```

Function GetLuas(ByVal n1 As Object, ByVal n2 As Object) As Integer
    Return n1 + n2
End Function

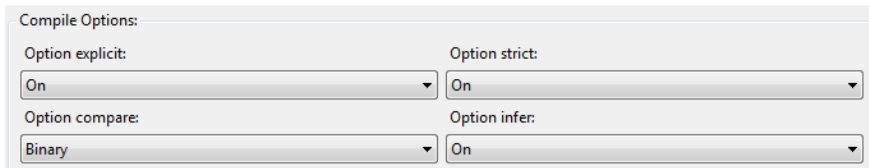
'menggunakan late binding
Dim luas As Integer = GetLuas(12, 34)
MessageBox.Show(luas)

```

Function GetLuas() memiliki parameter yang bertipe object yang dapat diisi tipe data yang diinginkan.

Karena beberapa masalah yang dapat ditimbulkan maka anda disarankan menggunakan Option Strict 'On', namun pada kasus tertentu jika anda membutuhkan *late binding* anda dapat mengubah setting Option Strict menjadi 'Off'.

Cara untuk mengubah Option Strict ada 2 macam, cara yang pertama klik kanan pada project ► pilih Properties ► pilih tab compile, disana anda dapat mengatur Option Strict menjadi 'On' atau 'Off' untuk pengaturan default Option Strict pada program yang anda buat.



Gambar 8.5 Pengaturan Option

Cara yang kedua anda dapat menuliskannya secara explicit di dalam program anda dibagian paling atas (berlaku tiap untuk tiap file .vb).

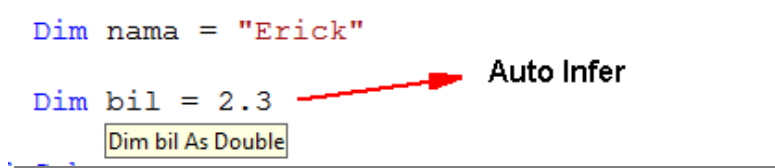
```

Option Strict Off
Public Class Form1
    .....
End Class

```

Satu lagi pengaturan yang digunakan adalah Option Infer, nilai Option Infer defaultnya adalah 'On'. Option Infer ada sejak VB9 (.NET 3.5). pada VB9 anda dapat mendeklarasikan variable tanpa menyebut tipe datanya namun harus diberi nilai default, maka kompiler VB akan dapat secara otomatis

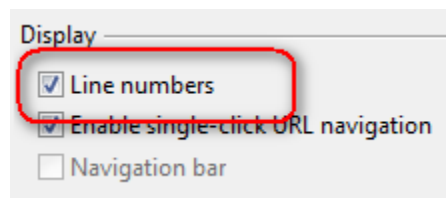
menginfer tipe datanya. Fitur ini sebenarnya disiapkan untuk mendukung LINQ agar dapat digunakan dengan lebih mudah. Pengaturan Option Infer disarankan untuk 'On'.



Gambar 8.6 Auto Infer

Line Number

Secara default Visual Studi tidak menampilkan line number pada tampilan kode editor. Namun anda dapat menampilkannya dengan cara klik menu tools ► Options ► pilih Text Editor ► Basic ► beri tanda check pada Line Number. Menampilkan line number penting karena setiap error yang terdeteksi akan ditunjukan dengan line number.



Gambar 8.7 Mengaktifkan Line Number

Visual Studio Debugging Tools

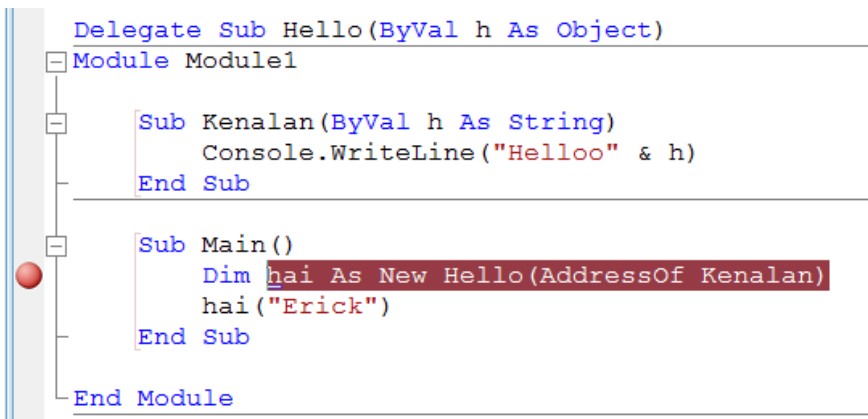
Visual Studio menyediakan fasilitas untuk melakukan debugging. Fasilitas yang disediakan antara lain:

- Breakpoint : digunakan untuk menelusuri alur dari program anda secara step by step.
- Watch Window : digunakan untuk memeriksa nilai yang ditampung dalam variable pada saat program dijalankan (runtime).

Melihat Jalannya Program

Anda dapat menggunakan Visual Studio untuk melihat jalannya program yang sedang anda buat, caranya:

5. Untuk melihat jalannya program yang anda buat, anda harus meletakkan tanda breakpoint pada salah satu baris pada program anda. Untuk menambahkan breakpoint caranya klik pada bagian kiri kode anda atau tekan F9.
6. Pada saat program di jalankan maka program akan berhenti sementara pada baris yang anda tandai dengan breakpoint. Baris tersebut secara otomatis akan di highlight dengan warna kuning.



Gambar 8.7 Menambahkan Breakpoint

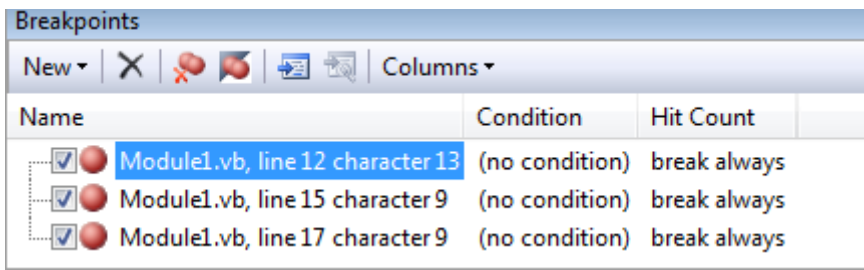
7. Kemudian anda dapat melakukan pelacakan pada program anda baris per baris dengan menekan tombol F11. Pada saat anda melakukan step into (F11) maka anda dapat mengetahui isi dari setiap variable per baris ketika program sedang berjalan.

Perintah-perintah pada Breakmode

Pada saat program dalam keadaan pause, anda dapat menjalankan perintah sebagai berikut:

- Toggle Breakpoint : untuk memberi tanda pada bagian program yang akan diberi breakpoint.
- Step Into (F11) : perintah ini digunakan untuk mengecek baris per baris dari program.
- Step Over (F10) : perintah ini mirip dengan Step Into yang digunakan untuk mengecek program perbaris bedanya Step Over tidak mengecek kode yang berada pada kalang subroutine dan function.

Anda dapat melihat baris kode yang diberi breakpoint dengan cara menggunakan breakpoint window. Klik pada menu Debug ► Windows ► Breakpoints



Gambar 8.8 Breakpoint Window

Exception Handling

Ketika anda membuat program tidak semua kesalahan dapat anda tangani karena mungkin kesalahan tersebut bukan dikarenakan anda salah menuliskan kode namun dari inputan user, kondisi jaringan, atau faktor lain diluar kode anda. Misal program anda mencoba membaca file sedangkan file tersebut sudah dihapus maka akan menyebabkan program error.

Pada VB9 anda dapat menggunakan exception handling untuk mengecek apakah kode blok yang anda buat menimbulkan kesalahan. Misalnya anda mempunyai program sebagai berikut:

```

Class Mahasiswa
    Public nim As String
End Class

```

```

Module Module1
    Sub CobaKesalahan()
        Dim objMhs As Mahasiswa = Nothing
        objMhs.nim = 12
    End Sub

    Sub Main()
        'jika dijalankan akan terjadi error
        CobaKesalahan()
    End Sub
End Module

```

Program diatas jika dijalankan akan menyebabkan terjadinya error `System.NullReferenceException` karena object `objMhs` belum dibuat tapi sudah diberi nilai. Error tersebut akan membuat aplikasi anda terhenti.

Anda dapat menggunakan struktur `Try Catch` untuk menangkap error yang terjadi dan memberi solusi terhadap error tersebut. Program yang akan anda periksa dapat dimasukan kedalam blok `Try`, sedangkan jika terjadi kesalahan dapat ditangkap atau dideteksi di kalang `Catch`.

```

Try
    CobaKesalahan()
Catch ex As Exception
    Console.WriteLine("Kesalahan : " & ex.Message)
End Try

```

Jika method `CobaKesalahan()` dijalankan dan menyebabkan error maka akan dicek errornya di kalang `Catch` untuk ditampilkan atau ditangani lebih lanjut.

Anda juga dapat menggunakan blok `Finally` yang adapat dituliskan setelah blok `Catch`. `Finally` adalah blok yang pasti akan dijalankan meskipun tidak terjadi error. Kode yang dimasukan kedalam blok `Finally` biasanya digunakan untuk menutup koneksi ke resource seperti database atau text.

```

Dim baris As String
Dim myFile As System.IO.StreamReader = Nothing
Try
    myFile =System.IO.File.OpenText("tidakada.txt")

```

```

        baris = myFile.ReadLine()
Catch ex As Exception
    Console.WriteLine("Error :" & ex.Message)
Finally
    'jika filestream terbuka akan ditutup disini
    If Not myFile Is Nothing Then
        myFile.Close()
    End If
End Try

```

Pada contoh diatas jika object myFile ada isinya maka ketika sampai pada kalang Finally akan ditutup koneksinya dengan resource file. Jika file tidak ada maka akan ditampilkan errornya pada kalang Catch.

Menangkap Lebih Dari Satu Kesalahan

Class Exception adalah base class dari semua Exception yang ada pada .NET. Jenis error yang mungkin terjadi ketika anda menulis program amat sangat banyak, pada .NET setiap error yang terjadi mempunyai nomor error yang berbeda dan anda dapat mengetahuinya dengan jelas, misal pada saat anda membuat aplikasi yang terhubung ke database, kemungkinan ditemukan banyak error mulai dari kegagalan koneksi, primary key yang terduplikasi, dll. Untuk menangani kesalahan yang terjadi secara spesifik anda dapat menggunakan lebih dari satu kalang Catch.

```

Dim strConn = "Data Source=Erick-PC;Integrated
Security=SSPI;Initial Catalog=Northwind"
Dim conn As New SqlConnection(strConn)
Dim sql = "insert into Customers(CustomerId)
        values('ALFKI')"
Dim cmd As New SqlCommand(sql, conn)
Try
    conn.Open()
    cmd.ExecuteNonQuery()
Catch sqlEx As SqlException
    If sqlEx.Number = 515 Then
        Console.WriteLine("Kolom CustomerName tidak boleh
        NULL..")
    End If
End Try

```

```
ElseIf sqlEx.Number = 2627 Then
    Console.WriteLine("Kode Customer sudah ada...")
Else
    Console.WriteLine("Error : " & sqlEx.Message & "
        Number :" & sqlEx.Number)
End If
Catch ex As Exception
    Console.WriteLine("Error : " & ex.Message)
End Try
```

Pada kode diatas anda dapat menggunakan beberapa kalang Catch untuk menangkap error yang lebih spesifik. Misal untuk menagkap error yang terjadi karena koneksi atau kesalahan perintah query anda dapat menggunakan `SqlException`. Dengan menggunakan `SqlException` anda dapat mengambil error number dari kesalahan yang mungkin terjadi menggunakan properti `Number`. Misal dari program diatas untuk kesalahan duplikasi primary key error numbertnya 2627 sedangkan untuk kesalahan kolom tidak boleh bernilai NULL error numbertnya 515. Dengan mengetahui kesalahan secara spesifik anda dapat memberi error handler satu persatu untuk kesalahan tersebut.

Mengapa Tidak Menggunakan On Error Goto

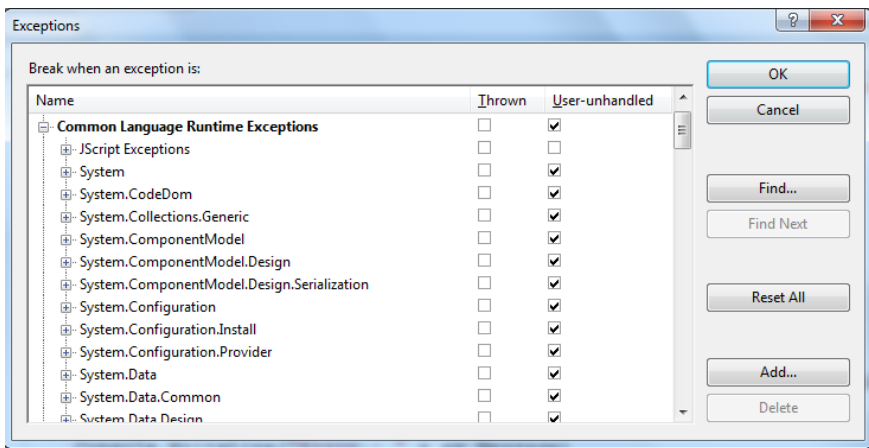
Sebenarnya VB9 masih mensupport penggunaan perintah `On Error Goto` untuk backward compatibility dengan VB6, namun ada beberapa alasan yang menyebabkan penggunaan `On Error Goto` tidak disarankan yaitu:

- **Spaghetti Code** : jika anda ingin menghandle banyak error pada VB6 maka anda harus menuliskan banyak statement `On Error Goto` yang akan tercampur dengan logic program anda yang lain. Ini akan sangat menyulitkan dalam pembacaan program.
- **Single Error Object** : VB6 hanya memiliki sebuah object untuk menghandle error yaitu object `Err` jadi sulit sekali untuk menangkap beberapa error secara spesifik untuk ditangani.

- Batasan Bahasa : object Exception ada didalam .NET Framework sehingga didukung semua bahasa yang messuport .NET. Anda dapat throw exception menggunakan VB9 dan catch exception menggunakan C#.
- Keterbatasan Informasi : Errobject pada VB6 tidak mempunyai cukup informasi tentang error yang terjadi.

Type Exception Pada VB9

Jika anda ingin tahu jenis exception apa saja yang mungkin terjadi ketika menulis kode anda dapat menampilkan Exception Window, caranya klik pada menu Debug ► pilih Exceptions, maka akan tampil window berikut.



Gambar 8.9 Error Window

Throw Exception

Jika anda membuat class maka anda harus melakukan throw exception jika terjadi kesalahan pada kode di class tersebut. Untuk itu gunakan keyword Throw. Perhatikan kode dibawah ini:

```

Public Class Mahasiswa
    Private _nim As String
    Public Property Nim() As String
        Get
            Return _nim
        End Get
        Set(ByVal value As String)
            If value.Length <> 8 Then
                Throw New Exception("Panjang Nim tidak
                    boleh lebih dari 8 karakter")
            Else
                _nim = value
            End If
        End Set
    End Property
End Class

```

Pada class Mahasiswa akan dilakukan pengecekan apakah panjang dari nilai yang dimasukkan kedalam fields Nim 8 karakter, jika lebih atau kurang dari 8 karakter maka class tersebut akan melempar kesalahan. Anda dapat menangkap kesalahan yang dilempar oleh class Mahasiswa dengan cara:

```

Dim objMhs As New Mahasiswa
Try
    objMhs.Nim = "220023211"
Catch ex As Exception
    MessageBox.Show("Error : " & ex.Message)
End Try

```

UnhandledException Event

Jika aplikasi anda gagal menangkap exception yang terjadi maka aplikasi anda akan berakhir. Seperti yang sudah kita bahas diatas anda dapat menggunakan structure exception handling dan melakukan debugging pada kode anda. Meskipun anda sudah sangat teliti namun kemungkinan error yang tidak diduga masih dapat terjadi. Untuk pertahanan terakhir terhadap error anda dapat memanfaatkan event UnhandledException.

Untuk menambahkan event `UnhandledException` double click folder `MyProject` pada solution explorer ► pilih `Application` tab ► klik pada tombol `View Application Event` pada project properties window ► kemudian pada drop-down list pilih event `UnhandledException`. Maka Visual Studio akan menggenerate method event handler, masukan kode untuk handle kesalahan pada method tersebut.

```
Private Sub MyApplication_UnhandledException(ByVal sender
As Object, ByVal e As
Microsoft.VisualBasic.ApplicationServices.UnhandledExceptio
nEventArgs) Handles Me.UnhandledException
    MessageBox.Show("Kesalahan ini tidak dapat dihandle
        oleh aplikasi anda", e.Exception.GetType.Name)
    If TypeOf e.Exception Is ApplicationException Then
        e.ExitApplication = False
    End If
End Sub
```

Untuk mencoba program diatas jalankan .exe dari program tersebut (bukan dari visual studio). Jika terjadi error yang tidak dapat ditangani oleh kode yang anda buat dalam aplikasi anda maka akan muncul pesan diatas dan program akan diakhiri.

BAB 9

ADO .NET

ADO.NET (ActiveX Data Object .NET) merupakan kumpulan object-object yang digunakan untuk akses data dan manipulasi data. Sejak pertama kali di rilis pada tahun 2002, ADO.NET ini telah mengalami perubahan versi dari mulai versi 1.0 sampai yang terakhir sekarang pada saat buku ini ditulis telah mencapai versi 3.5. Tiap versi memiliki fitur barunya masing-masing yang ditujukan untuk lebih meningkatkan performansi dan dukungan terhadap database provider yang ada.

Programmer VB telah mengenal ADO sebelum ADO.NET dirilis. Pada saat itu ADO cukup banyak digunakan seiring dengan popularitas VB 6.0 yang pada saat itu semakin populer dikarenakan kemudahannya dalam membuat aplikasi yang berbasis bisnis khususnya. Sehingga disebutlah VB 6.0 sebagai RAD (Rapid Application Development) Tool.

Lalu apa yang kurang dari ADO sehingga Microsoft memutuskan untuk membuat komponen data akses yang sama sekali berbeda dan bukan sekedar upgrade versi? Mari kita lihat beberapa kekurangan yang dimiliki oleh ADO

- ADO menggunakan object Recordset untuk akses data yang notabene ditujukan untuk “connected data access” artinya untuk akses data maka object tersebut harus selalu terkoneksi ke database, meskipun sebenarnya Recordset juga bisa digunakan untuk “disconnected data access”.
- Recordset tidak mendukung penyimpanan / akses data untuk tabel lebih dari satu. Bila kita ingin

melakukan hal tersebut maka query nya harus dimodifikasi dengan menggunakan perintah Join.

- ADO tidak cocok digunakan untuk transfer data dalam jumlah record yang besar, terutama pertukaran data antar “distributed application”. Hal itu disebabkan karena ADO menggunakan teknik “COM Marshalling”. Teknik tersebut tidak efisien, karena terjadi proses konversi tipe data antar aplikasi. Selain itu juga terdapat masalah security firewall.
- Dukungan terhadap XML terbatas.

Mungkin itu hanya beberapa kekurangan ADO apabila dibandingkan dengan ADO.Net yang penulis rasakan. Semua kekurangan diatas dijawab oleh Microsoft dengan adanya ADO.Net. Sekarang saatnya anda berpaling ke ADO.Net.

Arsitektur

Secara umum ADO.Net itu terdiri dari dua komponen utama, yaitu :

1. ADO.Net Data Provider
2. DataSet

ADO.Net Data Provider merupakan komponen yang terdiri dari object-object yang spesifik terhadap masing-masing database provider dan sifatnya Connected, sehingga sering disebut dengan istilah “Connected Data Access”.

Komponen ADO.Net Data Provider terdiri dari object-object:

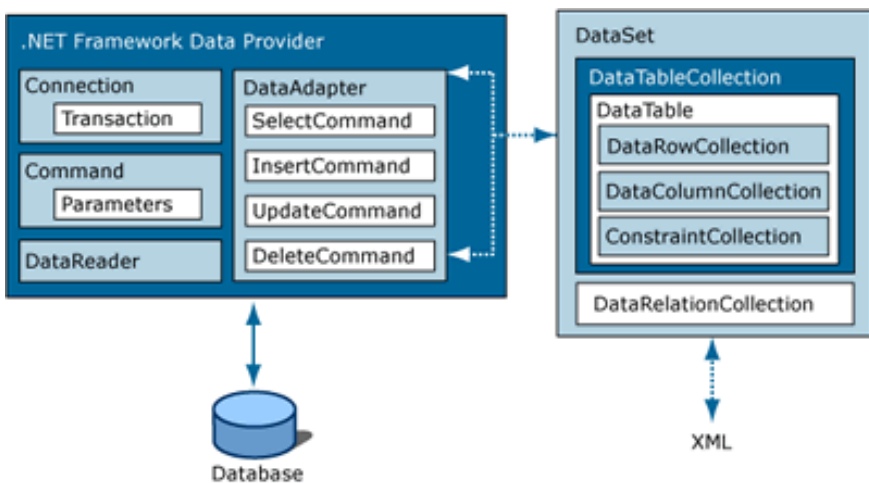
- Connection
- Command

- DataReader
- DataAdapter

Sedangkan DataSet terdiri dari object-object sebagai berikut :

- DataSet
- DataTable(s)
- DataRelation(s)

Bagan dari arsitektur ADO.Net dapat dilihat dari gambar 1.1



Gambar 9.1 Arsitektur ADO.NET

Penjelasan untuk object-object ADO.NET Data Provider :

- Connection : Object yang digunakan untuk melakukan koneksi ke sumber data.
- Command : Object yang digunakan untuk melakukan eksekusi query SQL Statement.

- **DataReader** : Object yang digunakan untuk membaca data.
- **DataAdapter** : Object yang menjembatani antara sumber data dan DataSet dalam pengambilan dan manipulasi data.

Keterangan lebih detail mengenai semua object diatas akan diterangkan pada bab berikutnya.

Berikut penjelasan untuk object-object yang terdapat di dalam DataSet :

- **DataSet** : Object yang digunakan untuk menyimpan data dalam mode disconnected. DataSet ini sebagai kontainer untuk object DataTable dan DataRelation yang disimpan didalam memory. DataSet dapat memiliki lebih dari satu DataTable.
- **DataTable** : Object ini hampir sama fungsinya seperti DataSet. Untuk membuat DataTable tidak harus selalu membuat object DataSet terlebih dahulu, dalam arti lain bahwa DataTable itu dapat berdiri sendiri. Object ini merupakan kumpulan dari DataRow, DataColumn dan Constraint yang merupakan representasi dari row / record, kolom, dan konstrain yang terdapat pada tabel di database.
- **DataRelation** : Object ini merupakan representasi relationship antar tabel yang terdapat di database.

Komponen ADO.NET Data Provider terdapat di dalam masing-masing namespace yang bersesuaian dengan database provider. Misalnya jika database yang digunakan SQL Server versi 7.0 ke atas maka namespace yang digunakan yaitu System.Data.SqlClient, sedangkan jika databasenya SQL

Server versi 7.0 ke bawah atau MS.Access maka namespace yang digunakan yaitu System.Data.OleDb. Begitu juga jika aplikasi yang digunakan masih menggunakan teknologi ODBC maka namespace yang digunakan yaitu System.Data.Odbc.

ADO.NET DataSet menggunakan namespace yang berbeda dengan ADO.NET Data Provider, karena komponen ini tidak tergantung dari jenis database provider. .Net framework menyediakannya didalam namespace System.Data.

Pada ADO.NET versi 2.0 disediakan data akses komponen dengan model factory. Komponen ini digunakan jika database yang digunakan belum ditentukan terlebih dahulu pada saat aplikasi sedang dibangun, atau jika anda ingin membuat aplikasi dimana database yang digunakan dapat dirubah secara fleksibel. Namespace yang digunakan yaitu System.Data.Common. Semua namespace yang berhubungan dengan Data dan spesifik terhadap database provider tertentu diturunkan dari class yang terdapat didalam namespace tersebut.

Teknologi Data Akses

Pada awalnya tiap-tiap data provider membuat sendiri Application Programming Interface (API) untuk akses data terhadap database system yang mereka buat. Hal ini menyulitkan para programmer pembuat aplikasi. Setiap kali database yang digunakan itu berubah, atau beralih ke database provider yang lain maka pada saat itu juga programmer harus mempelajari API baru agar dapat menggunakan database system yang akan dipakai. Oleh karena itu Microsoft membuat beberapa database driver (database API) untuk membuat antar muka yang seragam, sehingga tidak menyulitkan lagi para pembuat aplikasi dalam mempelajari API yang baru.

ODBC

Open Database Connectivity (ODBC) merupakan API pertama yang dibuat oleh Microsoft sebagai langkah awal untuk menyeragamkan antarmuka semua database provider. Sehingga semua database provider yang ingin sistem databasenya dapat digunakan harus membuat driver tersendiri yang mendukung teknologi ODBC. Hal ini memudahkan pengguna, karena mereka tidak harus lagi langsung berbicara dengan sistem database, melainkan menggunakan driver ODBC yang telah “membungkus” semua fungsionalitas sistem database yang dipakai apabila dibandingkan dengan harus berinteraksi langsung dengan databasenya.

Pada saat ini teknologi ODBC hanya digunakan untuk database-database lama, misalnya Access, SQL Server ataupun Oracle dengan versi yang terdahulu. ODBC ini memiliki beberapa kelemahan diantaranya yaitu tidak adanya dukungan untuk mengakses data yang siftanya hirarki, jadi teknologi ini hanya mendukung relational database.

OLE DB

Object Linking And Embedded Database (OLE DB) merupakan teknologi data akses terbaru setelah ODBC. Teknologi ini menghapus semua kekurangan-kekurangan yang dimiliki oleh ODBC, diantaranya ialah masalah akses data terhadap database yang sifatnya hirarki dan juga tingkat performansi akses data lebih ditingkatkan lagi.

Teknologi ini digunakan untuk database versi terbaru pada saat itu, dan sampai sekarang pun masih digunakan. Dengan teknologi ini kita dapat membuat aplikasi yang independent terhadap database yang digunakan.

SQL Native Client

Teknologi ini merupakan teknologi data akses terbaru seiring dengan dikeluarkannya SQL Server 2005 dan hanya

digunakan untuk akses data terhadap SQL Server 2005. ODBC dan OLE DB merupakan komponen yang termasuk ke dalam MDAC (Microsoft Data Access Component) yang juga merupakan komponen yang termasuk kedalam operating system windows.

SQL Native Client (SQLNCLI.dll) ini merupakan library yang berdiri sendiri dan merupakan penggabungan antara ODBC dan OLE DB. Library ini dapat dicari pada installer SQL Server 2005 dengan nama file sqlncli.msi. Jika anda melakukan instalasi SQL Server 2005 maka secara default file tersebut terinstal.

Data akses ini mengekspose fitur-fitur baru yang terdapat di dalam SQL Server 2005 seperti MARS (Multiple Active Result Set), tipe data xml di SQL Server 2005, User Defined Data Type (UDT), Query Notification, dsb.

Aplikasi tidak langsung berinteraksi dengan ketiga teknologi data akses tersebut, melainkan melalui sekumpulan library yang kita kenal dengan ADO.NET untuk versi data akses terbaru. Sebelumnya kita mengenal beberapa library yang terdapat didalam DAO (Data Access Object), RDO (Remote Data Object), ADO (ActiveX Data Object).

LINQ (Language Integrated Query) merupakan teknik data query terbaru seiring dengan dikeluarkannya .Net Framework 3.5. Teknik ini merupakan alternatif lain selain ADO.NET.

ADO.NET Data Provider

Komponen-komponen yang termasuk ke dalam ADO.NET Data Provider adalah : Connection, Command, DataReader, dan DataAdapter.

Connection Object

Object ini merupakan object yang penting, karena untuk menggunakan object ADO.NET Data Provider lainnya kita

harus mengetahui terlebih dahulu penggunaan dari object Connection.

Connection digunakan untuk melakukan koneksi terhadap database. Metoda yang paling sering digunakan yaitu Open dan Close. Object ini terdapat pada masing-masing namespace yang spesifik terhadap database provider. Sebelum menggunakan object connection kita harus mengetahui terlebih dahulu connection string. Connection string ini berisi informasi mengenai database yang dituju, nama server database, jenis autentikasi, informasi user name beserta password.

Connection String

Berikut beberapa atribut umum connection string yang sering digunakan :

- Database atau Initial Catalog : Diisi dengan nama database yang digunakan.
- Server atau Data Source : Diisi dengan nama server atau instance database.
- Integrated Security : Diisi dengan SSPI. Atribut ini digunakan jika jenis autentikasi yang digunakan untuk melakukan koneksi terhadap databasenya yaitu windows.
- Trusted Connection : Diisi dengan True atau False. Atribut ini fungsinya sama dengan Integrated Security. Jadi anda bisa memilih salah satunya.
- User ID atau UID : Diisi dengan nama user name yang terdaftar di database. Opsi ini digunakan jika jenis autentikasi yang digunakan untuk melakukan koneksi terhadap databasenya yaitu SQL Server Login (jika database yang digunakan SQL Server).

- Password atau PWD : Diisi dengan password dari User ID yang digunakan.

Contoh deklarasi object Connection dengan connection string untuk database SQL Server :

```

'====sql server authentication====
Private strConn1 As String = _
"database=northwind;server=localhost;uid=sa;pwd=sql2005"

Private strConn2 As String = _
"initial catalog=northwind;data
source=localhost;uid=sa;pwd=sql2005"

'====windows authentication===
Private strConn3 As String = _
"database=northwind;server=localhost\sql2008;integrated
security=true"

Private strConn4 As String = _
"database=northwind;server=localhost\sql2008;integrated
security=sspi"

```

Pada kode diatas dapat kita lihat beberapa alternatif penulisan connection string untuk SQL Authentication dan Windows Authentication. Salah satu parameter yang harus diperhatikan yaitu properti server atau data source. Kedua properti tersebut diisi dengan nama instance dari database yang digunakan.

SqlConnection Object

SqlConnection merupakan object yang digunakan untuk terhubung ke database SQL Server versi 7.0 dan versi diatasnya. Object ini terdapat didalam namespace System.Data.SqlClient, sehingga untuk menggunakannya kita harus Imports terlebih dahulu namespace tersebut.

Berikut adalah contoh program penggunaan SqlConnection object untuk membuka dan menutup koneksi ke database SQL Server.

1. Buat sebuah Windows Forms Application project template, berikan nama ConnectionObject pada project tersebut.
2. Tambahkan sebuah control button ke dalam form.
3. Ubah nama button tersebut menjadi btnOpenConnection.
4. Tambahkan prosedur event handler untuk event Click button dengan melakukan double klik pada button tersebut.
5. Tambahkan setting Option dan namespace berikut ini di baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

6. Ketikkan kode berikut :

```
Private sqlConn1 As New SqlConnection(strConn1)

Private Sub btnOpenConnection_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
    Handles btnOpenConnection.Click

    Try
        If sqlConn1.State <> ConnectionState.Open Then
            sqlConn1.Open()
            MessageBox.Show("Connection Open!")
        End If
    Catch argEx As ArgumentException
        MessageBox.Show(argEx.ToString)
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    Finally
        If sqlConn1 IsNot Nothing Then
            sqlConn1.Dispose()
```

```
        MessageBox.Show("Connection State : " & _  
                        & sqlConn1.State.ToString)  
    End If  
End Try  
  
End Sub
```

Dari kode diatas dapat kita lihat bahwa SqlConnection object membutuhkan connection string yang telah didefinisikan sebelumnya sebagai informasi tentang database yang dituju.

Method State digunakan untuk mengecek state dari koneksi yang akan digunakan. Method ini berguna agar apabila koneksinya sudah terbuka maka kita ga perlu untuk membukanya kembali.

Method Open digunakan untuk membuka koneksi ke database. Sedangkan untuk menutup koneksi terdapat dua alternatif pilihan, yaitu Close atau Dispose.

Penggunaan SqlConnection object diatas ditempatkan didalam sebuah structure exception handling yang menggunakan perintah Finally. Kenapa? Perlu diketahui bahwa proses membuka koneksi itu membutuhkan resource yang cukup besar, dari mulai pengecekan koneksi jaringan, handshaking, pengecekan instance database dan juga security nya.

Oleh karena itu SQL Server memiliki fitur Connection Pooling. Fitur ini digunakan untuk menyimpan koneksi yang sudah digunakan sebelumnya, sehingga apabila kita membutuhkannya lagi suatu saat maka tidak perlu dibuat dari awal untuk pembentukan koneksi tersebut.

Dengan adanya perintah finally maka koneksi selalu dipastikan ditutup setelah dibuka. Hal ini untuk mengurangi jumlah koneksi yang disimpan di pool dimana jumlah maksimumnya secara default yaitu 100. Apabila koneksi dibiarkan terbuka terus maka suatu saat akan mencapai batas

maksimum default jumlah koneksi yang disimpan di pool, sehingga apabila terdapat lagi koneksi lain yang akan terhubung ke database yang sama maka koneksi tersebut akan menunggu sampai jumlah koneksi di pool nya berkurang sampai melampaui batas time out connection.

Penggunaan exception bertingkat digunakan untuk mencegah kemungkinan-kemungkinan terjadinya error yang dapat disebabkan oleh beberapa hal, seperti kesalahan pengetikkan properti string koneksi, kesalahan autentikasi, security, dll.

Sebagai best practice untuk penggunaan connection object, buka koneksi sesaat sebelum digunakan dan tutup koneksi segera setelah digunakan.

ConnectionStringBuilder

Pada contoh diatas sebelumnya object connection menggunakan connection string yang ditulis secara hard-coded. Hal ini dapat menyebabkan kesalahan pengetikkan properti connection string.

Mulai di .NET 2.0 disediakan ConnectionStringBuilder object. Diharapkan dengan adanya object ini maka akan terhindar dari kesalahan penulisan properti connectionstring nya. Berikut adalah contoh penggunaan object tersebut :

1. Tambahkan satu button kedalam form yang sama pada lab sebelumnya diatas.
2. Ubah nama button tersebut menjadi btnConnStrBuilder.
3. Ketikkan kode berikut pada prosedur event handler untuk event Click button yang bersangkutan :

```
Private Sub btnConnStrBuilder_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) _  
Handles btnConnStrBuilder.Click
```

```
Dim connBuilder As New SqlConnectionStringBuilder
```

```
connBuilder.DataSource = "localhost"
connBuilder.InitialCatalog = "northwind"
connBuilder.UserID = "sa"
connBuilder.Password = "sql2005"
'connBuilder.IntegratedSecurity = True

Dim sqlConn As SqlConnection = Nothing

Try
    sqlConn = New
SqlConnection(connBuilder.ToString)
    sqlConn.Open()
    MessageBox.Show("Connection Open!")
Catch sqlEx As SqlException
    MessageBox.Show(sqlEx.ToString)
Catch ex As Exception
    MessageBox.Show(ex.ToString)
Finally
    If sqlConn IsNot Nothing Then
        sqlConn.Dispose()
        MessageBox.Show("Connection State : " & _
            & sqlConn.State.ToString)
    End If
End Try

End Sub
```

Kode diatas yang diberikan komentar digunakan apabila jenis autentikasi yang ingin digunakan yaitu Windows, sehingga properti UserID dan Password harus dihilangkan.

Perintah Using

Seperti yang telah dijelaskan sebelumnya, untuk menutup koneksi yang sudah dibuka dapat digunakan salah satu dari method yang disediakan, yaitu Close atau Dispose. Sehingga kita harus memanfaatkan Exception Handling untuk memastikan bahwa objek koneksi yang digunakan selalu ditutup setelah selesai digunakan.

Mulai di .NET 2.0 diberikan perintah baru yaitu Using. Perintah ini digunakan untuk melakukan auto disposing sebuah object yang implemen IDisposable interface. Sehingga setelah object tersebut out of scope maka otomatis perintah Dispose akan dipanggil secara implisit untuk object tersebut. Berikut contoh penggunaan Using didalam pembuatan objek koneksi :

1. Tambahkan satu button kedalam form yang sama pada lab sebelumnya diatas.
2. Ubah nama button tersebut menjadi btnUsing.
3. Ketikkan kode berikut pada prosedur event handler untuk event Click button yang bersangkutan :

```
Private Sub btnUsing_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnUsing.Click
```

```
    Using sqlConn As New SqlConnection(strConn1)  
        sqlConn.Open()  
        MessageBox.Show("Connection Open!")
```

```
    End Using
```

```
End Sub
```

Setelah variabel sqlConn tersebut keluar dari blok Using maka otomatis akan di dispose.

My.Settings Namespace

Pada contoh-contoh sebelumnya, connection string yang digunakan masih diketik secara hard-coded, sehingga nantinya apabila terdapat perubahan maka kode programnya harus dibuka, diubah kemudian di kompilasi ulang kemudian di deploy kembali. Sungguh sebuah pekerjaan yang tidak seharusnya dilakukan.

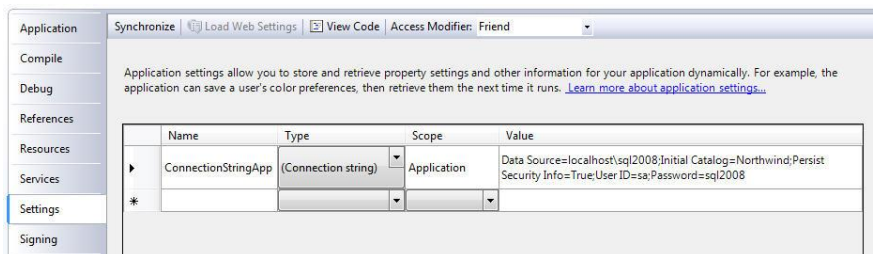
Mulai di Visual Basic 2005 telah disediakan My namespace. Namespace ini berisi shortcut-shortcut ke beberapa namespace-namespaces yang terdapat di dalam .NET framework. Salah satu class yang terdapat didalam namespace tersebut yaitu My.Settings.

Class tersebut digunakan untuk mengakses dan memanipulasi data yang disimpan di dalam file configuration (file exe.config) dan dibentuk dengan format XML.

Dengan menyimpan connection string di dalam file configuration maka apabila terjadi perubahan maka kita tinggal merubah file tersebut tanpa harus membuka kembali kode programnya.

Berikut contoh program penggunaan file configuration untuk menyimpan connection string dan diakses lewat My.Settings namespace :

1. Double klik My Project node didalam Solution Explorer pada project yang telah dibuat sebelumnya atau klik kanan project nya dan pilih menu Properties.
2. Pada bagian settings tambahkan variabel untuk menyimpan connection string seperti yang terlihat pada gambar dibawah ini :



Gambar 9.2 Project Settings

3. Tambahkan satu button kedalam form yang sama pada lab sebelumnya diatas.
4. Ubah nama button tersebut menjadi btnMyNamespace.

5. Ketikkan kode berikut pada prosedur event handler untuk event Click button yang bersangkutan :

```
Private Sub btnMyNamespace_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
Handles btnMyNamespace.Click

    Using sqlConn As New
SqlConnection(My.Settings.ConnectionStringApp)
        sqlConn.Open()
        MessageBox.Show("Connection Open!")

    End Using

End Sub
```

Encrypt Connection String

Connection string menyimpan informasi yang sifatnya penting dari segi keamanan, informasi tersebut diantaranya adalah : nama user, password, instance server dan database. Apabila informasi tersebut dapat dibaca langsung oleh user yang tidak berkepentingan maka hal tersebut dapat membahayakan data yang disimpan di database.

Untuk menghindari hal-hal yang tidak diinginkan, .NET framework sudah menyediakan library yang digunakan untuk memproteksi dengan melakukan enkripsi connection string yang digunakan.

Terdapat dua provider library yang disediakan untuk enkripsi connection string, yaitu : `RsaProtectedConfigurationProvider` dan `DataProtectedConfigurationProvider`. Informasi kedua provider tersebut dapat dilihat di file `machine.config` yang terdapat di lokasi `C:\Windows\Microsoft.NET\Framework\v2.0.50727\CONFIG`.

```

<configProtectedData defaultProvider="RsaProtectedConfigurationProvider">
  <providers>
    <add name="RsaProtectedConfigurationProvider"
        type="System.Configuration.RsaProtectedConfigurationPr
        description="uses RsaCryptoServiceProvider to
        cspProviderName="" useMachineContainer="true"
    <add name="DataProtectionConfigurationProvider"
        type="System.Configuration.DpapiProtectedConfiguratio
        description="Uses CryptProtectData and CryptUnProtect
        keyEntropy="" />
  </providers>
</configProtectedData>

```

Gambar 9.3 Provider Data Enkripsi di file machine.config

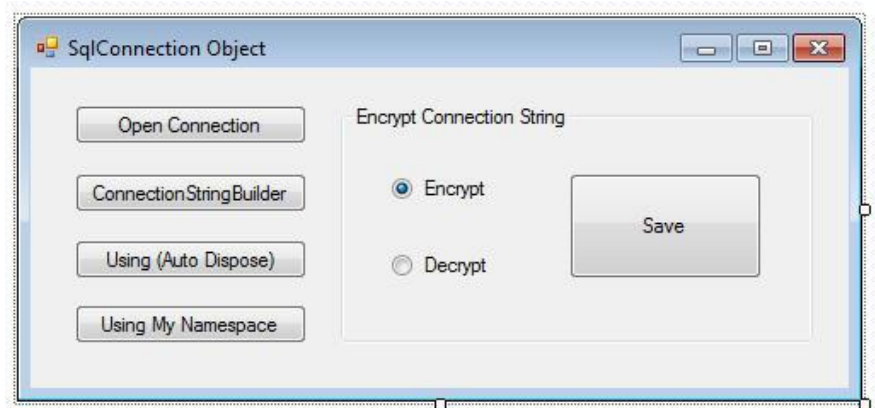
Default provider yang digunakan yaitu RsaProtectedConfigurationProvider. Perbedaan diantara kedua provider tersebut yaitu data hasil enkripsi dengan menggunakan Rsa dapat digunakan di komputer lain apabila aplikasi tersebut diinstal pada komputer yang berbeda. Hal tersebut dimungkinkan karena key enkripsi juga disimpan didalam data connection string yang di enkripsi. Namun tidak demikian dengan DataProtectedConfigurationProvider. Data hasil enkripsi untuk connection string tidak dapat digunakan langsung di komputer yang berbeda, karena data enkripsi yang dihasilkan tersebut bergantung pada CPU yang digunakan. Sehingga datanya harus di dekripsi terlebih dahulu, setelah itu baru dienkripsi.

Programmer tidak membutuhkan lagi proses untuk melakukan dekripsi connection string yang telah di enkripsi. Proses deksripsi dilakukan secara otomatis ketika data tersebut digunakan.

Berikut ini contoh program untuk melakukan enkripsi connection string dengan kedua provider diatas :

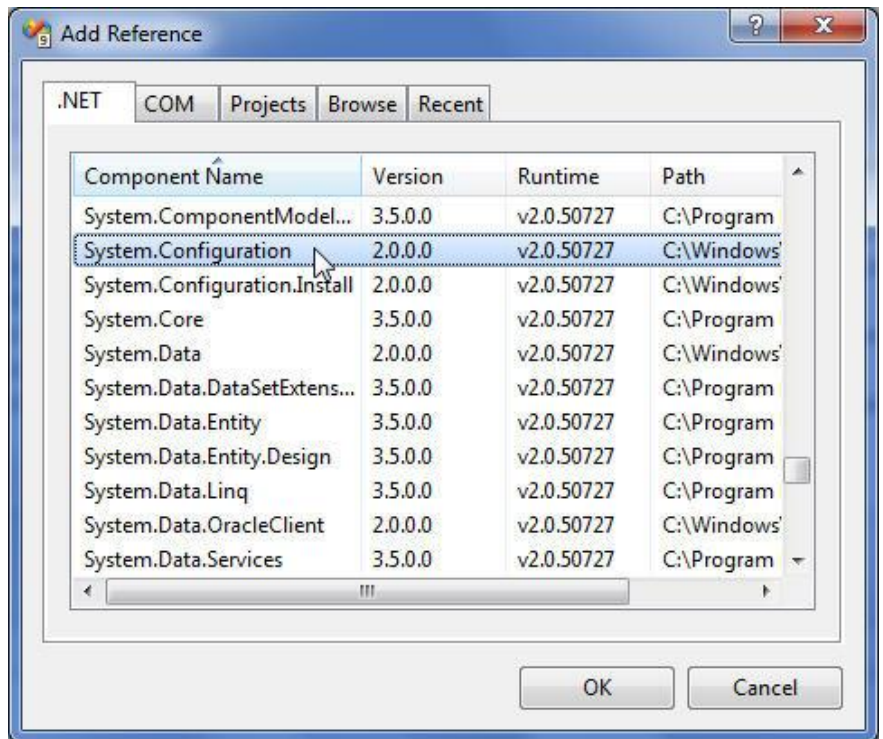
1. Tambahkan sebuah GroupBox, dua RadioButton dan satu Button.
2. Ubah text radio button pertama menjadi Encrypt dan name nya diubah menjadi rbEncrypt.
3. Ubah text radio button kedua menjadi Decrypt dan name nya diubah menjadi rbDecrypt.
4. Ubah name button menjadi btnEncryptProcess

5. Design form dapat dilihat seperti dibawah ini (gabungan dari lab pertama sampai yang sekarang) :



Gambar 9.4 Design Form

6. Tambahkan referensi ke library System.Configuration.dll seperti pada gambar dibawah ini :



Gambar 9.5 Referensi ke System.Configuration.dll

7. Imports namespace berikut :

Imports System.Configuration

8. Buat prosedur dibawah ini :

Private Sub EncryptConnection(**ByVal** Encrypt **As Boolean**)

Try

Dim configFile = _

ConfigurationManager.OpenExeConfiguration(ConfigurationUser
Level.None)

```

        Dim connectionElement =
configFile.GetSection("connectionStrings")

        If Encrypt Then

'connectionElement.SectionInformation.ProtectSection( _
            "DataProtectionConfigurationProvider")

            'RsaProtectedConfigurationProvider

connectionElement.SectionInformation.ProtectSection(Nothing
)
            Else
connectionElement.SectionInformation.UnprotectSection()
            End If

            connectionElement.SectionInformation.ForceSave
= True
            configFile.Save(ConfigurationSaveMode.Modified)

            MessageBox.Show("Encrypt : " &
Encrypt.ToString)
        Catch ex As Exception
            MessageBox.Show(ex.ToString)
        End Try

    End Sub

```

Prosedur diatas digunakan untuk melakukan enkripsi data connection string yang disimpan di file .config. Apabila input parameter yang digunakan untuk method ProtectSection yaitu Nothing maka default provider yang digunakan RsaProtectedConfigurationProvider. Apabila provider yang ingin digunakan merupakan DataProtectedConfigurationProvider maka anda harus menggunakan nama provider tersebut untuk passing input parameter method ProtectSection seperti yang dapat dilihat pada bagian kode yang di berikan komentar.

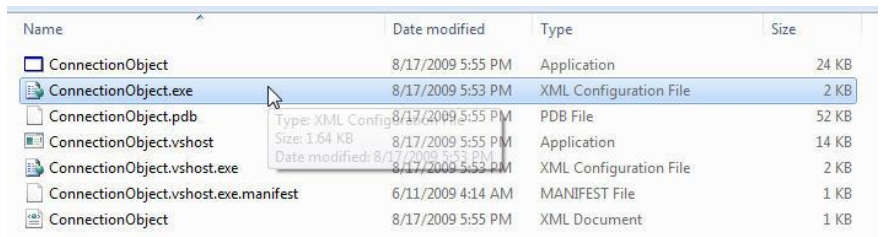
9. Tambahkan kode berikut pada prosedur event handler untuk event click button btnEncryptProcess :

```
Private Sub btnEncryptProcess_Click(ByVal sender As _  
System.Object, ByVal e As System.EventArgs) _  
Handles btnEncryptProcess.Click
```

```
EncryptConnection(rbEncrypt.Checked)
```

```
End Sub
```

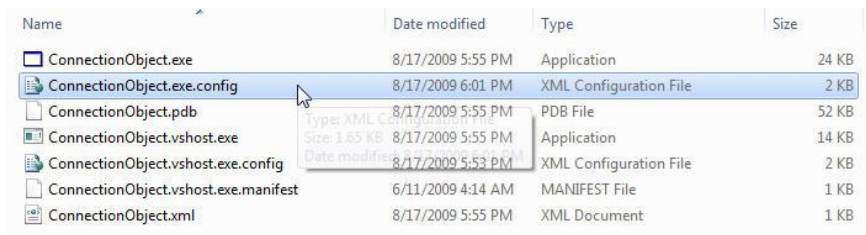
10. Jalankan program. Lakukan proses enkripsi dan dekripsi.
11. Buka file konfigurasi pada folder aplikasi :



Name	Date modified	Type	Size
ConnectionObject	8/17/2009 5:55 PM	Application	24 KB
ConnectionObject.exe	8/17/2009 5:53 PM	XML Configuration File	2 KB
ConnectionObject.pdb	8/17/2009 5:55 PM	PDB File	52 KB
ConnectionObject.vshost	8/17/2009 5:55 PM	Application	14 KB
ConnectionObject.vshost.exe	8/17/2009 5:53 PM	XML Configuration File	2 KB
ConnectionObject.vshost.exe.manifest	6/11/2009 4:14 AM	MANIFEST File	1 KB
ConnectionObject	8/17/2009 5:55 PM	XML Document	1 KB

Gambar 9.6 File Configuration

12. Apabila setting show extension file nya diaktifkan pada Folder Options maka file nya akan terlihat seperti dibawah ini :



Name	Date modified	Type	Size
ConnectionObject.exe	8/17/2009 5:55 PM	Application	24 KB
ConnectionObject.exe.config	8/17/2009 6:01 PM	XML Configuration File	2 KB
ConnectionObject.pdb	8/17/2009 5:55 PM	PDB File	52 KB
ConnectionObject.vshost.exe	8/17/2009 5:55 PM	Application	14 KB
ConnectionObject.vshost.exe.config	8/17/2009 5:53 PM	XML Configuration File	2 KB
ConnectionObject.vshost.exe.manifest	6/11/2009 4:14 AM	MANIFEST File	1 KB
ConnectionObject.xml	8/17/2009 5:55 PM	XML Document	1 KB

Gambar 9.7 File Configuration

13. Hasil enkripsi Connection String untuk Rsa provider :

```

<connectionStrings configProtectionProvider="RsaProtectedConfigurationProvider">
  <EncryptedData Type="http://www.w3.org/2001/04/xmlenc#Element"
    xmlns="http://www.w3.org/2001/04/xmlenc#">
    <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#tripledes-cbc" />
    <KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#">
      <EncryptedKey xmlns="http://www.w3.org/2001/04/xmlenc#">
        <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#rsa-1_5" />
        <KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#">
          <KeyName>Rsa Key</KeyName>
        </KeyInfo>
        <CipherData>
          <CipherValue>SvpFaDdiIugCuakv72PUFGq8gkg3jrr4kGSRM30fobvZnSRYtLwae10j:
        </CipherValue>
        </CipherData>
      </EncryptedKey>
    </KeyInfo>
    <CipherData>
      <CipherValue>RIIVxvFcW2jgfrUFdk4D9UTzEMRFwqatRqD1rILA3abtIOnV1x4rjvBmUqhNIsA1v
    </CipherValue>
    </CipherData>
  </EncryptedData>
</connectionStrings>

```

Gambar 9.8 Data hasil enkripsi Rsa Provider

14. Hasil enkripsi Connection String untuk Data provider :

```

<connectionStrings configProtectionProvider="DataProtectionConfigurationProvider">
  <EncryptedData>
    <CipherData>
      <CipherValue>AQAAANCMnd8BFdERjHoAwE/Cl+sBAAAA5mb3pgYv2uq1QcK+wj00YwQAAA
    </CipherData>
  </EncryptedData>
</connectionStrings>

```

Gambar 9.9 Data hasil enkripsi Data Provider

Command Object

Command object digunakan untuk melakukan eksekusi query SQL statement dari kode program. Object ini dapat digunakan untuk membaca dan memanipulasi data. Selain itu juga perintah Data Definition Language (DDL) dapat juga di eksekusi lewat object ini, misalnya perintah untuk Create database atau table.

Manipulasi Data

Command object sering digunakan untuk melakukan manipulasi data terhadap tabel-tabel yang terdapat didalam database. Perintah Data Manipulation Language (DML) yang dapat digunakan yaitu : Update, Delete dan Insert.

Berikut ini contoh program untuk melakukan manipulasi data dengan menggunakan perintah Update. Perintah DML lainnya pada prinsipnya sama saja, hanya merubah string

untuk properti CommandText dari object Command yang digunakan :

1. Buat sebuah Windows Forms Application project template baru, beri nama CommandObject untuk project tersebut.
2. Tambahkan sebuah Button pada form, ubah nilai properti name nya menjadi btnDataManipulation.
3. Buat connection string di Settings Project terhadap database Northwind seperti yang sudah dilakukan pada lab pembahasan Connection Object.
4. Tambahkan perintah dibawah ini pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

5. Tambahkan kode berikut pada event Click button diatas :

```
Private Sub btnDataManipulation_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnDataManipulation.Click
```

```
    Using sqlConn As New
SqlConnection(My.Settings.ConnectionStringApp)
        Using sqlCmd As New SqlCommand
            sqlCmd.Connection = sqlConn
            sqlCmd.CommandType = CommandType.Text
            sqlCmd.CommandText = _
                String.Format("Update Customers Set
City='Bandung',Country = 'Indonesia'"
& " Where CustomerID = '{0}'", "ALFKI")

            Try
                sqlConn.Open()
                Dim intRec = sqlCmd.ExecuteNonQuery

                MessageBox.Show("Updated!...Records
Affected : " & intRec.ToString)
```

```
        Catch sqlEx As SqlException
            MessageBox.Show(sqlEx.ToString)
        Catch ex As Exception
            MessageBox.Show(ex.ToString)
    End Try
End Using

End Using
```

```
End Sub
```

Beberapa properti yang harus di atur nilainya yaitu Connection yang diisi dengan variabel Connection object, CommandType yang diisi sesuai dengan jenis SQL statement yang digunakan apakah itu AdHoc query (Text) atau Stored Procedure, dan CommandText yang diisi dengan perintah SQL atau nama Stored Procedure yang digunakan.

Method Command yang digunakan untuk melakukan eksekusi AdHoc query atau stored procedure yaitu ExecuteNonQuery. Prosedur tersebut merupakan sebuah function yang mengembalikan nilai integer yang merepresentasikan jumlah record yang terlibat sebagai hasil dari eksekusi query tersebut.

SqlParameter

Apabila query yang digunakan memiliki input parameter maka object command tersebut membutuhkan SqlParameter apabila database yang digunakannya yaitu SQL Server 7 dan di atasnya.

Penggunaan SqlParameter ini dapat digunakan bukan hanya untuk passing input parameter pada query yang disimpan di dalam Stored Procedure melainkan juga dapat digunakan untuk query yang sifatnya Ad-hoc. Selain itu penggunaan SqlParameter ini dapat menghindari terjadinya Sql Injection yang dapat merusak data.

Berikut adalah contoh program penggunaan SqlParameter pada Ad-hoc query :

1. Tambahkan sebuah button pada form yang telah dibuat sebelumnya.
2. Ubah name properti button tersebut menjadi btnSqlParameter.
3. Tambahkan kode berikut pada event click button tersebut :

```
Private Sub btnSqlParameter_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnSqlParameter.Click

    Using sqlConn As New
SqlConnection(My.Settings.ConnectionStringApp)
        Using sqlCommand As New SqlCommand
            sqlCommand.Connection = sqlConn
            sqlCommand.CommandType = CommandType.Text
            sqlCommand.CommandText = _
                "Update Customers Set
City='Bandung',Country = 'Indonesia' Where CustomerID =
@id"

            Dim idParam = New SqlParameter("@id",
SqlDbType.NChar)
            idParam.Value = "Alfki"

            sqlCommand.Parameters.Add(idParam)

            'cara lain untuk passing SqlParameter yaitu
            'sqlCmd.Parameters.AddWithValue("@id",
            "Alfki")

            Try
                sqlConn.Open()
                Dim intRec = sqlCommand.ExecuteNonQuery

                MessageBox.Show("Updated!...Records
Affected : " & intRec.ToString)
            Catch sqlEx As SqlException
                MessageBox.Show(sqlEx.ToString)
            Catch ex As Exception
```

```
        MessageBox.Show(ex.ToString)
    End Try
End Using

End Using
End Sub
```

Dari kode diatas dapat dilihat bahwa untuk menggunakan SqlParameter terdapat beberapa alternatif, yang pertama yaitu kita buat terlebih dahulu SqlParameter object nya kemudian ditambahkan pada Parameters collection Command object tersebut. Cara kedua yaitu dapat langsung memanfaatkan method AddWithValue.

Eksekusi Stored Procedure

Apabila kita menggunakan Stored Procedure untuk melakukan manipulasi atau membaca data maka properti Command Object yang harus dirubah yaitu properti CommandType yang diisi dengan nilai StoredProcedure.

Berikut adalah contoh penggunaan Stored Procedure yang dieksekusi lewat Command object :

1. Buat sebuah stored procedure baru di database Northwind seperti dibawah ini :

```
CREATE PROCEDURE AddCustomer
@id nchar(5),
@name nvarchar(40)
AS
Insert Into Customers (CustomerID,CompanyName) Values
(@id,@name)
```

2. Tambahkan sebuah button pada form yang telah dibuat sebelumnya.
3. Ubah name properti button menjadi btnStoredProc.
4. Tambahkan kode berikut pada event click button tersebut :

```

Private Sub btnStoredProc_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnStoredProc.Click

    Using sqlConn As New
SqlConnection(My.Settings.ConnectionStringApp)
        Using sqlCmd As New SqlCommand
            sqlCmd.Connection = sqlConn
            sqlCmd.CommandType =
CommandType.StoredProcedure
            sqlCmd.CommandText = "AddCustomer"

            sqlCmd.Parameters.AddWithValue("@id",
"Rully")
            sqlCmd.Parameters.AddWithValue("@name",
"Native-Enterprise")

            Try
                sqlConn.Open()
                sqlCmd.ExecuteNonQuery()

                MessageBox.Show("Save!")

            Catch sqlEx As SqlException
                MessageBox.Show(sqlEx.ToString)
            Catch ex As Exception
                MessageBox.Show(ex.ToString)
            End Try
        End Using
    End Using

End Sub

```

ExecuteNonQuery

Method ini digunakan untuk mengembalikan nilai berupa sebuah record yang hanya memiliki satu kolom dan satu baris. Fungsi ini biasanya digunakan untuk eksekusi Aggregate function seperti Sum, Min, Max, Count, dan Average.

Return value dari fungsi ini berupa object sehingga kita harus lakukan konversi ke tipe data yang diinginkan.

Berikut adalah contoh program dalam penggunaan fungsi tersebut dimana aggregate function yang digunakan yaitu Sum:

1. Tambahkan sebuah button pada form yang telah dibuat sebelumnya.
2. Ubah name properti button tersebut menjadi btnSum.
3. Tambahkan kode berikut pada event click button tersebut :

```
Private Sub btnSum_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btnSum.Click

    Using sqlConn As New
SqlConnection(My.Settings.ConnectionStringApp)
        Using sqlCmd As New SqlCommand
            sqlCmd.Connection = sqlConn
            sqlCmd.CommandType = CommandType.Text
            sqlCmd.CommandText = _
String.Format("Select SUM(OD.Quantity) From
Orders As O Inner Join [Order Details] AS OD" _
& " On O.OrderID = OD.OrderID Where O.CustomerID = '{0}'
And OD.ProductID = {1}", "Alfki", 28)
            Try
                sqlConn.Open()
                Dim intTotQty =
Convert.ToInt32(sqlCmd.ExecuteScalar)

                MessageBox.Show("Total Quantity : " &
intTotQty.ToString)
            Catch sqlEx As SqlException
                MessageBox.Show(sqlEx.ToString)
            Catch ex As Exception
                MessageBox.Show(ex.ToString)
            End Try
        End Using
    End Using
```

Hasil eksekusi fungsi `ExecuteNonQuery` diatas yaitu berupa penjumlahan total `Quantity ProductID 28` untuk customer Alfki.

Asynchronous Processing

Contoh-contoh kode di atas dilakukan secara `synchronous`, artinya apabila terdapat lebih dari satu query yang akan dijalankan secara simultan maka salah satu query harus selesai di proses terlebih dahulu untuk melakukan eksekusi query berikutnya.

Proses eksekusi query dimungkinkan secara `asynchronous`. Fitur ini memang sudah disediakan oleh `.NET` framework itu sendiri. Sehingga satu proses query tidak harus menunggu proses query sebelumnya selesai untuk melakukan eksekusi query berikutnya.

Method yang sifatnya `asynchronous` ditandai dengan awalan `Begin...` dan `End...` misalnya `BeginExecuteReader` dan `EndExecuteReader`.

Untuk menggunakan fitur `asynchronous` maka harus ditambahkan atribut `Asynchronous Processing = true` didalam `connection string` yang digunakan. Selain itu juga harus dibuat sebuah prosedur yang nantinya akan di eksekusi apabila prosesnya telah selesai. Method tersebut harus memiliki satu buah input parameter berupa `IAsyncResult`. Dimana input parameter tersebut nantinya akan digunakan sebagai object yang akan di convert ke tipe semula untuk mengakhiri proses yang sudah selesai dilaksanakan.

Untuk menampilkan hasil eksekusi `asynchronous` ke dalam control yang terdapat didalam form yang sama maka harus digunakan method `Invoke` form yang bersangkutan. Hal

ini diperlukan untuk mengembalikan proses thread yang terpisah ke dalam main thread windows.

Berikut di bawah ini ada contoh penggunaan Asynchronous Processing dalam melakukan proses pembacaan dua buah query yang dilakukan secara simultan.

1. Deklarasikan variabel delegate berikut pada form yang sama yang digunakan pada lab sebelumnya :

```
Private Delegate Sub GetReaderDelegate(ByVal TheDataReader  
As SqlDataReader)
```

2. Tambahkan dua control ListBox ke dalam form, ubah properti name nya menjadi lbCustomers dan lbOrders.
3. Buat dua buah prosedur yang digunakan untuk menampilkan hasil eksekusi query ke dalam kontrol ListBox :

```
Private Sub GetCustomers(ByVal CustomerReader As  
SqlDataReader)
```

```
    Dim dt As New DataTable("customers")  
    dt.Load(CustomerReader)
```

```
    lbCustomers.DataSource = dt  
    lbCustomers.DisplayMember = "CompanyName"  
    lbCustomers.ValueMember = "CustomerID"  
    lbCustomers.Refresh()
```

```
    CustomerReader.Close()
```

```
End Sub
```

```
Private Sub GetOrders(ByVal OrdersReader As  
SqlDataReader)
```

```
    While OrdersReader.Read
```

```
        lbOrders.Items.Add(OrdersReader("OrderID").ToString & "-"  
& OrdersReader("CustomerID").ToString)
```

```

        lbOrders.TopIndex = lbOrders.Items.Count - 1

    End While

    OrdersReader.Close()

End Sub

```

4. Buat dua buah prosedur yang nantinya prosedur ini akan dieksekusi apabila prosesnya telah selesai :

```

Private Sub ProcessingCustomers(ByVal ar As
IAsyncResult)
    Dim cmd As SqlCommand = Nothing
    Dim dr As SqlDataReader = Nothing

    Try
        cmd = DirectCast(ar.AsyncState, SqlCommand)
        dr = cmd.ExecuteReader(ar)

        Me.Invoke(New GetReaderDelegate(AddressOf
GetCustomers), New Object() {dr})
    Catch ex As Exception
        If cmd IsNot Nothing Then cmd.Dispose()
        If dr IsNot Nothing Then dr = Nothing

        MessageBox.Show("Error from ProcessingCustomers
Procedure : " & ex.ToString)
    End Try
End Sub

Private Sub ProcessingOrders(ByVal ar As IAsyncResult)
    Dim cmd As SqlCommand = Nothing
    Dim dr As SqlDataReader = Nothing

    Try
        cmd = DirectCast(ar.AsyncState, SqlCommand)
        dr = cmd.ExecuteReader(ar)

```

```

        Me.Invoke(New GetReaderDelegate(AddressOf
GetOrders), New Object() {dr})
    Catch ex As Exception
        If cmd IsNot Nothing Then cmd.Dispose()
        If dr IsNot Nothing Then dr = Nothing

        MessageBox.Show("Error from ProcessingOrders
Procedure : " & ex.ToString)
    End Try
End Sub

```

5. Buat sebuah prosedur yang akan melakukan eksekusi dua query sekaligus secara asynchronous :

```

Private Sub CallCustomersAndOrdersWithoutMARS()
    Dim connBuilder As New SqlConnectionStringBuilder
    connBuilder.InitialCatalog = "Northwind"
    connBuilder.DataSource = ".\sql2008"
    connBuilder.IntegratedSecurity = True
    connBuilder.AsynchronousProcessing = True

    Dim sqlConn1 As New
SqlConnection(connBuilder.ToString)

    Dim cmdCustomers = New SqlCommand
    cmdCustomers.Connection = sqlConn1
    cmdCustomers.CommandType = CommandType.Text
    cmdCustomers.CommandText = "Select * From
Customers"

    Dim sqlConn2 As New SqlConnection( _
    "initial catalog=northwind;data source=.\sql2008;"
    -
    & "integrated security=true;asynchronous
processing=true")

    Dim cmdOrders = New SqlCommand
    cmdOrders.Connection = sqlConn2

```

```

cmdOrders.CommandType = CommandType.Text
cmdOrders.CommandText = "Select * From Orders"

Try
    sqlConn1.Open()
    sqlConn2.Open()

    cmdCustomers.BeginExecuteReader( _
        AddressOf ProcessingCustomers,
cmdCustomers,
        CommandBehavior.CloseConnection)

    cmdOrders.BeginExecuteReader( _
        AddressOf ProcessingOrders, cmdOrders, _
        CommandBehavior.CloseConnection)

Catch ex As Exception
    If cmdCustomers IsNot Nothing Then
cmdCustomers.Dispose()
    If cmdOrders IsNot Nothing Then
cmdOrders.Dispose()

    If sqlConn1 IsNot Nothing Then
sqlConn1.Dispose()
    If sqlConn2 IsNot Nothing Then
sqlConn2.Dispose()

        Throw New Exception("Error From
CallCustomersAndOrders Procedure...", ex)
    End Try

End Sub

```

6. Tambahkan satu button ke dalam form, ubah properti name nya menjadi btnAsyncProc.
7. Ketikkan kode berikut pada event click button :

```

Private Sub btnAsyncProc_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnAsyncProc.Click

```

```

Try
    CallCustomersAndOrdersWithoutMARS()
Catch sqlEx As SqlException
    MessageBox.Show(sqlEx.ToString)
Catch ex As Exception
    MessageBox.Show(ex.ToString)
End Try
End Sub

```

Kode diatas dibuat dengan menggunakan dua buah Connection object untuk masing-masing Command object. Karena proses pembacaan data dengan DataReader tidak dapat dilakukan secara bersamaan sehingga masing-masing reader harus memiliki koneksi yang berbeda.

.NET 2.0 telah menyediakan fitur MARS (Multiple Active Result Sets). Fitur ini memungkinkan untuk membaca data dengan beberapa DataReader secara simultan yang hanya menggunakan satu koneksi saja.

Fitur MARS diaktifkan dengan menambahkan atribut MultipleActiveResultSets = true di connection string nya.

Berikut adalah modifikasi kode program sebelumnya yang menggunakan dua koneksi menjadi satu koneksi dengan memanfaatkan fitur MARS :

```

Private Sub CallCustomersAndOrdersUsingMARS()
    Dim sqlConn1 As New SqlConnection( _
        "initial catalog=northwind;data source=localhost;"
        & "integrated security=true;asynchronous
processing=true;MultipleActiveResultSets=true")

    Dim cmdCustomers = New SqlCommand
    cmdCustomers.Connection = sqlConn1
    cmdCustomers.CommandType = CommandType.Text
    cmdCustomers.CommandText = "Select * From
Customers"

    Dim cmdOrders = New SqlCommand

```

```

cmdOrders.Connection = sqlConn1
cmdOrders.CommandType = CommandType.Text
cmdOrders.CommandText = "Select * From Orders"

Try
    sqlConn1.Open()

    cmdCustomers.BeginExecuteReader( _
        AddressOf ProcessingCustomers,
cmdCustomers)

    cmdOrders.BeginExecuteReader( _
        AddressOf ProcessingOrders, cmdOrders)

Catch ex As Exception
    If cmdCustomers IsNot Nothing Then
cmdCustomers.Dispose()
    If cmdOrders IsNot Nothing Then
cmdOrders.Dispose()

    If sqlConn1 IsNot Nothing Then
sqlConn1.Dispose()

        Throw New Exception("Error From
CallCustomersAndOrders Procedure...", ex)
    End Try

End Sub

```

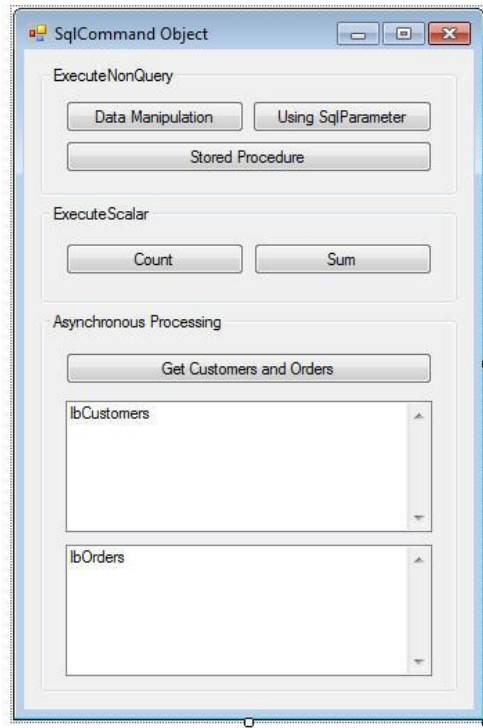
Kode pada langkah-7 dapat di ubah menjadi :

```

Private Sub btnAsyncProc_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnAsyncProc.Click
    Try
        CallCustomersAndOrdersUsingMARS()
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    End Try

```

Berikut adalah design form secara keseluruhan :



Gambar 9.10 Design Form

DataReader

DataReader merupakan object yang digunakan untuk membaca data. Object ini sifatnya read only dan forward only. Read Only dalam artian object ini tidak dapat digunakan untuk melakukan manipulasi data. Sedangkan Forward Only dalam artian bahwa selama dalam proses pembacaan record maka datareader tersebut tidak dapat melakukan navigasi record ke record sebelumnya yang sudah dibaca.

Berikut adalah contoh kode program penggunaan DataReader :

1. Tambahkan Windows Forms Application Project template baru, beri nama DataReaderObject.
2. Tambahkan satu Button, ubah properti name nya menjadi btnUsingReader.
3. Tambahkan ListBox ke dalam form yang sama, ubah properti name nya menjadi lbCustomers.
4. Buat satu buah variabel bertipe SqlConnection pada Settings Project Properties seperti yang sudah dilakukan sebelumnya. Beri nama SqlConnectionApp dan isi sesuai dengan database yang dituju yaitu Northwind beserta informasi lainnya seperti jenis autentikasi.
5. Tambahkan perintah berikut pada baris paling atas di class form tersebut :

Option Strict On

Imports System.Data.SqlClient

6. Ketikkan kode berikut pada event click button :

```
Private Sub btnUsingReader_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnUsingReader.Click
```

```
    lbCustomers.Items.Clear()
```

```
    Using sqlConn As New
SqlConnection(My.Settings.ConnectionStringApp)
        Using sqlCmd As New SqlCommand
            sqlCmd.Connection = sqlConn
            sqlCmd.CommandType = CommandType.Text
            sqlCmd.CommandText = "Select
CustomerID,CompanyName From Customers"
```

```

        Dim dr As SqlDataReader = Nothing

        Try
            sqlConn.Open()
            dr = sqlCommand.ExecuteReader

            While dr.Read
                Dim custID = dr(0).ToString
                Dim compName =
dr("CompanyName").ToString

                lbCustomers.Items.Add(custID & "-"
& compName)
            End While

            Catch sqlEx As SqlException
                MessageBox.Show(sqlEx.ToString)
            Catch ex As Exception
                MessageBox.Show(ex.ToString)
            Finally
                If dr IsNot Nothing Then dr.Close()
                If sqlConn IsNot Nothing Then
sqlConn.Dispose()
            End Try
        End Using
    End Using

End Sub

```

Untuk membaca data dari datareader dibutuhkan perintah looping While terhadap method Read dari datareader tersebut. Sedangkan untuk membaca data kolom digunakan indexer dengan memberikan index dari urutan kolom yang terdapat didalam query, index ini dimulai dari Nol. Selain itu juga dapat menggunakan nama kolom itu sendiri untuk membaca data kolom tersebut seperti yang dicontohkan pada kode diatas.

Berikut adalah contoh penggunaan fitur MARS yang telah saya bahas sebelumnya di lab terdahulu. Skenarionya adalah

proses pembacaan data customers dan orders nya sekaligus dengan menggunakan dua datareader dan satu koneksi :

1. Tambahkan satu button kedalam form yang sama, beri nama btnReaderMars.
2. Tambahkan ListBox, beri nama lbCustomerMARS.
3. Buat satu buah variabel bertipe ConnectionString pada Settings Project Properties seperti yang sudah dilakukan sebelumnya. Beri nama MARSConnection dan isi sesuai dengan database yang dituju yaitu Northwind beserta informasi lainnya seperti jenis autentikasi dan setting MARS (lihat lab sebelumnya).
4. Tambahkan kode berikut pada event click button :

```
Private Sub btnReaderMARS_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnReaderMARS.Click
    lbCustomerMARS.Items.Clear()

    Using sqlConn As New
SqlConnection(My.Settings.MARSConnection)
        Using cmdCustomers As New SqlCommand
            cmdCustomers.Connection = sqlConn
            cmdCustomers.CommandType = CommandType.Text
            cmdCustomers.CommandText = "Select
CustomerID From Customers"

            Using cmdOrders As New SqlCommand
                cmdOrders.Connection = sqlConn
                cmdOrders.CommandType =
CommandType.Text
                cmdOrders.CommandText = "Select * From
Orders Where CustomerID=@id"
                cmdOrders.Parameters.Add("@id",
SqlDbType.NChar, 5)

                Dim customerReader As SqlDataReader =
Nothing
```

```

Dim orderReader As SqlDataReader =
Nothing

Try
    sqlConn.Open()
    customerReader =
cmdCustomers.ExecuteReader

    While customerReader.Read
        Dim custID =
customerReader("CustomerID").ToString
lbCustomerMARS.Items.Add(custID)

cmdOrders.Parameters("@id").Value = custID

        orderReader =
cmdOrders.ExecuteReader
        While orderReader.Read
            Dim orderID =
orderReader("OrderID").ToString
            Dim orderDate =
Convert.ToDateTime(orderReader("OrderDate"))
            Dim lines = New
String("=", 20)

lbCustomerMARS.Items.Add(lines & orderID & ", " _
&
orderDate.ToString("MMM/dd/yyyy"))
lbCustomerMARS.TopIndex =
lbCustomerMARS.Items.Count - 1
        End While
        orderReader.Close()

    End While

Catch sqlEx As SqlException
    If orderReader IsNot Nothing Then
orderReader.Close()

```

```

        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        If orderReader IsNot Nothing Then
orderReader.Close()

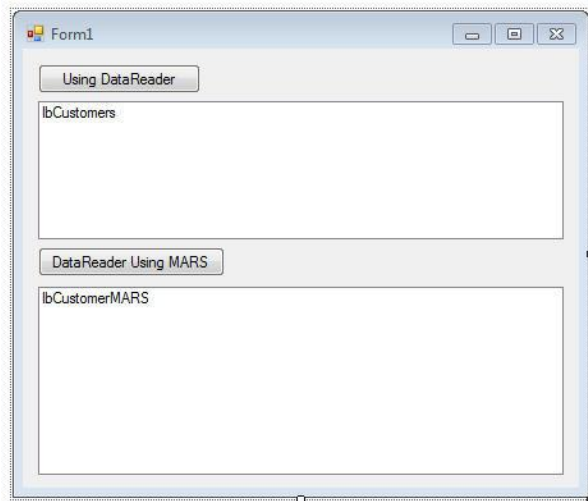
        MessageBox.Show(ex.ToString)
    Finally
        If customerReader IsNot Nothing
Then customerReader.Close()

        If sqlConn IsNot Nothing Then
sqlConn.Dispose()
    End Try
    End Using

    End Using
End Using

End Sub

```



Gambar 9.11 Design Form

Dengan menggunakan fitur MARS maka kita dapat bekerja dengan lebih dari satu datareader yang bekerja secara bersamaan dengan menggunakan satu koneksi.

Transaction

Transaction merupakan teknik yang digunakan untuk menjaga integritas data agar tidak terdapat data yang tidak valid, seperti misalnya data Orders yang tidak memiliki records di Order Details table untuk salah satu OrderID numbernya.

Dengan transaction ini maka semua proses eksekusi query yang dijadikan kedalam satu unit transaction harus success semua atau gagal. Artinya apabila terjadi error dalam sebuah proses query maka proses query sebelumnya yang success akan di rollback dikembalikan lagi ke kondisi sebelumnya, dan query berikutnya yang belum di proses tidak akan pernah di eksekusi.

Terdapat dua object ADO.NET yang dapat digunakan untuk melakukan transaction ini yaitu, SqlTransaction (untuk SQL Server 7.0 dan versi diatasnya) dan TransactionScope class.

TransactionScope class terdapat di dalam .NET versi 2.0. Untuk menggunakan class ini harus dilakukan referensi terlebih dahulu terhadap System.Transaction.dll. Penggunaan class ini lebih simpel dibandingkan dengan SqlTransaction. Dengan class ini kita tidak perlu secara eksplisit mengatur properti transaction terhadap Command object yang terlibat didalam unit transaction tersebut. Selain itu juga didalam transactionscope class method rollback akan dipanggil secara otomatis apabila terjadi error.

SqlTransaction

Berikut dibawah ini adalah contoh program penggunaan SqlTransaction untuk membuat satu unit transaksi yang akan melakukan proses eksekusi dua query yang berbeda.

1. Tambahkan satu buat Windows Forms Application baru, beri nama project tersebut TransactionObject.
2. Tambahkan satu button, beri nama btnSqlTrans.
3. Tambahkan dua radiobutton, ubah name yang pertama menjadi rbSetError dan text menjadi Set Error. Untuk radiobutton yang kedua ubah properti name nya menjadi rbSetSuccess dan properti text nya menjadi Set Success.
4. Tambahkan perintah berikut di baris paling atas :

Option Strict On

Imports System.Data.SqlClient

5. Deklarasikan dua buah variabel berikut pada class level variabel :

Private strSQL1, strSQL2 As String

6. Ketikkan kode berikut untuk menghandle event checkedchanged dari kedua radiobutton :

```
Private Sub RadioButtonCheckedChanged(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
    Handles rbSetError.CheckedChanged, _
    rbSetSuccess.CheckedChanged
```

```
    Dim rButton = DirectCast(sender, RadioButton)
    If rButton.Name.ToLower.Equals("rbseterror")
AndAlso rButton.Checked Then
```

```
        strSQL1 = "Update Customers Set CustomerID =
'ALFRD' Where CompanyName = 'Alfreds Futterkiste'"
```

```
strSQL2 = "Update [Order Details] Set Quantity  
= 10 Where OrderID = 10248 And ProductID = 11"
```

```
ElseIf rButton.Name.ToLower.Equals("rbsetsuccess")  
AndAlso rButton.Checked Then
```

```
strSQL1 = "Update Customers Set ContactName =  
'Anders' Where CustomerID = 'Alfki'"
```

```
strSQL2 = "Update [Order Details] Set Quantity  
= 10 Where OrderID = 10248 And ProductID = 11"
```

```
End If
```

```
End Sub
```

Kode diatas digunakan untuk melakukan simulasi transaksi, dimana radiobutton Set Error dibuat untuk melakukan transaksi yang nantinya akan di rollback karena error dalam pencarian CustomerID sehingga query yang kedua tidak akan pernah dieksekusi. Untuk radiobutton Set Success sebaliknya, yaitu query dibuat agar eksekusi transaksinya sukses.

7. Ketikkan kode berikut pada event click button btnSqlTrans
:

```
Private Sub btnSqlTrans_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnSqlTrans.Click
```

```
If strSQL1 Is Nothing AndAlso strSQL2 Is Nothing  
Then
```

```
strSQL1 = "Update Customers Set CustomerID =  
'ALFRD' Where CompanyName = 'Alfreds Futterkiste'"
```

```
strSQL2 = "Update [Order Details] Set Quantity  
= 10 Where OrderID = 10248 And ProductID = 11"
```

```
End If
```

```
Using sqlConn As New  
SqlConnection(My.Settings.ConnectionStringApp)
```

```
Dim cmdCustomer As New SqlCommand
cmdCustomer.Connection = sqlConn
cmdCustomer.CommandType = CommandType.Text
cmdCustomer.CommandText = strSQL1

Dim cmdOrderDetails As New SqlCommand
cmdOrderDetails.Connection = sqlConn
cmdOrderDetails.CommandType = CommandType.Text
cmdOrderDetails.CommandText = strSQL2

Dim sqlTrans As SqlTransaction = Nothing

Try
    sqlConn.Open()
    sqlTrans =
sqlConn.BeginTransaction(IsolationLevel.Serializable)

    cmdCustomer.Transaction = sqlTrans
    cmdOrderDetails.Transaction = sqlTrans

    cmdCustomer.ExecuteNonQuery()
    cmdOrderDetails.ExecuteNonQuery()

    sqlTrans.Commit()

    MessageBox.Show("Updated!")
Catch sqlEx As SqlException
    sqlTrans.Rollback()

    MessageBox.Show(sqlEx.ToString)
Catch ex As Exception
    sqlTrans.Rollback()

    MessageBox.Show(ex.ToString)
End Try

End Using

End Sub
```

Penempatan transaction harus berada di dalam blok Try...Catch...sehingga apabila terjadi error kita cukup panggil method Rollback untuk mengembalikan data pada kondisi semula.

Perintah Commit digunakan untuk menandakan bahwa tidak terjadi error selama proses eksekusi query tersebut.

TransactionScope

Berikut adalah contoh penggunaan TransactionScope class untuk melakukan unit transaksi :

1. Tambahkan satu button kedalam form yang sama, beri nama btnTransScope.
2. Tambahkan referensi pada System.Transaction.dll.
3. Ketikkan kode berikut pada event click button diatas :

```
Private Sub btnTransScope_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnTransScope.Click
```

```
    Using ts As New
System.Transactions.TransactionScope()
        Using sqlConn As New
SqlConnection(My.Settings.ConnectionStringApp)
            strSQL1 = "Update Customers Set ContactName
= 'Anders' Where CustomerID = 'Alfki'"
            strSQL2 = "Update [Order Details] Set
Quantity = 10 Where OrderID = 10248 And ProductID = 11"
```

```
            Dim cmdCustomer As New SqlCommand
cmdCustomer.Connection = sqlConn
cmdCustomer.CommandType = CommandType.Text
cmdCustomer.CommandText = strSQL1
```

```
            Dim cmdOrderDetails As New SqlCommand
```

```

        cmdOrderDetails.Connection = sqlConn
        cmdOrderDetails.CommandType =
CommandType.Text
        cmdOrderDetails.CommandText = strSQL2

    Try
        sqlConn.Open()

        cmdCustomer.ExecuteNonQuery()
        cmdOrderDetails.ExecuteNonQuery()

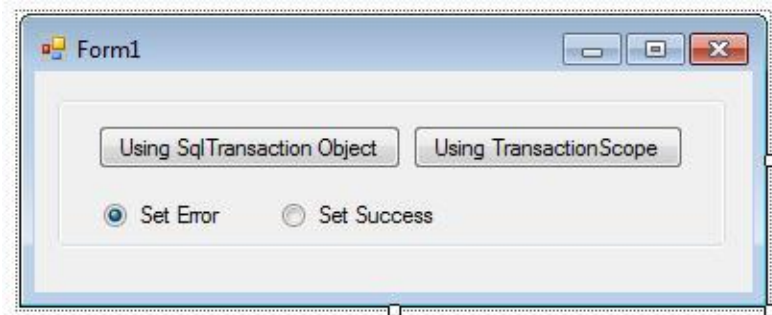
        ts.Complete()

        MessageBox.Show("Updated!")
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    End Try
End Using
End Using

End Sub

```

Dari kode diatas dapat dilihat bahwa penggunaan TransactionScope class lebih simpel dibandingkan dengan transaksi sebelumnya. Untuk menandakan bahwa transaksi tersebut sukses yaitu dengan dipanggilnya method Complete. Di dalam kode tersebut tidak terdapat perintah Rollback yang dipanggil, karena hal tersebut sudah secara otomatis dipanggil apabila terjadi error.



Gambar 9.12 Design Form

DataSet

Dataset merupakan object ADO.NET yang sifatnya disconnected. Dengan object ini kita dapat membuat aplikasi yang sifatnya offline (tidak selalu terhubung ke database). Data yang disimpan didalam dataset ini sifatnya In Memory data.

Dataset dapat dibayangkan sebagai sebuah miniatur database yang merupakan representasi dari database sebenarnya. Komponen yang membentuknya berupa DataTables, dan DataRelations.

Didalam DataTables tersebut terdiri dari DataColumnns dan DataRows. Sedangkan DataRelations merupakan class yang digunakan untuk menghubungkan antar DataTable yang terdapat di dalam DataSet apabila terdapat sebuah relationship.

Terdapat dua jenis DataSet, yaitu UntypedDataSet dan TypedDataSet

UntypedDataSet merupakan dataset yang dibuat langsung dari class System.Data.DataSet. Dataset ini sifatnya untyped dikarenakan tidak memiliki schema yang berisi informasi tentang struktur datatable yang terdapat didalam dataset tersebut, seperti misalnya nama kolom, constraint,

relationship,dll. Sehingga pengaksesan object-object tersebut dilakukan secara manual dan memungkinkan terjadinya kesalahan pengaksesan object-object yang terdapat didalam dataset. Error hanya dapat diketahui pada saat run time saja, misalnya terjadi kesalahan perujukan nama kolom atau nama datatable.

TypedDataset merupakan class yang diturunkan dari System.Data.Dataset class. Class ini sudah merupakan class modifikasi dari baseclass nya. Dengan jenis Dataset ini maka semua kekurangan yang terdapat didalam Untyped Dataset tertutupi semua. Properti seperti nama kolom, nama tabel akan muncul di jendela Intellisense. Hal tersebut terjadi karena jenis dataset ini memiliki schema berupa file .xsd pada saat design time yang berisi informasi tentang struktur datatable yang terdapat di dalam dataset tersebut.

Dataset membutuhkan komponen DataAdapter yang merupakan komponen yang menjembatani antara dataset dan sumber data. Selain untuk mengambil data tugas lainnya dari dataadapter yaitu melakukan update data (insert,update,dan delete) yang terjadi didalam dataset ke database.

Untyped Dataset

Berikut adalah contoh program penggunaan UntypedDataset :

1. Buat sebuah Windows Forms Application project baru, beri nama project tersebut UntypedDataset.
2. Tambahkan perintah dibawah ini pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

3. Deklarasikan variable berikut pada class level variabel :

```
Private ds As DataSet
Private WithEvents da As SqlDataAdapter
Private cb As SqlCommandBuilder
```

4. Tambahkan sebuah button, beri nama btnLoad.
5. Tambahkan sebuah DataGridView ke dalam form.
6. Buat sebuah prosedur seperti ini :

```
Private Sub LoadCustomers()
    da = New SqlDataAdapter("Select * From Customers",
My.Settings.ConnectionStringApp)
    da.UpdateBatchSize = 0

    cb = New SqlCommandBuilder(da)

    ds = New DataSet
    da.Fill(ds, "customerdata")

    DataGridView1.DataSource =
ds.Tables("customerdata")

    'atau :
    'DataGridView1.DataSource = ds.Tables(0)

    'atau :
    'DataGridView1.DataSource = ds
    'DataGridView1.DataMember = "customerdata"
End Sub
```

Prosedur diatas digunakan untuk load data dari tabel customers dengan menggunakan dataadapter untuk ditampung kedalam dataset.

Properti UpdateBatchSize digunakan untuk mengatur jumlah query yang terlibat dalam satu kali batch update ke database. Perlu diketahui bahwa apabila didalam dataset terdapat lima record yang dimanipulasi maka akan terdapat

juga lima kali round trip ke database untuk melakukan update data tersebut. Dengan merubah nilai properti tersebut menjadi angka nol maka dataadapter akan mengambil jumlah record maksimum yang dimanipulasi. Misalnya di dalam dataset tersebut terdapat sepuluh record yang mengalami manipulasi data maka hanya akan terdapat satu batch pengiriman data ke database untuk kesepuluh record tersebut. Fitur ini baru terdapat di ADO.NET 2.0.

CommandBuilder digunakan untuk generate command text yang berhubungan dengan manipulasi data seperti Insert, Update dan Delete. Command text tersebut dihasilkan pada saat run time.

Method Fill digunakan untuk menyimpan data yang telah di load ke dalam dataset. Selain itu juga kita dapat memberikan nama datatable yang otomatis akan dibuatkan ketika dataset diisi dengan data.

Dari kode diatas dapat dilihat beberapa cara untuk akses datatable yang terdapat didalam dataset. Anda dapat menggunakan nomor index yang menunjukkan urutan datatable yang terdapat didalam dataset sesuai dengan urutan pemanggilan Fill method atau dengan nama datatable tersebut.

7. Tambahkan kode berikut pada event click btnLoad :

```
Private Sub btnLoadData_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnLoadData.Click
```

```
LoadCustomers()
```

```
btnGetBuilder.Enabled = True
btnCancel.Enabled = True
btnUpdate.Enabled = True
```

```
End Sub
```

8. Tambahkan kontrol button, berikan nama btnUpdate, dan ketikkan kode berikut pada event click :

```
Private Sub btnUpdate_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnUpdate.Click  
  
Try  
    Dim records =  
da.Update(ds.Tables("customerdata"))  
  
    MessageBox.Show("Updated " & records.ToString &  
" record(s).")  
    Catch sqlEx As SqlException  
        MessageBox.Show(sqlEx.ToString)  
    Catch ex As Exception  
        MessageBox.Show(ex.ToString)  
    End Try  
End Sub
```

Kode diatas digunakan untuk melakukan update semua perubahan yang terjadi didalam dataset ke database. Method yang digunakan yaitu fungsi Update dengan menyebutkan nama datatable yang akan di update. Fungsi tersebut mengembalikan nilai jumlah record yang dimanipulasi.

9. Tambahkan satu button, beri nama btnCancel dan ketikkan kode berikut pada event click :

```
Private Sub btnCancel_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnCancel.Click  
Try  
    ds.Tables("customerdata").RejectChanges()  
    Catch ex As Exception  
        MessageBox.Show(ex.ToString)  
    End Try  
End Sub
```

Dengan memanggil method RejectChanges maka semua perubahan yang terjadi didalam dataset sebelum di commit ke database akan di reset ke keadaan semula.

10. Tambahkan satu button, berikan nama btnGetBuilder, ketikkan kode berikut pada event click :

```
Private Sub btnGetBuilder_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnGetBuilder.Click
    Try
        Dim sb As New System.Text.StringBuilder

        sb.Append("Delete Command : " & vbNewLine)
        sb.Append(cb.GetDeleteCommand.CommandText &
vbNewLine)
        sb.Append(vbNewLine)
        sb.Append("Update Command : " & vbNewLine)
        sb.Append(cb.GetUpdateCommand.CommandText &
vbNewLine)
        sb.Append(vbNewLine)
        sb.Append("Insert Command : " & vbNewLine)
        sb.Append(cb.GetInsertCommand.CommandText &
vbNewLine)

        MessageBox.Show(sb.ToString)

    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    End Try
End Sub
```

Kode diatas digunakan untuk menunjukkan perintah-perintah CoomandText yang digenerate secara run time oleh CommandBuilder.

11. Tambahkan kode berikut untuk mengetahui proses batch update yang terjadi di dalam dataadapter ketika proses

commit semua perubahan data yang terdapat didalam dataset ke database terjadi.

```
Private Sub da_RowUpdating(ByVal sender As Object, ByVal e  
As System.Data.SqlClient.SqlRowUpdatingEventArgs) Handles  
da.RowUpdating
```

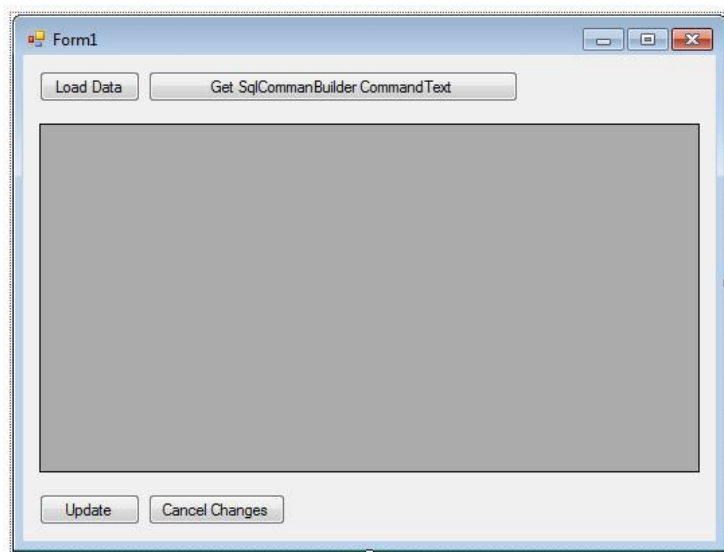
```
    Console.WriteLine("Processing : " &  
e.StatementType.ToString)
```

```
End Sub
```

```
Private Sub da_RowUpdated(ByVal sender As Object, ByVal  
e As System.Data.SqlClient.SqlRowUpdatedEventArgs) Handles  
da.RowUpdated
```

```
    Console.WriteLine("Records Affected : " &  
e.RecordsAffected.ToString)
```

```
End Sub
```



Gambar 9.13 Design Form

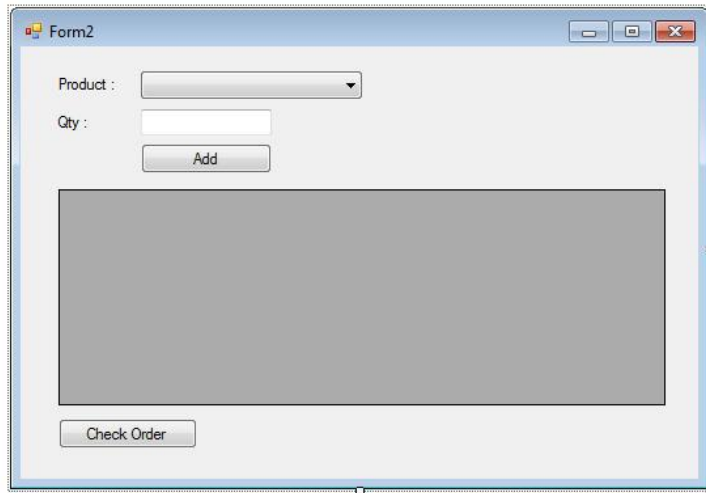
Membuat DataTable Secara Programmatically

Pada contoh diatas datatable secara otomatis akan terbentuk ketika method Fill dari dataadapter dipanggil. Lalu apakah datatable bisa dibuat secara programmatically tanpa harus tergantung dari data yang terdapat di dalam database? Jawabannya bisa.

Datatable dapat anda bayangkan sebagai object table yang terdapat didalam database. Semua fitur-fitur yang terdapat didalam table database juga hampir semua ada di fitur Datatable. Pemakaian Datatable ini sangat berguna untuk menyimpan data secara sementara selama aplikasi berjalan sebelum kemudian disimpan ke dalam database atau file xml.

Berikut adalah contoh penggunaan Datatable dimana datacolumnnya kita definisikan sendiri, sementara itu datatable ini nantinya digunakan sebagai data source untuk control Datagridview :

1. Tambahkan form baru kedalam project yang sama.
2. Tambahkan Combobox, beri nama cboProducts.
3. Tambahkan Textbox, beri nama txtQty.
4. Tambahkan Button, beri nama btnAdd.
5. Tambahkan Datagridview.
6. Tambahkan Button, beri nama btnCheck.
7. Hasil design form dapat dilihat pada gambar dibawah ini :



Gambar 9.14 Design Form

8. Buat sebuah class dibawah deklarasi class form :

Public Class Products

```

Private strName As String
Private intPrice As Integer

Public ReadOnly Property ProductName() As String
    Get
        Return strName
    End Get
End Property

Public ReadOnly Property UnitPrice() As Integer
    Get
        Return intPrice
    End Get
End Property

```

```
Public Sub New(ByVal ProdName As String, ByVal Price As Integer)
    strName = ProdName
    intPrice = Price
End Sub
```

End Class

Class tersebut nantinya akan digunakan sebagai sumber data control Combobox untuk menampilkan daftar nama-nama produk beserta harganya.

9. Buat prosedur dibawah ini di dalam class form :

```
Private Sub LoadProducts()
    Dim productList As New List(Of Products)

    productList.AddRange(New Products() _
        {New Products("Mouse", 40000),
        _ New Products("Keyboard", 15000),
        _ New Products("Flash Disc", 75000)}) _
    )

    cboProducts.DataSource = productList
    cboProducts.DisplayMember = "ProductName"
    cboProducts.ValueMember = "UnitPrice"
End Sub
```

10. Deklarasikan class level variabel berikut :

```
Private dtOrders As DataTable
```

11. Buat prosedur dibawah ini untuk membuat Datatable secara programmatically. Datatable ini nantinya dijadikan sebagai sumber data untuk Datagridview :

```
Private Sub CreateOrdersTable()
```

```

        Dim dcName = New DataColumn("ProductName",
GetType(String))
        dcName.AllowDBNull = False

        Dim dcPrice = New DataColumn("Price",
GetType(Integer))
        dcPrice.AllowDBNull = False

        Dim dcQty = New DataColumn("Qty", GetType(Integer))
        dcQty.AllowDBNull = False

        Dim dcTotal = New DataColumn("Total",
GetType(Integer))
        dcTotal.Expression = "Price * Qty"

        dtOrders = New DataTable("Orders")

        dtOrders.Columns.AddRange(New DataColumn() {dcName,
dcPrice, dcQty, dcTotal})

        dtOrders.PrimaryKey = New DataColumn() {dcName}

        DataGridView1.DataSource = dtOrders
End Sub

```

Perhatikan properti Expression yang dimiliki oleh DataColumn. Properti tersebut digunakan untuk membuat kolom yang sifatnya merupakan hasil perhitungan dari kolom-kolom yang lain.

12. Buat prosedur dibawah ini untuk setting tampilan kolom Datagridview yang menampilkan angka :

```

Private Sub DataGridViewSettings()

DataGridView1.Columns("Price").DefaultCellStyle.Format =
"#,##"
DataGridView1.Columns("Qty").DefaultCellStyle.Format =
"#,##"
DataGridView1.Columns("Total").DefaultCellStyle.Format =
"#,##"

```

End Sub

13. Panggil ketiga prosedur diatas pada event Load dari form :

```
Private Sub Form2_Load(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles MyBase.Load  
    LoadProducts()  
    CreateOrdersTable()  
    DataGridSettings()  
End Sub
```

14. Tambahkan kode berikut pada event click btnAdd :

```
Private Sub btnAdd_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnAdd.Click  
    If Not IsNumeric(txtQty.Text) Then  
        MessageBox.Show("Input harus angka")  
        Exit Sub  
    End If  
  
    Dim newRow As DataRow = dtOrders.NewRow  
    newRow.BeginEdit()  
    newRow("ProductName") = cboProducts.Text  
    newRow("Price") =  
Convert.ToInt32(cboProducts.SelectedValue)  
    newRow("Qty") = Convert.ToInt32(txtQty.Text)  
    newRow.EndEdit()  
  
    dtOrders.Rows.Add(newRow)  
End Sub
```

Prosedur diatas digunakan untuk menambahkan record baru ke datatable yang telah dibuat.

15. Tambahkan kode berikut pada event click btnCheck :

```

    Private Sub btnCheck_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnCheck.Click
    If dtOrders.Rows.Count < 1 Then
        MessageBox.Show("No Orders")
        Exit Sub
    End If

    Dim rowsColl As DataRowCollection = dtOrders.Rows
    Dim aRow As DataRow

    Dim sb As New System.Text.StringBuilder

    For Each aRow In rowsColl
        sb.AppendFormat("Product : {0}, Qty : {1},
Total : {2}", _
                        aRow("ProductName").ToString, _
                        aRow("Qty").ToString(), _
                        aRow("Total").ToString)
        sb.Append(vbNewLine)
    Next

    MessageBox.Show(sb.ToString)
End Sub

```

Kode diatas digunakan untuk membaca record yang telah masuk ke dalam datatable dengan menggunakan perintah looping for each terhadap koleksi row datatable.

Read Write XML dari Dataset

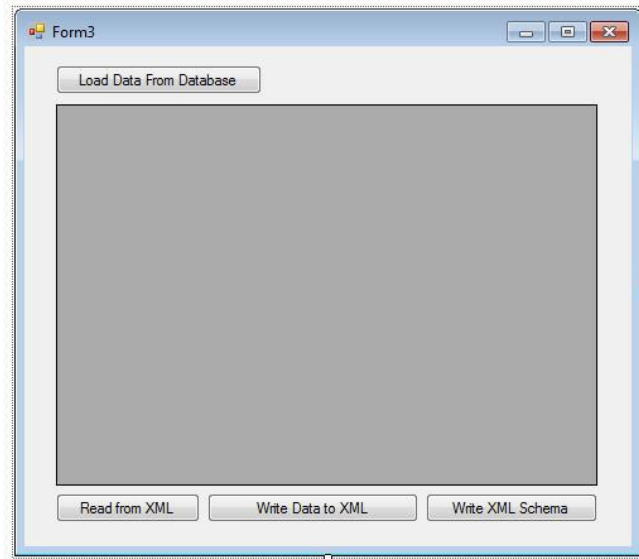
Salah satu kelebihan yang dimiliki oleh dataset yaitu dukungannya terhadap xml cukup baik. Hal ini diwujudkan dengan disediakannya method untuk menulis dan membaca data dari dan ke file xml. Method tersebut yaitu ReadXML dan WriteXML. Selain itu dataset memiliki kemampuan juga untuk menuliskan schema data yang terdapat didalam dataset yang berisi informasi struktur datatable didalamnya. Schema xml sangat berguna untuk validasi data xml sebelum dibaca agar

memenuhi aturan yang telah ditetapkan sebelumnya. Hal tersebut diimplementasikan dengan adanya method ReadXMLSchema.

Penggunaan file xml yang berasal dari dataset sangat berguna untuk membuat aplikasi yang sifatnya occasionally connected application, artinya aplikasi tersebut tidak selalu harus terkoneksi ke database, sehingga datanya sementara dapat disimpan didalam file xml.

Berikut adalah contoh program untuk menuliskan data yang terdapat didalam dataset yang diambil dari database kedalam file xml dan juga bagaimana caranya membaca data tersebut dari file xml kembali ke dataset. Selain itu juga diberikan contoh penulisan xml schema dan pembacaannya untuk validasi xml file yang dibaca.

1. Tambahkan form baru kedalam project yang sama.
2. Tambahkan button, beri nama btnLoadData.
3. Tambahkan Datagridview control.
4. Tambahkan button, beri nama btnReadXML.
5. Tambahkan button, beri nama btnWriteXML.
6. Tambahkan button, beri nama btnWriteSchema.
7. Tampilan design form dapat dilihat seperti ini :



Gambar 9.15 Design Form

8. Tambahkan perintah ini pada baris paling atas :

Option Strict On

Imports System.Data.SqlClient

9. Deklarasikan class level variabel berikut :

```
Private ds As DataSet
Private da As SqlDataAdapter
```

10. Buat prosedur dibawah ini untuk membaca data dari tabel database yang nantinya disimpan kedalam dataset untuk dituliskan kedalam file xml :

```
Private Sub LoadCustomers()
    ds = New DataSet("CustomersDataSet")
    da = New SqlDataAdapter("Select * From Customers",
My.Settings.ConnectionStringApp)
```

```
ds = New DataSet
da.Fill(ds, "Customers")

DataGridView1.DataSource = ds.Tables("Customers")
End Sub
```

11. Panggil prosedur tersebut pada event click btnLoadData :

```
Private Sub btnLoadData_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnLoadData.Click
    LoadCustomers()
End Sub
```

12. Ketikkan kode berikut pada event click btnReadXML :

```
Private Sub btnReadXML_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btnReadXML.Click

    If
My.Computer.FileSystem.FileExists("C:\customers.xml")
AndAlso
My.Computer.FileSystem.FileExists("C:\customerschema.xml")
Then
        ds = New DataSet("CustomersDataSet")
        ds.ReadXmlSchema("C:\customerschema.xml")
        ds.ReadXml("C:\customers.xml")

        DataGridView1.DataSource =
ds.Tables("Customers")
    Else
        MessageBox.Show("C:\customers.xml and (or)
C:\customerschema.xml file could not be found")
    End If
End Sub
```

13. Ketikkan kode berikut pada event click btnWriteXML :

```
Private Sub btnWriteXML_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnWriteXML.Click
```

```
    If ds IsNot Nothing Then  
        ds.WriteXml("C:\customers.xml")  
        Process.Start("C:\customers.xml")  
    Else  
        MessageBox.Show("Load data first")  
    End If
```

```
End Sub
```

14. Ketikkan kode berikut pada event click btnWriteSchema :

```
Private Sub btnWriteSchema_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnWriteSchema.Click
```

```
    If ds IsNot Nothing Then  
        ds.WriteXmlSchema("C:\customerschema.xml")  
        Process.Start("C:\customerschema.xml")  
    Else  
        MessageBox.Show("Load data first")  
    End If
```

```
End Sub
```

XML Diffgram

Method WriteXML dan ReadXML diatas menuliskan file xml dan membaca file xml secara keseluruhan, artinya tidak terdapat informasi data-data mana saja yang dimanipulasi.

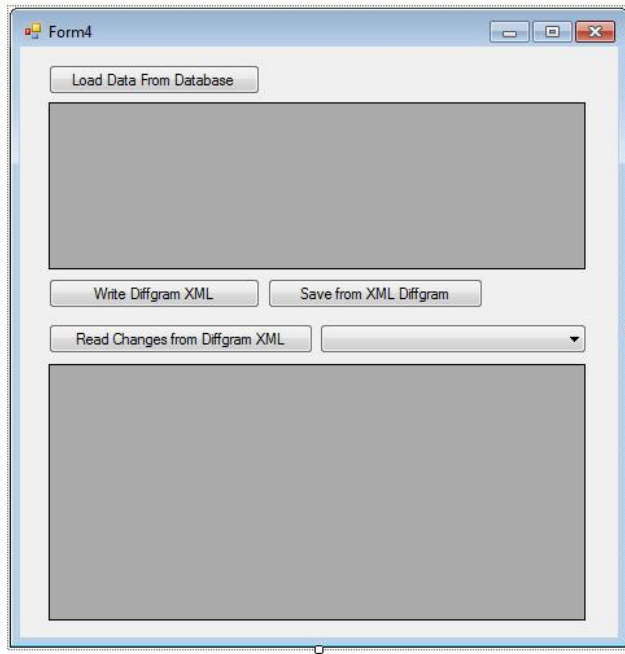
Dataset menyediakan method Diffgram yang digunakan untuk menyimpan data kedalam file xml dengan menyimpan informasi data-data yang dimanipulasi. Selain itu juga kita

dapat membaca data dari file xml tersebut hanya untuk data-data yang telah dimanipulasi saja.

Method tersebut berguna untuk menyimpan perubahan data secara sementara sebelum akhirnya nanti di upload ke database. Untuk menggunakan fitur tersebut cukup dengan menambahkan mode Diffgram pada write mode dan read mode.

Berikut adalah contoh program yang memanfaatkan teknik Diffgram :

1. Tambahkan form baru kedalam project yang sama.
2. Tambahkan button, beri nama btnLoadData.
3. Tambahkan button, beri nama btnWriteDiffgram.
4. Tambahkan button, beri nama btnSave.
5. Tambahkan button, beri nama btnReadChanges.
6. Tambahkan Combobox, beri nama cboState.
7. Tambahkan dua DataGridView.
8. Atur control seperti yang tampak pada gambar ini :



Gambar 9.16 Design Form

9. Ketikkan perintah berikut pada baris code paling atas :

Option Strict On

Imports System.Data

Imports System.Data.SqlClient

10. Deklarasikan class level variabel dibawah ini :

Private ds As DataSet

Private da As SqlDataAdapter

Private cb As SqlCommandBuilder

11. Buat prosedur seperti dibawah ini :

```

Private Sub LoadCustomersAndWriteXMLSchema()

    ds = New DataSet("CustomersDataSet")
    da = New SqlDataAdapter("Select * From Customers",
My.Settings.ConnectionStringApp)

    cb = New SqlCommandBuilder(da)

    ds = New DataSet
    da.Fill(ds, "Customers")

    ds.WriteXmlSchema("C:\customerschema.xml")

    DataGridView1.DataSource = ds.Tables("Customers")

End Sub

```

12. Tambahkan kode berikut pada event click btnLoadData:

```

Private Sub btnLoadData_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnLoadData.Click

    LoadCustomersAndWriteXMLSchema()

End Sub

```

13. Tambahkan kode berikut pada event click btnWriteDiffgram. Kode ini digunakan untuk menuliskan data ke file xml dengan mode Diffgram.

```

Private Sub btnWriteDiffgram_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnWriteDiffgram.Click

    If ds IsNot Nothing Then
        ds.WriteXml("C:\customersdiffgram.xml",
XmlWriteMode.DiffGram)
        Process.Start("C:\customersdiffgram.xml")
    End If
End Sub

```

```
Else
    MessageBox.Show("Load data first")
End If
```

```
End Sub
```

14. Tambahkan kode berikut pada event click btnReadChanges
:

```
Private Sub btnReadChanges_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnReadChanges.Click

    If cboState.SelectedIndex < 0 Then
        MessageBox.Show("Choose the Row State firts")
        Exit Sub
    End If

    If
My.Computer.FileSystem.FileExists("C:\customersdiffgram.xml
") AndAlso _

My.Computer.FileSystem.FileExists("C:\customerschema.xml")
Then

        Dim dsDiff = New DataSet("CustomersDataSet")
        dsDiff.ReadXmlSchema("C:\customerschema.xml")
        dsDiff.ReadXml("C:\customersdiffgram.xml",
XmlReadMode.DiffGram)

        Dim dv = dsDiff.Tables(0).DefaultView

        Dim dvRowState As DataRowState = Nothing

        Select Case cboState.SelectedIndex
            Case 0
                dv.RowStateFilter =
DataRowState.Added
```

```

        Case 1
            dv.RowStateFilter =
DataRowState.CurrentRows
        Case 2
            dv.RowStateFilter =
DataRowState.Deleted
        Case 3
            dv.RowStateFilter =
DataRowState.ModifiedCurrent
        Case 4
            dv.RowStateFilter =
DataRowState.ModifiedOriginal
        Case 5
            dv.RowStateFilter =
DataRowState.OriginalRows
        Case 6
            dv.RowStateFilter =
DataRowState.Unchanged
    End Select

    DataGridView2.DataSource = dv
Else
    MessageBox.Show("C:\customers.xml and (or)
C:\customerschema.xml file could not be found. Write to XML
first.")
End If

End Sub

```

Kode diatas digunakan untuk membaca data file xml sesuai dengan RowStateFilter yang diinginkan. Dengan filter tersebut kita dapat mengetahui data-data mana saja yang mengalami manipulasi atau bahkan data original sekalipun kita masih dapat mengetahuinya. Filtering tersebut harus dilakukan lewat object DataView.

15. Tambahkan kode berikut pada event click btnSave :

```

Private Sub btnSave_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btnSave.Click

    If
My.Computer.FileSystem.FileExists("C:\customersdiffgram.xml
") AndAlso _

My.Computer.FileSystem.FileExists("C:\customerschema.xml")
AndAlso ds IsNot Nothing Then

        ds.Clear()
        ds.ReadXmlSchema("C:\customerschema.xml")
        ds.ReadXml("C:\customersdiffgram.xml",
XmlReadMode.DiffGram)

        Try
            da.Update(ds.Tables(0))
            ds.AcceptChanges()
            ds.WriteXml("C:\customersdiffgram.xml",
XmlWriteMode.DiffGram)
            Process.Start("C:\customersdiffgram.xml")
        Catch ex As Exception
            MessageBox.Show(ex.ToString)
        End Try
    Else
        MessageBox.Show("C:\customers.xml and (or)
C:\customerschema.xml file could not be found. Write to XML
first.")
    End If

End Sub

```

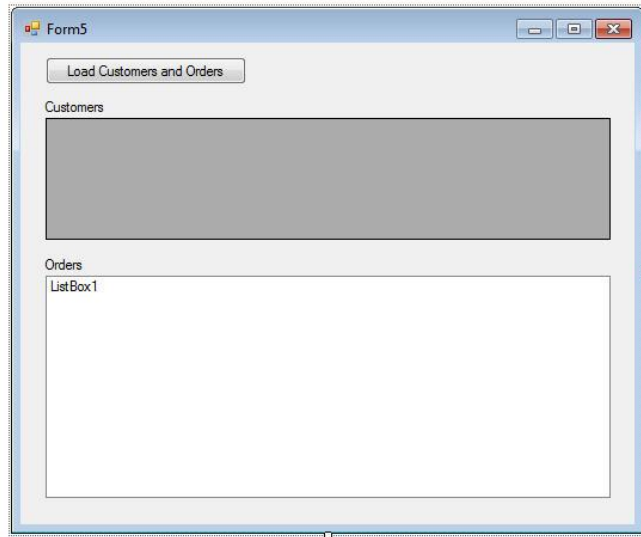
Kode diatas digunakan untuk melakukan update terhadap data-data yang dimanipulasi ke database dan ke file xml dengan mode Diffgram.

DataRelation

Datarelation digunakan untuk menghubungkan datatable-datatable yang terdapat didalam dataset apabila diantara beberapa table tersebut memiliki relationship yang menghubungkan antara primary key dengan foreign key nya. Untuk membuat relation ini dibutuhkan minimal dua datatable dan kolom yang bertindak sebagai primary key dan foreign key. Relation ini berguna dalam hal navigasi record yang memiliki sifat master-details.

Berikut adalah contoh program bagaimana caranya menghubungkan dua datatable didalam dataset dan melakukan navigasi record dari master table terhadap child table yang berhubungan. Data diambil dari table Customers dan Orders didalam database Northwind, sehingga nantinya kita dapat melakukan navigasi record untuk melihat data Orders per CustomerID :

1. Tambahkan form baru kedalam project yang sama.
2. Tambahkan button, beri nama btnLoadData.
3. Tambahkan DataGridView.
4. Tambahkan ListBox.
5. Tampilan design form dapat dilihat seperti dibawah ini:



Gambar 9.17 Design Form

6. Tambahkan perintah berikut pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

7. Deklarasikan class level variabel berikut :

```
Private ds As DataSet
Private daCust, daOrders As SqlDataAdapter
Private dr As DataRelation
```

8. Buat prosedur berikut yang digunakan untuk membuat Datarelation antar datatable customers dan orders :

```
Private Sub LoadCustomerOrders()
    ds = New DataSet
```

```

        daCust = New SqlDataAdapter("Select
CustomerID,CompanyName From Customers",
My.Settings.ConnectionStringApp)

        daCust.MissingSchemaAction =
MissingSchemaAction.AddWithKey
        daCust.Fill(ds, "customers")

        daOrders = New SqlDataAdapter("Select * From
Orders", My.Settings.ConnectionStringApp)
        daOrders.Fill(ds, "Orders")

        Dim dcPK As DataColumn =
ds.Tables("customers").Columns("customerid")
        Dim dcFK As DataColumn =
ds.Tables("orders").Columns("customerid")

        dr = New DataRelation("RelCustOrders", dcPK, dcFK)

        ds.Relations.Add(dr)

        DataGridView1.DataSource = ds.Tables("customers")

End Sub

```

MissingSchemaAction properti yang dimiliki oleh DataAdapter digunakan untuk menambahkan informasi key yang dimiliki oleh tabel tersebut.

9. Tambahkan kode berikut pada event click btnLoadData:

```

Private Sub btnLoadData_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnLoadData.Click

```

```

    LoadCustomerOrders()

```

```

End Sub

```

10. Ketikkan kode berikut pada event CellEnter DataGridView
:

```
Private Sub DataGridView1_CellEnter(ByVal sender As Object,
ByVal e As System.Windows.Forms.DataGridViewCellEventArgs)
Handles DataGridView1.CellEnter

    ListBox1.Items.Clear()

    Dim custID =
DataGridView1.Rows(e.RowIndex).Cells(0).Value.ToString
    Dim rowID =
ds.Tables("customers").Rows.Find(custID)

    If rowID IsNot Nothing Then
        Try
            Dim dt As New DataTable("orders")

            Dim childRows = rowID.GetChildRows(dr)

            For Each aRow In childRows

ListBox1.Items.Add(aRow("OrderID").ToString & ", " _
&
Convert.ToDateTime(aRow("OrderDate")).ToLongDateString)
                Next
            Catch ex As Exception
                MsgBox(ex.Message)
            End Try
        End If

End Sub
```

Kode tersebut akan dieksekusi ketika user melakukan navigasi pada record yang terdapat di Datagridview dengan mouse atau cursor. Untuk mendapatkan child records digunakan fungsi GetChildRows yang mengembalikan nilai berupa collection of rows dari record hasil filtering yang terdapat didalam master datatable.

Filtering Record Dengan DataSet dan DataView

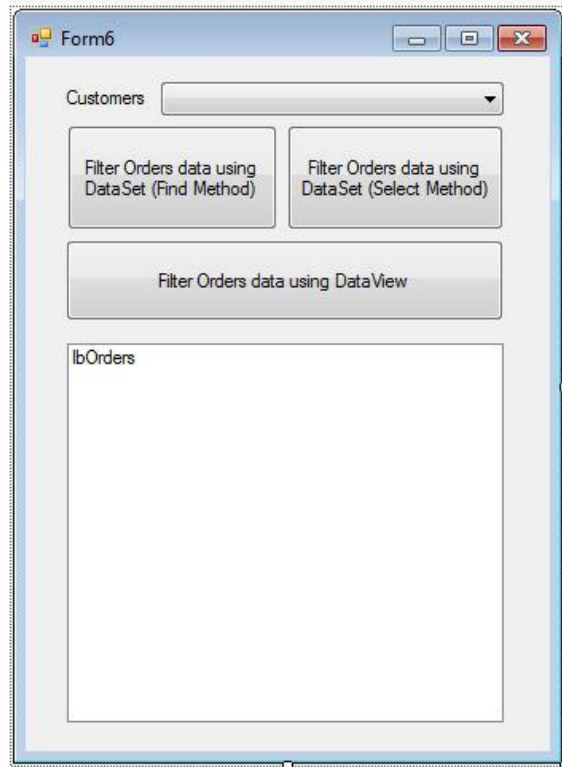
Dataset dan DataView sudah menyediakan fitur yang dapat digunakan untuk melakukan pencarian data. Dataset lewat datatable nya menyediakan dua method untuk pencarian data yaitu Find dan Select. Sedangkan dataview menyediakan properti RowFilter untuk melakukan pencarian datanya.

Method Find digunakan untuk pencarian data berdasarkan kolom primary key yang mengembalikan single record. Untuk menggunakan method tersebut dataadapter harus memberikan informasi mengenai primary key yang digunakan dengan mengatur properti MissingSchemaAction yang diisi dengan nilai MissingSchemaAction.AddWithKey.

Method Select akan memberikan hasil berupa array dari DataRow. Method ini tidak membutuhkan informasi primary key datatable.

Berikut adalah contoh penggunaan ketiga method diatas untuk melakukan pencarian data terhadap datatable dan dataview :

1. Tambahkan Combobox, beri nama cboCustomers.
2. Tambahkan button, beri nama btnDataSetFind.
3. Tambahkan button, beri nama btnDataSetSelect.
4. Tambahkan button, beri nama btnDataView.
5. Tambahkan Listbox, beri nama lbOrders.
6. Tampilan design form dapat dilihat seperti dibawah ini:



Gambar 9.18 Design Form

7. Tambahkan perintah berikut pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

8. Deklarasikan class level variabel berikut :

Private dv As DataView
Private ds As DataSet

9. Buat prosedur berikut untuk menampilkan data customer di combobox :

```
Private Sub LoadCustomers()  
  
    Using da As New SqlDataAdapter("Select  
CustomerID,CompanyName From Customers",  
My.Settings.ConnectionStringApp)  
        Dim dsCust = New DataSet  
        da.Fill(dsCust)  
        cboCustomers.DataSource = dsCust.Tables(0)  
        cboCustomers.DisplayMember = "CompanyName"  
        cboCustomers.ValueMember = "CustomerID"  
    End Using  
  
End Sub
```

10. Ketikkan prosedur berikut ini untuk membuat dataset dan dataview :

```
Private Sub CreateDataViewAndDataSet()  
  
    Using da As New SqlDataAdapter("Select * From  
Orders", My.Settings.ConnectionStringApp)  
        ds = New DataSet  
        dv = New DataView  
  
        da.MissingSchemaAction =  
MissingSchemaAction.AddWithKey  
        da.Fill(ds)  
  
        Dim dsTemp As DataSet = ds.Copy  
        dv.Table = dsTemp.Tables(0)  
    End Using  
  
End Sub
```

11. Panggil kedua prosedur diatas pada event Load dari form :

```
Private Sub Form6_Load(ByVal sender As Object, ByVal e As  
System.EventArgs) Handles Me.Load
```

```
    LoadCustomers()  
    CreateDataViewAndDataSet()
```

```
End Sub
```

12. Tambahkan kode berikut pada event click btnDataView:

```
Private Sub btnDataView_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnDataView.Click  
    lbOrders.Items.Clear()  
  
    dv.RowFilter = "CustomerID = '" &  
cboCustomers.SelectedValue.ToString & "'"
```

```
    For i = 0 To dv.Count - 1  
  
lbOrders.Items.Add(dv(i).Item("OrderID").ToString & ", " _  
                    &  
Convert.ToDateTime(dv(i).Item("OrderDate")).ToLongDateStrin  
g)  
    Next
```

```
End Sub
```

13. Tambahkan kode berikut pada event click btnDataSetFind
:

```
Private Sub btnDataSetFind_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnDataSetFind.Click
```

```
    lbOrders.Items.Clear()
```

```
    Dim OrderID = InputBox("Masukkan No.OrderID  
disini.", "Find DataSet's Method")
```

```

        If IsNumeric(OrderID) Then
            Dim row = ds.Tables(0).Rows.Find(OrderID)

            lbOrders.Items.Add(row("CustomerID").ToString &
", " &
Convert.ToDateTime(row("OrderDate")).ToLongDateString)
        End If
    End Sub

```

14. Tambahkan kode berikut pada event click btnDataSetSelect :

```

Private Sub btnDataSetSelect_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnDataSetSelect.Click

    lbOrders.Items.Clear()

    Dim rows() = ds.Tables(0).Select("CustomerID = '" &
cboCustomers.SelectedValue.ToString & "'")

    If rows.Length > 0 Then
        For i = 0 To rows.GetUpperBound(0)

lbOrders.Items.Add(rows(i).Item("OrderID").ToString & ", "
-
&
Convert.ToDateTime(rows(i).Item("OrderDate")).ToLongDateStr
ing)
            Next
        End If
    End Sub

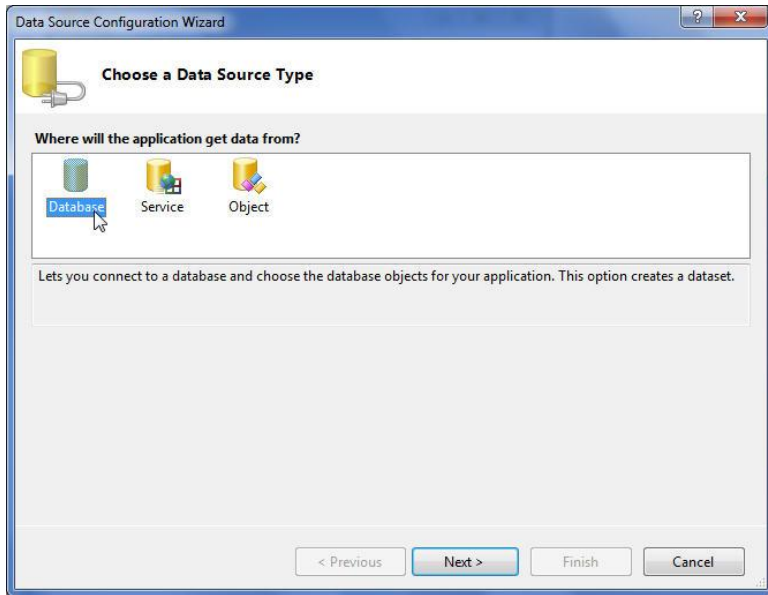
```

Typed DataSet

Seperti yang telah disinggung sebelumnya bahwa typed dataset merupakan class yang diturunkan dari System.Data.DataSet class dan merupakan hasil modifikasi. Dataset ini memiliki schema berupa file .xsd yang berisi informasi mengenai object-object yang terdapat di dalam dataset tersebut seperti datatable, nama kolom, tipe data kolom, constraint, dll. Semua object tersebut dapat diakses lewat jendela intellisense, sehingga memiliki keuntungan dalam hal terhindar dari run time error karena kesalahan akses member atau object dataset. Dataset ini dapat dibentuk secara manual from the scratch (dari awal) atau menggunakan data wizard.

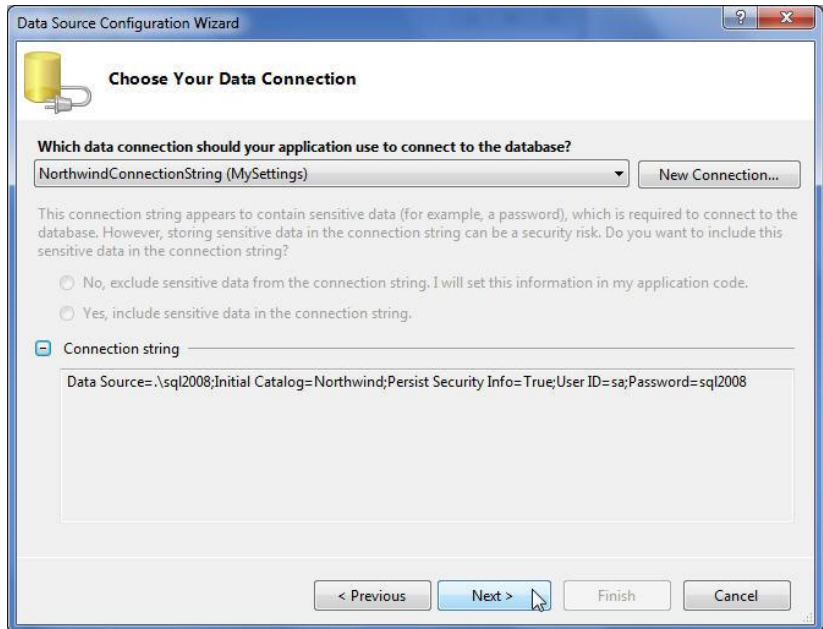
Berikut adalah contoh membuat typeddataset dengan menggunakan wizard :

1. Buat project Windows Forms Application baru, beri nama project tersebut dengan UntypedDataSet.
2. Klik menu Add New Data Source didalam menu Data di toolbar visual studio.
3. Pilih Database sebagai sumber data :



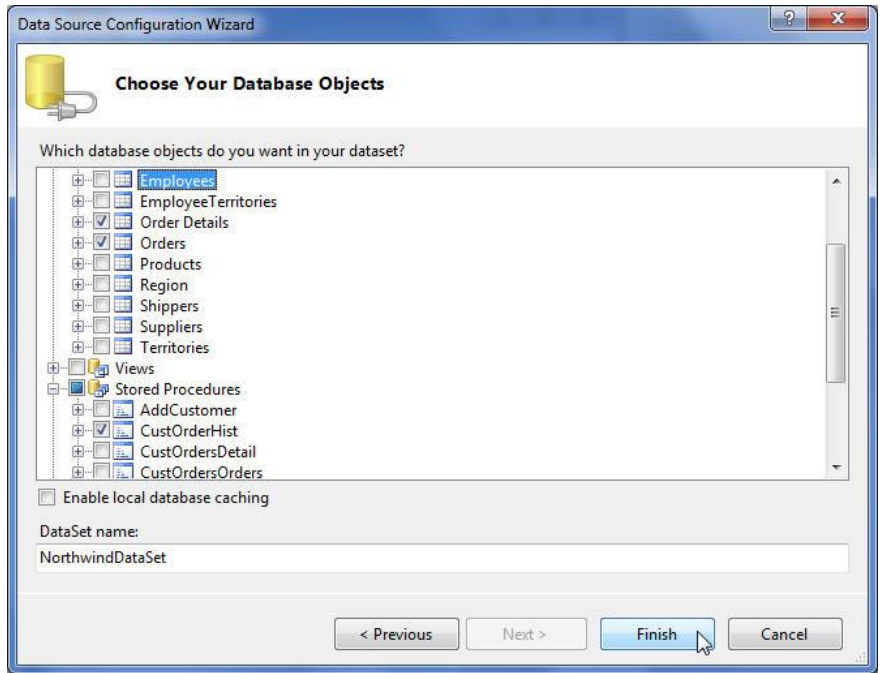
Gambar 9.19 Database sebagai sumber data

4. Pilih connection string yang sesuai dengan instance server yang digunakan. Pilih database Northwind beserta jenis autentikasinya.



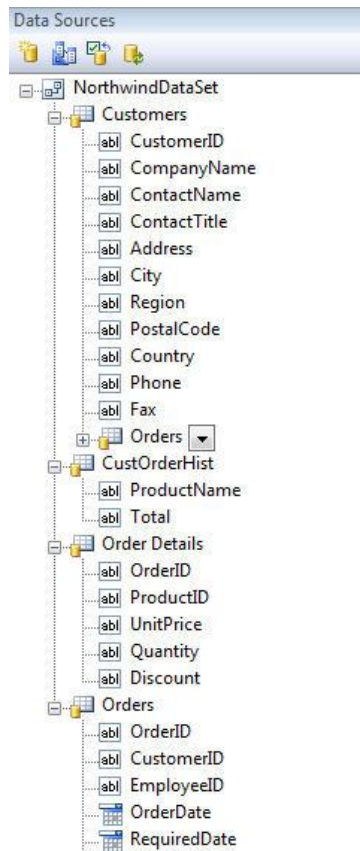
Gambar 9.20 Connection String wizard

5. Pilih table Customers, Orders, OrderDetails, dan satu stored procedure CustOrderHist :



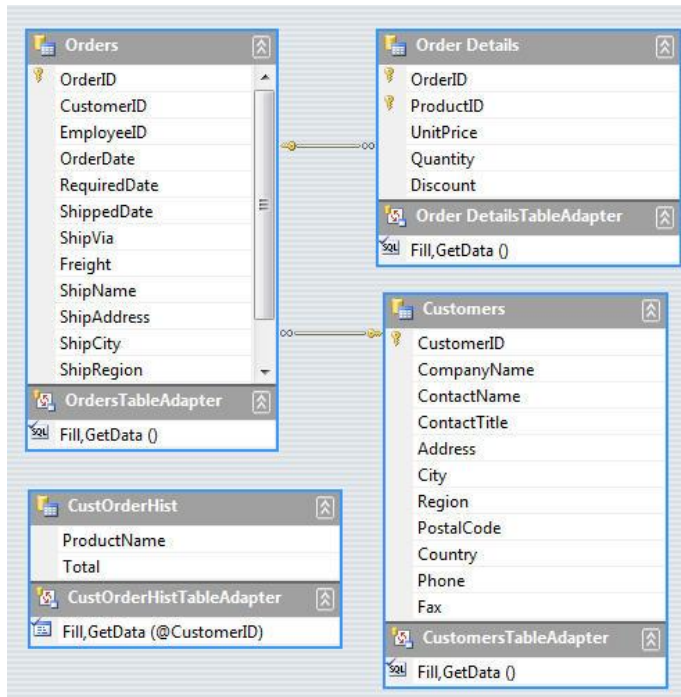
Gambar 9.21 Object database

6. Dataset yang terbentuk dapat dilihat di DataSources window :



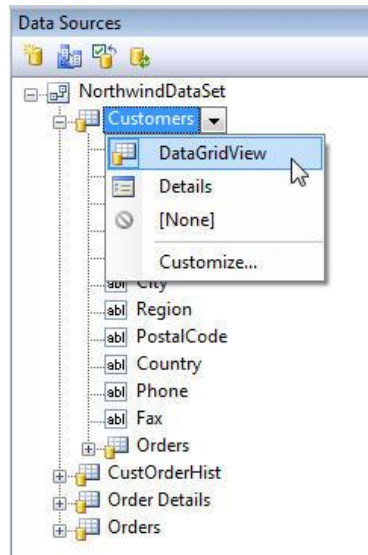
Gambar 9.22 Data Sources Window

7. Visualisasi Northwind.xsd file yang terbentuk :



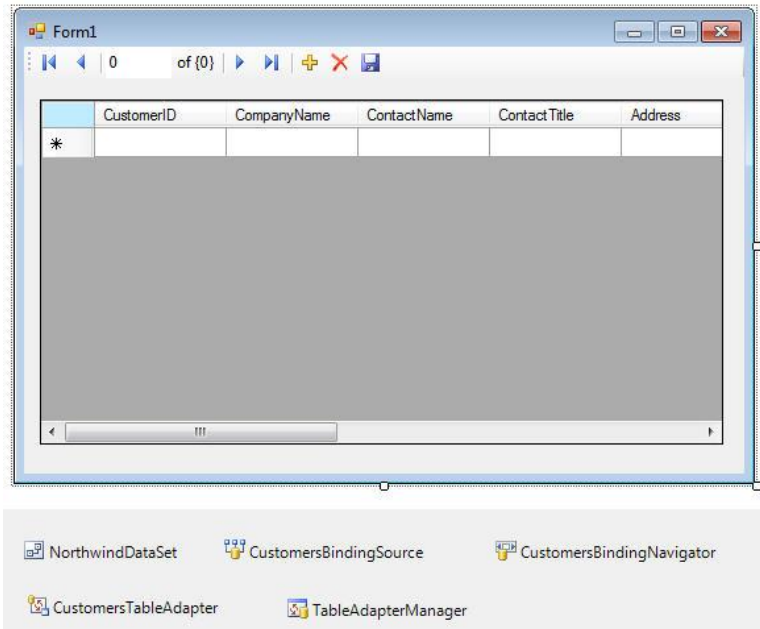
Gambar 9.23 Northwind.xsd

8. Drag dan drop customers table dari DataSources window dalam mode DataGridView ke form :



Gambar 9.24 Customers table dalam mode DataGridView

9. Tampilan form setelah menggunakan Customers table dengan mode DataGridView :



Gambar 9.25 Form hasil Data Wizard

Dari form diatas dapat dilihat terdapat beberapa komponen yang digenerate secara otomatis yaitu, Navigator control yang digunakan untuk melakukan navigasi record, button untuk melakukan manipulasi data (save, add, delete), TableAdapter yang bertugas untuk mengambil data dari database dan melakukan update data dari dataset ke database. Apabila anda membuka kode form tersebut maka akan terdapat beberapa kode yang dihasilkan seperti kode untuk load data pada even Load form dan kode untuk melakukan update data pada event click button save.

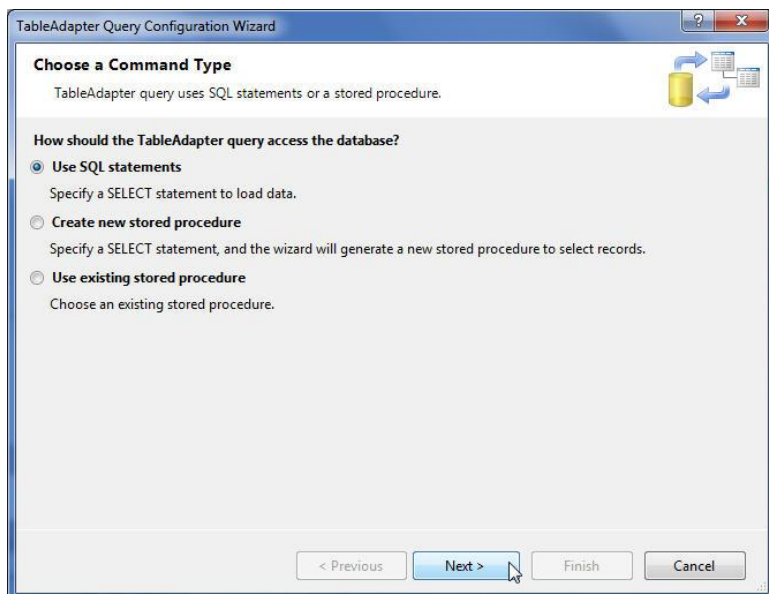
TableAdapter Query

Kita dapat membuat query lain pada tableadapter yang telah terbentuk. Query tersebut digunakan untuk mengambil

data dari database. Misalnya kita ingin membuat query yang dapat menampilkan data customers berdasarkan country. Jenis query yang dapat digunakan bukan hanya query yang mengembalikan rows saja, namun juga seperti query yang menghitung nilai pada kolom tertentu dengan fungsi aggregate.

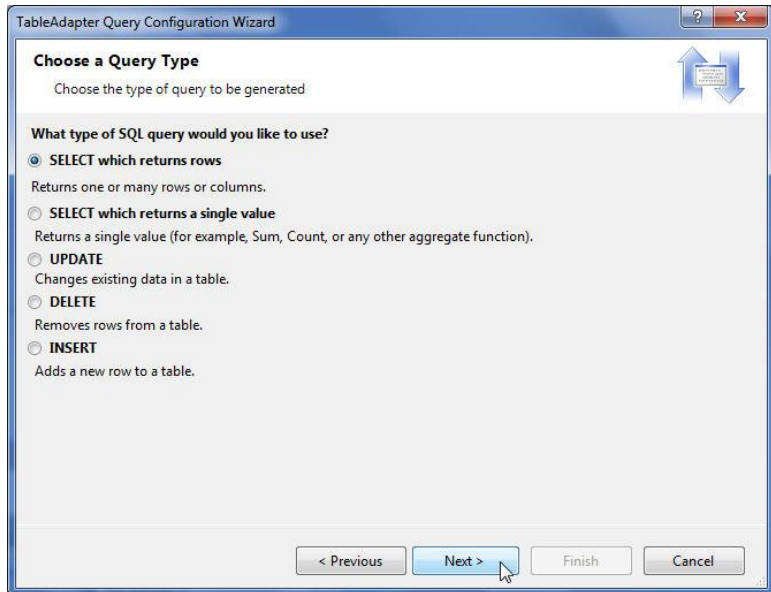
Berikut adalah contoh pembuatan query tableadapter yang akan mengambil data customers berdasarkan country dari database northwind.

1. Klik kanan Customers table pada designer Northwind.xsd kemudian pilih menu Add – Query.
2. Terdapat beberapa pilihan command type. Anda bisa menggunakan stored procedure yang sudah ada atau membuat stored procedure yang baru. Untuk kali ini pilih Use SQL Statements :



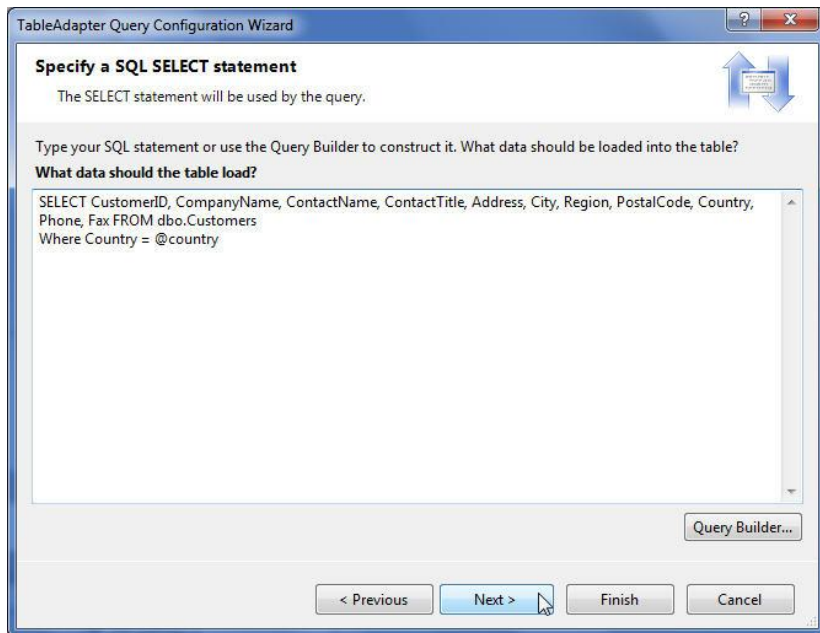
Gambar 9.26 Command Type yang disediakan wizard

3. Pilih Query Type “Select which returns rows” :



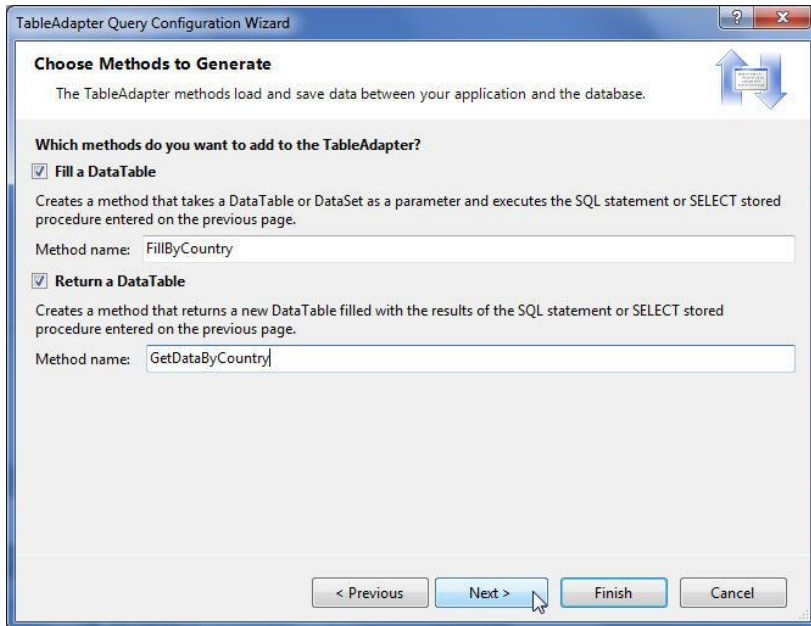
Gambar 9.27 Query Type yang disediakan wizard

4. Ketikkan query berikut :



Gambar 9.28 Select statement

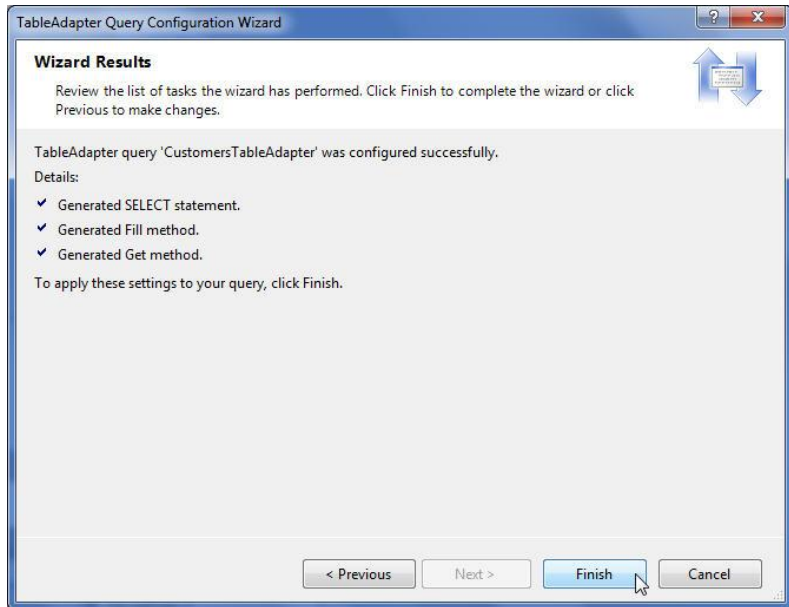
5. Berikan nama untuk kedua method yang nantinya akan digunakan seperti dibawah ini :



Gambar 9.29 Method yang disediakan wizard

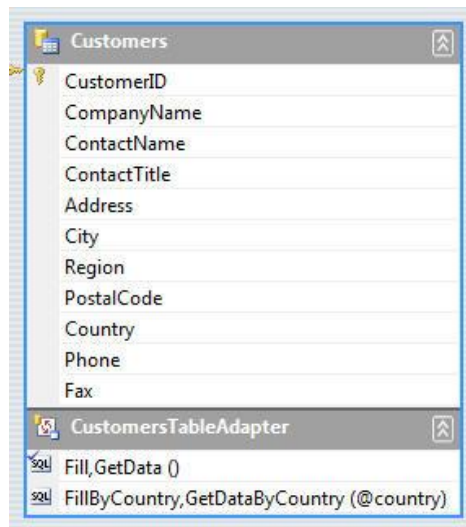
Dari wizard diatas tampak bahwa kita disediakan dua method yaitu Fill method yang digunakan untuk mengambil data dari database dan disimpan di dataset dan satu fungsi yang mengembalikan nilai sebuah datatable yang nantinya akan berisi data sesuai dengan query yang di definisikan sebelumnya.

6. Berikut informasi hasil wizard yang telah dilakukan :



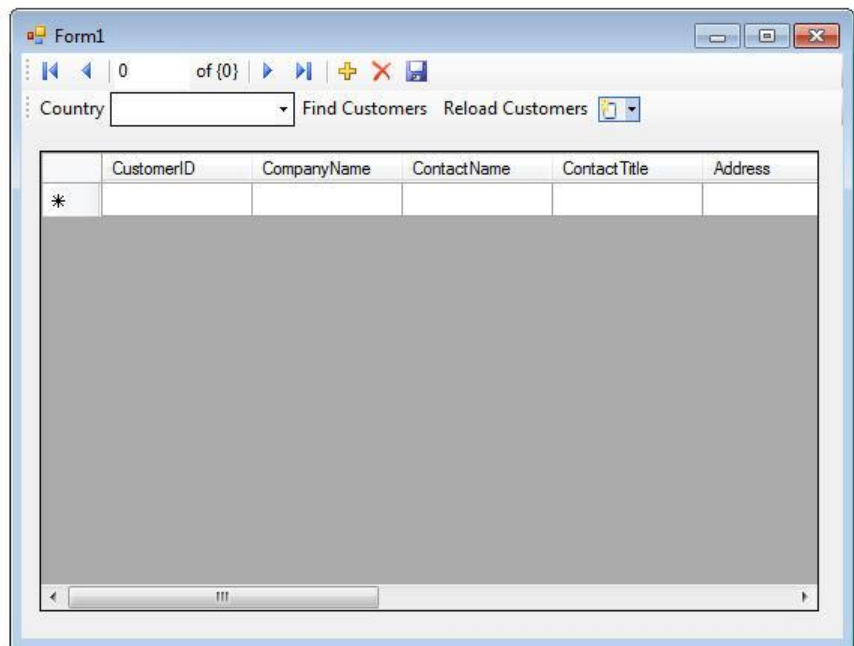
Gambar 9.30 Hasil wizard

7. Hasil akhir modifikasi Customers datatable :



Gambar 9.31 Hasil modifikasi Customers Datatable

8. Tambahkan ToolStrip pada form yang telah dibuat sebelumnya.
9. Tambahkan ToolStriplabel, ToolStripcombobox dan ToolStripbutton pada ToolStrip yang telah ditambahkan kedalam form. Atur tampilannya seperti dibawah ini :



Gambar 9.32 Form hasil modifikasi

10. Tambahkan perintah berikut pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

11. Buat prosedur berikut untuk load data country ke toolstripcombobox :

```
Private Sub LoadComboCustomers()  
  
    Using sqlConn As New  
        SqlConnection(My.Settings.NorthwindConnectionString)  
        Using sqlCmd As New SqlCommand  
            sqlCmd.Connection = sqlConn  
            sqlCmd.CommandType = CommandType.Text  
            sqlCmd.CommandText = "Select Distinct  
Country From Customers Where Country Is Not Null"  
  
            Dim reader As SqlDataReader = Nothing  
  
            Try  
                sqlConn.Open()  
  
                reader = sqlCmd.ExecuteReader  
                While reader.Read  
  
                    ToolStripComboBox1.Items.Add(reader(0).ToString)  
                End While  
  
                Catch sqlEx As SqlException  
                    MessageBox.Show(sqlEx.ToString)  
                Catch ex As Exception  
                    MessageBox.Show(ex.ToString)  
                Finally  
                    If reader IsNot Nothing Then  
reader.Close()  
                    End Try  
  
                End Using  
            End Using  
  
End Sub
```

12. Panggil prosedur di atas pada event Load form :

```
Private Sub Form1_Load(ByVal sender As System.Object, ByVal  
e As System.EventArgs) Handles MyBase.Load
```

```
Me.CustomersTableAdapter.Fill(Me.NorthwindDataSet.Customers  
)  
    LoadComboCustomers()
```

```
End Sub
```

13. Tambahkan kode berikut pada event click
ToolStripbutton1:

```
Private Sub ToolStripButton1_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
ToolStripButton1.Click
```

```
Me.CustomersTableAdapter.FillByCountry(Me.NorthwindDataSet.  
Customers, ToolStripComboBox1.Text)
```

```
End Sub
```

Kode diatas digunakan untuk melakukan proses filtering
data berdasarkan country yang dipilih dari toolstripcombobox.

14. Tambahkan kode berikut pada event click ToolStripbutton2
:

```
Private Sub ToolStripButton2_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
ToolStripButton2.Click
```

```
Me.CustomersTableAdapter.Fill(Me.NorthwindDataSet.Customers  
)
```

End Sub

Kode diatas digunakan untuk load data customers tanpa adanya proses filtering.

15. Modifikasi kode untuk button save seperti dibawah ini :

```
Private Sub CustomersBindingNavigatorSaveItem_Click(ByVal sender As System.Object, _  
ByVal e As System.EventArgs) _  
Handles CustomersBindingNavigatorSaveItem.Click  
  
    Dim confirm = MessageBox.Show("Save Data?",  
"Confirmation", _  
    MessageBoxButtons.YesNo, MessageBoxIcon.Question)  
    If confirm = Windows.Forms.DialogResult.Yes Then  
        Try  
            Me.Validate()  
            Me.CustomersBindingSource.EndEdit()  
  
Me.TableAdapterManager.UpdateAll(Me.NorthwindDataSet)  
  
            MessageBox.Show("Data Saved!")  
        Catch ex As Exception  
            MessageBox.Show(ex.Message)  
        End Try  
    End If
```

End Sub

Modifikasi kode tersebut dilakukan dengan menambahkan kotak dialog konfirmasi sebelum di save dan juga dengan adanya tambahan exception handling.

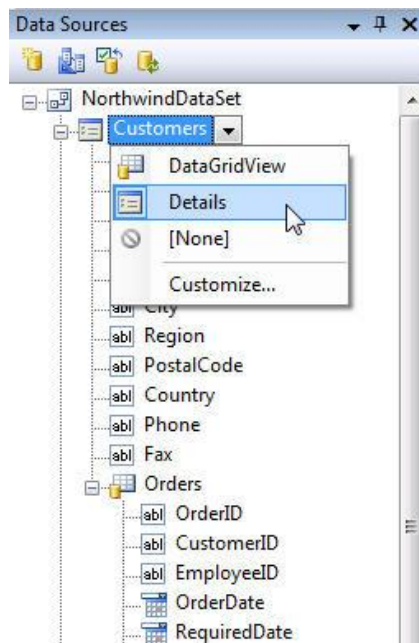
Master-Detail Typed DataSet

Pada contoh sebelumnya data yang digunakan hanya diambil dari satu tabel saja. Lalu bagaimana caranya kalau

kita ingin membuat form input dengan menggunakan lebih dari satu tabel, misalnya dua tabel, antara tabel customers dengan tabel orders, atau bahkan tiga tabel dengan tabel OrderDetail? Hal tersebut tidak sulit untuk dilakukan dengan menggunakan data wizard.

Berikut adalah contoh penggunaan Master Detail data dengan menggunakan TypedDataset dimana data yang digunakan berasal dari tabel Customers dan Orders.

1. Tambahkan satu form baru kedalam project yang sama.
2. Drag dan Drop datatable customers dan orders (tepat berada didalam node customers datatable) ke dalam form dengan merubah terlebih dahulu mode customer datatable menjadi details :

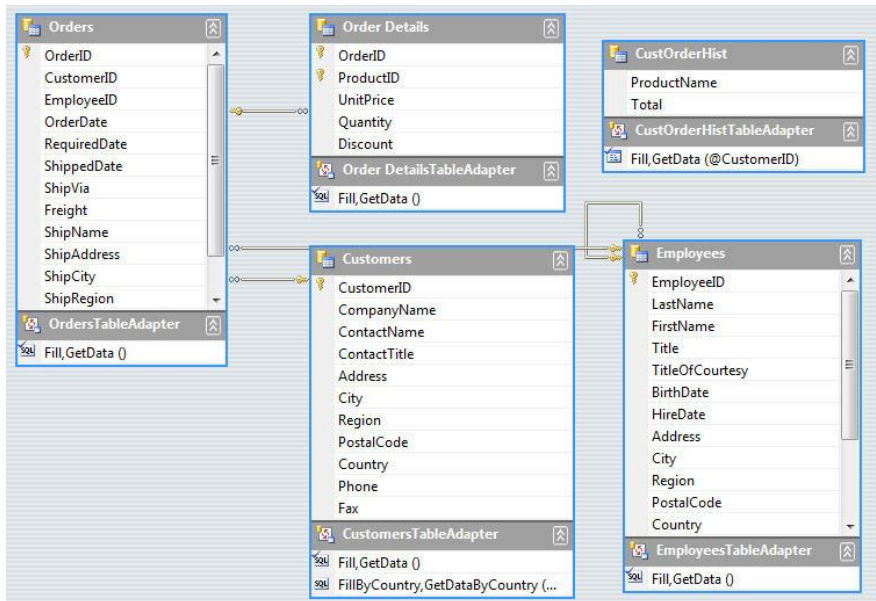


Gambar 9.33 Details mode datatable customers

3. Atur design form seperti pada gambar dibawah ini :

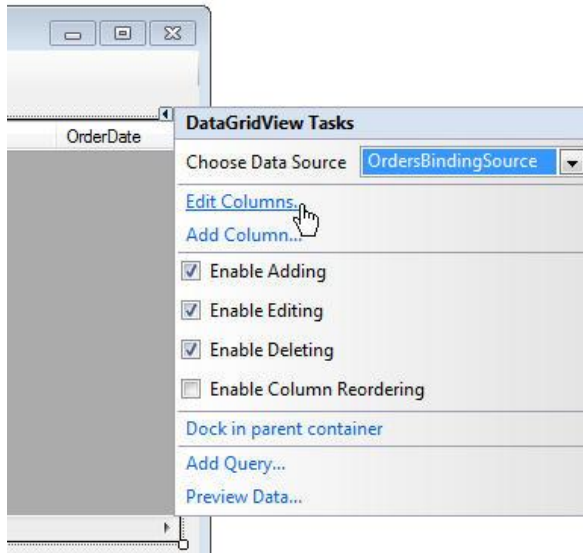
Gambar 9.34 Master Details form

4. Tambahkan tabel Employees dari jendela Server Explorer database Northwind ke dalam Northwind.xsd designer. Datatable employees yang nantinya di hasilkan akan digunakan sebagai sumber data Datagridviewcomboboxcolumn di datagridview orders untuk menampilkan nama employee :



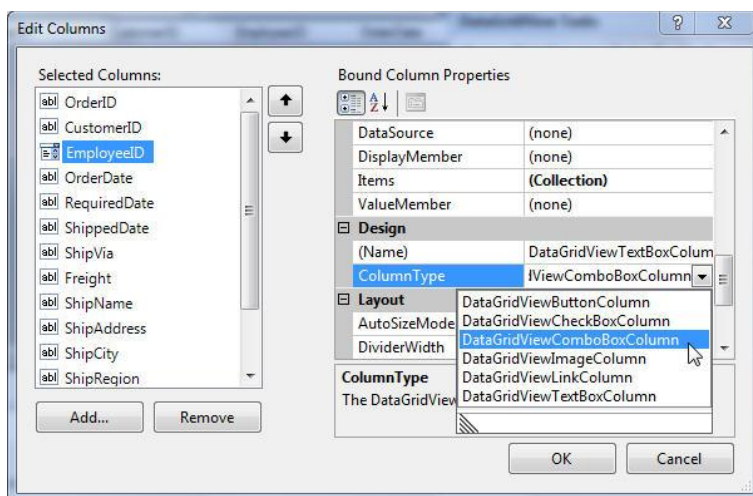
Gambar 9.35 Northwind Datasource

5. Lakukan modifikasi pada kolom EmployeeID di datagridview sehingga nantinya data yang ditampilkan bukan data employeeid melainkan firstname namun nilai yang disimpan tetap data employeeid :



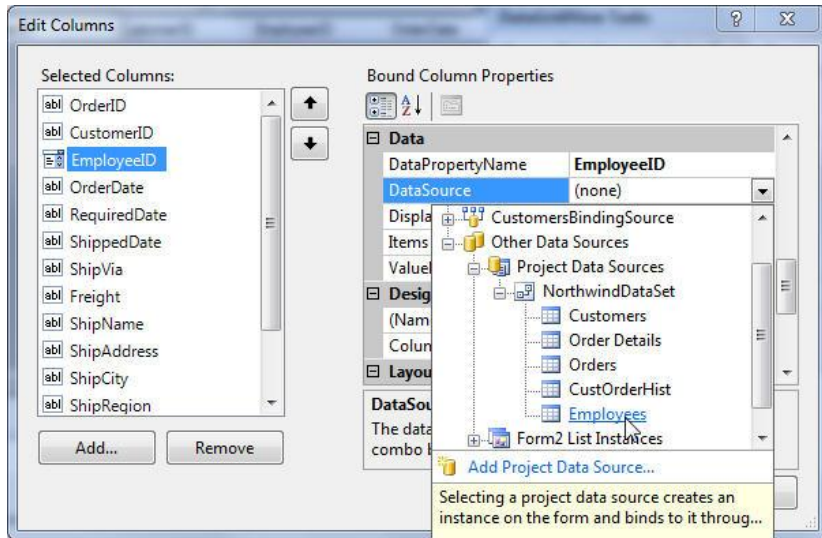
Gambar 9.36 Editing datagridview kolom

- Ubah ColumnType EmployeeID menjadi DataGridViewComboBoxColumn :



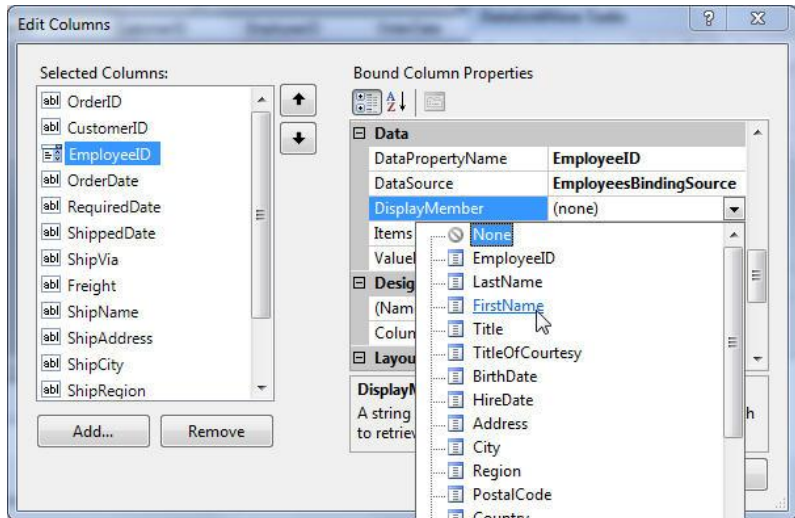
Gambar 9.37 DataGridViewComboBoxColumn

- Ubah DataSource kolom EmployeeID ke Employees table yang terdapat didalam Project Data Sources :



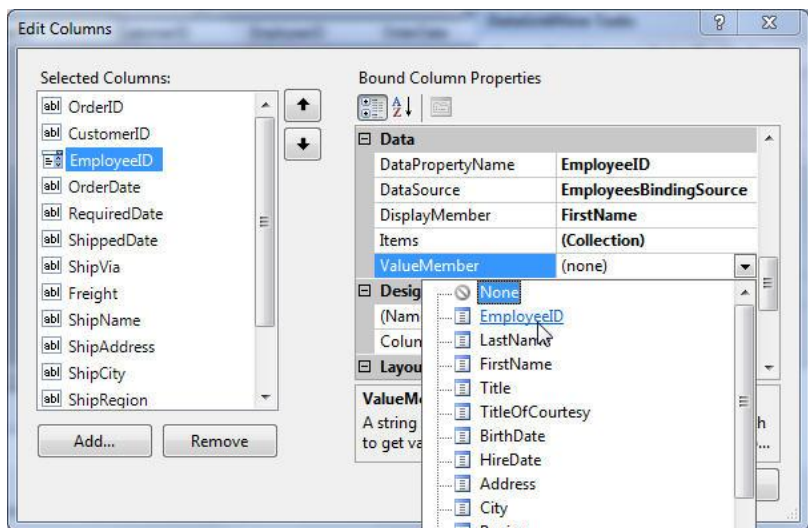
Gambar 9.38 Project Data Sources

- Ubah DisplayMember kolom EmployeeID menjadi FirstName :



Gambar 9.39 DisplayMember properti

9. Ubah ValueMember kolom EmployeeID menjadi EmployeeID :



Gambar 9.40 ValueMember properti

10. Buka source code file form tersebut kemudian lakukan modifikasi pada prosedur save button seperti berikut :

```
Private Sub CustomersBindingNavigatorSaveItem_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) _ Handles CustomersBindingNavigatorSaveItem.Click

    Dim confirm = MessageBox.Show("Save Data?", _
    "Confirmation", _
    MessageBoxButtons.YesNo, MessageBoxIcon.Question)

    If confirm = Windows.Forms.DialogResult.Yes Then
        Try
            Me.Validate()

            Me.CustomersBindingSource.EndEdit()
            Me.OrdersBindingSource.EndEdit()

            Me.TableAdapterManager.UpdateAll(Me.NorthwindDataSet)

            MessageBox.Show("Data Saved!")
        Catch ex As Exception
            MessageBox.Show(ex.Message)
        End Try
    End If

End Sub
```

11. Form akan tampak seperti berikut apabila program di jalankan :

OrderID	CustomerID	EmployeeID	OrderDate
10643	ALFKI	Michael	8/25/1997
10692	ALFKI	Margaret	10/3/1997
10702	ALFKI	Margaret	10/13/1997
10835	ALFKI	Nancy	1/15/1998
10952	ALFKI	Nancy	3/16/1998
11011	ALFKI	Janet	4/9/1998

Gambar 9.41 Form Master Details

Membaca Record TypedDataSet

Pada pembahasan sebelumnya mengenai UntypedDataSet telah diberikan contoh bagaimana caranya membaca record yang terdapat didalam DataTable UntypedDataSet.

Berikut adalah contoh kode program bagaimana caranya membaca dan menampilkan record yang terdapat didalam DataTable TypedDataSet :

1. Tambahkan satu form kedalam project yang sama.
2. Tambahkan button, beri nama btnGetCustomers.
3. Tambahkan Listbox kedalam form.
4. Buka source code form, tambahkan kode berikut pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

5. Deklarasikan class level variabel berikut :

```
Private dsNorthwind As New NorthwindDataSet
```

```
Private taCust As New  
NorthwindDataSetTableAdapters.CustomersTableAdapter
```

```
Private taOrdHist As New  
NorthwindDataSetTableAdapters.CustOrderHistTableAdapter
```

6. Tambahkan kode berikut pada event Load form :

```
Private Sub Form3_Load(ByVal sender As Object, ByVal e As  
System.EventArgs) Handles Me.Load
```

```
    taCust.Fill(dsNorthwind.Customers)
```

```
End Sub
```

7. Tambahkan kode berikut pada event click
btnGetCustomers :

```
Private Sub btnGetCustomers_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) _  
Handles btnGetCustomers.Click
```

```
    ListBox1.Items.Clear()
```

```
    Dim custRows As NorthwindDataSet.CustomersRow  
    Dim rowsColl As NorthwindDataSet.CustomersDataTable  
    = dsNorthwind.Customers
```

```
    For Each custRows In rowsColl  
        ListBox1.Items.Add(custRows.CustomerID & ", " &  
custRows.CompanyName)  
    Next
```

```
End Sub
```

Dari kode diatas dapat kita lihat perbedaan cara akses datarow untyped dataset dengan typed dataset. Pada typed dataset semua member atau object yang terdapat didalam dataset tersebut akan muncul di dalam jendela intellisense sehingga memudahkan kita untuk mengaksesnya tanpa harus diketik secara manual seperti yang dilakukan pada untyped dataset.

Stored Procedure Typed DataSet

Pada wizard sebelumnya kita sudah menambahkan sebuah stored procedure CustOrderHist. Stored procedure tersebut digunakan untuk menampilkan data product name beserta quantity nya yang pernah dipesan oleh customerid tertentu. Terdapat dua method yang disediakan untuk eksekusi stored procedure, yaitu Fillxxx method dan GetDataxxx method. GetDataxxx method merupakan fungsi yang mengembalikan return value berupa datatable yang berisi data berdasarkan query yang telah didefinisikan.

Berikut adalah contoh kode program bagaimana caranya menggunakan method FillData tersebut :

1. Tambahkan button ke dalam form yang sama beri nama btnExecGetData.
2. Tambahkan DataGridView ke dalam form.
3. Ketikkan kode berikut untuk event click btnExecSPFill:

```
Private Sub btnExecSPFill_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
Handles btnExecSPFill.Click

    Dim custID = InputBox("Masukkan CustomerID", _
& "Find Customer By ID")

    taOrdHist.Fill(dsNorthwind.CustOrderHist, custID)
```

```
        DataGridView1.DataSource =  
dsNorthwind.CustOrderHist
```

```
End Sub
```

Method Fill diatas membutuhkan input parameter berupa customerid yang di supply dari InputBox.

Berikut adalah contoh kode program untuk eksekusi method GetData dari stored procedure CustOrderHist yang terdapat didalam Typed DataSet :

1. Tambahkan button kedalam form yang sama, beri nama btnExecGetData dan ketikkan kode berikut pada event click :

```
Private Sub btnExecGetData_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) _  
    Handles btnExecGetData.Click  
  
    Dim custID = InputBox("Masukkan CustomerID", _  
        & "Find Customer By ID")  
  
    Dim orderHist As  
NorthwindDataSet.CustOrderHistDataTable  
    orderHist = taOrdHist.GetData(custID)  
  
    DataGridView1.DataSource = orderHist  
  
End Sub
```

Method GetData merupakan sebuah fungsi yang mengembalikan nilai datatable.

Filtering Typed DataSet Data

Apabila tabel yang digunakan didalam typed dataset memiliki kolom primary key maka otomatis akan disediakan sebuah method berupa fungsi yang akan mengembalikan nilai datarow. Nama method tersebut disesuaikan dengan nama kolom yang bertindak sebagai primary key. Misalnya pada tabel customers memiliki kolom primary key berupa kolom CustomerID, maka di dalam typed dataset tersebut akan diberikan sebuah method dengan nama FindByCustomerID yang akan meminta sebuah input parameter berupa nilai customerid.

Berikut adalah contoh penggunaan method tersebut :

1. Tambahkan button kedalam form yang sama, beri nama btnFindByID dan ketikkan kode berikut pada event click button tersebut :

```
Private Sub btnFindByID_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnFindByID.Click

    Dim custID = InputBox("Masukkan CustomerID", _
        & "Find Customer By ID")

    Dim custRow As NorthwindDataSet.CustomersRow = _
        dsNorthwind.Customers.FindByCustomerID(custID)

    If custRow IsNot Nothing Then
        Dim sb As New System.Text.StringBuilder

        sb.AppendFormat("Company Name : {0}",
custRow.CompanyName)
        sb.Append(vbNewLine)
        sb.AppendFormat("City : {0}", custRow.City)
        sb.Append(vbNewLine)
        sb.AppendFormat("Country : {0}",
custRow.Country)
        sb.Append(vbNewLine)
```

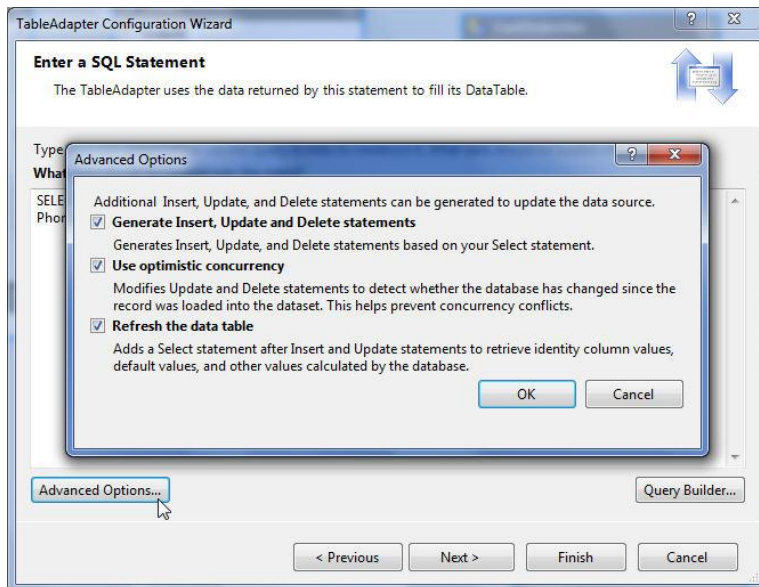
```
        MessageBox.Show(sb.ToString)
    End If
```

```
End Sub
```

Hasil eksekusi fungsi tersebut akan mengembalikan data berupa CustomersRow.

Typed DataSet Insert Method

Apabila kita melakukan setting opsi untuk generate method pada typed dataset yang digunakan maka otomatis akan dihasilkan method untuk melakukan manipulasi data seperti Insert, Update dan Delete. Setting opsi tersebut dilakukan dengan memilih setting opsi Advanced Option seperti yang dapat kita lihat pada gambar dibawah ini :



Gambar 9.42 Setting Advanced Option Typed DataSet

Anda dapat melihat setting tersebut dengan melakukan klik kanan salah satu datatable yang terdapat didalam Northwind.xsd designer kemudian pilih menu Configure. Apabila dataset tersebut hanya digunakan untuk menampilkan data saja maka anda dapat menghilangkan opsi Generate Insert, Update and Delete Statements.

Berikut adalah contoh penggunaan Insert method untuk customers tabel :

1. Tambahkan button, beri nama btnAddUsingInsert, dan ketikkan kode berikut untuk event click button tersebut :

```
Private Sub btnAddUsingInsert_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
    Handles btnAddUsingInsert.Click

    Try
        taCust.Insert("NTV1", "Native-Enterprise 1", _
Nothing, Nothing, Nothing, _
& "Bandung", Nothing, Nothing, "Indonesia", Nothing, _
Nothing)

        MessageBox.Show("Data Saved!")
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    End Try

End Sub
```

Untuk kolom yang tidak akan di supply nilainya anda cukup memberikan nilai tersebut dengan Nothing. Jumlah kolom yang dijadikan input parameter tergantung dari query yang didefinisikan pada wizard.

Typed DataSet Insert Row Method

Tidak seperti contoh diatas yang dapat langsung insert data ke tabel customers, method lainnya yang dapat kita gunakan untuk menambah data baru yaitu dengan menggunakan insert row method yang harus didefinisikan terlebih dahulu datarow nya.

Method insert row berguna untuk menyimpan data sementara di dalam dataset sebelum data tersebut disimpan kedalam database.

Setelah datanya ditampung didalam dataset tersebut maka untuk melakukan update semua perubahan yang terjadi di dataset ke database dilakukan lewat pemanggilan fungsi Update dari object TableAdapter.

Berikut adalah contoh penggunaan method tersebut untuk menambahkan data baru kedalam tabel customers :

1. Tambahkan button kedalam form yang sama, beri nama btnInsertRow dan ketikkan kode berikut pada event click button tersebut :

```
Private Sub btnInsertRow_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
Handles btnInsertRow.Click

    Dim newRow As NorthwindDataSet.CustomersRow = _
        dsNorthwind.Customers.NewCustomersRow

    newRow.CustomerID = "NTV2"
    newRow.CompanyName = "Native-Enterprise 2"
    newRow.City = "Bandung"
    newRow.Country = "Indonesia"

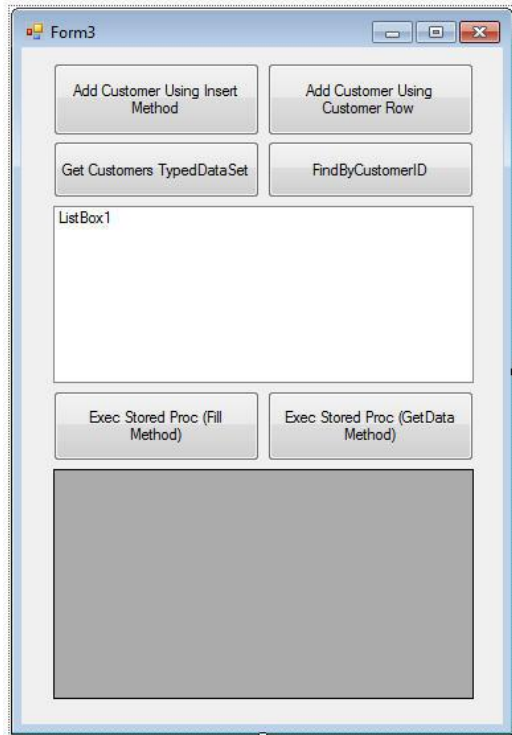
    Try
        dsNorthwind.Customers.AddCustomersRow(newRow)
        taCust.Update(dsNorthwind.Customers)

        MessageBox.Show("Data Saved!")
    End Try
End Sub
```

```
Catch sqlEx As SqlException
    MessageBox.Show(sqlEx.ToString)
Catch ex As Exception
    MessageBox.Show(ex.ToString)
End Try
```

End Sub

Berikut adalah hasil akhir design form yang digunakan didalam lab ini :



Gambar 9.43 Design Form

BAB 10

Crystal Reports

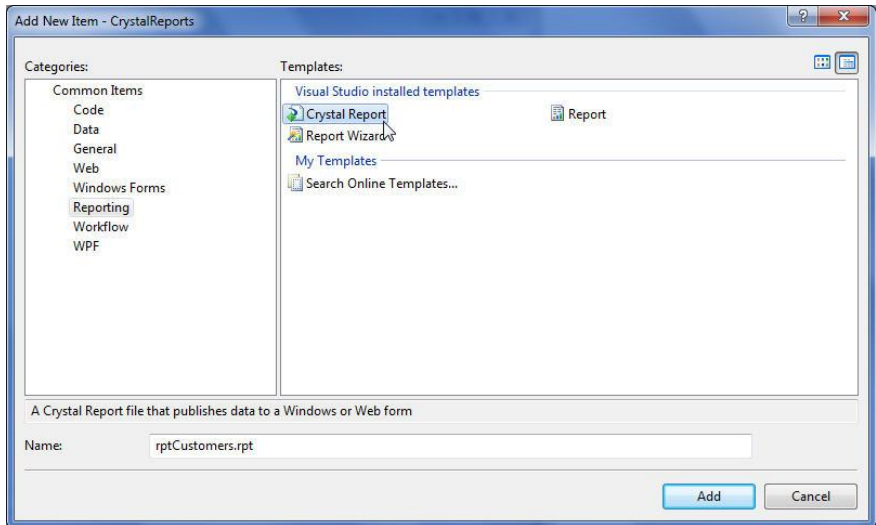
Crystal Reports merupakan salah satu reporting tools yang disediakan mulai di .NET versi pertama keluar yaitu .NET versi 1.0. Sebelum .NET muncul crystal reports merupakan reporting tools yang harus diinstal secara terpisah dan di refrensi secara manual library nya apabila ingin digunakan. Hal tersebut sudah tidak berlaku lagi semenjak kemunculan .NET pertama sehingga crystal reports sudah di include kan didalam Visual Studio.NET dan tidak perlu diinstal secara terpisah.

Crystal Reports yang terdapat didalam Visual Studio 2008 merupakan crystal reports versi 2008 Basic Edition. Penggunaan crystal reports pada versi sebelum .NET muncul sangat berbeda sekali. .NET framework menyediakan library yang berbeda dengan library crystal reports yang biasa digunakan pada Visual Studio 6 dengan VB 6 nya.

Crystal Reports Item

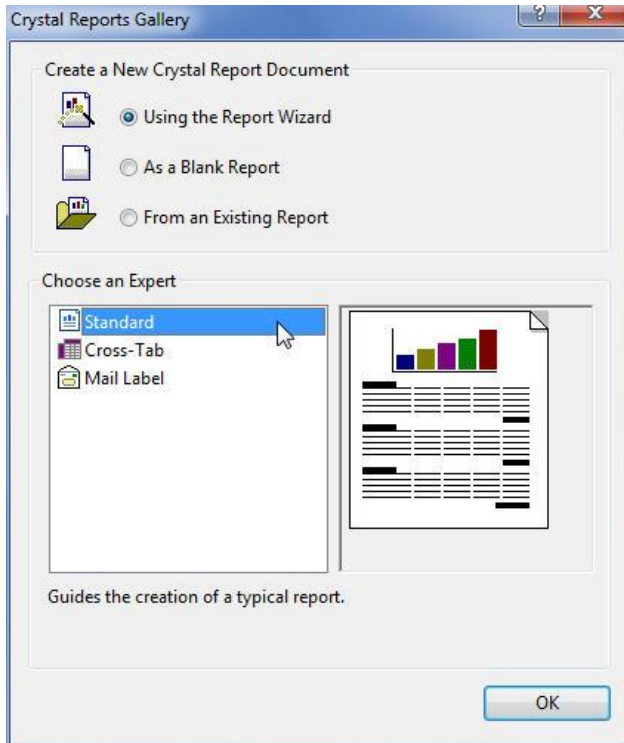
Untuk membuat crystal reports pada visual studio 2008 terlebih dahulu kita harus menambahkan item baru kedalam project. Berikut adalah contoh bagaimana menambahkan item crystal reports kedalam project :

1. Buat sebuah Windows Forms Application project template baru, beri nama Crystal Reports.
2. Klik kanan project, pilih menu Add – New Item. Pada kotak dialog Add New Item pilih Crystal Reports dan beri nama rptCustomers.rpt.



Gambar 10.1 Crystal Reports item

3. Terdapat beberapa pilihan untuk membuat crystal reports seperti pada gambar di bawah ini :

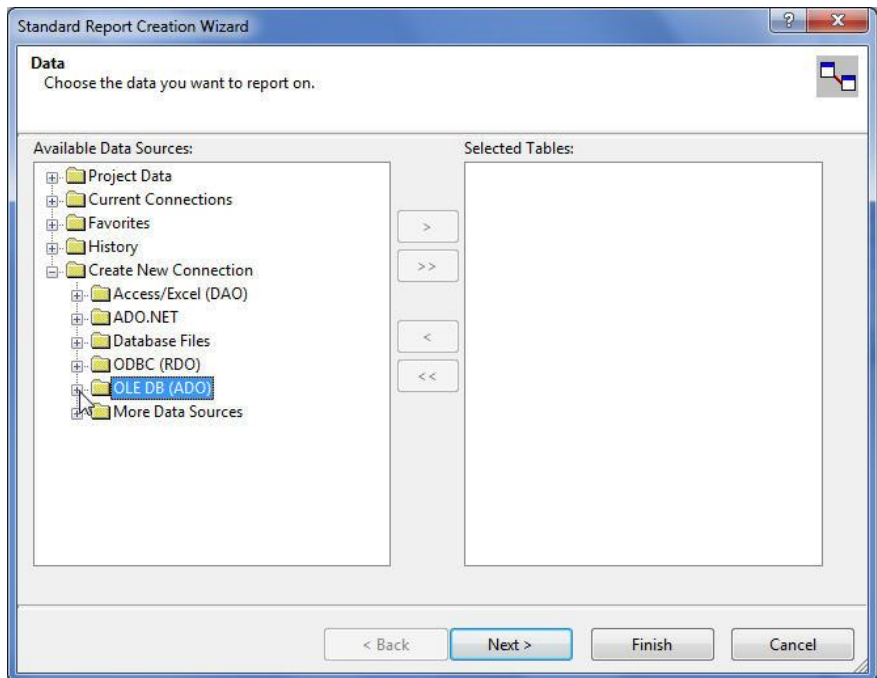


Gambar 10.2 Crystal Reports Gallery

Crystal Reports memberikan beberapa pilihan yaitu wizard yang akan dipandu dengan jendela-jendela wizard yang nantinya report yang dihasilkan akan secara otomatis di design oleh visual studio. Selain itu diberikan juga pilihan blank report apabila kita ingin design dengan template report designer yang kosong atau menggunakan report yang telah ada dari pilihan from an existing report.

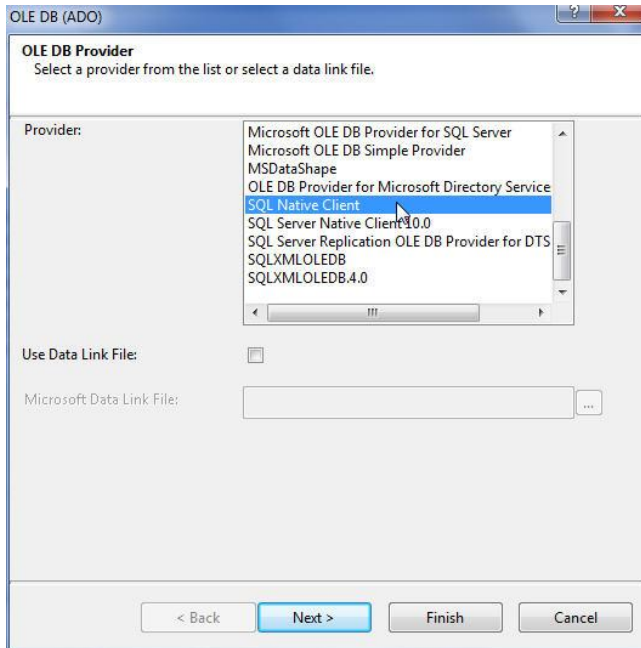
Model report juga disediakan tiga pilihan yaitu Standard, Cross Tab dan Mail Label. Untuk kali ini pilih model Standard.

4. Pilih OLE DB (ADO) data source :



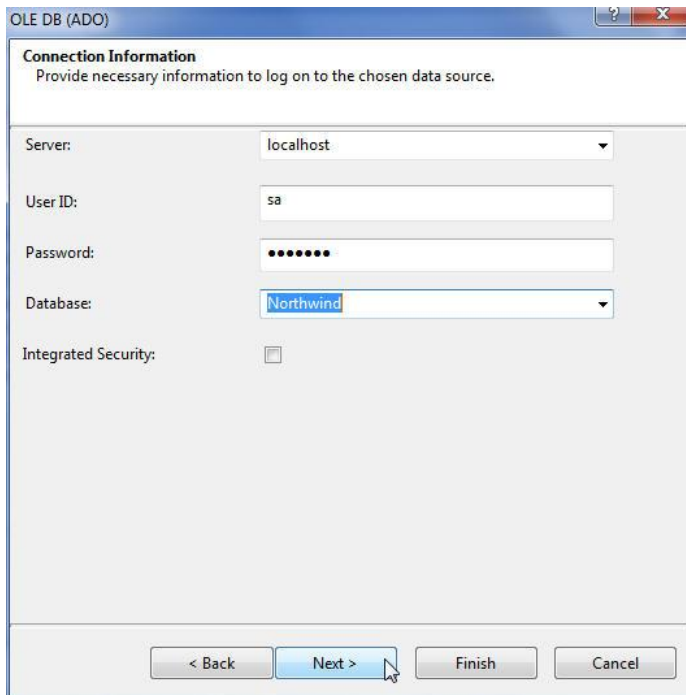
Gambar 10.3 OLE DB source

5. Pilih SQL Native Client untuk database SQL Server 2005 atau versi di atasnya.



Gambar 10.4 SQL Native Client provider

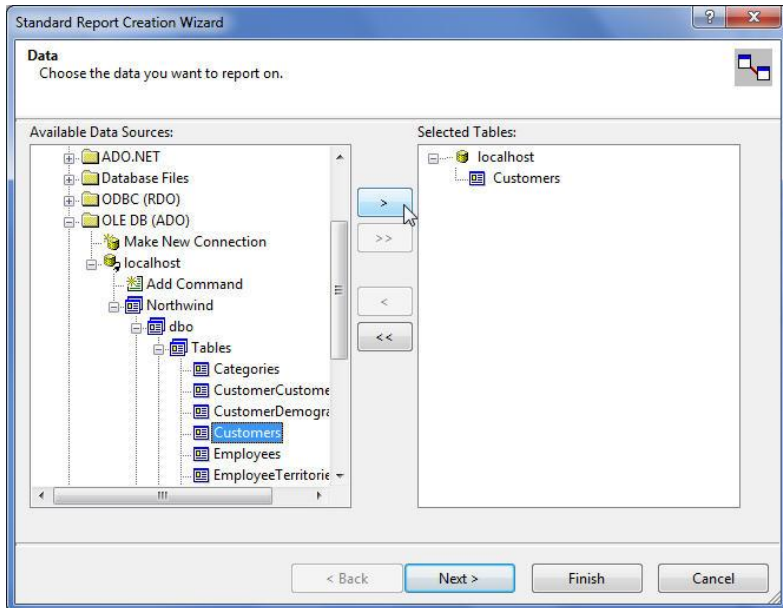
6. Koneksikan wizard ke database northwind sesuai dengan setting autentikasi yang dimiliki.



The image shows a Windows dialog box titled "OLE DB (ADO)". Inside, there is a section titled "Connection Information" with the instruction "Provide necessary information to log on to the chosen data source." Below this, there are four input fields: "Server:" with a dropdown menu showing "localhost"; "User ID:" with a text box containing "sa"; "Password:" with a text box containing ten dots; and "Database:" with a dropdown menu showing "Northwind". Below these fields is a checkbox labeled "Integrated Security:" which is currently unchecked. At the bottom of the dialog, there are four buttons: "< Back", "Next >" (which is highlighted with a mouse cursor), "Finish", and "Cancel".

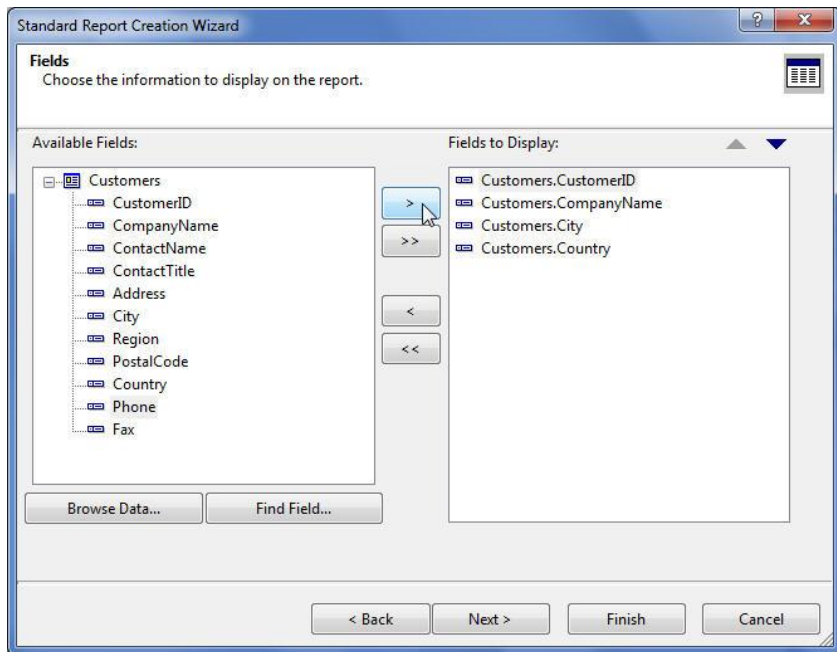
Gambar 10.5 Informasi koneksi ke database server

7. Pilih tabel Customers sebagai sumber data



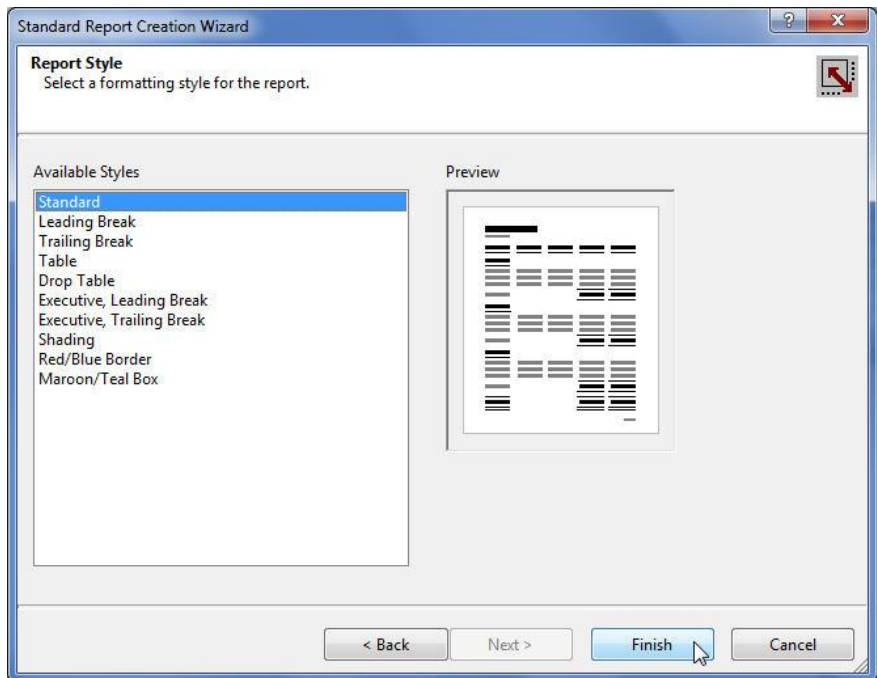
Gambar 10.6 Tabel Customers sebagai sumber data.

8. Pilih kolom-kolom berikut yang akan ditampilkan di report.



Gambar 10.7 Fields tabel customers yang dipilih.

9. Untuk jendela wizard Grouping dan Record Selection dibuat datanya kosong.
10. Pilih mode Standard untuk style nya :



Gambar 10.8 Report Style

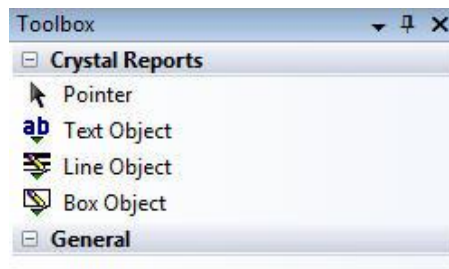
11. Berikut adalah report hasil wizard :

Section1 (Report Header)				
Section2 (Page Header)				
Customers Report				Print Date
ID	Name	City	Country	
Section3 (Details)				
CustomerID	CompanyName	City	Country	
Section4 (Report Footer)				
Section5 (Page Footer)				
Page Number				

Gambar 10.9 Report hasil wizard

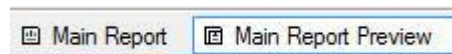
Tentunya kita dapat merubah design tersebut, seperti yang penulis lakukan dengan menambahkan control TextBox untuk memberi judul report, atau merubah lokasi penempatan kolom atau object report lainnya seperti PrintDate dan PageNumber object.

Berikut adalah object crystal reports yang dapat ditambahkan pada crystal reports designer :



Gambar 10.10 Toolbox Crystal Reports Object

12. Untuk melihat preview hasil design dapat dilakukan dengan klik tombol Main Report Preview pada bagian bawah crystal reports designer :



Gambar 10.11 Main Report Preview

13. Hasil preview report dapat dilihat pada gambar dibawah ini :

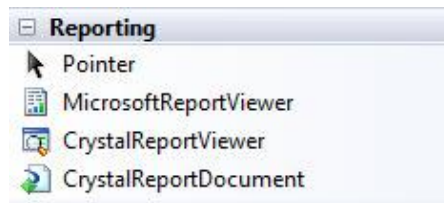
9/6/2009

<u>Country</u>	<u>CustomerID</u>	<u>CompanyName</u>	<u>City</u>
Argentina			
Argentina	CACTU	Cactus Comidas para llevar	Buenos Aires
Argentina	OCEAN	Océano Atlántico Ltda.	Buenos Aires
Argentina	RANCH	Rancho grande	Buenos Aires
Argentina			
Austria			
Austria	PICCO	Piccolo und mehr	Salzburg
Austria	ERNSH	Ernst Handel	Graz
Austria			
Belgium			
Belgium	MAISD	Maison Dewey	Bruxelles
Belgium	SUPRD	Suprêmes délices	Charleroi
Belgium			

Gambar 10.12 Preview report

Menampilkan Crystal Reports dari Form

Untuk memanggil crystal reports dari form terlebih dahulu harus ditambahkan CrystalReportViewer control dari Toolbox Reporting kedalam form.



Gambar 10.13 Reporting Toolbox

Sumber data yang dapat digunakan untuk menampilkan report cukup beragam salah satunya yaitu DataTable atau DataView. DataTable tersebut digunakan untuk menampung data secara temporary dari database server.

Berikut adalah contoh penggunaan CrystalReportViewer untuk menampilkan CrystalReports dan DataTable sebagai sumber datanya :

1. Tambahkan CrystalReportViewer control kedalam form yang ada dari reporting toolbox.
2. Tambahkan button kedalam form tersebut, beri nama btnLoadCustomers.
3. Ketikkan perintah berikut pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

4. Deklarasikan class level variabel berikut :

```
Private rpt As New rptCustomers
Private dt As New DataTable
```

5. Ketikkan prosedur berikut yang digunakan untuk load data dari database yang disimpan di DataView :

```
Private Function LoadDataTableCustomersReport() As
DataTable
```

```
    Using sqlConn As New SqlConnection( _
        My.Settings.NorthwindConnectionString)

        Using sqlDa As New SqlDataAdapter( _
            "Select * From Customers", sqlConn)
            sqlDa.Fill(dt)

        Return dt

    End Using
```

End Using

End Function

6. Ketikkan kode berikut untuk melakukan setting crystal report data source dan autentikasi yang dibutuhkan untuk terhubung ke database :

```
Private Sub SetReportSource(ByVal ViewSource As DataView)
```

```
    rpt.SetDatabaseLogon("sa", "sql2005", "localhost",  
"northwind")  
    rpt.SetDataSource(ViewSource)  
    CrystalReportViewer1.ReportSource = rpt
```

End Sub

SetDatabaseLogon method digunakan sebagai autentikasi ke database server. Input paramater yang dibutuhkan yaitu user id, password, instance server dan nama database. Sudah pasti kita harus memikirkan penggunaan password diatas yang tidak terenkripsi. Kita dapat menyimpannya di registry atau file yang sudah dienkripsi.

7. Ketikkan kode berikut pada event click dari btnLoadCustomers :

```
Private Sub btnLoadCustomers_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnLoadCustomers.Click
```

```
    If dt.Rows.Count = 0 Then
```

```
        SetReportSource(LoadDataTableCustomersReport.DefaultView)  
    Else  
        dt.DefaultView.RowFilter = ""
```

```
SetReportSource(dt.DefaultView)
End If
```

```
End Sub
```

Filtering Data Report

Untuk melakukan filtering data di crystal reports kita dapat memanfaatkan fitur RowFilter yang dimiliki oleh DataView. Dengan menggunakan properti RowFilter maka data akan di filter sesuai dengan kriteria pencarian data sehingga data yang ditampilkan di crystal reports akan sesuai dengan data yang terdapat didalam DataView.

Berikut adalah contoh penggunaan properti RowFilter dari DataView untuk melakukan filtering data :

1. Tambahkan combobox kedalam form yang sama.
2. Buat prosedur berikut untuk menampilkan data country dari tabel customers ke combobox :

```
Private Sub LoadCustomersCountry()

    Using sqlConn As New SqlConnection( _
        My.Settings.NorthwindConnectionString)

        Using sqlCmd As New SqlCommand
            sqlCmd.Connection = sqlConn
            sqlCmd.CommandType = CommandType.Text
            sqlCmd.CommandText = _
                "Select Distinct Country From Customers"

            Dim dr As SqlDataReader = Nothing

            Try
                sqlConn.Open()
                dr = sqlCmd.ExecuteReader
                While dr.Read
                    ComboBox1.Items.Add(dr(0).ToString)
                End While
            Catch ex As Exception
            End Try
        End Using
    End Using
```

```

        End While
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    Finally
        If dr IsNot Nothing Then dr = Nothing
        If sqlConn IsNot Nothing Then
sqlConn.Dispose()
        End Try
    End Using

End Using

End Sub

```

3. Panggil prosedur tersebut didalam event Load form :

```

Private Sub Form1_Load(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles MyBase.Load

    LoadCustomersCountry()

End Sub

```

4. Ketikkan kode berikut pada event SelectedIndexChanged combobox :

```

Private Sub ComboBox1_SelectedIndexChanged(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
Handles ComboBox1.SelectedIndexChanged

    If ComboBox1.Text <> "" Then
        If dt.Rows.Count = 0 Then

```

```

LoadDataTableCustomersReport.DefaultView.RowFilter = _
    "Country = '" & ComboBox1.Text & "'"
Else
    dt.DefaultView.RowFilter = _
    "Country = '" & ComboBox1.Text & "'"
End If
SetReportSource(dt.DefaultView)
End If

End Sub

```

Untuk mendapatkan object DataView kita dapat memanfaatkannya secara langsung dengan menggunakan properti DefaultView dari DataTable yang digunakan.

Sorting Data Report

DataView juga menyediakan fitur untuk melakukan sorting data. Properti Sort digunakan untuk sorting data secara Ascending atau Descending, sehingga nantinya data yang ditampilkan di report juga akan ikut tersorting.

Berikut adalah contoh kode program untuk melakukan sorting data report dengan menggunakan properti Sort dari DataView :

1. Tambahkan combobox kedalam form yang sama.
2. Tambahkan item berikut kedalam Items Collection combobox : "" (nilai string kosong), CustomerID, CompanyName, City, Coutry.
3. Ketikkan kode berikut pada event SelectedIndexChanged combobox :

```

Private Sub ComboBox2_SelectedIndexChanged(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
Handles ComboBox2.SelectedIndexChanged

```

```

If dt.Rows.Count = 0 Then
    LoadDataTableCustomersReport.DefaultView.RowFilter =
    ""
End If

dt.DefaultView.Sort = ComboBox2.Text
SetReportSource(dt.DefaultView)

End Sub

```

Untuk mengubah kembali data yang telah di sorting ke kondisi semula atur nilai dari properti Sort ke "" (string kosong).

Preview form untuk filtering dan sorting report dapat dilihat pada gambar dibawah ini :

The screenshot shows a Windows application window titled 'Form1'. At the top, there are two buttons: 'Load All Customers' and 'Load Customers By Country :'. To the right of the second button is a dropdown menu currently set to 'USA'. Further right is another dropdown menu labeled 'Order By :', which is currently set to 'City'. Below these controls is a toolbar with various icons. The main area of the window is titled 'Main Report' and contains a table titled 'Customers Report'. The table has four columns: 'ID', 'Name', 'City', and 'Country'. The data is sorted by city. At the bottom of the window, there is a status bar showing 'Current Page No.: 1', 'Total Page No.: 1', and 'Zoom Factor: 100%'.

ID	Name	City	Country
RATTC	Rattlesnake Canyon Grocery	Albuquerque	USA
OLDWO	Old World Delicatessen	Anchorage	USA
SAVEA	Save-a-lot Markets	Boise	USA
THECR	The Cracker Box	Butte	USA
HUNGC	Hungry Coyote Import Store	Elgin	USA
GREAL	Great Lakes Food Market	Eugene	USA
TRAIH	Trail's Head Gourmet Provisioner	Kirkland	USA
SPLIR	Split Rail Beer & Ale	Lander	USA
LONEP	Lonesome Pine Restaurant	Portland	USA

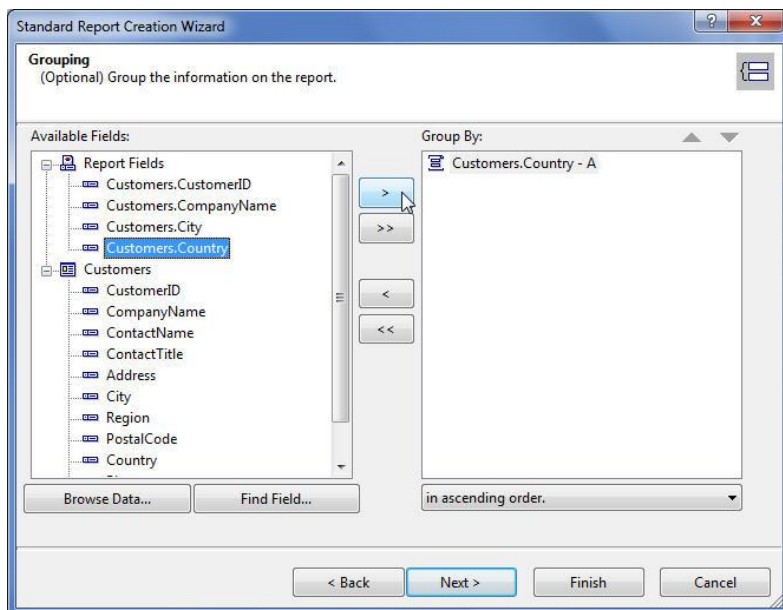
Gambar 10.14 Filtering dan Sorting report

Grouping Report

Crystal reports menyediakan fitur untuk melakukan grouping atau pengelompokan data berdasarkan kolom tertentu. Grouping dapat dilakukan pada saat proses wizard pembuatan report atau modifikasi report yang sudah ada.

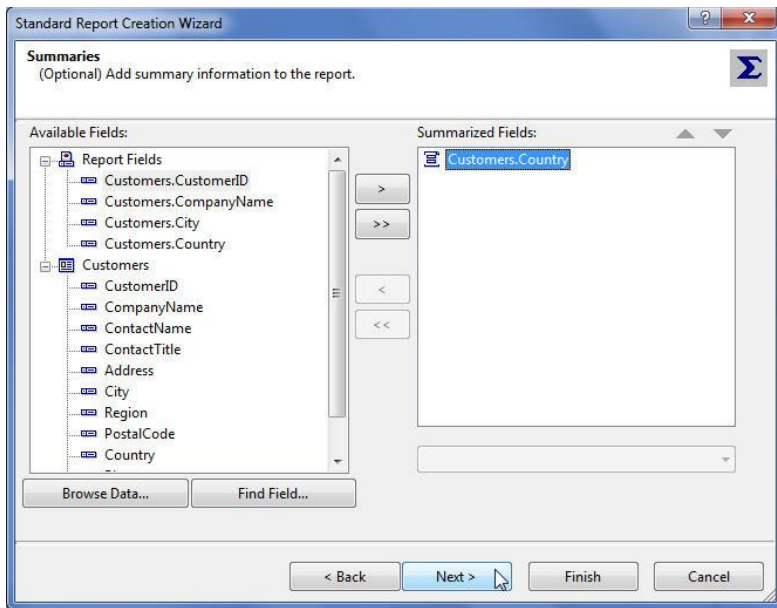
Berikut adalah contoh bagaimana membuat grouping pada crystal reports :

1. Tambahkan form baru kedalam project yang sama.
2. Tambahkan crystal reports baru kedalam project, beri nama rptCustGrouping.rpt.
3. Lakukan proses wizard seperti yang telah dilakukan pada lab pertama.
4. Lakukan setting grouping pada jendela wizard grouping dengan kolom Country seperti dibawah ini :



Gambar 10.15 Grouping Report

5. Lakukan Summarize fields untuk kolom Country :



Gambar 10.16 Summarize fields report

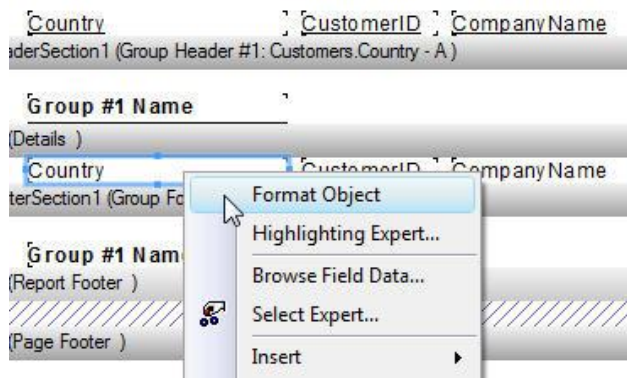
6. Pada Record Selection wizard biarkan nilainya kosong.
7. Pilih mode Standard untuk style report nya.
8. Preview report hasil grouping tampak seperti dibawah ini :

9/6/2009

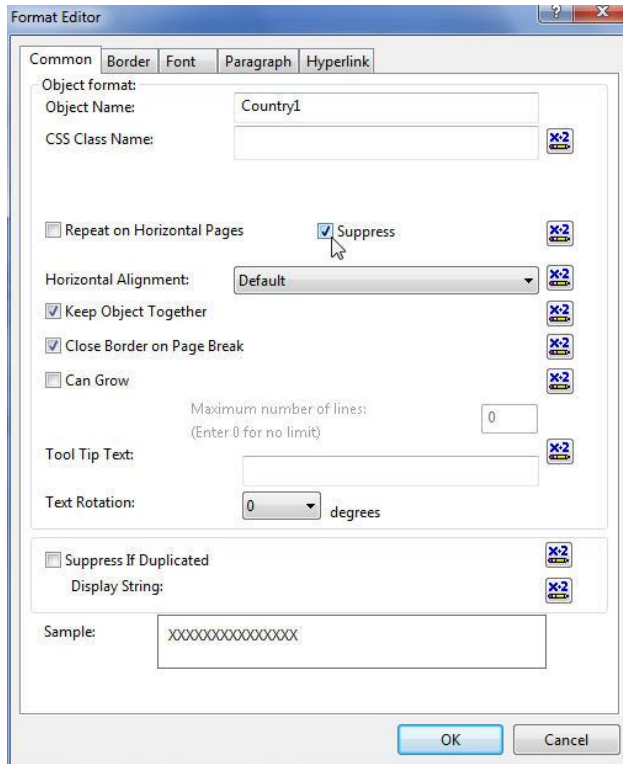
<u>Country</u>	<u>CustomerID</u>	<u>CompanyName</u>	<u>City</u>
Argentina			
Argentina	CACTU	Cactus Comidas para llevar	Buenos Aires
Argentina	OCEAN	Océano Atlántico Ltda.	Buenos Aires
Argentina	RANCH	Rancho grande	Buenos Aires
Argentina			
Austria			
Austria	PICCO	Piccolo und mehr	Salzburg
Austria	ERNSH	Ernst Handel	Graz
Austria			
Belgium			
Belgium	MAISD	Maison Dewey	Bruxelles
Belgium	SUPRD	Suprêmes délices	Charleroi
Belgium			

Gambar 10.17 Report hasil grouping country

9. Lakukan modifikasi pada design report untuk menghilangkan data country yang tampil berulang dengan melakukan klik kanan field country pada section details dan pilih menu Format Object. Pada jendela Format Object aktifkan setting Suppress untuk menyembunyikan field tersebut agar tidak tampil di preview report.

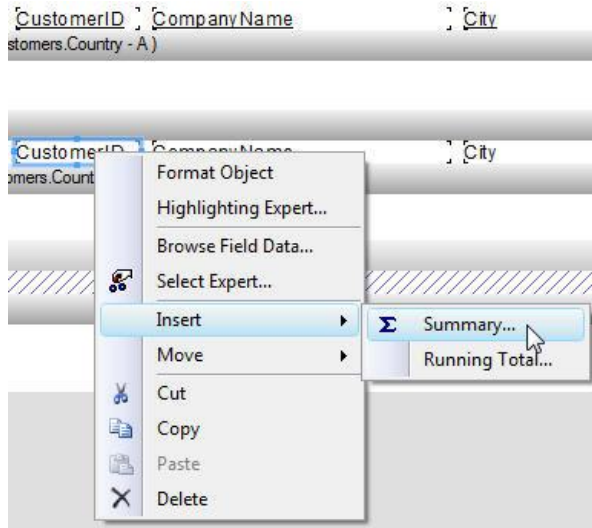


Gambar 10.18 Format object country field



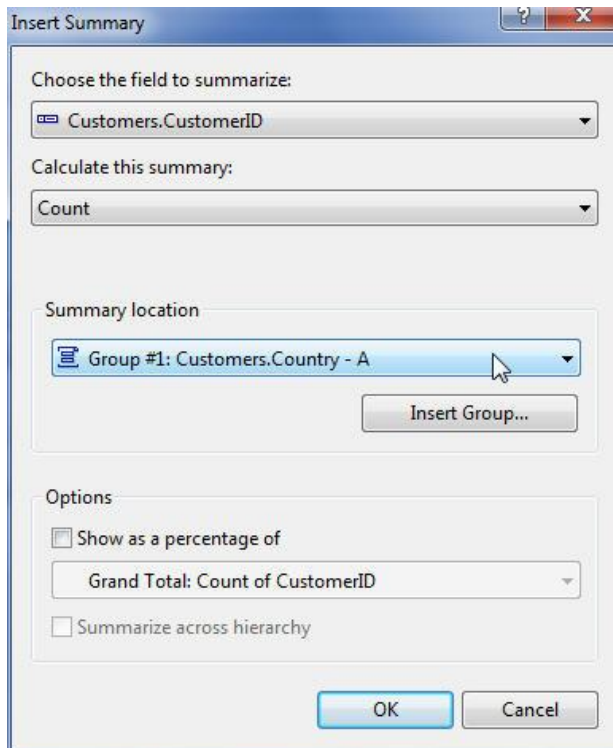
Gambar 10.19 Setting Suppress country field

10. Hapus GroupName field pada GroupFooterSection1 dan tambahkan Text Object dari Toolbox Crystal Reports, isi teks nya dengan “Total : “.
11. Klik kana field CustomerID pada section details pilih menu Insert – Summary. Proses ini dilakukan untuk menghitung jumlah customer per country :



Gambar 10.20 Insert Summary field

12. Lakukan setting berikut pada jendela Insert Summary.



Gambar 10.21 Insert Summary Count field CustomerID

13. Design report hasil modifikasi penambahan Summary field count untuk CustomerID :

<u>Country</u>	<u>CustomerID</u>	<u>CompanyName</u>
derSection1 (Group Header #1: Customers.Country - A)		
Group #1 Name		
Details)		
<u>Country</u>	<u>CustomerID</u>	<u>CompanyName</u>
erSection1 (Group Footer #1: Customers.Country - A)		
	Total	Count of Cu\$
Report Footer)		
Page Footer)		

Gambar 10.22 Design report hasil modifikasi

14. Preview report di Visual Studio :

9/6/2009			
<u>Country</u>	<u>CustomerID</u>	<u>CompanyName</u>	<u>City</u>
Argentina			
	CACTU	Cactus Comidas para llevar	Buenos Aires
	OCEAN	Océano Atlántico Ltda.	Buenos Aires
	RANCH	Rancho grande	Buenos Aires
	Total :	3	
Austria			
	PICCO	Piccolo und mehr	Salzburg
	ERNSH	Ernst Handel	Graz
	Total :	2	
Belgium			
	MAISD	Maison Dewey	Bruxelles
	SUPRD	Suprêmes délices	Charleroi
	Total :	2	

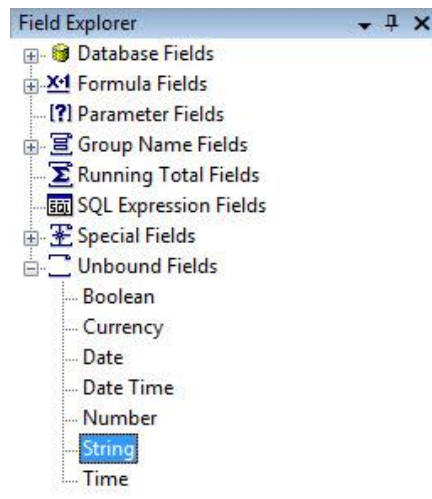
Gambar 10.23 Preview report grouping

Mengubah Grouping Secara Dinamis

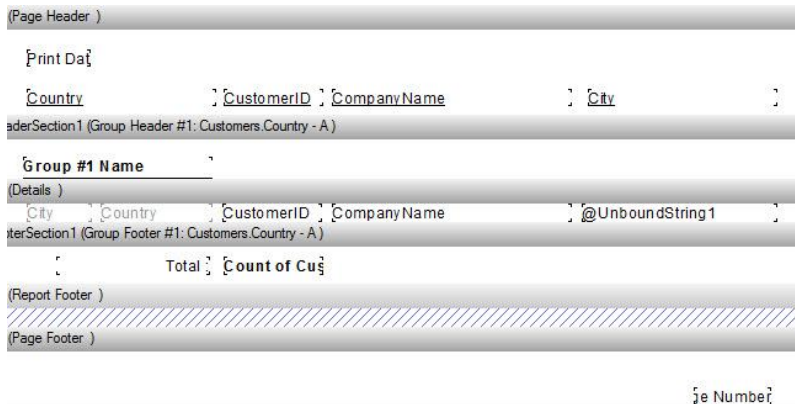
Grouping di report yang telah di design sebelumnya dengan menggunakan field tertentu dapat diubah secara runtime berdasarkan field lainnya. Untuk melakukan hal tersebut kita harus melakukan modifikasi pada report, diantaranya adalah menambahkan field yang belum ditambahkan dari tabel kedalam report yang akan digunakan untuk grouping dan menambahkan object UnboundField pada kolom yang akan diubah nilainya secara dinamis.

Berikut adalah contoh untuk merubah grouping report secara dinamis.

1. Lakukan modifikasi pada file report rptCustGrouping yang telah dibuat sebelumnya dengan menambahkan UnboundString object dari Field Explorer dan tempatkan pada kolom City dengan sebelumnya memindahkan terlebih dahulu field City ke bagian kiri. Aktifkan setting Suppress untuk field City.



Gambar 10.24 Field Explorer



Gambar 10.25 Penambahan UnboundString object

2. Tambahkan form baru kedalam project yang sama.
3. Tambahkan control CrystalReportViewer, dan button. Beri nama btnLoadCustCountry untuk button tersebut.
4. Ketikkan perintah berikut pada baris kode paling atas :

Option Strict On

```
Imports System.Data.SqlClient
Imports CrystalDecisions.CrystalReports.Engine
Imports CrystalDecisions.Shared
```

5. Deklarasikan class level variabel berikut :

```
Private rpt As New rptCustGrouping
Private dt As New DataTable
```

6. Ketikkan prosedur berikut yang digunakan untuk load data dari tabel customer ke dalam DataTable yang nantinya dijadikan sebagai sumber data crystal report :

```
Private Function LoadDataTableCustomersReport() As  
DataTable
```

```
    Using sqlConn As New SqlConnection( _  
        My.Settings.NorthwindConnectionString)  
    Using sqlDa As New SqlDataAdapter( _  
        "Select * From Customers", sqlConn)  
        sqlDa.Fill(dt)  
  
        Return dt  
    End Using  
End Using
```

```
End Function
```

7. Tambahkan prosedur berikut untuk setting autentikasi ke database server dan crystal report datasource. Selain itu di kode tersebut terdapat perintah yang digunakan untuk akses UnboundString object dan menampilkan nilai default berupa field City.

```
Private Sub SetReportSource(ByVal SourceTable As DataTable)
```

```
    rpt.SetDatabaseLogon("sa", "sql2005", _  
        "localhost", "northwind")  
    rpt.SetDataSource(SourceTable)  
  
    Dim formulas = rpt.DataDefinition.FormulaFields  
    Dim ffDef = formulas("UnboundString1")  
    ffDef.Text = "{Customers.City}"  
  
    CrystalReportViewer1.ReportSource = rpt
```

```
End Sub
```

8. Ketikkan kode berikut untuk merubah group field secara dinamis :

```

Private Sub ChangeGroupField(ByVal ColumnName As String)

    Dim secGroups As Groups
    secGroups = rpt.DataDefinition.Groups

    Dim objField As FieldDefinition
    objField = _
rpt.Database.Tables("Customers").Fields(ColumnName)

    For Each aGroup As Group In secGroups
        aGroup.ConditionField = objField
    Next

    Dim txtHeader1 As TextObject
    txtHeader1 =
CType(rpt.ReportDefinition.ReportObjects("txtHeader1"),
TextObject)

    Dim txtHeader2 As TextObject
    txtHeader2 =
CType(rpt.ReportDefinition.ReportObjects("txtHeader2"),
TextObject)

    txtHeader2.Text = txtHeader1.Text
    txtHeader1.Text = ColumnName

    Dim formulas = rpt.DataDefinition.FormulaFields
    Dim ffDef = formulas("UnboundString1")
    Try
        Select Case ComboBox1.Text.ToUpper
            Case "COUNTRY"
                ffDef.Text = "{Customers.City}"
            Case "CITY"
                ffDef.Text = "{Customers.Country}"
        End Select
    Catch ex As Exception
    End Try

    CrystalReportViewer1.ReportSource = rpt
    CrystalReportViewer1.RefreshReport()

```

End Sub

Nilai UnboundString object diatur secara runtime berdasarkan inputan dari combobox begitu juga dengan field yang digunakan untuk mengubah grouping field.

9. Tambahkan Combobox kedalam form, isi Items Collectionnya dengan nilai City dan Country dan ketikkan kode berikut pada event SelectedIndexChanged :

```
Private Sub ComboBox1_SelectedIndexChanged(ByVal sender As
Object, ByVal e As System.EventArgs) _
Handles ComboBox1.SelectedIndexChanged

    If ComboBox1.Text <> "" Then
        If CrystalReportViewer1.ReportSource IsNot
Nothing Then
            ChangeGroupField(ComboBox1.Text)
        Else
            MessageBox.Show("Load Customers first")
        End If
    End If

End Sub
```

End Sub

Mengubah Sort Order Group Secara Dinamis

Sort order group pada crystal reports dapat diubah secara dinamis. Kita dapat merubahnya untuk sort order ascending atau descending berdasarkan kolom grouping yang sedang digunakan.

Berikut adalah contoh penggunaannya :

1. Tambahkan Combobox pada form yang digunakan sebelumnya dan tambahkan Items Collection dengan nilai ASC dan DESC.
2. Tambahkan prosedur berikut ini untuk mengubah sort order group secara dinamis :

```
Private Sub SortReport(ByVal ASC As Boolean)

    Dim allSortField As SortFields
    allSortField = rpt.DataDefinition.SortFields

    For Each aSortField As SortField In allSortField
        If (aSortField.SortType =
SortFieldType.GroupSortField) Then
            If (ASC) Then
                aSortField.SortDirection =
SortDirection.AscendingOrder
            Else
                aSortField.SortDirection =
SortDirection.DescendingOrder
            End If
        End If
    Next

    CrystalReportViewer1.ReportSource = rpt
    CrystalReportViewer1.RefreshReport()

End Sub
```

3. Ketikkan kode berikut pada event SelectedIndexChanged combobox tersebut :

```
Private Sub ComboBox2_SelectedIndexChanged(ByVal sender As
Object, ByVal e As System.EventArgs) _
Handles ComboBox2.SelectedIndexChanged

    If ComboBox2.Text <> "" Then
        If CrystalReportViewer1.ReportSource IsNot
Nothing Then
```

```

        Select Case ComboBox2.Text.ToUpper
            Case "ASC"
                SortReport(True)
            Case "DESC"
                SortReport(False)
        End Select
    Else
        MessageBox.Show("Load Customers first")
    End If
End If

End Sub

```

Berikut adalah preview form report untuk Change Group dan Sort Order :

9/15/2009

Load Customers By Country

Change Group : Order By :

Main Report

Country	CustomerID	CompanyName	City
Argentina	CACTU	Cactus Comidas para llevar	Buenos Aires
	OCEAN	Océano Atlántico Ltda.	Buenos Aires
	RANCH	Rancho grande	Buenos Aires
Total :			3
Austria	ERNSH	Ernst Handel	Graz
	PICCO	Piccolo und mehr	Salzburg
Total :			2
Belgium			

Current Page No.: 1 Total Page No.: 1+ Zoom Factor: 100%

Gambar 10.27 Report By Country

Form2

Load Customers By Country Change Group : City Order By :

Main Report

9/15/2009

City	CustomerID	CompanyName	Country
<u>Aachen</u>	DRACD	Drachenblut Delikatessen	Germany
Total :	1		
<u>Albuquerque</u>	RATTC	Rattlesnake Canyon Grocery	USA
Total :	1		
<u>Anchorage</u>	OLDWO	Old World Delicatessen	USA
Total :	1		

Current Page No.: 1 Total Page No.: 1+ Zoom Factor: 100%

Gambar 10.28 Report By City

Form2

Load Customers By Country Change Group : City Order By : DESC

Main Report

9/15/2009

City	CustomerID	CompanyName	Country
<u>Warszawa</u>	WOLZA	Wolski Zajazd	Poland
Total :	1		
<u>Walla Walla</u>	LAZYK	Lazy K Kountry Store	USA
Total :	1		
<u>Versailles</u>	LACOR	La come d'abondance	France
Total :	1		

Current Page No.: 1 Total Page No.: 1+ Zoom Factor: 100%

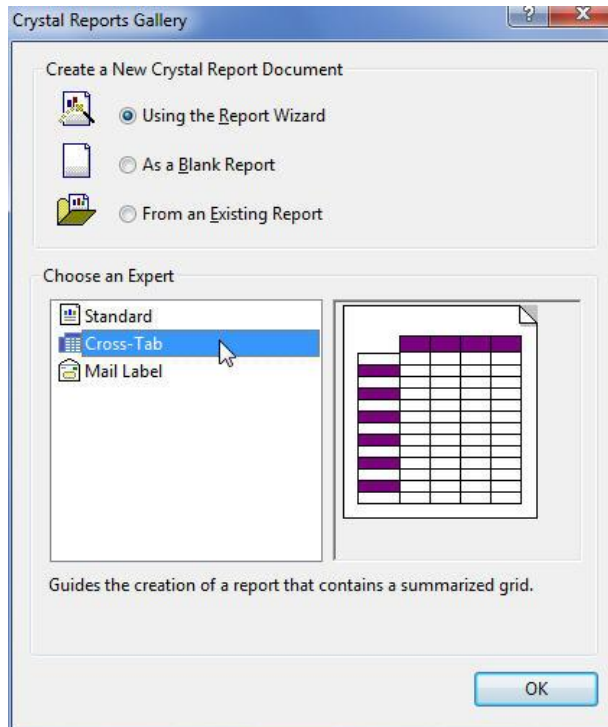
Gambar 10.29 Descending Sort Order

Cross-Tab Report

Cross-tab report merupakan style report yang digunakan untuk menampilkan data secara tabular berupa baris dan kolom dimana data yang ditampilkan merupakan data summary dari sebuah field yang dilihat dari dimensi field pada baris dan kolom.

Berikut adalah contoh pembuatan cross-tab report :

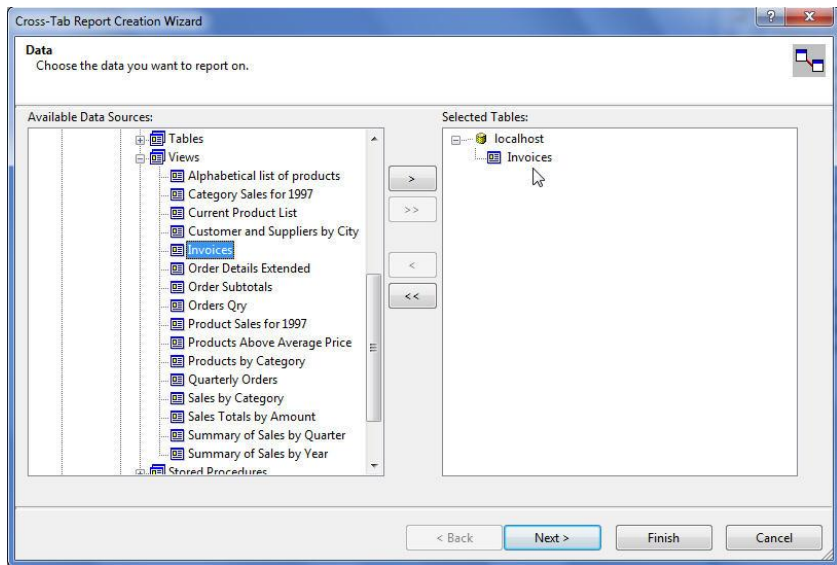
1. Tambahkan CrystalReports item baru kedalam project yang sama dan beri nama rptCrossTab.rpt.
2. Pilih Cross-tab report model :



Gambar 10.30 Cross-tab report model

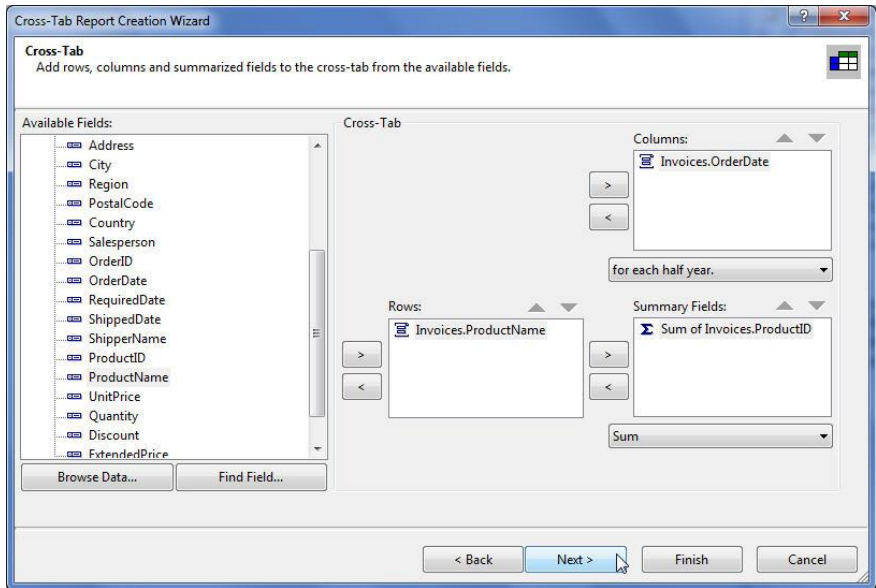
3. Pilih OLE DB Connection dan gunakan SQL Native Client sebagai database provider untuk SQL Server. Database yang digunakan yaitu Northwind.

4. Pilih object view Invoices :



Gambar 10.31 View Invoices

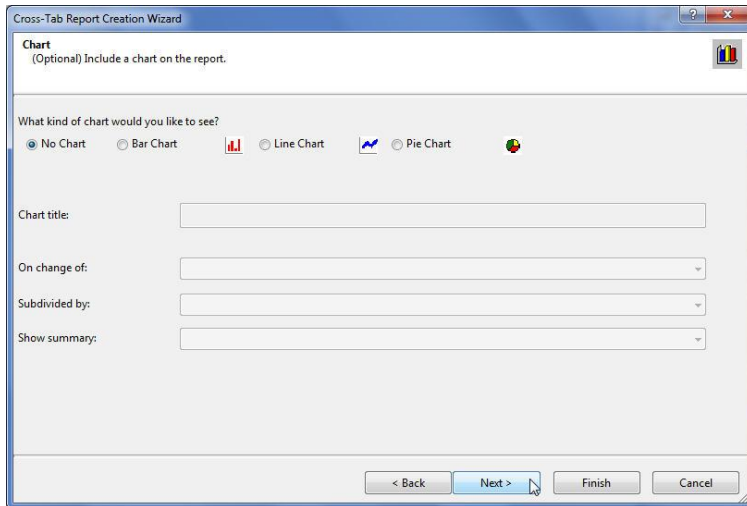
5. Lakukan setting pemilihan field untuk baris, kolom dan fungsi summary yang digunakan seperti pada gambar dibawah ini :



Gambar 10.32 Setting baris, kolom dan summary fields cross-tab.

Cross tab report tersebut akan menampilkan jumlah product yang dibeli oleh semua customers berdasarkan ProductName dan OrderDate untuk setiap setengah tahun. Tentunya anda dapat menggunakan pilihan lainnya untuk OrderDate selain untuk perhitungan setiap setengah tahun. Pastikan summary fields yang dipilih yaitu Sum.

6. Pilih mode No Chart untuk style report yang akan digunakan :



Gambar 10.33 Cross-tab chart model

7. Report yang dihasilkan akan tampak seperti berikut ini:

▼ Section1 (Report Header)

	Column #1	Total
Row #1 Name	ProductID	ProductID
Total	ProductID	ProductID

▼ Section2 (Page Header)

Print Data

▼ Section3 (Details)

▼ Section4 (Report Footer)

▼ Section5 (Page Footer)

Page Num

Gambar 10.34 Design Cross-tab report

8. Preview report dari Visual Studio :

	7/1996	1/1997	7/1997	1/1998	Total
Alice Mutton	136	153	153	187	629
Aniseed Syrup	3	9	12	12	36
Boston Crab Meat	240	560	440	400	1,640
Camembert Pierrot	660	480	780	1,140	3,060
Camarvon Tigers	90	54	162	180	486
Chai	6	6	10	16	38
Chang	16	12	24	36	88
Chartreuse verte	312	195	312	351	1,170
Chef Anton's Cajun Seasoning	20	16	24	20	80
Chef Anton's Gumbo Mix	20	0	10	20	50
Chocolate	0	144	96	48	288
Côte de Blaye	190	228	152	342	912

Gambar 10.35 Preview cross-tab report

Berikut adalah langkah-langkah untuk menampilkan report tersebut dari form :

1. Tambahkan form baru kedalam project yang sama.
2. Tambahkan crystalreportviewer ke dalam form.
3. Tambahkan button, dan beri nama btnLoadReport.
4. Ketikkan kode berikut pada baris kode paling atas :

Option Strict On

Imports System.Data.SqlClient

5. Deklarasikan class level variabel berikut :

Private rpt As New rptCrossTab

Private dt As New DataTable

6. Ketikkan kode berikut untuk membaca data dari database server yang disimpan kedalam DataTable :

Private Function LoadDataTableCustomersReport() As DataTable

```
    Using sqlConn As New
SqlConnection(My.Settings.NorthwindConnectionString)
        Using sqlDa As New SqlDataAdapter( _
            "Select * From Invoices", sqlConn)
            sqlDa.Fill(dt)

        Return dt

    End Using
End Using
```

End Function

7. Ketikkan kode berikut untuk setting autentikasi report dan datasource :

Private Sub SetReportSource(ByVal SourceTable As DataTable)

```
    rpt.SetDatabaseLogon("sa", "sql2005", _
        "localhost", "northwind")
    rpt.SetDataSource(SourceTable)
    CrystalReportViewer1.ReportSource = rpt
```

End Sub

8. Ketikkan kode berikut pada event click untuk button btnLoadReport :

```

Private Sub btnLoadReport_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
Handles btnLoadReport.Click

    SetReportSource(LoadDataTableCustomersReport)
End Sub

```

Berikut adalah preview dari form cross-tab report :

	7/1996	1/1997	7/1997	1/1998	Total
Alice Mutton	136	153	153	187	629
Aniseed Syrup	3	9	12	12	36
Boston Crab Meat	240	560	440	400	1,640
Camembert Pierrot	660	480	780	1,140	3,060
Camarvon Tigers	90	54	162	180	486
Chai	6	6	10	16	38
Chang	16	12	24	36	88
Chartreuse verte	312	195	312	351	1,170
Chef Anton's Cajun Seasoning	20	16	24	20	80

Gambar 10.36 Preview form cross-tab report

LINQ To SQL sebagai DataSource Report

Linq To SQL dapat juga digunakan sebagai datasource untuk report. Pada contoh-contoh sebelumnya datasource yang digunakan yaitu berasal dari ADO.NET object seperti DataTable dan DataView.

Berikut adalah contoh penggunaan Linq To SQL yang dijadikan sebagai datasource untuk crystal reports :

1. Tambahkan Linq To SQL Classes item baru kedalam project yang sama, dan beri nama Northwind.dbml.
2. Tambahkan Customers tabel dari jendela Server Explorer ke Linq To SQL O/R designer.
3. Tambahkan CrystalReports item baru kedalam project. Lakukan setting pada wizard seperti yang telah kita lakukan pada lab sebelumnya. Koneksikan ke database server, pilih database Northwind dan Customers tabel. Gunakan kolom CustomerID, CompanyName, City dan Country untuk field yang akan ditampilkan di report.
4. Tambahkan form baru kedalam project.
5. Tambahkan crystalreportviewer pada form.
6. Tambahkan button dan beri nama btnCustLINQ.
7. Tambahkan Combobox pada form.
8. Ketikkan perintah berikut pada baris kode paling atas :

Option Strict On

Imports System.Linq

9. Deklarasikan class level variabel berikut :

```
Private db As New NorthwindDataContext
Private rpt As New rptCustLinq
Private customers As IQueryable(Of Customer)
```

10. Ketikkan prosedur berikut yang digunakan untuk menampilkan data country di combobox :

```
Private Sub LoadCustomersCountry()

    Dim countries = (From c In db.Customers _
                     Where c.Country IsNot Nothing _
```

```
Select c.Country).Distinct
```

```
ComboBox1.DataSource = countries.ToList  
ComboBox1.DisplayMember = "Country"  
ComboBox1.SelectedIndex = -1
```

```
End Sub
```

11. Ketikkan prosedur berikut untuk setting autentikasi database dan datasource report :

```
Private Sub SetReportSource(ByVal customerSource As _  
IQueryable(Of Customer))
```

```
    rpt.SetDatabaseLogon("sa", "sql2005", _  
                        "localhost", "northwind")  
    rpt.SetDataSource(customerSource)
```

```
    CrystalReportViewer1.ReportSource = rpt
```

```
End Sub
```

12. Tambahkan kode berikut ini pada event Load form untuk memanggil prosedur yang digunakan untuk menampilkan data country ke combobox :

```
Private Sub Form4_Load(ByVal sender As Object, _  
ByVal e As System.EventArgs) Handles Me.Load
```

```
    LoadCustomersCountry()
```

```
End Sub
```

13. Ketikkan kode berikut pada event click button btnCustLINQ. Hasil eksekusi perintah ini akan

menampilkan report berisi data customers dimana data tersebut di supply dari Linq To SQL object.

```
Private Sub btnCustLINQ_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnCustLINQ.Click
```

```
    customers = From c In db.Customers _  
                Select c
```

```
    SetReportSource(customers)
```

```
End Sub
```

14. Tambahkan perintah berikut pada event SelectedIndexChanged untuk kontrol Combobox yang sebelumnya telah ditambahkan pada form :

```
Private Sub ComboBox1_SelectedIndexChanged(ByVal sender As  
Object, ByVal e As System.EventArgs) _  
Handles ComboBox1.SelectedIndexChanged
```

```
    If ComboBox1.Text <> "" Then  
        customers = From c In db.Customers _  
                    Where c.Country = ComboBox1.Text _  
                    Select c
```

```
        SetReportSource(customers)
```

```
    Else
```

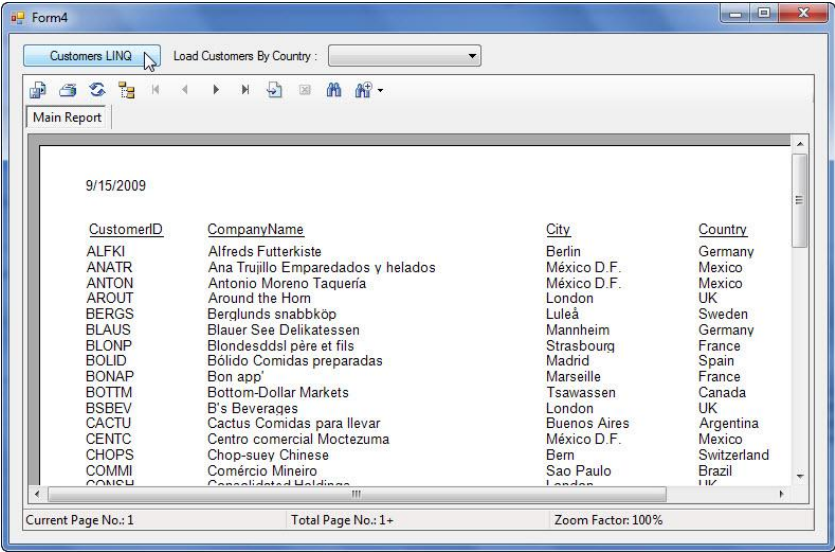
```
        CrystalReportViewer1.ReportSource = Nothing
```

```
    End If
```

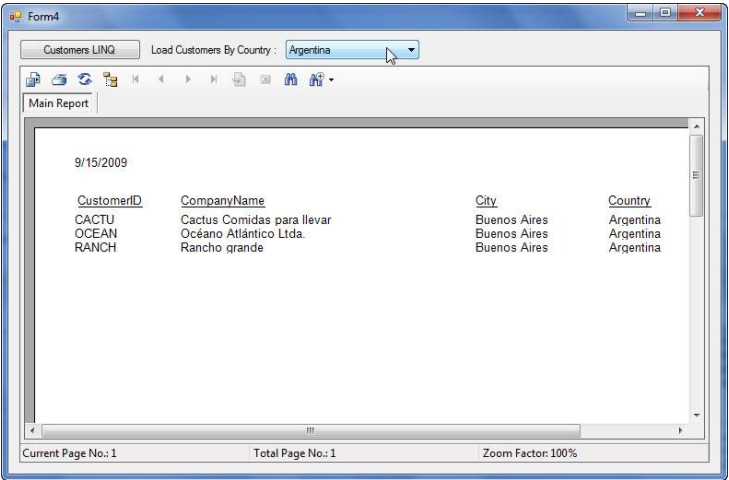
```
End Sub
```

Kode diatas digunakan untuk melakukan filtering data customers berdasarkan kolom country yang dipilih dari combobox.

Preview dari form report tersebut dapat dilihat pada gambar dibawah ini :



Gambar 10.37 Preview Linq To SQL form report



Gambar 10.38 Preview Linq To SQL form report hasil filtering

BAB 11

.NET Assemblies

Assembly merupakan satuan unit deployment dan merupakan unit yang membentuk aplikasi. Assembly dibentuk oleh file-file dan resources yang memiliki versi tertentu. Assembly tersebut dapat berupa file .dll atau file .exe.

Dari sisi lokasi penyimpanan maka assembly tersebut dibagi kedalam dua kelompok yaitu private assembly dan shared assembly.

Private assembly merupakan jenis assembly yang disimpan di lokasi direktori aplikasi yang digunakan. Keuntungan dari jenis assembly ini yaitu kemudahannya dalam mendeploy aplikasi tersebut. Kita cukup melakukan copy satu folder aplikasi yang akan dideploy ke lokasi direktori lain selama assembly yang direferensi masih berada didalam direktori aplikasi tersebut.

Shared assembly merupakan assembly yang disimpan di lokasi khusus. Keuntungan dari jenis assembly ini yaitu dapat digunakan oleh lebih dari satu aplikasi dalam satu komputer yang sama. Selain itu kita dapat mendeploy assembly yang sama dengan versi yang berbeda-beda tanpa mengganggu kompatibilitas, istilah ini biasanya disebut dengan side by side versioning.

Private Assembly

Aplikasi yang mereferensi ke sebuah assembly akan disimpan secara copy lokal ke direktori aplikasi tersebut apabila assembly yang digunakan bukan shared assembly. CLR menggunakan caranya tersendiri dalam mencari lokasi assembly yang digunakan oleh aplikasi tersebut.

Untuk mengetahui prinsip kerja CLR dalam mencari lokasi assembly yang direferensi digunakan tools fuslogvw.exe yang diakses dari Visual Studio command prompt.

Berikut adalah contoh simulasi penggunaan tools fuslogvw.exe dalam mencari lokasi assembly yang direferensi oleh sebuah aplikasi. Jenis assembly yang digunakan yaitu private assembly.

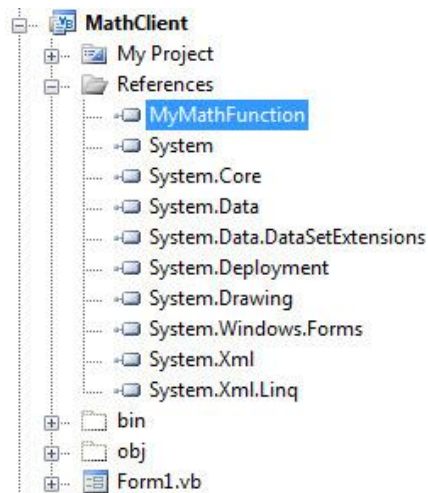
1. Buat sebuah Class Library project, beri nama MyMathFunction.
2. Ubah nama file class yang terdapat didalam project tersebut dari Class1.vb menjadi MyMathFunction.vb, sehingga nanti nama class nya juga akan berubah menjadi nama file tersebut.
3. Buka project properties dengan melakukan double klik My Project node pada Solution Explorer. Buka bagian Application, dan hapus Root Namespace menjadi kosong kemudian save. Dengan menghapus root namespace ini maka untuk akses kepada class-class yang terdapat didalam project tersebut dapat langsung dilakukan dengan memanggil nama class yang akan digunakan.
4. Ketikkan kode berikut pada class diatas. Fungsi yang dibuat ini untuk menghitung nilai rata-rata dari nilai-nilai yang diinput.

```
Public Function GetAverage(ByVal ParamArray Numbers() As _  
Integer) As Double
```

```
    Dim intCount = Numbers.Count  
    Dim intSum = 0  
    For i = 0 To intCount - 1  
        intSum += Numbers(i)  
    Next  
    Return intSum / intCount
```

```
End Function
```

5. Build project tersebut untuk menghasilkan MyMathFunction.dll
6. Buat sebuah Windows Forms Application baru kedalam solution yang sama dan beri nama MathClient.
7. Tambahkan referensi ke file dll yang dihasilkan oleh project MyMathFunction.



Gambar 11.1 Private assembly yang direferensi

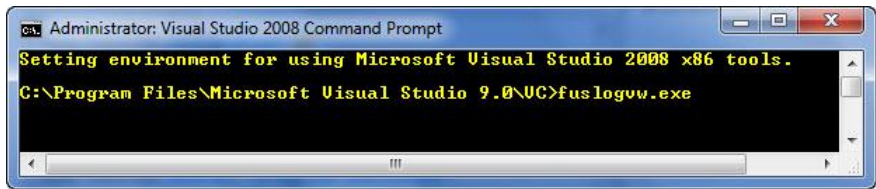
8. Tambahkan sebuah button kedalam form, dan ketikkan kode berikut pada event click button :

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
```

```
    Dim mathFunc As New MyMathFunction
    Dim dblAvg = mathFunc.GetAverage(2, 4, 6, 10)
    MessageBox.Show(dblAvg.ToString)
```

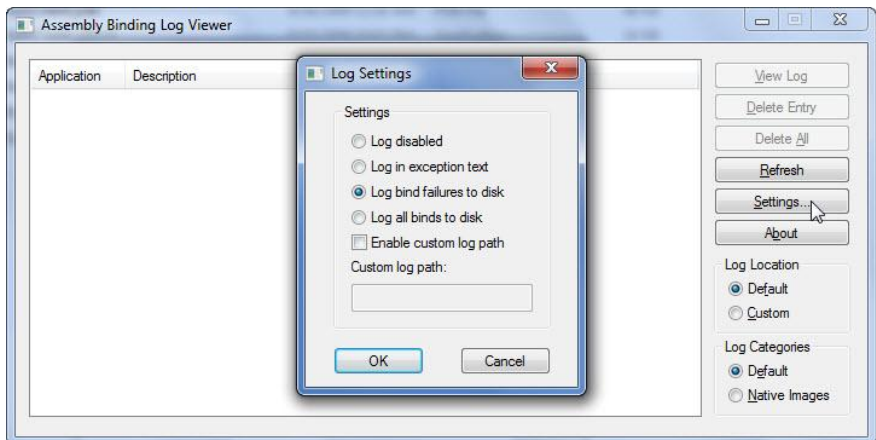
```
End Sub
```

9. Build aplikasi tersebut dan jalankan.
10. Jalankan tools fuslogvw.exe dengan mengetikkan nama tools tersebut pada Visual Studio command prompt.



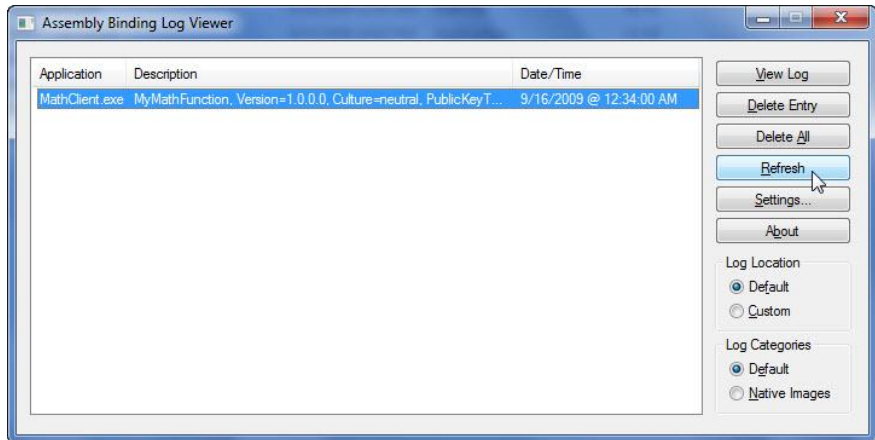
Gambar 11.2 Fuslogvw.exe tools

11. Pilih setting berikut pada tools tersebut :



Gambar 11.3 Settings fuslogvw.exe

12. Tutup kembali window tools tersebut.
13. Cari file dll yang direferensi pada lokasi direktori aplikasi windows tersebut kemudian hapus.
14. Jalankan file exe windows forms application dari lokasi direktori bin/Debug atau bin/Release dan klik tombol button yang ada di form untuk menghitung nilai rata-rata.
15. Jalankan kembali fuslogvw.exe dan klik tombol Refresh :



Gambar 11.4 Refresh log file

16. Klik tombol View Log untuk menampilkan log filenya :

```
*** Assembly Binder Log Entry (9/16/2009 @ 12:34:00 AM) ***

The operation failed.
Bind result: hr = 0x80070002. The system cannot find the file specified.

Assembly manager loaded from: C:\Windows\Microsoft.NET\Framework\v2.0.50727\mscorwks.dll
Running under executable D:\[redacted]\VB2008\MathClient\bin\Debug\MathClient.exe
--- A detailed error log follows.

=== Pre-bind state information ===
LOG: User = [redacted]
LOG: DisplayName = MyMathFunction, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
(Fully-specified)
LOG: Appbase = file:///D:/[redacted]\VB2008\MathClient\bin\Debug/
LOG: Initial PrivatePath = NULL
LOG: Dynamic Base = NULL
LOG: Cache Base = NULL
LOG: AppName = NULL
Calling assembly : MathClient, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null.
=====
LOG: This bind starts in default load context.
LOG: No application configuration file found.
LOG: Using machine configuration file from C:\Windows\Microsoft.NET\Framework\v2.0.50727\config\machine.config.
LOG: Policy not being applied to reference at this time (private, custom, partial, or location-based assembly bind).
LOG: Attempting download of new URL file:///D:/[redacted]\VB2008\MathClient\bin\Debug\MyMathFunction.DLL.
LOG: Attempting download of new URL file:///D:/[redacted]\VB2008\MathClient\bin\Debug\MyMathFunction.DLL.
LOG: Attempting download of new URL file:///D:/[redacted]\VB2008\MathClient\bin\Debug\MyMathFunction.EXE.
LOG: Attempting download of new URL file:///D:/[redacted]\VB2008\MathClient\bin\Debug\MyMathFunction\MyMathFunction.EXE.
LOG: All probing URLs attempted and failed.
```

Gambar 11.5 Log file

Dari log file tersebut dapat kita lihat variasi lokasi direktori yang digunakan untuk mencari private assembly yang direferensi oleh aplikasi.

Shared Assembly

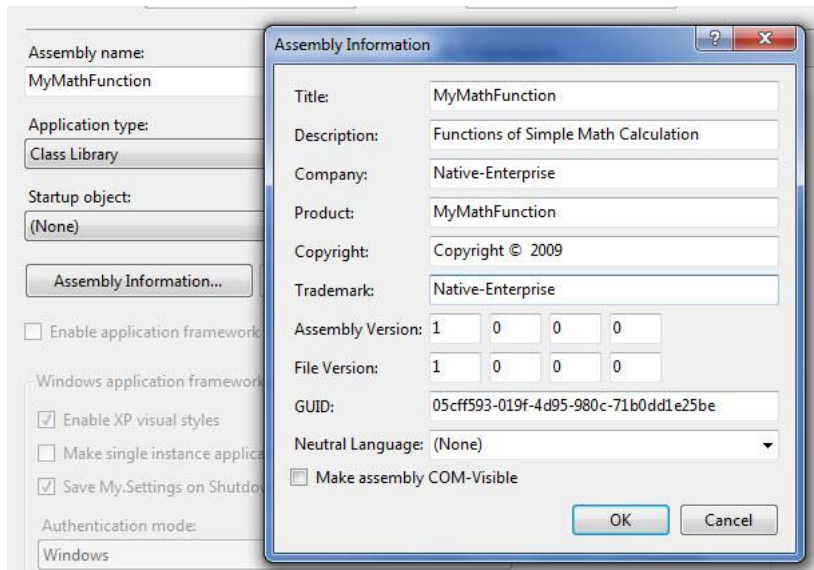
Untuk membuat shared assembly ada beberapa syarat yang harus dipenuhi yaitu Strong Name. Strong name tersebut

terdiri dari beberapa item yang harus dipenuhi, item-item tersebut adalah : Assembly Name, Assembly Version dan Key File.

Side by side versioning akan terpenuhi apabila salah satu dari ketiga item tersebut memiliki nilai yang berbeda dari assembly-assembly yang akan di shared.

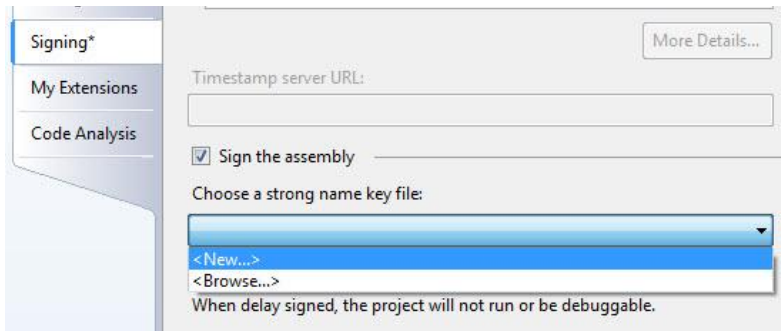
Berikut adalah langkah-langkah untuk membuat shared assembly dan di deploy ke lokasi Global Assembly Cache (GAC):

1. Buka kembali class library project MyMathFunction.
2. Klik dua kali My Project node di solution explorer.
3. Pada bagian Application tab klik tombol Assembly Information, isi beberapa informasi yang dibutuhkan terutama Assembly Version. Biarkan nilai version tetap default.



Gambar 11.6 Assembly Information

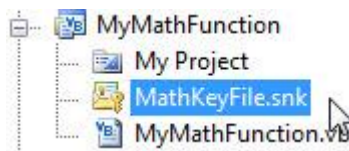
4. Pada bagian Signing, aktifkan pilihan Sign the assembly dan pilih menu New... dan isi jendela tersebut dengan setting seperti berikut ini :



Gambar 11.7 Aktivasi Sign the assembly

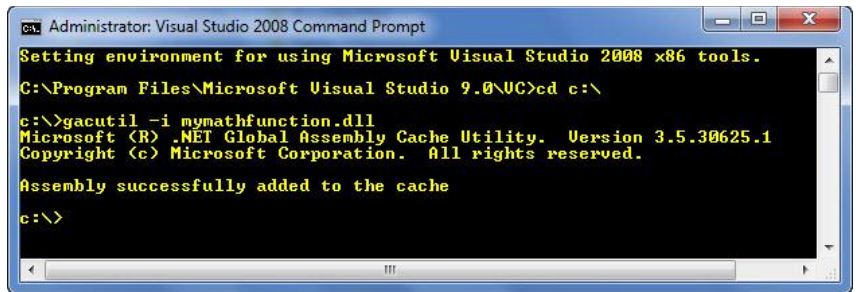


Gambar 11.8 Key File Name



Gambar 11.9 Strong Name Key file

5. Build project tersebut.
6. Buka visual studio command prompt. Ubah lokasi direktori ke lokasi file MyMathFunction.dll disimpan. Misalkan file tersebut saya pindahkan ke direktori C:\MyMathFunction.dll.
7. Ketikkan perintah gacutil.exe -i pada visual studio command prompt seperti pada gambar berikut ini :



Gambar 11.10 Gacutil (Global Assembly Cache Utility).exe

8. Cari lokasi file dll tersebut di direktori C:\Windows\Assembly :

msddsp	9.0.0.0	b03f5f7f11d50a3a	MSIL
msddsp	8.0.0.0	b03f5f7f11d50a3a	MSIL
MyMathFunction	1.0.0.0	8bfb4fd6ea3f5e2b	MSIL
napcrypt	6.1.0.0	31bf3856ad364e35	x86

Gambar 11.11 GAC direktori

Menambahkan Info Assembly ke Registry

Assembly yang sudah di deploy ke GAC belum dapat langsung digunakan dari Visual Studio karena assembly tersebut belum dapat dilihat dari .NET reference tab form. Untuk menampilkan assembly tersebut di .NET reference kita harus menambahkan key baru kedalam registry.

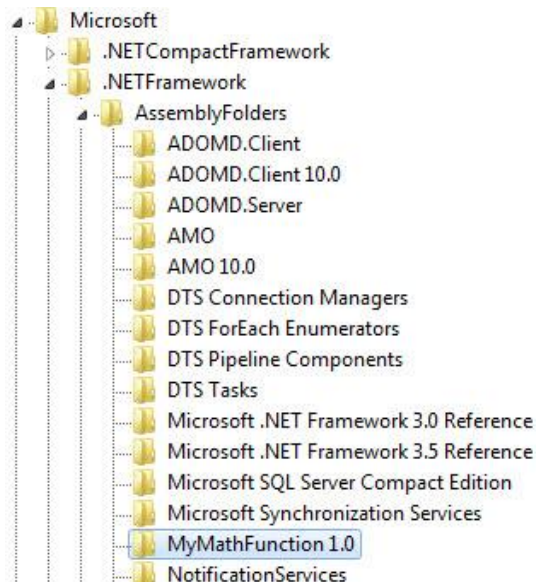
Berikut adalah langkah-langkah untuk menambahkan info assembly cache ke dalam registry :

1. Buka registry tools dengan mengetikkan regedit pada command prompt windows.
2. Cari path berikut ini :

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\NETFramework\AssemblyFolders

3. Buat key baru pada path tersebut, misalnya :

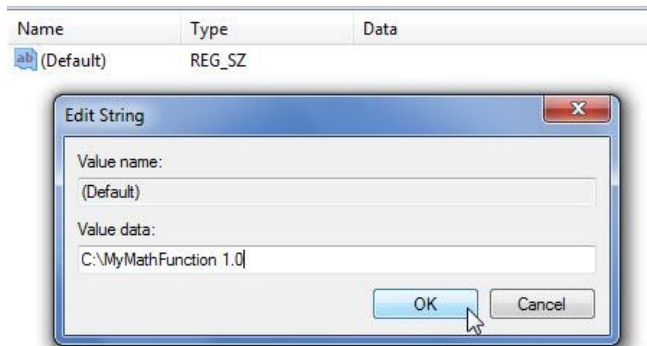
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\NETFramework\AssemblyFolders\MyMathFunction 1.0



Gambar 11.12 Key baru di registry

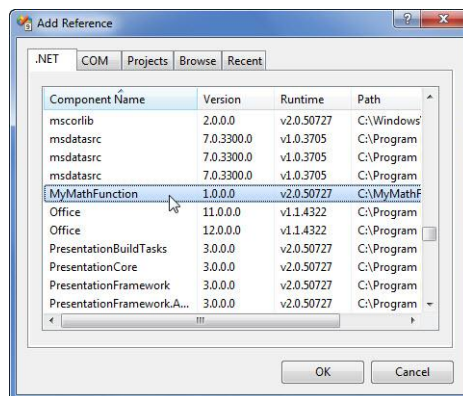
4. Ubah nilai Default string key tersebut yang diisi dengan lokasi penyimpanan assembly, misalnya

MyMathFunction.dll saya simpan di C:\MyMathFunction 1.0



Gambar 11.13 Mengubah nilai default string

5. Tutup semua Visual Studio yang terbuka kemudian buka kembali.
6. Tambahkan referensi ke assembly tersebut (MyMathFunction.dll) dari MathClient project yang telah kita buat sebelumnya.
7. Kita dapat melihat assembly tersebut dalam list .NET assembly tab :



Gambar 11.14 .NET Shared Assembly list

8. Aplikasi tersebut sudah tidak lagi membutuhkan private assembly yang di copy lokal ke direktori aplikasi.

Side by Side Versioning

Seperti yang telah saya jelaskan sebelumnya bahwa salah satu keuntungan dari shared assembly yaitu side by side versioning. Dengan keuntungan tersebut kita dapat membuat assembly dengan nama assembly yang sama namun memiliki version number yang berbeda.

Misalkan kita menemukan sebuah bugs didalam komponen yang dideploy atau akan menambahkan fungsionalitas baru kedalam komponen tersebut tanpa harus mengganggu aplikasi yang telah diinstal sebelumnya yang mereferensi ke assembly version yang lama. Hal tersebut dapat dilakukan tanpa menyebabkan masalah conflict versioning.

Berikut adalah contoh simulasi penjelasan side by side versioning :

1. Buat Class Library project baru pada Solution yang sama dengan Solution yang telah dibuat sebelumnya. Beri nama project tersebut dengan MyMathFunction20.
2. Hapus file Class1.vb di project tersebut.
3. Tambahkan file class MyMathFunction.vb yang telah dibuat pada project MyMathFunction class library dengan klik kanan project dan pilih menu Add – Existing Item....cari file class MyMathFunction.vb di project class library sebelumnya.
4. Lakukan modifikasi fungsi yang ada di dalam class tersebut menjadi seperti ini :

```
Public Function GetAverage(ByVal ParamArray Numbers() As _  
Integer) As Double
```

```
    Dim intCount = Numbers.Count  
    Dim intSum = 0  
    For i = 0 To intCount - 1  
        intSum += Numbers(i)  
    Next
```

```
    Dim strMessage = _  
    "GetAverage function was executed on : " &  
    & Now.ToString() & Environment.NewLine
```

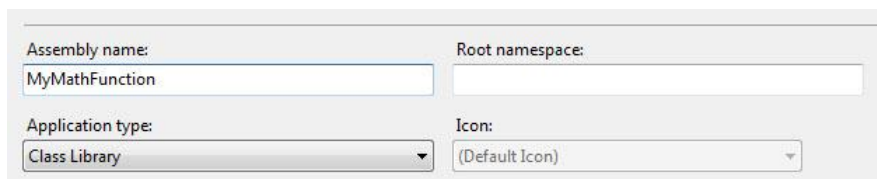
```
    My.Computer.FileSystem.WriteAllText(_  
        "C:\MyLogFile.txt", strMessage, True)
```

```
    Return intSum / intCount
```

```
End Function
```

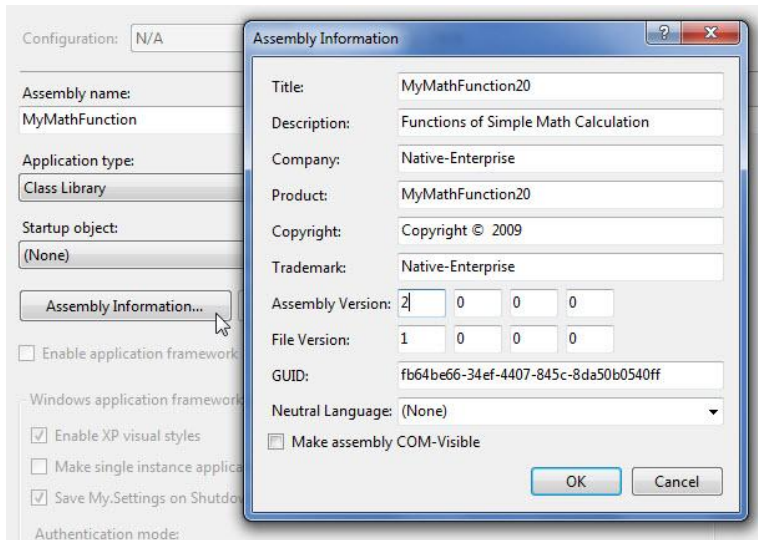
Kode diatas dimodifikasi dengan menambahkan info pada log file yang disimpan di lokasi direktori komputer yang digunakan.

5. Buka project properties dengan double klik My Project node.
6. Buka bagian Application kemudian hapus Root namespace dan ganti Assembly Name menjadi MyMathFunction.



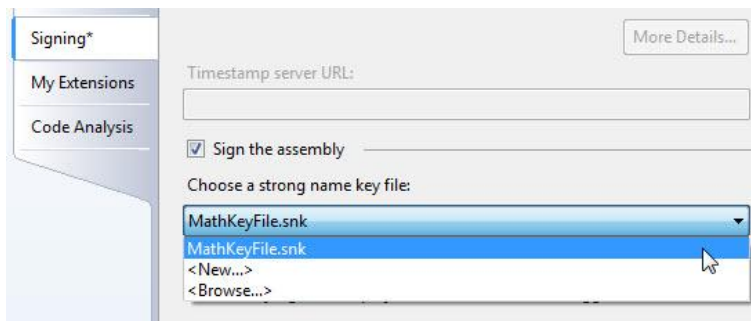
Gambar 11.15 Modifikasi Root namespace dan Assembly Name

7. Ubah Assembly Information seperti berikut ini :



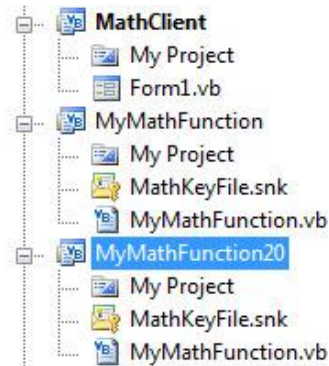
Gambar 11.16 Modifikasi Assembly Information

8. Tambahkan key file yang telah digunakan pada class library project sebelumnya dengan menambahkan existing item pada project ini. Lakukan signing assembly dengan menggunakan key file yang telah di import :



Gambar 11.17 Signing assembly dengan key file

9. Struktur project dapat dilihat pada gambar berikut ini :



Gambar 11.18 Struktur project

10. Build project.

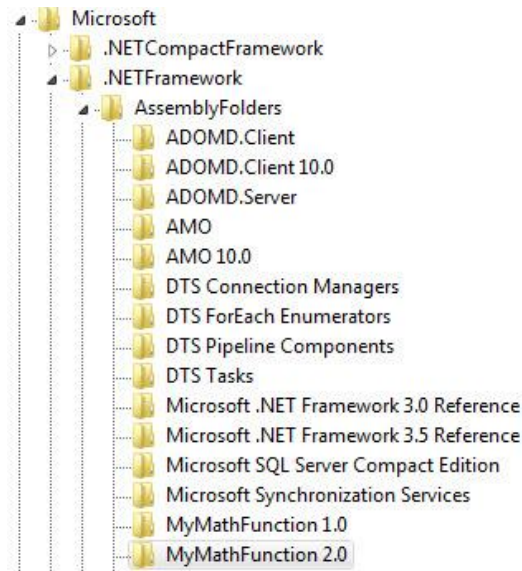
11. Tambahkan library hasil kompilasi tersebut ke GAC seperti yang telah dijelaskan sebelumnya. Hasilnya akan nampak seperti dibawah ini :

MyMathFunction	2.0.0.0	8bf4fd6ea3f5e2b	MSIL
MyMathFunction	1.0.0.0	8bf4fd6ea3f5e2b	MSIL

Gambar 11.19 Side by side versioning assembly

12. Buat folder baru, misalnya di C:\MyMathFunction 2.0 kemudian pindahkan assembly dari project ke lokasi direktori tersebut.

13. Tambahkan info assembly tersebut ke dalam registry seperti yang telah dijelaskan sebelumnya.



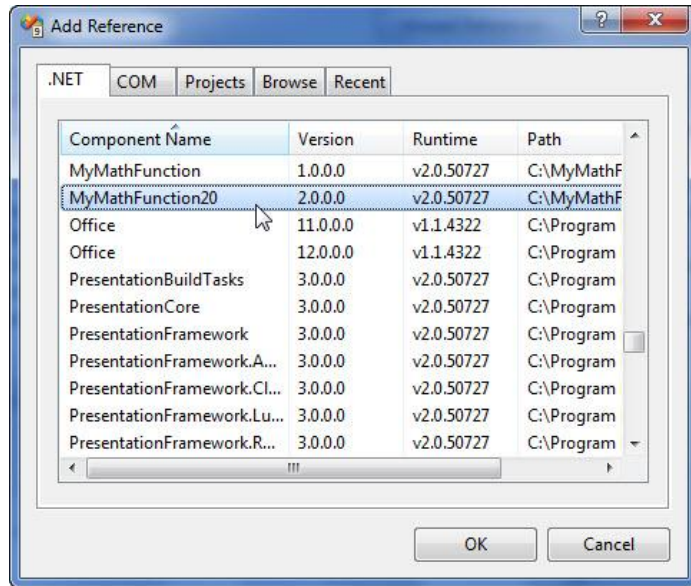
Gambar 11.20 Penambahan key baru di registry

14. Ubah nilai default string nya menjadi lokasi direktori penyimpanan assembly tersebut.

Name	Type	Data
ab (Default)	REG_SZ	C:\MyMathFunction 2.0

Gambar 11.21 Modifikasi nilai default string

15. Tutup semua Visual Studio yang terbuka.
16. Buka kembali project solution sebelumnya dan tambahkan Windows Forms Application project baru dan beri nama MyMathClient20.
17. Tambahkan referensi ke assembly MyMathFunction20.dll.



Gambar 11.22 .NET Shared Assembly

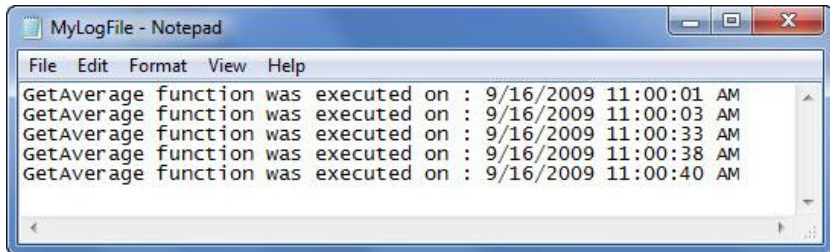
18. Tambahkan button kedalam form tersebut, dan ketikkan kode berikut pada event click button :

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
```

```
    Dim mathFunc As New MyMathFunction
    Dim dblAvg = mathFunc.GetAverage(2, 4, 6, 10)
    MessageBox.Show(dblAvg.ToString)
```

```
End Sub
```

19. Jalankan aplikasi kemudian klik button tersebut.
 20. Buka file MyLogFile.txt pada direktori C:\. Isi file tersebut akan nampak seperti berikut ini :



Gambar 11.23 Log file

Publisher Policy

Bagaimana caranya apabila aplikasi yang telah diinstal sebelumnya dan menggunakan assembly versi tertentu ingin di redirect ke assembly versi yang lain dengan tidak membongkar kembali source code yang digunakan? Jawabannya adalah Publisher policy.

Publisher Policy (PP) merupakan sebuah file XML seperti halnya file app.config yang di compile kedalam sebuah assembly dan disimpan di Global Assembly Cache. PP ini berisi informasi yang diberikan kepada CLR untuk redirect assembly version yang digunakan pada aplikasi yang sudah diinstal ke assembly versi lain di Global Assembly Cache (GAC). Misalnya aplikasi A mereferensi ke sebuah assembly A dengan versi 1.0.0.0 kemudian tanpa harus membongkar kembali aplikasinya maka kita bisa redirect ke assembly A version 2.0.0.0.

Proses redirect assembly tersebut dilakukan melalui tahap-tahap sbb :

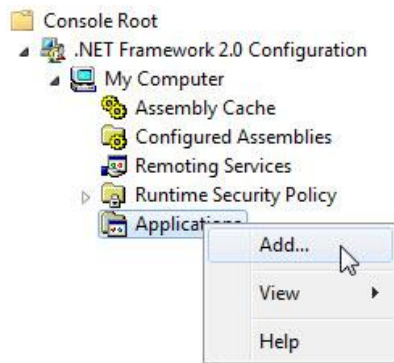
1. Cek Application Policy (app.config).
2. Cek Publisher Policy yang terdapat didalam GAC.
3. Cek Administrator Policy (machine.config). Ini merupakan hasil binding terakhir.

Berikut penjelasannya :

1. CLR akan cek terlebih dahulu policy yang terdapat di dalam app.config. Didalam app.config ini kita bisa tambahkan path assembly version tertentu yang akan digunakan oleh CLR untuk mencari assembly yang di reference.
2. Apabila didalam PP (disimpan di GAC) terdapat perintah redirect ke assembly version tertentu maka setting yang terdapat di app.config akan di override. Akan tetapi kalau kita tidak menginginkan setting yang terdapat di file app.config di override kita tinggal ubah nilai elemen <publisherPolicy apply="no" /> di app.config.
3. Kedua setting di atas tidak akan bekerja apabila didalam file machine.config terdapat perintah untuk redirect ke assembly version tertentu. Jadi ini merupakan proses binding terakhir.

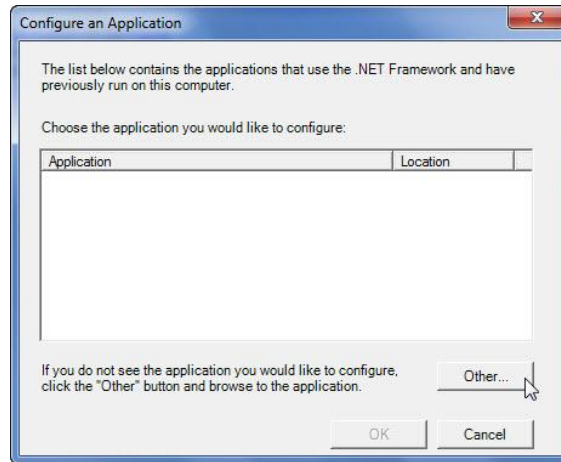
Berikut adalah contoh penggunaan file Publisher Policy dan cara pembuatannya :

1. Buka Control Panel – Administrative Tools – Microsoft .NET framework 2.0 configuration.
2. Tambahkan aplikasi yang akan di konfigurasi :



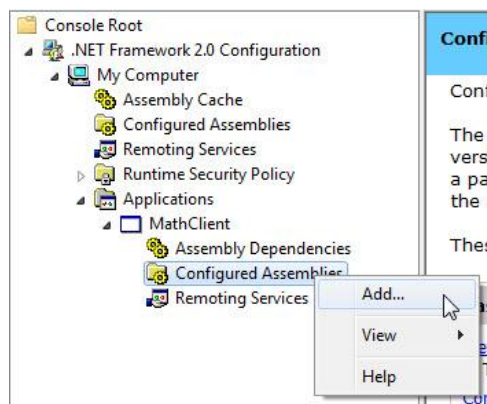
Gambar 11.24 Add aplikasi yang akan di konfigurasi

3. Klik tombol Other... cari file MathClient.exe.



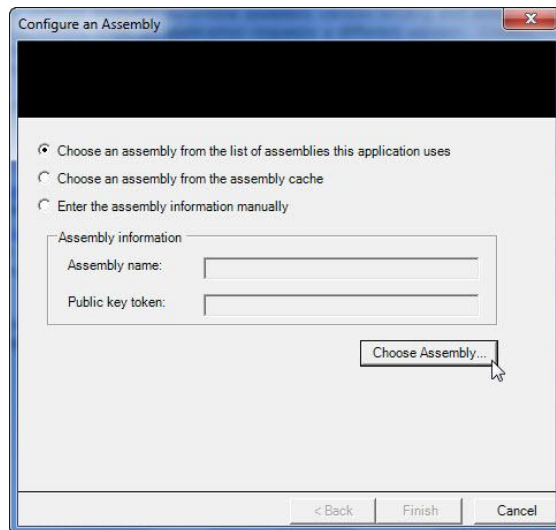
Gambar 11.25 Pencarian file aplikasi yang akan di konfigurasi

4. Klik kanan node Configure Assemblies untuk menambahkan assembly yang digunakan oleh aplikasi tersebut yang akan di konfigurasi :



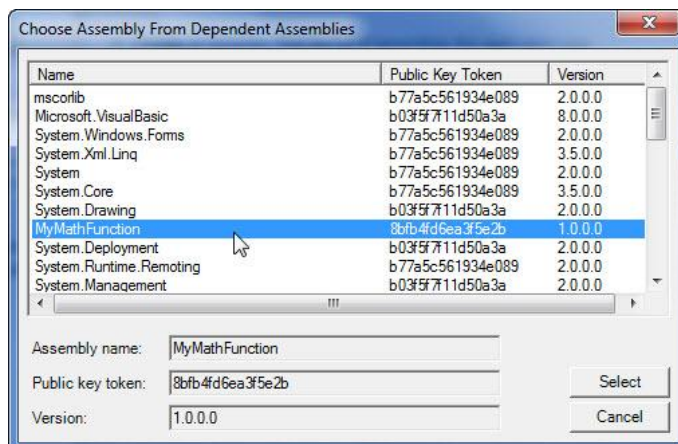
Gambar 11.26 Add Configured Assemblies

5. Aktifkan pilihan pertama dan klik tombol Choose Assembly.

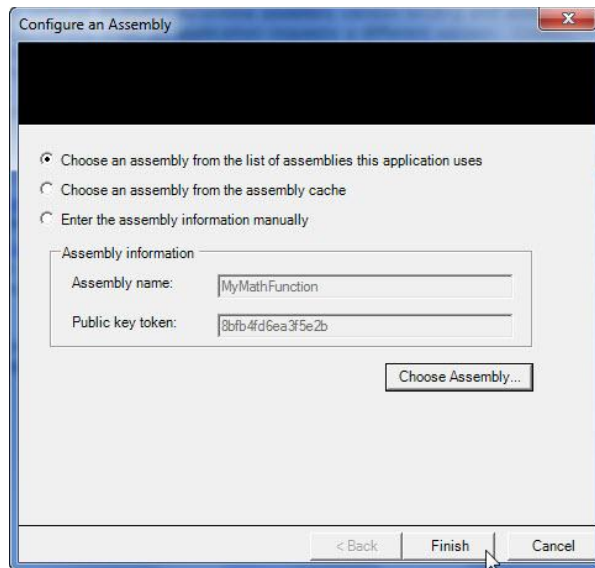


Gambar 11.27 Choose Assembly

6. Pilih MathFunction.dll (versi 1.0.0.0)

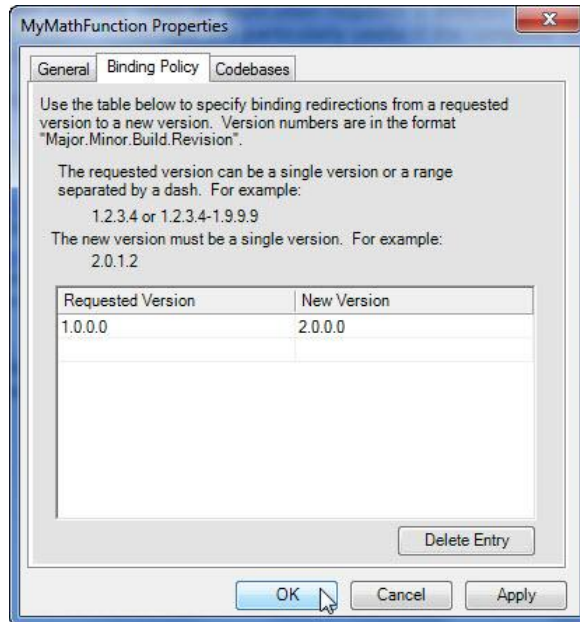


Gambar 11.28 Pilih assembly yang akan di konfigurasi



Gambar 11.29 Assembly yang akan di konfigurasi

7. Pada tab Binding Policy ketikkan Requested Version dan Newer Version dengan versi yang diinginkan seperti yang dapat dilihat dibawah ini :



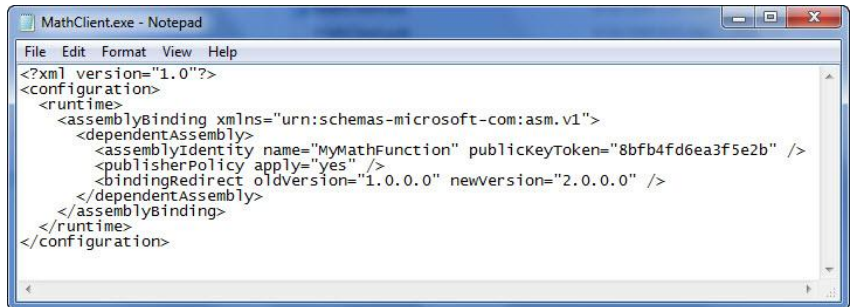
Gambar 11.30 Binding Policy

8. Buka direktori bin\debug atau bin\release maka akan tampak sebuah file XML configuration yang dihasilkan.

MathClient	9/16/2009 9:55 AM	Application	20 KB
MathClient.exe	9/16/2009 1:40 PM	XML Configuratio...	1 KB
MathClient.pdb	9/16/2009 9:55 AM	PDB File	48 KB
MathClient.vshost	9/16/2009 10:40 AM	Application	14 KB
MathClient.vshost.exe.manifest	6/11/2009 4:14 AM	MANIFEST File	1 KB
MathClient	9/16/2009 9:55 AM	XML Document	1 KB

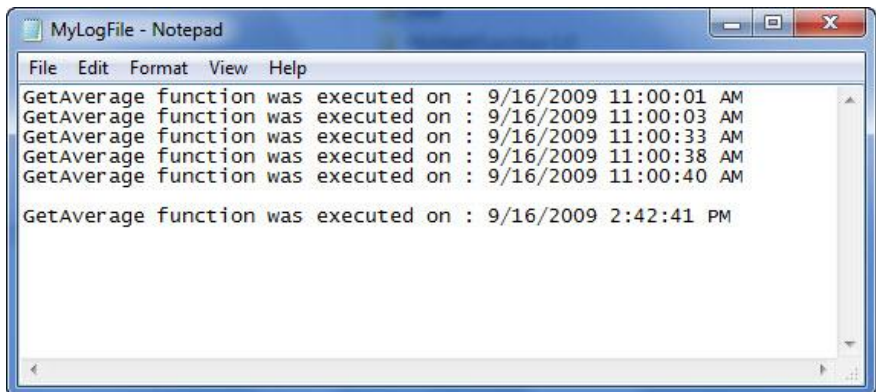
Gambar 11.31 File Publisher Policy

9. Buka file xml configuration tersebut dengan Notepad :



Gambar 11.32 File Publisher Policy

10. Jalankan MathClient.exe dari direktori bin\Debug atau bin\Release. Kemudian klik button yang terdapat didalam form tersebut untuk eksekusi kode program didalamnya.
11. Buka file MyLogFile.txt



Gambar 11.33 Entri baru di MyLogFile.txt

Entri baru di file log tersebut sebagai hasil dari redirect assembly yang digunakan oleh aplikasi MathClient.exe ke assembly MyMathFunction.dll versi 2.0 dari versi 1.0.

Untuk membuat Publisher Policy yang akan di deploy ke GAC harus mengikuti format :

policy.majorVersion.minorVersion.AssemblyName.dll

MajorVersion merupakan major number assembly version yang akan di redirect.

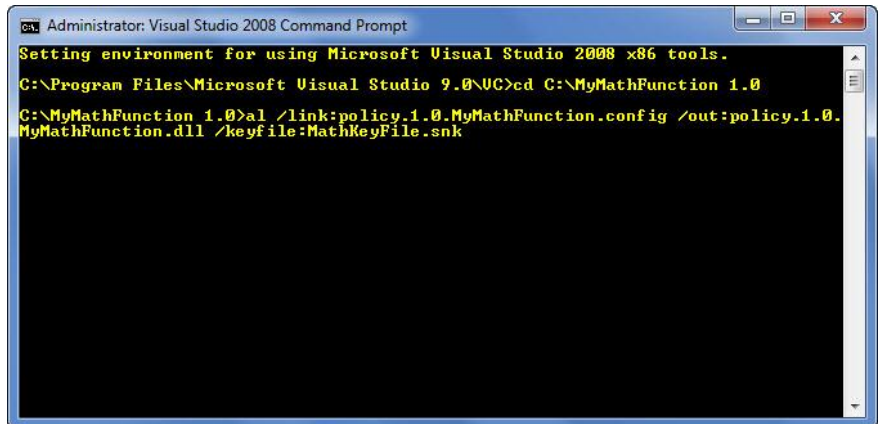
MinorVersion merupakan minor number assembly version yang akan di redirect.

AssemblyName merupakan nama assembly yang akan di redirect.

Berikut dijelaskan langkah-langkah untuk membuat Publisher Policy yang akan di deploy ke GAC :

1. Buat file `policy.1.0.MyMathFunction.config` di folder dimana assembly versi tersebut berada dengan isi yang sama dengan file `app.config` yang ada di Gambar 11.32.
2. Misalkan assembly `MyMathFunction.dll` v 1.0 disimpan di folder `C:\MyMathFunction 1.0` maka file policy yang dibuat disimpan di folder tersebut.
3. Copy kan file `MathKeyFile.snk` yang telah digunakan pada `MyMathFunction` class library project ke direktori yang sama dengan file policy diatas.
4. Untuk menghasilkan assembly dari file policy tersebut digunakan tools `al.exe` (Assembly Linker) yang diakses lewat visual studio command prompt.
5. Ketikkan perintah `al.exe` berikut, sesuaikan dengan lokasi direktori dimana file assembly dan file policy berada:

```
al /link:policy.1.0.MyMathFunction.config/  
out:policy.1.0.MyMathFunction.dll /keyfile:MathKeyFile.snk
```



```
Administrator: Visual Studio 2008 Command Prompt
Setting environment for using Microsoft Visual Studio 2008 x86 tools.
C:\Program Files\Microsoft Visual Studio 9.0\VC>cd C:\MyMathFunction 1.0
C:\MyMathFunction 1.0>al /link:policy.1.0.MyMathFunction.config /out:policy.1.0.
MyMathFunction.dll /keyfile:MathKeyFile.snk
```

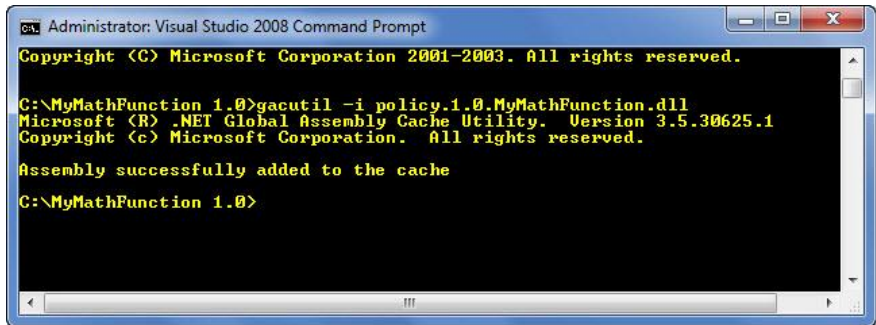
Gambar 11.34 Perintah al.exe

6. Berikut file-file yang terdapat didalam lokasi direktori C:\MyMathFunction 1.0

Name	Date modified	Type	Size
 MathKeyFile	9/16/2009 1:35 AM	Visual Studio Stro...	1 KB
 MyMathFunction.dll	9/16/2009 1:38 AM	Application extens...	15 KB
 policy.1.0.MyMathFunction	9/16/2009 3:35 PM	XML Configuratio...	1 KB
 policy.1.0.MyMathFunction.dll	9/16/2009 3:52 PM	Application extens...	3 KB

Gambar 11.35 File assembly hasil tools al.exe

7. Copy kan file assembly dll tersebut diatas ke GAC dengan menggunakan perintah gacutil.exe seperti yang telah dijelaskan sebelumnya :



```
Administrator: Visual Studio 2008 Command Prompt
Copyright (C) Microsoft Corporation 2001-2003. All rights reserved.

C:\MyMathFunction 1.0>gacutil -i policy.1.0.MyMathFunction.dll
Microsoft (R) .NET Global Assembly Cache Utility. Version 3.5.30625.1
Copyright (c) Microsoft Corporation. All rights reserved.

Assembly successfully added to the cache
C:\MyMathFunction 1.0>
```

Gambar 11.36 Perintah gacutil.exe

8. Buka direktori C:\Windows\Assembly dan perhatikan gambar dibawah ini :

 Policy.1.0.Microsoft.Interop.Security.AzRoles	6.1.760...	31bf3856ad364e35	x86
 policy.1.0.MyMathFunction	0.0.0.0	8bfb4fd6ea3f5e2b	MSIL
 Policy.1.2.Microsoft.Interop.Security.AzRoles	6.1.760...	31bf3856ad364e35	x86

Gambar 11.37 File Publisher Policy yang telah di simpan di GAC

9. Hapus file configuration yang terdapat di lokasi direktori MathClient project (bin\Debug atau bin\Release).
10. Jalankan kembali file MathClient.exe dan klik button yang terdapat didalam form tersebut.
11. Buka kembali file MyLogFile.txt. Dan perhatikan entri baru yang merupakan output dari eksekusi function yang terdapat didalam MyMathFunction.dll v 2.0 tetap dituliskan kedalam file log tersebut :



Gambar 11.38 Output log dari MyMathFunction.dll v 2.0

Apabila kita tidak menginginkan setting yang terdapat di file app.config di override oleh policy yang terdapat di GAC kita tinggal ubah nilai elemen `<publisherPolicy apply="no" />` di app.config. Sudah tentu kita harus mengembalikan kembali file configuration yang telah dihapus diatas ke project.

BAB 12

Fitur Baru VB9

Pada bab ini anda akan diajak untuk mempelajari fitur-fitur terbaru yang ada di VB9. VB9 (VB 2008) adalah release ketiga dari VB yang berjalan diatas platform .NET. VB9 mempunyai beberapa fitur tambahan yang menarik dan sangat memudahkan anda untuk membuat program. Adapun fitur-fitur yang akan kita bahas adalah:

- Implicitly typed local variable
- Object initializer
- Anonymous types
- Nullable types
- If Ternary operator
- Partial Method
- Extension Method
- Relaxed Delegates
- Lambda Expression
- LINQ Query
- XML Literal

Implicitly Typed Local Variable

Pada VB9 anda dapat mendeklarasikan variable tanpa harus mendefinisikan tipe datanya terlebih dahulu. Kompiler VB secara cerdas akan mengetahui tipe data dari variable tersebut dengan cara melihat nilai yang dimasukan kedalam variabelnya. Kompiler VB akan meng-infer secara otomatis tipe data berdasarkan nilai yang dimasukan.

Pada versi VB yang sebelumnya (VB8 .NET 2.0), anda dapat mendeklarasikan variable dengan cara

```
Dim bil As Integer = 99
Dim nama As String = "Erick"
Dim objList As List(Of String) = New List(Of String)
```

Mungkin anda merasa deklarasi variable seperti diatas terlalu panjang, lebih mudah jika kompiler dapat secara langsung menebak tipe datanya sesuai dengan nilai yang anda inputkan. Sebenarnya pada VB8 anda dapat menuliskan:

```
Dim bil = 99
Dim nama = "Erick"
Dim nama As Object
```

Gambar 12.1 Late Binding

Lebih simple kan? namun pada VB8 jika ada mendeklarasikan variable seperti diatas maka compiler secara otomatis akan merubah tipenya menjadi tipe "object" / menganggapnya sebagai "late binding" (option strict = off).

Ini akan sangat tidak efektif karena kompiler harus melakukan proses "boxing" dan "unboxing" untuk merubah dari tipe string ke object, object ke string yang akan menyebabkan performa program menjadi lambat.

Pada VB9 diperkenalkan satu fitur baru yaitu "Option Infer", dengan mengaktifkan option ini menjadi "on" maka Kompiler secara otomatis dapat menebak tipe data apa yang dimiliki oleh variabel dari nilai yang kita berikan di sisi kanan. Secara default VB9 mengaktifkan pilihan option infer "on" atau anda juga dapat menuliskannya secara eksplisit pada kode.

Option Infer On

Jika anda sudah mengaktifkan Option Infer menjadi 'On' sekarang anda dapat mengecek lagi kode yang tadi sudah anda buat di VB8, coba lihat tipe data dari variable yang ada di sisi kiri, maka tipe datanya akan secara otomatis menyesuaikan dengan nilai yang anda berikan disisi kanan.

```
Dim bil = 99
Dim nama = "Erick"
Dim nama As String
```

Auto Infer Menjadi String

Gambar 12.2 Automatic Infer

Object Initializer

Mungkin anda pernah menggunakan VB8 sebelumnya dan bekerja dengan class, jika anda ingin melakukan inisialisasi variabel dalam class maka anda harus membuat overloading constructor, atau harus menuliskan code yang cukup panjang untuk menginisialisasi property satu-persatu, sebagai contoh:

Class Mahasiswa

```
Private _nim As String
Private _ipk As Single

Public Property Nim() As String
    Get
        Return _nim
    End Get
    Set(ByVal value As String)
        _nim = value
    End Set
End Property

Public Property Ipk() As Single
    Get
        Return _ipk
    End Get
    Set(ByVal value As Single)
        _ipk = value
    End Set
End Property
End Class
```

Kemudian untuk membuat object dan menginisialisasi class tersebut anda harus melakukan inisialisasi per fields.

'inisialisasi object biasa
Dim mhs1 As New Mahasiswa()

```
mhs1.Nim = "22002321"  
mhs1.Ipk = 3.5  
  
'atau dapat ditulis  
Dim mhs2 As New Mahasiswa()  
With mhs2  
    .Nim = "22002321"  
    .Ipk = 3.5  
End With
```

Pada VB9 dengan menggunakan Object initializer anda dapat menuliskannya dengan jauh lebih mudah dan simple.

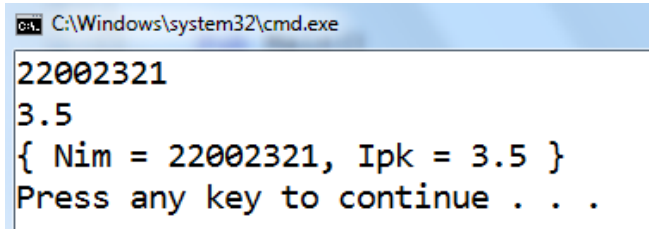
```
'dengan menggunakan object initializer  
Dim mhs3 As New Mahasiswa() With {.Nim = "22002321", .Ipk =  
3.5}
```

Anonymous Type

Pada VB9 Anonymous type sebenarnya dibutuhkan ketika anda menggunakan LINQ (Language Integrated Query) statements. Sintaks dari anonymous type mirip dengan sintaks dari object initializer. Dengan Anonymous Types anda dapat membuat object tanpa harus mendefinisikan class secara eksplisit. Untuk mencoba menggunakan anonymous type buat sebuah console application baru kemudian tulis kode berikut:

```
Sub Main()  
    Dim mhs = New With {.Nim = "22002321", .Ipk = 3.5}  
    Console.WriteLine(mhs.Nim)  
    Console.WriteLine(mhs.Ipk)  
    Console.WriteLine(mhs.ToString())  
End Sub
```

Anda dapat melihat bahwa variable mhs diisi dengan object baru yang langsung dibuat tanpa membuat instan classnya dahulu. Itu yang disebut sebagai anonymous type. Jika dijalankan programnya akan ditampilkan output berikut:



```
C:\Windows\system32\cmd.exe
22002321
3.5
{ Nim = 22002321, Ipk = 3.5 }
Press any key to continue . . .
```

Gambar 12.3 Output Anonymous Type

Dapat anda lihat bahwa variabel `mhs` dapat digunakan dan berisi property `Nim` dan `Nama` padahal anda tidak membuat object Mahasiswa dari class Mahasiswa. Anda juga dapat menggunakan object `mhs` dari anonymous type untuk membuat object baru:

```
Dim mhs2 = New With {.Nim = mhs.Nim, .Nama = "Erick", .Ipk  
= mhs.Ipk}  
Console.WriteLine(mhs2.ToString())
```

Jika anda membandingkan dua object yang sama yang dibuat dengan anonymous type

```
Dim mhs1 = New With {.Nim = "22002321", .Ipk = 3.5}  
Dim mhs2 = New With {.Nim = "22002321", .Ipk = 3.5}  
Console.WriteLine(mhs1.Equals(mhs2))
```

Maka statement diatas akan menghasilkan nilai false biarpun kedua object tersebut isinya sama persis karena kedua object tersebut tidak menunjuk pada referensi yang sama. Hasilnya akan bernilai true hanya jika dibandingkan dengan referensi dari object itu sendiri.

```
Dim mhs1 = New With {.Nim = "22002321", .Ipk = 3.5}  
Dim mhs2 = mhs1  
'menghasilkan nilai true  
Console.WriteLine(mhs1.Equals(mhs2))
```

Ada fitur yang unik di VB9, yaitu anda dapat menggunakan keyword **key** jika anda ingin membuat anonymous type yang mempunyai object referensi yang sama, caranya:

```
'menggunakan keyword key  
Dim mhs1 = New With {Key .Nim = "22002321", .Ipk = 3.5}
```

```
Dim mhs2 = New With {Key .Nim = "22002321", .Ipk = 3.5}  
'menghasilkan nilai true  
Console.WriteLine(mhs1.Equals(mhs2))
```

```
Dim mhs3 = New With {Key .Nim = "22002321", .Ipk = 3.5}  
Dim mhs4 = New With {Key .Nim = "22002322", .Ipk = 3.4}  
'menghasilkan false  
Console.WriteLine(mhs3.Equals(mhs4))
```

Pada aplikasi yang sebenarnya, anonymous types digunakan untuk mensupport LINQ statement untuk menampung hasil kembalian dari LINQ contohnya :

```
Dim mhs = From m In mahasiswa Select New With {m.Nim,  
m.Ipk}
```

Hasil dari query diatas adalah anonymoust type yang ditampung dalam object mhs.

Nullable Types

Seperti kita ketahui pada CLS terdapat dua tipe data yang disupport yaitu value type (integer,double,boolean,etc) dan reference type (String, StringBuilder, etc). Untuk tipe reference (tipe object) jika anda tidak mau memberi nilai maka anda dapat memberi nilai "Nothing", tapi lain halnya dengan value type anda tidak bisa tidak meberi nilai, anda harus memberi salah satu nilai yang sesuai dengan range nilainya (misal untuk tipe short dari -32000 sd 32000). Ini akan dapat menimbulkan masalah ketika kita melakukan mapping dan mengambil data dari database (pada Object Relational Mapping), misal pada table Mahasiswa field Ipik bernilai null, nah kita akan kesulitan untuk merepresentasikan nilai data field yang kosong tersebut dalam object Mahasiswa.

Nullable types digunakan agar anda dapat memberi nilai null pada value type, sebenarnya pada VB8 anda juga sudah dapat menggunakan nullabe type dengan generic Nullable(Of T).

```
Sub Main()  
'pada VB8
```

```
Dim bill As Nullable(Of Integer)
bill = Nothing

'akan bernilai false
Dim cek As Boolean = bill.HasValue
bill = 999
'akan bernilai true
cek = bill.HasValue End Sub
End Sub
```

Tetapi dengan menggunakan nullable type di VB9 penulisannya menjadi lebih mudah.

'nullable pada VB9
Dim i As Integer?

```
Console.WriteLine(i.HasValue) 'akan bernilai false
i = 999
Console.WriteLine(i.HasValue) 'akan bernilai true
```

Nullable types akan sangat berguna ketika anda melakukan mapping table pada LINQ to SQL, atau mapping menggunakan ORM (Object Relational Mapping) framework lain seperti Subsonic.

If Ternary Operator

Pada VB versi sebelumnya anda pasti pernah menggunakan perintah IIF untuk menuliskan single IF statement.

```
'penggunaan iif pada VB
Dim bill As Integer = 999
Dim ket As String = IIf(bill Mod 2 = 1, "Ganjil", "Genap")

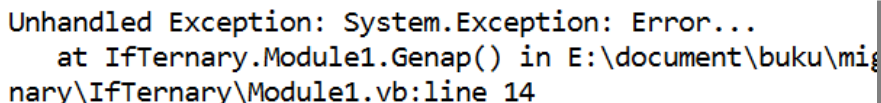
'menghasilkan output Ganjil
Console.WriteLine(ket)
```

Apakah ada yang salah dari kode diatas? memang tidak ada, outputnya benar dan sesuai yang kita harapkan. Tetapi sebenarnya IIF pada VB bukan merupakan built-in function tetapi hanya fungsi biasa yang dipanggil dari library. Seperti halnya regular fungsi maka ketika fungsi tersebut dipanggil

maka semua argumen dalam fungsi tersebut akan dicek oleh VB compiler sebelum dijalankan. Untuk lebih jelasnya coba buat aplikasi console untuk menuliskan kode berikut:

```
Sub Main()  
    Dim bil1 = 999  
    Dim ket As String = IIf(bil1 Mod 2 = 1, Ganjil(),  
        Genap())  
End Sub  
  
Function Ganjil() As String  
    Return "Ganjil"  
End Function  
  
Function Genap() As String  
    Throw New Exception("Error...")  
    Return "Genap"  
End Function
```

Kode diatas jika dijalankan akan menghasilkan error sebagai berikut:



Unhandled Exception: System.Exception: Error...
at IfTernary.Module1.Genap() in E:\document\buku\mig
nary\IfTernary\Module1.vb:line 14

Gambar 12.4 Error IIF

Jika kode diatas dijalankan maka akan menghasilkan error karna ada perintah Throw Exception pada function Genap() akan dieksekusi, meskipun hasil dari IIF diatas function Ganjil() namun function Genap() juga dibaca oleh VB sehingga Throw Exception dipanggil.

Jadi jika anda selama ini menganggap bahwa IIF sama dengan perintah IF biasa anda salah sangka. Kode dibawah ini tidak sama dengan kode diatas

```
Sub Main()  
    Dim bil1 = 999  
    Dim ket = ""  
    If bil1 Mod 2 = 1 Then  
        ket = Ganjil()  
    Else
```

```
        ket = Genap()  
    End If  
    Console.WriteLine(ket)  
End Sub
```

Kode diatas tidak akan menghasilkan error ketika dijalankan, karena function Genap() tidak dicek ketika IF dieksekusi. Ini juga sama dengan perintah IF Ternary pada VB9 (yang ini benar-benar ternary asli tidak seperti statement IIF sebelumnya).

```
Sub Main()  
    'Real IF Ternary pada VB 9  
    Dim bill = 999  
    Dim ket = If(bill Mod 2 = 1, Ganjil(), Genap())  
  
    'menghasilkan output Ganjil  
    Console.WriteLine(ket)  
End Sub
```

Kode diatas juga tidak akan menghasilkan error jika dijalankan karena menggunakan IF Ternary pada VB9. Dengan IF Ternary pada VB anda juga dapat menuliskan hanya 2 argumen dengan saja misal:

```
Dim nama As String = Nothing  
Dim strnama = If(nama, "Kosong")  
'outputnya Kosong  
Console.WriteLine(strnama)
```

```
nama = "Erick Kurniawan"  
strnama = If(nama, "Kosong")  
'outputnya Erick Kurniawan  
Console.WriteLine(strnama)
```

```
Dim bill As Integer? = Nothing  
Dim nilai = If(bill, -1)  
Console.WriteLine(nilai)
```

Harap diperhatikan bahwa tipe data yang dihasilkan dari IF harus kompatibel dengan nilai yang dibandingkan, jika anda menulis kode seperti dibawah ini maka akan menghasilkan error.

```
'error karena bill1 integer bukan string  
Dim nilai2 = If(bill1, "Kosong")
```

Partial Method

Partial Method pada VB9 adalah fitur yang hampir sama dengan Partial Class yang sudah ada pada VB8, Partial Class pada VB8 sangat berguna terutama untuk memisahkan kode yang digenerate secara otomatis oleh Visual Studio.

Partial method pada VB9 digunakan dengan tujuan memudahkan kita untuk merubah method yang ada pada class yang di generate oleh tools, misal pada penggunaan LINQ to SQL. Karena ketika kita melakukan perubahan kode terhadap class yang digenerate oleh tools (misal LINQ to SQL menggunakan SQL Metal) maka ketika kita menggenerate class baru lagi dengan menggunakan tools kemungkinan code kita yang lama akan tertumpuk oleh kode yang digenerate oleh tools tersebut.

Misal class yang digenerate oleh tools sebagai berikut

```
Class Mahasiswa  
    Private _nim As String  
  
    Public Property Nim() As String  
        Get  
            Return _nim  
        End Get  
        Set(ByVal value As String)  
            _nim = value  
        End Set  
    End Property  
End Class
```

Kemudian anda ingin menambahkan method dalam class yang sudah digenerate oleh tools tersebut.

```
Class Mahasiswa  
    Private _nim As String  
    Public Property Nim() As String  
        Get  
            Return _nim
```

```

        End Get
        Set(ByVal value As String)
            CekNim(value)
            _nim = value
        End Set
    End Property

    Private Sub CekNim(ByVal value As String)
        If value = String.Empty Then
            Console.WriteLine("Nim tidak boleh kosong !!")
        End If
    End Sub
End Class

```

Ketika anda melakukan mapping table baru kedalam class (misal pada LINQ to SQL) maka class diatas akan degenerate ulang oleh tools, dan method yang anda tambahkan akan hilang karena ditumpuk oleh kode hasil dari tools.

Jadi bagaimana caranya agar anda dapat menambahkan method pada kode hasil generate dari tools? jawabannya adalah dengan partial method, dengan partial method anda dapat memisahkan method tersebut kedalam file yang berbeda. Penggunaan partial method dapat dilihat pada kode berikut:

```

Class Mahasiswa
    Private _nim As String
    Public Property Nim() As String
        Get
            Return _nim
        End Get
        Set(ByVal value As String)
            CekNim(value)
            _nim = value
        End Set
    End Property

    'implementasi dari CekNim di split di file yang berbeda
    Partial Private Sub CekNim(ByVal value As String)
    End Sub
End Class

```

Partial Class Mahasiswa

```
'implementasi dari method CekNim di generated code
Private Sub CekNim(ByVal value As String)
    If value = String.Empty Then
        Console.WriteLine("Nim tidak boleh kosong !!")
    End If
End Sub
End Class
```

Fitur partial class ini akan sangat berguna ketika anda menggunakan LINQ to SQL dan ingin melakukan modifikasi dari kode class mapping yang digenerate oleh SQL Metal.

Extension Method

Extension method digunakan untuk menambahkan method kedalam class tanpa harus menulis ulang atau mengoverride class tersebut, biasanya digunakan untuk class-class bawaan dari .NET yang kita tidak bisa memodifikasinya langsung (misalnya String).

Untuk menambahkan extension method anda harus menambahkan keyword `<Extension()>`. Misalnya anda ingin membuat method untuk membalik kata, padahal pada class String belum disediakan method tersebut maka anda dapat menambahkannya dengan Extension Method.

Pada VB8 anda dapat menuliskannya sebagai berikut :

```
Sub Main()
    Dim nama = "Erick Kurniawan"
    Dim balikNama = BalikKata(nama).ToLower().Substring(0, 9)
End Sub

Function BalikKata(ByVal kata As String) As String
    Dim kar = kata.ToCharArray()
    Array.Reverse(kar)
    Return New String(kar)
End Function
```

Anda harus memanggil fungsi dengan cara diatas karena memang fungsi BalikKata tidak ada dalam class String, jika anda ingin menambahkan method BalikKata dalam class String maka anda dapat menuliskan:

```
Imports System.Runtime.CompilerServices
```

```
Module ExtensionMethod
```

```
    Sub Main()  
        Dim nama = "Erick Kurniawan"  
        Dim balikNama = _  
            nama.ToLower().BalikKata().Substring(0, 9)  
        Console.WriteLine(balikNama)  
    End Sub  
  
    <Extension()> _  
    Function BalikKata(ByVal kata As String) As String  
        Dim kar = kata.ToCharArray()  
        Array.Reverse(kar)  
        Return New String(kar)  
    End Function
```

```
End Module
```

Dengan menggunakan extension method anda dapat melakukan penulisan dengan cara "chaining" yang akan terasa lebih natural dengan menuliskan:

```
Dim balikNama = nama.ToLower().BalikKata().Substring(0, 9)
```

Relaxed Delegates

Pada VB8 jika anda mengaktifkan pilihan option strict menjadi "Off" maka compiler VB secara otomatis akan mengkonversi dari satu tipe data ke tipe data yang lain tanpa harus menuliskannya secara eksplisit (konversi menggunakan **Ctype()**), misal untuk melakukan narrowing conversion (dari double menjadi integer).

Pada VB9 fasilitas semacam itu juga ditambahkan namun untuk signature delegates. Pada VB8 jika anda menggunakan delegates seperti berikut:

```
Delegate Sub Hello(ByVal h As Object)  
Module Module1  
    Sub Kenalan(ByVal h As String)  
        Console.WriteLine("Helloo" & h)  
    End Sub
```

```

Sub Main()
    'error karena signature tidak sesuai
    Dim hai As New Hello(AddressOf Kenalan)
    hai("Erick")
End Sub
End Module

```

Maka kompiler VB akan memberikan pesan error bahwa tipe datanya tidak sama beda signature delegates antara "object" dan "string".

Tapi pada VB9 kasus seperti diatas tidak akan terjadi lagi karena compiler VB9 sudah cukup smart untuk mengkonversi dari "object" ke "string". Namun anda harus ingat bahwa relaxed delegates hanya dapat digunakan jika pengaturan Option Strict 'Off'.

Lambda Expression

Lambda Expression adalah istilah yang diambil dari functional programming, Lambda Expression digunakan untuk memudahkan anda membuat fungsi atau method. contoh penulisan Lambda Expression

Function (e) e + 1

Fungsi diatas mempunyai satu parameter e, dan mengembalikan e+1.

Untuk menunjukan penggunaan Lambda Expression pada VB, maka sebagai contoh kita akan membuat array berisi nilai integer dan memfilter nilai yang lebih besar dari 6 dengan Lambda Expression.

```

Sub Main()
    Dim arrBil() = {4, 5, 7, 9, 12, 3, 2}
    Dim filter = arrBil.Where(Function(b) b > 5)
    For Each i In filter
        Console.WriteLine(i)
    Next
End Sub

```

Lambda Expression dan Delegates

Lambda Expression dapat digunakan untuk mempermudah penggunaan delegates di VB. Tanpa Lambda anda harus menuliskan kode berikut:

```
'deklarasi delegates
Public Delegate Function UbahInt(ByVal x As Integer) As Integer

'buat fungsi yang akan ditunjuk oleh delegates
Function Gandakan(ByVal x As Integer) As Integer
    Return x * 2
End Function
```

Kemudian buat instance delegates sebagai berikut

```
Sub Main()
    Dim mydel As UbahInt = New UbahInt(AddressOf Gandakan)
    Console.WriteLine("{0}", mydel(5))
End Sub
```

Maka program diatas akan mencetak 10. Dengan menggunakan Lambda Expression anda dapat menuliskan :

```
Public Delegate Function UbahInt(ByVal x As Integer) As Integer
Sub Main()
    Dim mydel As UbahInt = Function(x) x * 2
    Console.WriteLine(mydel(5))
End Sub
```

Program diatas juga akan mencetak 10 sama dengan sebelumnya, tapi dengan menggunakan Lambda Expression penambahan method menjadi lebih mudah.

Menggunakan Lambda dengan Dua Parameter

Anda juga dapat menggunakan Lambda dengan dua parameter, cara penulisannya:

```
'delegates dengan dua parameter
Public Delegate Function Kali(ByVal x As Integer, ByVal y
As Integer) As Integer
```

```
'mencetak 5*6=30
Sub Main()
    Dim mydel = Function(x, y) x * y
    Console.WriteLine(mydel(5, 6))
End Sub
```

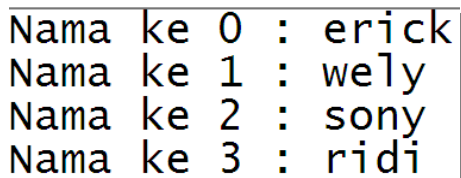
Anda juga dapat menggunakan `Select()` extension method dengan Lambda Expression yang mempunyai dua parameter, parameter pertama untuk nilai sedangkan parameter kedua untuk index

```
Dim nama() = {"erick", "wely", "sony", "ridi"}

Dim filter = nama.Select(Function(x, i) "Nama ke " +
i.ToString() + " : " + x)

For Each i In filter
    Console.WriteLine(i)
Next
```

Jika program dijalankan akan mencetak:



```
Nama ke 0 : erick
Nama ke 1 : wely
Nama ke 2 : sony
Nama ke 3 : ridi
```

Gambar 12.4 Output Lambda Dua Parameter

Statement Lambda Expression

VB9 belum mendukung Statement Lambda Expression seperti yang ada pada C# 3.0, VB akan mensupport fitur ini pada versi VB10. Jika menggunakan Statement Lambda Expression di C# anda dapat menuliskan:

```
//menggunakan Statement Lamda pada C#
string[] nama = new[] {
"erick","wely","sony","ridi","eriawan" };

```

```

foreach (string n in nama.Where(
    x => {
        if (x.Contains("eri"))
            return true;
        else
            return false;
    })
)

```

Console.WriteLine(n);

Dengan VB9 anda dapat menghasilkan output yang sama dengan menuliskan kode sebagai berikut:

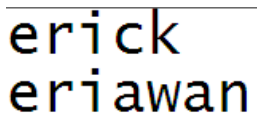
```

Function CekNama(ByVal nama As String)
    If nama.Contains("eri") Then
        Return True
    Else
        Return False
    End If
End Function

Sub Main()
    Dim nama() = {"erick", "wely", "sony", "eriawan"}
    Dim filter = nama.Where(Function(x) CekNama(x))
    For Each n In filter
        Console.WriteLine(n)
    Next
End Sub

```

maka akan dicetak output:



```

erick
eriawan

```

Gambar 12.5 Output Statement Lambda Expression

LINQ (Language Integrated Query)

LINQ (Language Integrated Query) adalah fitur yang mensupport sintaks query pada general purpose programming

language, anda dapat melakukan query ke berbagai sumber data seperti object, Database, XML, dll.

LINQ to Object / LINQ to Memory

Misal anda akan mengambil data dari object Array, anda dapat menuliskan kode:

```
Dim nilai() = {12, 34, 56, 11, 45, 66}
Dim query = From n In nilai _
             Where n Mod 2 = 0 _
             Select n
For Each bil In query
    Console.WriteLine(bil)
Next
```

Query diatas akan mengembalikan bilangan yang genap, anda dapat menuliskannya dalam bentuk Lambda sebagai berikut:

```
Dim query = nilai.Where(Function(x) x Mod 2 =
0).Select(Function(x) x)
For Each bil In query
    Console.WriteLine(bil)
Next
```

Object yang dapat diquery dengan LINQ harus bertipe IEnumerable(Of T), Select() dan Where() adalah Extension Method yang ditambahkan ke IEnumerable(Of T), asal dari Select() dan Where() sebenarnya dari System.Linq.Enumerable, aslinya anda dapat menuliskan:

```
Dim query = Enumerable.Select(Enumerable.Where(nilai,
Function(x) x Mod 2 = 0), Function(x) x)
For Each bil In query
    Console.WriteLine(bil)
Next
```

LINQ to SQL

Selain query ke object, anda dapat juga melakukan query ke database SQL Server, menggunakan OR Mapping tools yang bernama LINQ to SQL. ORM digunakan untuk mapping dari database relational ke object. Misal anda mempunyai database dengan nama MhsDb, kemudian dalam database tersebut terdapat satu table dengan nama Mahasiswa dan fields berikut (nim char(8) primary key, nama varchar(50), dan ipk float).

Pertama buat class dengan nama Mahasiswa.vb, kemudian mapping table Mahasiswa kedalam class:

```
Imports System.Data.Linq.Mapping
<Table(Name:="dbo.Mahasiswa")> _
Public Class Mahasiswa
    Private _nim As String
    <Column(IsPrimaryKey:=True)> _
    Public Property Nim() As String
        Get
            Return _nim
        End Get
        Set(ByVal value As String)
            _nim = value
        End Set
    End Property

    Private _nama As String
    <Column()> _
    Public Property Nama() As String
        Get
            Return _nama
        End Get
        Set(ByVal value As String)
            _nama = value
        End Set
    End Property

    Private _ipk As Single
    <Column()> _
    Public Property Ipk() As Double
        Get
```

```

        Return _ipk
    End Get
    Set(ByVal value As Double)
        _ipk = value
    End Set
End Property
End Class

```

Anda dapat juga melakukan mapping secara otomatis menggunakan fasilitas "LINQ to SQL Class" atau "SQL Metal".

Untuk mengakses data menggunakan LINQ to SQL kita harus membuat object DataContext terlebih dahulu, kemudian ambil table yang akan kita query

```

Imports System.Data
Imports System.Linq
Imports System.Data.Linq

Sub Main()
    Dim strConn = "Data Source=.\SQLEXPRESS;Integrated
        Security=SSPI;Initial Catalog=MhsDb;"
    Dim db As New DataContext(strConn)
    Dim tMahasiswa = db.GetTable(Of Mahasiswa)()

    Dim query = From m In tMahasiswa _
        Select m

    For Each mhs In query
        Console.WriteLine(mhs.Nim & " " & mhs>Nama & _
            " " & mhs.Ipk)
    Next
End Sub

```

LINQ Query Operator

Beberapa operator LINQ yang dapat digunakan adalah:

From : source dari query yang akan digunakan

From <tampung> [As<type>] In <source>

- Tampung berisi variable tampung yang akan digunakan dalam query
- Type berisi tipe data dari tampung (tapi tidak harus disebutkan karena vb akan secara otomatis

menggunakan local type inference untuk menentukan tipenya). biasanya dapat berupa IEnumerable(Of T) atau IQueryable(Of T)

- Source berisi referensi ke object source, dapat berupa in-memory collection atau IQueryable(Of T) object

Select : mendefinisikan projection clause dalam query

Select <projection>

- Projection dapat berisi expression yang mengembalikan satu object, dapat juga berupa tampung dari From clause, atau object yang lebih kompleks.

[alias =] <column>

- Anda juga dapat mendefinisikan alias (sifatnya optional), mengindikasikan nama dari property yang akan digenerate oleh anonymous type yang mewakili projection.
- Column mengacu ke fields atau property pada tampung, tapi dapat juga berupa expression yang lebih kompleks

Contoh:

```
'mencetak 9 8 7 6 5 4
Dim bil = From i In New Integer() {9, 8, 7, 6, 5, 4} Select
i
For Each i In bil
    Console.WriteLine(i)
Next

'menghasilkan anonymous type
{Dobel=2,Tripel=3},{Dobel=4,Tripel=6}
Dim bil2 = From i In New Integer() {1, 2} Select Dobel = 2
* i, Tripel = 3 * i
For Each i In bil2
    Console.WriteLine(i.Dobel & " " & i.Tripel)
Next
```

Order By : digunakan untuk mengurutkan hasil query

Order By <ordering> [Ascending] [Descending]

- Ordering berisi expression key yang digunakan untuk proses pengurutan

Contoh:

```
Dim mhs() = {New With {.Nim = "22081234", .IPK = 3.2}, _
             New With {.Nim = "23081235", .IPK = 3.5}, _
             New With {.Nim = "23081231", .IPK = 2.5}}
Dim query = From m In mhs Order By m.Nim Acending Order By
m.IPK Select m.Nim, m.IPK
For Each m In query
    Console.WriteLine(m.Nim & " " & m.IPK)
Next
```

Distinct : melakukan filter terhadap nilai yang terduplikasi dari query

Contoh:

```
'menghasilkan 1 2 3 4
Dim bil = From i In New Integer() {1, 1, 1, 2, 3, 4} Select
i Distinct
For Each b In bil
    Console.Write(b & " ")
Next
```

Skip dan **Take**: untuk memfilter range dari nilai hasil query

Take<number> : poisisi

Skip <number> : banyak nilai yang diambil

Contoh:

```
'menghasilkan 5,6 (ambil dari posisi ke 4 sebanyak 2 nilai)
Dim bil = From i In New Integer() {1, 2, 3, 4, 5, 6, 7, 8,
9, 10} Select i Skip (4) Take (2)
For Each b In bil
    Console.Write(b & " ")
Next
```

Aggregate : digunakan untuk operasi aggregation, contoh agregation

- Count

- Sum
- Min
- Max
- Average

Aggregate <tampung> In <source> Into <aggregation list>

Contoh:

'penggunaan Sum

```
Dim tot = Aggregate i In (From i In New Integer() {1, 2, 3, 4, 5, 6} Select i) Into Total = Sum()
Console.WriteLine(tot)
```

'penggunaan Count

```
Dim jml = Aggregate i In (From i In New Integer() {1, 2, 3, 4, 5, 6} Select i) Into Jumlah = Count()
Console.WriteLine(jml)
```

'penggunaan Min

```
Dim min = Aggregate i In (From i In New Integer() {1, 2, 3, 4, 5, 6} Select i) Into Minimal = Min()
Console.WriteLine(min)
```

'penggunaan Average

```
Dim rata = Aggregate i In (From i In New Integer() {1, 2, 3, 4, 5, 6} Select i) Into Minimal = Average()
Console.WriteLine(rata)
```

Grouping : digunakan untuk membuat group berdasarkan group expression yang dibuat

Group By <key> Into Aggregate

Contoh:

```
Dim mhs() = {New With {.Nim = "22081234", .Jurusan = "TI", .IPK = 3.2}, _
             New With {.Nim = "23081235", .Jurusan = "SI", .IPK = 3.5}, _
             New With {.Nim = "23081231", .Jurusan = "SI", .IPK = 2.5}}
Dim query = From m In mhs Group By m.Jurusan Into Anggota = Group
```



```

For Each jur In query
    Console.WriteLine(jur.Jurusan)
    For Each m In jur.Anggota
        Console.WriteLine("- NIM : " & m.Nim & " IPK : " & _
            m.IPK)
    Next
Next

```

Join : menggabungkan multiple source bersama berdasarkan kondisi join tertentu

Join <tampung> In <source> On <key1> equals <key2> and

```

Dim mhs() = {New With {.Nim = "22081234", .Jurusan = "TI",
    .IPK = 3.2}, New With {.Nim = "23081235", .Jurusan = "SI",
    .IPK = 3.5}, New With {.Nim = "23081231", .Jurusan = "SI",
    .IPK = 2.5}}

```

```

Dim kul() = {New With {.Kode = "IM2043", .Nama = "Web
Database"}, New With {.Kode = "IM2033", .Nama = "Rekayasa
Web"}}

```

```

Dim ambil() = {New With {.Id = 1, .Kode = "IM2043", .Nim =
    "22081234"}, New With {.Id = 2, .Kode = "IM2033", .Nim =
    "23081235"}, New With {.Id = 3, .Kode = "IM2043", .Nim =
    "23081231"}}

```

```

Dim query = From m In mhs _
    Join a In ambil On m.Nim Equals a.Nim _
    Join k In kul On a.Kode Equals k.Kode _
    Select m.Nim, k.Nama

```

```

For Each hsl In query
    Console.WriteLine("Mahasiswa dengan Nim : " &
        hsl.Nim & " mengambil Matakuliah : " & hsl.Nama)
Next

```

LINQ to XML dan XML Literal

XML Literal. Ini adalah fitur dari VB9 yang menurut saya paling "cool", pada VB9 XML menjadi "first class citizen" yang berarti anda dapat menuliskan XML secara Literal.

Pada versi sebelumnya (VB8) untuk membuat file XML anda harus menggunakan System.Xml untuk DOM manipulation, contoh kodenya:

```
Imports System.IO
Imports System.Xml
Module Module1
    Sub Main()
        Using fs As New FileStream("produksample.xml",
            FileMode.Create)
            Using wt As XmlWriter = XmlWriter.Create(fs)
                wt.WriteStartDocument()
                wt.WriteStartElement("Products")
                wt.WriteStartElement("Product")
                wt.WriteAttributeString("Id", "KB001")
                wt.WriteElementString("Name", "Laptop
                    Acer")
                wt.WriteElementString("Price", "7000000")
                wt.WriteEndElement()
                wt.Flush()
            End Using
        End Using
    End Sub
End Module
```

Kode diatas akan menghasilkan file "produksample.xml" sebagai berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<Products>
    <Product Id="KB001">
        <Name>Laptop Acer</Name>
        <Price>7000000</Price>
    </Product>
</Products>
```

LINQ to XML

Pada VB9 (.NET 3.5) anda dapat menuliskannya dengan cara yang baru. VB9 mendukung **XElement** class yang terdapat pada namespace System.Xml.Linq, dengan menggunakan cara penulisan ini code untuk pembuatan dokumen XML tampak menjadi lebih "clean". Class yang ada pada System.Xml.Linq adalah

Class	Deskripsi
XAttribute	Merepresentasikan atribut yang dibuat
XDocument	Merepresentasikan Complete XML Tree
XElement	Merepresentasikan XML Element, dan sebagai dasar untuk mengkonstruksi XML Tree
XName	Merepresentasikan atribut dan element names
XNode	Merepresentasikan base class dari XML Node

Untuk membuat dokumen xml seperti contoh sebelumnya dengan menggunakan XElement anda dapat menuliskan kode berikut:

```
Imports System.Xml.Linq
Module ContohXElement
    Sub Main()
        Dim xmlProd = New XElement("Products", _
            New XElement("Product", _
                New XAttribute("Id", "KB001"), _
                New XElement("Name", "Laptop Acer"), _
                New XElement("Price", "7000000")))
        xmlProd.Save("xelementProd.xml")
    End Sub
End Module
```

XML Literal

VB9 juga menawarkan cara yang lebih simple dan clean selain menggunakan XElement yaitu XML Literal, saat ini XML Literal hanya disupport oleh VB9, C# 3.0 belum menssupport fitur ini. dengan XML Literal anda dapat menuliskan kode berikut:

```
Imports System.Xml.Linq
Module ContohXElement
```

```

Sub Main()
    Dim xmlProd = <Products>
        <Product Id="KB001">
            <Name>Laptop Acer</Name>
            <Price>7000000</Price>
        </Product>
    </Products>
    xmlProd.Save("literalprod.xml")
End Sub
End Module

```

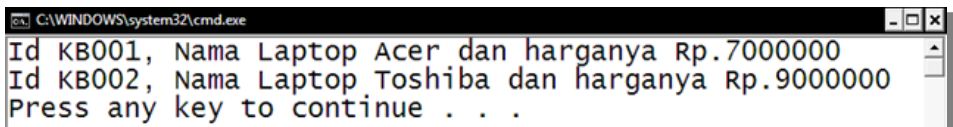
Anda juga dapat mengambil nilai dari attribute atau elemen dalam dokumen tersebut dengan iterasi

```

Dim xmlProd = <Products>
    <Product Id="KB001">
        <Name>Laptop Acer</Name>
        <Price>7000000</Price>
    </Product>
    <Product Id="KB002">
        <Name>Laptop Toshiba</Name>
        <Price>9000000</Price>
    </Product>
</Products>
For Each produk In xmlProd.<Product>
    Console.WriteLine("Id {0}, Nama {1} dan harganya
Rp.{2}", _
        produk.@Id, _
        produk.<Name>.Value, _
        produk.<Price>.Value)
Next

```

Hasilnya:



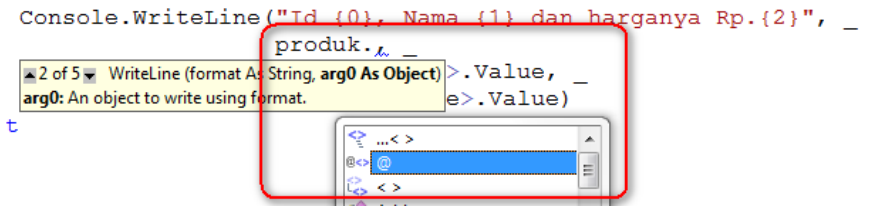
```

C:\WINDOWS\system32\cmd.exe
Id KB001, Nama Laptop Acer dan harganya Rp.7000000
Id KB002, Nama Laptop Toshiba dan harganya Rp.9000000
Press any key to continue . . .

```

Gambar 12.6 Output XML Literal

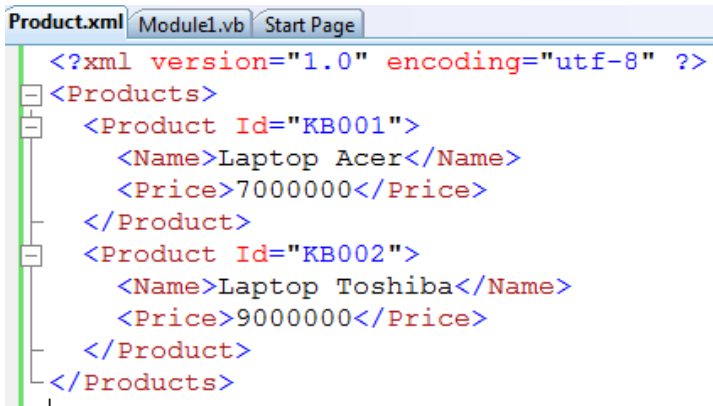
Sangat mudah bukan? jika kita perhatikan ketika anda melakukan iterasi dokumen xml diatas, VS 2008 akan secara otomatis memberikan intellisense untuk membantu anda menuliskan kode.



Gambar 12.7 Intellisense XML Literal

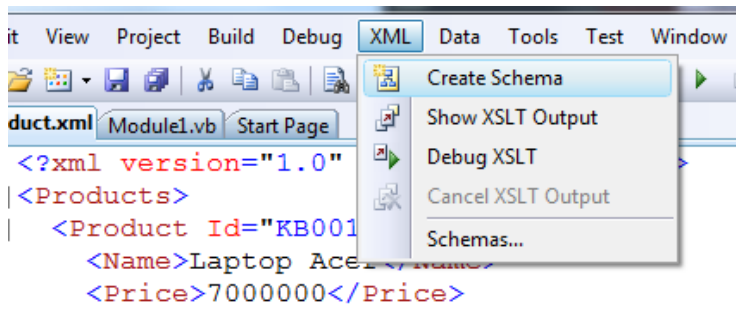
Namun intellisense tersebut tidak dapat menampilkan keterangan nama elemen atau atribut yang ada pada dokumen XML yang anda buat, karena IDE tidak memiliki informasi tentang XML Schema anda, untuk menyelesaikan masalah ini anda dapat mengimports xml shema dari dokumen. Caranya tambahkan XML Schema, cara yang paling mudah adalah dengan menggunakan fitur pada VS 2008.

Pertama copykan file XML yang anda buat kedalam sebuah file baru dengan nama 'Product.xml'. caranya klik kanan pada project ► Add ► New Item ► pilih XML File ► beri nama 'Product.xml'.



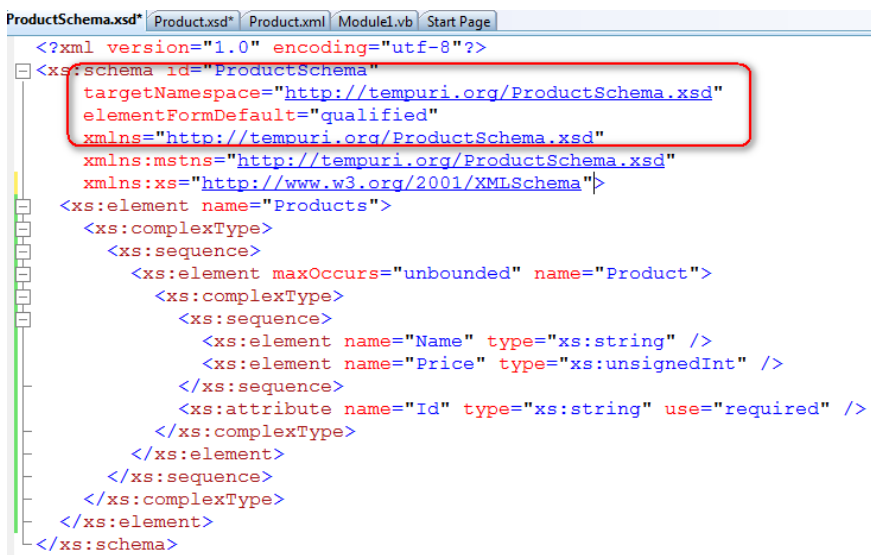
Gambar 12.8 Product.xml

Kemudian pilih menu XML ► Create Schema, untuk membuat XML Schema dari dokumen tersebut, maka otomatis Visual Studio akan menggenerate XML Schema untuk anda.



Gambar 12.9 Create Schema

Selanjutnya buat ProductSchema.xsd kemudian copykan hasil genereta schema pada langkah sebelumnya kedalam file xml schema tersebut.



Gambar 12.10 ProductSchema.xsd

Jangan lupa untuk mengisi property 'targetNamespace' pada XML Schema diatas. Setelah itu kembali ke kode program anda kemudian lakukan imports dokumen xml schema yang sudah anda buat.

```
Imports System.Xml.Linq
Imports <xmlns="http://tempuri.org/ProductSchema.xsd">
```

Gambar 12.11 Imports XML Schema

Dengan cara mengimports xml schema yang sudah anda buat maka intellisense pada Visual Studio dapat menampilkan bantuan secara lebih jelas terhadap dokumen xml yang sedang anda proses.

```
For Each produk In xmlProd.<
    Console.WriteLine("Id {1} dan harganya Rp.{2}", _
        produk.Id, produk.Price, produk.Product)
Next
```

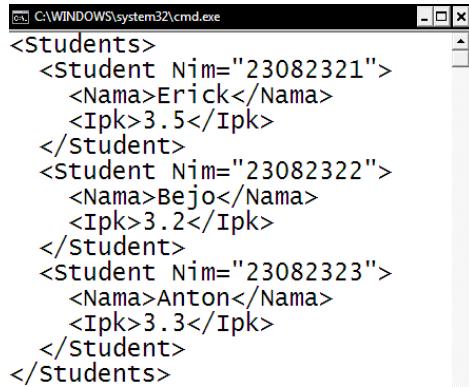
Gambar 12.12 Intellisense yang lebih baik

Embedded Expression

Anda juga dapat menambahkan embedded expression dalam XML Literal, jika anda pernah menggunakan ASP.NET pasti familiar dengan cara penulisan embedded expression menggunakan keyword `<%= %>`. Contoh penggunaannya.

```
Imports System.Xml.Linq
Module EmbeddedExpression
    Sub Main()
        Dim mahasiswa() = {New With {.Nim = "23082321",
            .Nama = "Erick", .Ipk = 3.5}, New With {.Nim =
            "23082322", .Nama = "Bejo", .Ipk = 3.2},
            New With {.Nim = "23082323", .Nama = "Anton",
            .Ipk = 3.3}}
        Dim xmlStudent = <Students>
            <%= From mhs In mahasiswa _
                Select _
                    <Student Nim=<%= mhs.Nim %>>
                        <Nama><%= mhs.Nama %></Nama>
                        <Ipk><%= mhs.Ipk %></Ipk>
                    </Student> _
                %>
            </Students>
```

```
xmlStudent.Save("students.xml")
Console.WriteLine(xmlStudent.ToString())
End Sub
End Module
```

A screenshot of a Windows command prompt window titled "C:\WINDOWS\system32\cmd.exe". The window displays the following XML output:

```
<Students>
  <Student Nim="23082321">
    <Nama>Erick</Nama>
    <Ipk>3.5</Ipk>
  </Student>
  <Student Nim="23082322">
    <Nama>Bejo</Nama>
    <Ipk>3.2</Ipk>
  </Student>
  <Student Nim="23082323">
    <Nama>Anton</Nama>
    <Ipk>3.3</Ipk>
  </Student>
</Students>
```

Gambar 12.13 Embedded Expression

Pada kode diatas kita menggenerate dokumen xml menggunakan nilai yang diambil dari anonymous types yang sudah kita buat sebelumnya, anda juga dapat menggenerate dokumen XML dengan data yang anda ambil dari database, dll. Embedded Expression juga dapat digunakan untuk menggenerate "VB View" secara dinamis pada ASP.NET MVC.

BAB 13

LINQ to SQL

Linq To Sql merupakan teknik terbaru untuk data access yang disediakan didalam .NET framework versi 3.5. Dengan teknik ini kita melakukan query atau manipulasi data dengan bahasa .NET itu sendiri yang selama ini harus dilakukan dengan menggabungkannya bersama bahasa SQL (Structured Query Language).

Ada beberapa keuntungan dari penggunaan Linq To SQL ini yaitu sifatnya yang strongly typed, programmer yang tidak familiar dengan bahasa SQL dapat memanfaatkan skill nya mereka di bahasa .NET nya masing-masing.

Dengan strongly typed nya Linq To Sql dapat terhindar dari kesalahan penetikkan perintah-perintah SQL atau pengaksesan nama kolom, dan tabel. Kelebihan lainnya kalau kita bandingkan dengan ADO.NET yaitu jumlah kode yang digunakan untuk akses dan manipulasi data jauh lebih sedikit dengan menggunakan Linq To SQL.

Dari sisi performansi untuk perbandingan antara ADO.NET dengan Linq To Sql tidak ada perbedaannya. Linq To Sql juga dapat memanfaatkan stored procedure.

Linq To Sql Designer

Untuk memudahkan dalam menggunakan Linq To Sql di Visual Studio 2008 dengan .NET framework 3.5 nya telah disediakan designer untuk membuat mapping object-object terhadap tabel-tabel yang terdapat didalam sebuah database.

Hal yang perlu dilakukan dalam melakukan design Linq To Sql terhadap object-object database yaitu dengan melakukan drag dan drop object-object database seperti tabel, stored procedure, view, dan function ke dalam O/R (Object/Relational) designer atau Linq To Sql designer.

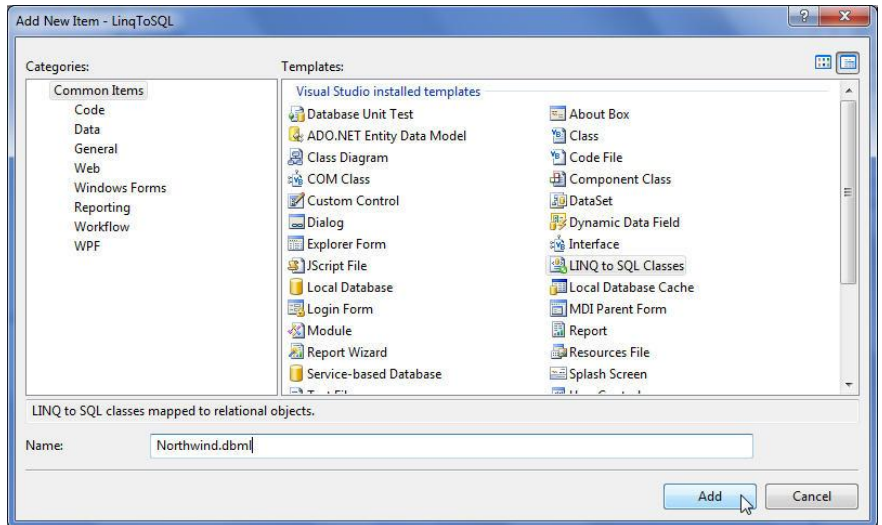
Semua object akan dibuatkan class, dimana di dalam class tersebut terdapat mapping object terhadap tabel, view, function, stored procedure, dll. File ekstension yang dihasilkan yaitu .dbml. Di dalam file itulah semua mapping object disimpan.

Class yang dihasilkan merupakan mapping terhadap masing-masing tabel. Properti yang dihasilkan di dalam class tersebut merupakan mapping terhadap kolom-kolom yang terdapat di dalam tabel tersebut.

Class tersebut dibuat dengan menggunakan teknik Partial Class. Partial class merupakan teknik untuk memisahkan atau memecah class-class kedalam file yang berbeda. Salah satu kegunaannya yaitu dapat kita lihat didalam implementasi Linq To Sql ini. Apabila kita melakukan drag dan drop ulang object-object database kedalam O/R designer maka kode-kode yang dihasilkan secara otomatis akan di tulis ulang. Hal tersebut dapat menimbulkan masalah apabila kita telah melakukan modifikasi ulang class tersebut. Maka semua kode yang kita buat akan hilang kembali. Oleh karena itulah disediakan partial class, dimana semua modifikasi kode yang kita lakukan untuk class yang bersangkutan disimpan di file yang berbeda.

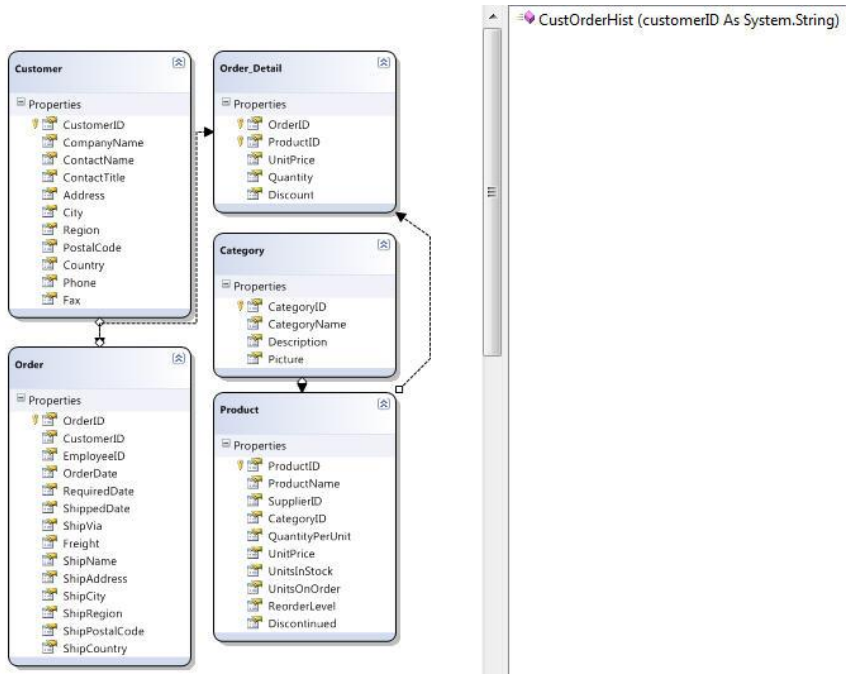
Berikut adalah contoh bagaimana menggunakan Linq To Sql untuk melakukan mapping object-object database :

1. Buat project baru Windows Forms Application, beri nama LinqToSQL.
2. Tambahkan item baru kedalam project tersebut dengan melakukan klik kanan project dan pilih Add - New Item...
3. Pilih LINQ To SQL Classes item dan beri nama Northwind.dbml



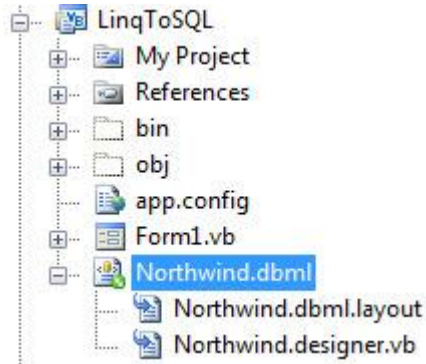
Gambar 13.1 LINQ To SQL Classes item

4. Buka jendela Server Explorer, koneksikan ke database Northwind yang terdapat didalam SQL Server.
5. Drag dan drop tabel Customers, Orders, OrderDetails, Category, dan Products. Selain itu juga drag dan drop stored procedure CustOrderHist pada panel kanan O/R Linq To SQL designer.



Gambar 13.3 Linq To SQL designer

- File Northwind.dbml dapat dilihat jendela Solution Explorer. Untuk melihat class yang dihasilkan oleh designer tersebut semua files yang terdapat didalam project tersebut harus ditampilkan semua dengan klik menu Show All Files di toolbar Solution Explorer.



Gambar 13.4 File Linq To SQL

Basic Query Expression

Untuk membaca data dengan menggunakan Linq dibutuhkan query expression. Query expression ini dibentuk dengan menggunakan fitur Local Typed Inference. Perintah minimal yang harus ada yaitu perintah From dimana From ini membaca data terhadap koleksi sumber data yang digunakan. Selain itu terdapat perintah Select yang sifatnya optional apabila kita ingin menampilkan semua member atau kolom didalam sebuah class yang merupakan mapping terhadap tabel di database.

DataContext adalah class utama yang berisi shortcut-shortcut terhadap class-class yang dihasilkan berdasarkan tabel-tabel yang di drag dan drop ke dalam O/R designer. Lewat class DataContext inilah kita lakukan pembacaan dan manipulasi data.

Berikut adalah contoh kode program bagaimana caranya melakukan query sederhana terhadap tabel customers yang terdapat didalam NorthwindDataContext class :

1. Tambahkan button ke dalam form yang telah dibuat pada project diatas, beri nama btnGetCustomers.
2. Deklarasikan class level variabel berikut ini :

```
Private db As New NorthwindDataContext
```

3. Tambahkan control ListBox kedalam form.
4. Ketikkan kode berikut pada event click btnGetCustomers :

```
Private Sub btnGetCustomers_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
    Handles btnGetCustomers.Click

    ListBox1.Items.Clear()

    Dim custData = From c In db.Customers Select c

    db.Log = Console.Out

    For Each cust In custData
        ListBox1.Items.Add(cust.CustomerID & "-" _
                            & cust.CompanyName)
    Next

End Sub
```

Query expression dibentuk dengan perintah Dim, From dan Select. Variabel custData akan membaca sendiri tipe data yang dikembalikan dari hasil query expression, fitur inilah yang dinamakan Local Type Inference.

Variabel c merupakan variabel elemen yang merujuk ke collection didalam DataContext class dalam hal ini Customers class. Perintah Select sebenarnya optional apabila kita ingin semua kolom yang terdapat didalam tabel customers tersebut ditampilkan semua maka perintah Select tidak perlu ditulis, kecuali apabila hanya beberapa kolom saja yang ingin ditampilkan.

Nama-nama kolom didalam tabel akan muncul semua sebagai properti di jendela intellisense, inilah salah satu

keuntungan dari Linq To Sql apabila dibandingkan dengan penggunaan bahasa SQL di ADO.NET.

Perintah Log yang ditampilkan ke Console.Out digunakan untuk menampilkan hasil transformasi perintah Linq To SQL kedalam perintah SQL nya itu sendiri. Anda dapat melihatnya di jendela Output Visual Studio seperti yang dapat dilihat pada gambar dibawah ini :

```
SELECT [t0].[CustomerID], [t0].[CompanyName], [t0].[ContactName], [t0].[ContactTitle],  
FROM [dbo].[Customers] AS [t0]
```

Gambar 13.5 Hasil transformasi perintah Linq To SQL ke perintah SQL

Filtering Record

Untuk menampilkan data berdasarkan kriteria atau kondisi tertentu digunakan perintah Where. Selain itu juga terdapat beberapa perintah lain seperti StartsWith, EndsWith, Equals yang dihubungkan dengan klausa Where untuk melakukan pencarian data.

Berikut adalah contoh kode program untuk melakukan filtering data.

1. Tambahkan button, beri nama btnFilteringRec dan tambahkan kode berikut pada event click :

```
Private Sub btnFilteringRec_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnFilteringRec.Click  
  
    ListBox1.Items.Clear()  
  
    Dim custData = From c In db.Customers _  
                    Where c.CompanyName.StartsWith("A") AndAlso  
-                    c.Country.Equals("UK") _  
                    Select c.CustomerID, c.CompanyName  
  
    db.Log = Console.Out
```

```
For Each cust In custData
    ListBox1.Items.Add(cust.CustomerID & "-" _
                      & cust.CompanyName)
Next
```

```
End Sub
```

Kode diatas digunakan untuk melakukan pencarian data customers dimana nama company name nya diawali dengan huruf A dan berada di negara UK. Kolom hasil yang ditampilkan hanya CustomerID dan CompanyName yang didefinisikan pada perintah Select. Anda dapat melihat hasil transformasi perintah Linq To SQL ke SQL di jendela Output.

Perintah Distinct

Perintah Distinct sama seperti halnya perintah Distinct yang terdapat di SQL, yaitu digunakan untuk membaca single value data. Apabila terdapat beberapa record yang memiliki nilai yang sama maka yang diambil hanya satu.

Berikut adalah contoh penggunaan perintah Distinct di Linq To SQL untuk menampilkan data country di tabel customers :

1. Tambahkan button, beri nama btnDistinct dan tambahkan kode berikut pada event click :

```
Private Sub btnDistinct_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnDistinct.Click

    ListBox1.Items.Clear()

    Dim custData = (From c In db.Customers _
                    Where c.Country <> "" _
                    Select c.Country).Distinct
```

```
db.Log = Console.Out

For Each cust In custData
    ListBox1.Items.Add(cust)
Next
```

```
End Sub
```

Perintah Single

Perintah single digunakan untuk mencari data yang sifatnya spesifik berdasarkan kriteria pencarian tertentu yang hanya mengembalikan satu record.

Berikut adalah contoh penggunaan perintah single untuk mencari data customers berdasarkan nilai customerid :

1. Tambahkan button, beri nama btnFindSingleRec dan tambahkan kode berikut pada event click :

```
Private Sub btnFindSingleRec_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnFindSingleRec.Click

    ListBox1.Items.Clear()

    Dim custData = (From c In db.Customers _
                    Where c.CustomerID.Equals("Alfki")
-
                    Select c).Single

    db.Log = Console.Out

    ListBox1.Items.Add(custData.CustomerID & "-" _
                        & custData.CompanyName)

End Sub
```

Berbeda dengan perintah sebelumnya, hasil perintah single ini hanya menghasilkan satu record sehingga kita tidak harus melakukan looping terhadap data hasil pencarian.

Perintah Count

Fungsi-fungsi aggregate yang kita kenal di SQL seperti Count, Sum, Min, Max, dan Avg dapat juga kita gunakan di Linq To SQL.

Berikut adalah contoh penggunaan perintah Count yang digunakan untuk menghitung berapa kali customerid tertentu melakukan transaksi :

1. Tambahkan button, beri nama btnAggFunc dan tambahkan kode berikut pada event click :

```
Private Sub btnAggFunc_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnAggFunc.Click  
  
    ListBox1.Items.Clear()  
  
    Dim orderData = (From o In db.Orders _  
                     Where o.CustomerID.Equals("Alfki")  
-                     Select o).Count  
  
    MessageBox.Show("Order Count for Alfki : " &  
orderData.ToString)  
  
End Sub
```

Perintah Join

Sama seperti halnya di SQL, di Linq To SQL kita juga dapat melakukan perintah Join untuk menggabungkan data dari beberapa tabel dengan menggunakan penghubung berupa nilai kolom yang sama dari tabel-tabel yang digabungkan.

Berikut adalah contoh penggunaan perintah Join yang digunakan untuk menampilkan data dari tabel customers dan orders dimana kolom penghubungnya yaitu kolom CustomerID:

1. Tambahkan button, beri nama btnJoin dan tambahkan kode berikut pada event click :

```
Private Sub btnJoin_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnJoin.Click

    ListBox1.Items.Clear()

    Dim custOrders = From c In db.Customers _
        Join o In db.Orders On _
        c.CustomerID Equals (o.CustomerID) _
        Select c.CustomerID, o.OrderID, o.OrderDate _
        Order By CustomerID, OrderID

    db.Log = Console.Out

    For Each item In custOrders
        ListBox1.Items.Add(item.CustomerID)
        ListBox1.Items.Add(New String("=", 20))
        ListBox1.Items.Add(item.OrderID & ", " _
            & Format(item.OrderDate, "MMM/dd/yyyy"))
        ListBox1.Items.Add(vbNewLine)

        ListBox1.TopIndex = ListBox1.Items.Count - 1
    Next

End Sub
```

Perintah Group

Linq To SQL juga menyediakan perintah untuk melakukan grouping seperti halnya yang dapat kita lakukan di SQL.

Berikut adalah contoh untuk melakukan grouping data Product berdasarkan CategoryName kolom :

1. Tambahkan button, beri nama btnGroup dan tambahkan kode berikut pada event click :

```
Private Sub btnGroup_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnGroup.Click

    ListBox1.Items.Clear()

    Dim prodCat = From p In db.Products _
        Join c In db.Categories On _
        p.CategoryID Equals c.CategoryID _
        Group By c.CategoryName Into Group

    db.Log = Console.Out

    For Each item In prodCat
        ListBox1.Items.Add(New String("="c, 30))
        ListBox1.Items.Add(item.CategoryName)
        ListBox1.Items.Add(New String("="c, 30))

        For Each prod In item.Group
            ListBox1.Items.Add(prod.p.ProductName)

            ListBox1.TopIndex = ListBox1.Items.Count -
1
        Next
    Next

End Sub
```

Paging Record Dengan Skip dan Take

Linq To SQL juga menyediakan perintah untuk melakukan paging data dengan jumlah record per page yang dapat kita atur. Hasil transformasi perintah skip dan take Linq To SQL ke perintah SQL untuk database SQL Server 2000 dan SQL Server 2005 terdapat perbedaan didalam query yang digunakan

untuk mendapatkan hasil paging. Anda dapat melihatnya dari window output.

Perintah Take digunakan untuk menentukan jumlah record per page yang ingin ditampilkan. Sedangkan perintah Skip digunakan untuk melewati record dengan jumlah baris yang ditentukan, misalnya melewati data untuk 10 baris pertama atau 20 baris pertama sehingga data yang tampil dimulai dari data berikutnya.

Berikut adalah contoh penggunaan perintah Skip dan Take untuk melakukan paging data terhadap tabel customers :

1. Deklarasikan class level variabel berikut ini :

```
Private intTake As Integer = 0
Private intSkip As Integer = 0
```

2. Tambahkan NumericUpDown control kedalam form dan tambahkan kode berikut pada event ValueChanged :

```
Private Sub NumericUpDown1_ValueChanged(ByVal sender As
Object, ByVal e As System.EventArgs) Handles
NumericUpDown1.ValueChanged
    intTake = 0
    intSkip = 0
End Sub
```

3. Tambahkan button, beri nama btnPrev dan tambahkan kode berikut pada event click :

```
Private Sub btnPrev_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnPrev.Click

    Dim customers As IQueryable(Of Customer) = Nothing
    Dim recCount =
Convert.ToInt32(NumericUpDown1.Value)
```

```

db.Log = Console.Out

If intTake = 0 AndAlso intSkip = 0 Then
    customers = From c In db.Customers _
                Take recCount

    intTake = recCount
Else
    If intTake > recCount Then intTake -= recCount
    If intSkip > 0 Then intSkip -= recCount

    customers = From c In db.Customers _
                Take intTake Skip intSkip
End If

If customers IsNot Nothing Then
    ListBox1.Items.Clear()
    For Each cust In customers
        ListBox1.Items.Add(cust.CustomerID & "-" _
                           & cust.CompanyName)
    Next
End If

End Sub

```

4. Tambahkan button, beri nama btnNext dan tambahkan kode berikut pada event click :

```

Private Sub btnNext_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnNext.Click

    Dim customers As IQueryable(Of Customer) = Nothing
    Dim recCount =
Convert.ToInt32(NumericUpDown1.Value)

    db.Log = Console.Out

    If intTake = 0 AndAlso intSkip = 0 Then
        customers = From c In db.Customers _
                    Take recCount

```

```

        intTake = recCount
Else
    Dim intCount = (From c In db.Customers).Count
    If intTake < intCount Then
        intTake += recCount
        intSkip += recCount

        customers = From c In db.Customers _
                    Take intTake Skip intSkip
    End If
End If

If customers IsNot Nothing Then
    ListBox1.Items.Clear()
    For Each cust In customers
        ListBox1.Items.Add(cust.CustomerID & "-" _
                            & cust.CompanyName)
    Next
End If

End Sub

```

Stored Procedure Linq To SQL

Object stored procedure yang disimpan didalam Linq To SQL akan di mapping menjadi method di dalam DataContext class. Untuk eksekusi stored procedure di dalam Linq To SQL maka kita cukup memanggil method tersebut lewat DataContext instance class.

Berikut adalah contoh eksekusi stored procedure CustOrderHist yang telah dimasukkan kedalam DataContext class lewat O/R Linq To SQL designer :

1. Tambahkan button ke dalam form yang sama, beri nama btnGetDataSP dan tambahkan kode berikut pada event click :

```

Private Sub btnGetDataSP_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnGetDataSP.Click

    ListBox1.Items.Clear()

    Dim custOrders As System.Data.Linq.ISingleResult( _
        Of CustOrderHistResult) = Nothing

    db.Log = Console.Out

    ListBox1.Items.Add("Order History for Alfki")
    ListBox1.Items.Add(New String("=", 30))

    Try
        custOrders = db.CustOrderHist("Alfki")

        For Each item In custOrders
            ListBox1.Items.Add(item.ProductName & " : "
_ & item.Total.Value.ToString)
        Next
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    End Try

End Sub

```

Stored procedure yang terdapat didalam database di mapping menjadi sebuah fungsi yang mengembalikan nilai berupa ISingleResult. Kode yang digunakan untuk eksekusi fungsi tersebut diatas menggunakan variabel db yang merupakan instance dari DataContext yang telah didefinisikan pada lab sebelumnya sebagai class level variabel.

Insert Data

Untuk menambahkan data baru ke database dari Linq To SQL maka terlebih dahulu harus dibuat instance object dari class yang merupakan mapping terhadap tabel yang akan digunakan. Misalnya kita akan menambahkan data baru customer maka kita harus buat terlebih dahulu instance dari class customers tersebut.

Linq To SQL menyediakan method InsertOnSubmit untuk menambahkan data baru lewat class yang bersangkutan dengan melakukan passing input parameter berupa instance class yang telah dibuat sebelumnya. Untuk membuat perubahan tersebut persistent Linq To SQL menyediakan method SubmitChanges dari instance class DataContext untuk semua perubahan baik itu Insert, Update dan Delete.

Berikut adalah contoh kode program untuk menambahkan data customer baru ke tabel customers :

1. Tambahkan button baru kedalam form yang sama, beri nama btnAddCust dan tambahkan kode berikut pada event click :

```
Private Sub btnAddCust_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnAddCust.Click
```

```
    Dim newCust As New Customer  
    newCust.CustomerID = "Ntv-1"  
    newCust.CompanyName = "Native-Enterprise 1"  
    newCust.City = "Bandung"  
    newCust.Country = "Indonesia"
```

```
    db.Log = Console.Out
```

```
    Try
```

```
        db.Customers.InsertOnSubmit(newCust)  
        db.SubmitChanges()
```

```
        MessageBox.Show( _  
            "New customer Native-Enterprise 1 was  
            succesfully added.")
```

```
Catch sqlEx As SqlException
    MessageBox.Show(sqlEx.ToString)
Catch ex As Exception
    MessageBox.Show(ex.ToString)
End Try
```

```
End Sub
```

Update Data

Cara yang digunakan untuk melakukan update data agak berbeda kalau dibandingkan dengan insert data baru. Untuk melakukan update harus terlebih dahulu mencari data yang sesuai dengan kriteria yang diinginkan. Apabila data yang diinginkan merupakan data tunggal maka kita dapat mencarinya dengan perintah Single seperti yang telah dibahas pada lab sebelumnya. Setelah itu anda cukup merubah value dari properti yang ingin diubah nilainya.

Berikut adalah contoh penggunaan method update data terhadap data customer dengan kriteria spesifik sehingga hanya data tunggal yang dikembalikan :

1. Tambahkan button kedalam form yang sama, beri nama btnUpdate dan tambahkan kode berikut pada event click :

```
Private Sub btnUpdate_Click(ByVal sender As System.Object,
-
    ByVal e As System.EventArgs) Handles btnUpdate.Click

    db.Log = Console.Out
    Try
        Dim custToUpdate = (From c In db.Customers _
            Where c.CustomerID.Equals("Alfki")).Single

        custToUpdate.ContactName = "Anders"

        db.SubmitChanges()
```

```

        MessageBox.Show( _
            "Contact Name for Alfki was changed from Maria
Anders to Anders")
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    End Try

End Sub

```

Delete Data

Untuk menghapus data telah disediakan method DeleteOnSubmit dari class yang digunakan. Sedangkan untuk membuat perubahannya persistent digunakan method SubmitChanges dari instance DataContext class. Hal ini mirip dengan teknik untuk melakukan insert data baru.

Berikut adalah contoh simulasi untuk melakukan delete data terhadap tabel customers, dimana sebelumnya ditambahkan data baru terlebih dahulu setelah itu data baru tersebut dihapus kembali :

1. Tambahkan button ke dalam form yang sama, beri nama btnDelete dan tambahkan kode berikut pada event click :

```

Private Sub btnDelete_Click(ByVal sender As System.Object,
    ByVal e As System.EventArgs) Handles btnDelete.Click

    db.Log = Console.Out

    Dim newCust As New Customer
    newCust.CustomerID = "Ntv-2"
    newCust.CompanyName = "Native-Enterprise 2"

    Try
        db.Customers.InsertOnSubmit(newCust)
        db.SubmitChanges()
    
```

```

        MessageBox.Show( _
            "New customer Native-Enterprise 2 was saved.")

        Dim custToDelete = (From c In db.Customers _
            Where c.CustomerID.Equals("Ntv-2")).Single

        db.Customers.DeleteOnSubmit(custToDelete)
        db.SubmitChanges()

        MessageBox.Show("Customer Native-Enterprise 2
was deleted.")
    Catch sqlEx As SqlException
        MessageBox.Show(sqlEx.ToString)
    Catch ex As Exception
        MessageBox.Show(ex.ToString)
    End Try

End Sub

```

Manipulasi Data Dengan Stored Procedure

Pada contoh manipulasi data sebelumnya query yang digunakan merupakan ad-hoc query. Linq To SQL dapat memanfaatkan stored procedure untuk melakukan manipulasi data seperti Insert, Update dan Delete. Stored Procedure yang ditambahkan ke Linq To SQL akan di mapping menjadi sebuah method.

Berikut adalah contoh penggunaan stored procedure didalam Linq To SQL untuk menambahkan data customer baru ke tabel customers di database northwind.

1. Buat stored procedure baru untuk menambahkan data customer baru di database northwind seperti berikut ini :

```

CREATE PROCEDURE AddCustomer
    @id nchar(5),
    @name nvarchar(40)
AS

```

Insert Into Customers (CustomerID,CompanyName)
Values (@id,@name)

2. Drag dan drop stored procedure tersebut ke dalam Linq To SQL designer.
3. Tambahkan button kedalam form yang sama, beri nama btnAddSP dan tambahkan kode berikut pada event click :

```
Private Sub btnAddSP_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles btnAddSP.Click  
  
    Try  
        Dim intResult = db.AddCustomer( _  
            "Ntv-3", "Native-Enterprise 3")  
  
        MessageBox.Show( _  
            "New customer Native-Enterprise 3 was added by  
Stored Proc.")  
  
        Catch sqlEx As SqlException  
            MessageBox.Show(sqlEx.ToString)  
        Catch ex As Exception  
            MessageBox.Show(ex.ToString)  
        End Try  
  
End Sub
```

Transaction

Untuk menggunakan fitur transaction didalam Linq To SQL kita dapat memanfaatkan TransactionScope class yang terdapat didalam System.Transaction.dll v 2.0. Oleh karena itu untuk menggunakan class tersebut harus ditambahkan referensi terlebih dahulu ke assembly tersebut dari project yang digunakan.

Berikut adalah simulasi contoh penggunaan TransactionScope class di Linq To SQL :

1. Tambahkan button ke dalam form yang sama, beri nama btnCommitTrans dan tambahkan kode berikut pada event click :

```
Private Sub btnCommitTrans_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) _
Handles btnCommitTrans.Click

    Dim newCust As New Customer
    newCust.CustomerID = "C" & New Random(1).Next(1,
100).ToString
    newCust.CompanyName = "Company " & New
Random(1).Next(1, 100).ToString

    Dim newProd As New Product
    newProd.ProductName = "P" & New Random(1).Next(1,
100).ToString
    newProd.CategoryID = New Random(1).Next(1, 8)
    newProd.SupplierID = New Random(1).Next(1, 29)

    Using ts As New TransactionScope
        Try
            db.Customers.InsertOnSubmit(newCust)
            db.Products.InsertOnSubmit(newProd)
            db.SubmitChanges()

            ts.Complete()

            MessageBox.Show("Transaction Complete!")
        Catch sqlEx As SqlException
            MessageBox.Show("Transaction Rollback : " &
sqlEx.ToString)
        Catch ex As Exception
            MessageBox.Show("Transaction Rollback : " &
ex.ToString)
        End Try
    End Using

End Sub
```

Mengenai penggunaan `TransactionScope` class sudah dibahas sebelumnya di bab ADO.NET. Kode diatas diasumsikan berhasil di commit untuk melakukan insert data baru ke tabel customer dan tabel products sekaligus.

2. Tambahkan button berikut pada form yang sama, beri nama `btnRollback` dan ketikkan kode berikut pada event click :

```
Private Sub btnRollback_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnRollback.Click

    Dim newProd As New Product
    newProd.ProductName = "P" & New Random(1).Next(1,
100).ToString
    newProd.CategoryID = New Random(1).Next(1, 8)
    newProd.SupplierID = New Random(1).Next(1, 29)

    Dim newCust As New Customer
    newCust.CustomerID = "Alfki"
    newCust.CompanyName = "Alfki Company"

    Using ts As New TransactionScope
        Try
            db.Products.InsertOnSubmit(newProd)
            db.Customers.InsertOnSubmit(newCust)
            db.SubmitChanges()

            ts.Complete()

            MessageBox.Show("Transaction Complete!")
        Catch sqlEx As SqlException
            MessageBox.Show("Transaction Rollback : " &
sqlEx.ToString)
        Catch ex As Exception
            MessageBox.Show("Transaction Rollback : " &
ex.ToString)
        End Try
    End Using
```

End Using

End Sub

Kode di atas sengaja dibuat untuk mensimulasikan error yang terjadi didalam sebuah transaction. Apabila salah satu proses query gagal di eksekusi maka proses lainnya tidak akan pernah dijalankan dan proses query yang berhasil akan di rollback.

Pada proses yang pertama yaitu pada penambahan data product baru tidak terjadi masalah, namun pada proses yang kedua yaitu pada penambahan data customer baru akan mengalami error karena adanya penambahan data customer baru dengan customerid yang telah terdaftar didalam tabel tersebut. Sehingga data product yang telah berhasil ditambahkan akan di rollback pada keadaan semula.

Berikut adalah design form untuk lab pada bab ini :

The image shows a Windows application window titled "Form1". The interface is organized into several sections:

- Top Section:** Contains six buttons arranged in two rows: "Get Customers", "Filtering Record", "Distinct Record", "Find Single Record", "Aggregate Function", and "Join".
- Second Row:** Contains two buttons: "Group By" and "Get Data By Stored Proc".
- Data Manipulation Section:** A group box containing four buttons: "Add Customer", "Update Customer", "Delete Customer", and "Add Customer (Using Stored Proc)".
- Paging with Take-Skip Section:** A group box containing two buttons, "Previous" and "Next", followed by the text "Set record count per page :" and a spin box currently set to "1".
- Transaction Section:** A group box containing two buttons: "Commit Transaction" and "Rollback Transaction".
- List Box:** A large rectangular area at the bottom labeled "ListBox1" for displaying data.

Gambar 13.6 Design form

BAB 14

LINQ to XML

Linq To XML merupakan teknik yang digunakan untuk melakukan query dan manipulasi data terhadap file xml. Fitur ini mulai disediakan di .NET framework versi 3.5. Selama ini kita mengenal Xquery untuk melakukan query terhadap file xml, selain itu juga kita telah mengenal sebelumnya object seperti XmlDocument untuk load file xml dan manipulasi data didalamnya.

Dengan Linq To XML kita diberikan kemudahan-kemudahan untuk melakukan query dan manipulasi data terutama di VB 9.0 yang juga menyediakan fitur baru yaitu XML Literal yang dapat digunakan untuk membuat file xml dengan sangat mudah sekali.

Dalam pembuatan file xml selain dengan menggunakan XML Literal disediakan juga XDocument dan XElement class yang terdapat di dalam .NET framework. XDocument class memiliki karakteristik yang berbeda apabila dibandingkan dengan XmlDocument class yang terdapat di dalam .NET framework versi sebelumnya.

XDocument class memiliki sifat elemen centric, sedangkan XmlDocument memiliki sifat document centric dalam hal pembuatan file xml. Tentunya dengan kemampuan elemen centric tersebut XDocument memiliki kemudahan didalam pembuatan file xml.

XDocument

Class ini digunakan untuk meload dan membuat file xml. Sifatnya yang element centric memudahkan kita untuk membuat file xml. Kita dapat membuat xml declaration, xml comment, xml element dan xml attribute dengan mudah.

Berikut adalah contoh kode program yang digunakan untuk membuat file xml dengan menggunakan XDocument :

1. Buat sebuah Windows Forms Application project template baru, beri nama LinqToXML.
2. Tambahkan button kedalam form dan beri nama btnXDoc, kemudian ketikkan kode berikut pada event click :

```
Private Sub btnXDoc_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnXDoc.Click

    Dim bookList = New XDocument( _
        New XDeclaration("1.0", "utf-8", "yes"), _
        New XComment("Daftar buku programming"), _
        New XElement("BookList", _
            New XElement("Book", _
                New XElement("ISBN", "1"), _
                New XElement("Title", _
                    "Visual Basic 2008")), _
            New XElement("Book", _
                New XElement("ISBN", "2"), _
                New XElement("Title", _
                    "Visual C# 2008")), _
            New XElement("Book", _
                New XElement("ISBN", "3"), _
                New XElement("Title", "ADO.NET 3.5"))
        -
    ))

    bookList.Save("C:/ProgrammingBooks1.xml")
    Process.Start("C:/ProgrammingBooks1.xml")

End Sub
```

Kode diatas akan menghasilkan file xml dengan BookList sebagai root document dan Book sebagai child element dari BookList. Book element memiliki dua buah child element yaitu

ISBN dan Title. XMLDeclaration digunakan untuk membuat deklarasi xml, sedangkan XComment digunakan untuk membuat komentar dan XElement digunakan untuk membuat xml element. Penggunaan XElement ini sifatnya nested, artinya untuk membuat nested elemen kita cukup menambahkan XElement didalam XElement yang akan digunakan sebagai parent elemennya.

XElement

Sama halnya seperti XDocument, XElement ini dapat digunakan juga untuk membuat file xml. Apabila anda tidak ingin mendeklarasikan file xml secara eksplisit dengan XDeclaration maka XElement ini dapat dijadikan sebagai alternatif lain.

Berikut adalah contoh penggunaan XElement untuk membuat file xml dengan struktur yang sama dengan project sebelumnya diatas :

1. Tambahkan button kedalam form, beri nama btnXElement dan ketikkan kode berikut pada event click :

```
Private Sub btnXElement_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnXElement.Click

    Dim bookList = New XElement("BookList", _
        New XComment("Daftar buku programming."),
    -
        New XElement("Book", _
            New XElement("ISBN", "1"), _
            New XElement("Title", _
                "Visual Basic 2008")), _
        New XElement("Book", _
            New XElement("ISBN", "2"), _
            New XElement("Title", _
                "Visual C# 2008")), _
```

```
        New XElement("Book", _  
            New XElement("ISBN", "3"), _  
            New XElement("Title", "ADO.NET  
3.5"))))
```

```
bookList.Save("C:/ProgrammingBooks2.xml")  
Process.Start("C:/ProgrammingBooks2.xml")
```

End Sub

XAttribute

Untuk membuat attribute didalam sebuah xml elemen .NET framework sudah menyediakan class nya, yaitu XAttribute.

Berikut adalah contoh penggunaan XAttribute untuk membuat attribute ISBN pada elemen Book :

1. Tambahkan button kedalam form yang sama, beri nama btnXElementAtt dan ketikkan kode berikut di event click :

```
Private Sub btnXElementAtt_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnXElementAtt.Click  
  
    Dim bookList = New XElement("BookList", _  
        New XComment("Daftar buku programming."), _  
        New XElement("Book", _  
            New XAttribute("ISBN", "1"), _  
            New XElement("Title", "Visual Basic 2008")),  
        _  
        New XElement("Book", _  
            New XAttribute("ISBN", "2"), _  
            New XElement("Title", "Visual C# 2008")), _  
        New XElement("Book", _  
            New XAttribute("ISBN", "3"), _  
            New XElement("Title", "ADO.NET 3.5")))
```

```
bookList.Save("C:/ProgrammingBooks3.xml")
Process.Start("C:/ProgrammingBooks3.xml")
```

End Sub

Untuk menambahkan attribute pada sebuah elemen anda tambahkan XAttribute class didalam XElement yang bersangkutan dengan supply input parameter nama attribute dan nilainya.

XML Literal

Seperti yang telah disinggung sebelumnya bahwa selain dengan menggunakan XDocument dan XElement class untuk membuat file xml di VB 9.0 disediakan fitur baru yaitu XML Literal.

Penggunaan XML Literal ini sangat mudah semudah anda mendeklarasikan variabel string. Implementasi dari XML Literal ini sangat beragam, mulai dari pembuatan file xml, pembuatan file html, dan juga pembuatan file excel. Deklarasi dari variabel XML Literal tersebut akan di infer ke tipe data XElement.

Berikut adalah contoh penggunaan XML Literal yang digunakan untuk membuat file xml Book seperti yang telah kita lakukan pada lab sebelumnya :

1. Tambahkan button kedalam form yang sama, beri nama btnXMLLiteral dan ketikkan kode berikut pada event click :

```
Private Sub btnXMLLiteral_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnXMLLiteral.Click
```

```
    Dim bookList = <BookList>
                    <!--Daftar buku Programming-->
                    <Book>
                        <ISBN>1</ISBN>
```

```

        <Title>Visual Basic 2008</Title>
    </Book>
    <Book>
        <ISBN>2</ISBN>
        <Title>Visual C# 2008</Title>
    </Book>

    <Book>
        <ISBN>3</ISBN>
        <Title>ADO.NET 3.5</Title>
    </Book>
</BookList>

```

```

bookList.Save("C:/ProgrammingBooks4.xml")
Process.Start("C:/ProgrammingBooks4.xml")

```

End Sub

Dari kode diatas dapat kita lihat begitu mudahnya untuk membuat file xml dengan XML Literal semudah mendeklarasikan variabel biasa.

Berikut adalah contoh lain penggunaan XML Literal dimana elemen ISBN pada file diatas diubah menjadi attribute :

1. Tambahkan button kedalam form yang sama, beri nama btnXMLitAtt dan ketikkan kode berikut pada event click :

```

Private Sub btnXMLitAtt_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnXMLitAtt.Click

```

```

    Dim bookList = <BookList>
        <!--Daftar buku Programming-->
        <Book ISBN="1">
            <Title>Visual Basic 2008</Title>
        </Book>
        <Book ISBN="2">
            <Title>Visual C# 2008</Title>
        </Book>
    </BookList>

```

```
<Book ISBN="3">  
  <Title>ADO.NET 3.5</Title>  
</Book>  
</BookList>
```

```
bookList.Save("C:/ProgrammingBooks5.xml")  
Process.Start("C:/ProgrammingBooks5.xml")
```

End Sub

Untuk menambahkan atribut pada sebuah elemen anda tinggal menuliskan atribut tersebut pada elemen yang diinginkan.

Membaca File XML

File xml yang akan dibaca datanya pertama kali di load terlebih dahulu dengan menggunakan XDocument class. Setelah itu kita gunakan Linq To XML untuk melakukan query.

Query terhadap file xml dengan menggunakan Linq To XML ini sedikit unik kalau dibandingkan dengan Linq object lainnya. Linq To XML di VB 9.0 memiliki kemampuan untuk membaca file xml dengan properti axis element dengan menggunakan simbol .<> dan ...<> atau @ untuk pembacaan atribut. Hal tersebut membuat pembacaan kode elemen dan attribute lebih mudah.

Berikut adalah contoh penggunaan Linq To XML dalam membaca file xml :

1. Tambahkan button kedalam form yang sama, beri nama btnReadXML dan ketikkan kode berikut pada event click :

```
Private Sub btnReadXML_Click(ByVal sender As System.Object,  
ByVal e As System.EventArgs) Handles btnReadXML.Click
```

```
Dim fileName = "C:/ProgrammingBooks5.xml"
```

```

If My.Computer.FileSystem.FileExists(fileName) Then

    Dim xDoc = XDocument.Load(fileName)

    Dim books = From b In xDoc...<Book> _
                Select b

    ListBox1.Items.Clear()

    For Each book In books
        ListBox1.Items.Add("ISBN : " & book.@ISBN _
                           & " Title : " & book.<Title>.Value)
    Next

Else
    MessageBox.Show(fileName _
                    & " file could not found.")
End If

End Sub

```

Simbol ...<> merupakan descendant dan digunakan untuk menuju elemen tertentu relatif berada dibawah elemen root document. Untuk membaca nilai child value elemen yang tepat berada satu tingkat dibawah elemen yang aktif digunakan simbol .<>, sedangkan simbol @ digunakan untuk membaca nilai attribut.

Mencari Data Tunggal

Untuk mencari data tunggal di dalam file xml anda dapat menggunakan fungsi Single. Nilai yang dikembalikan dari hasil tersebut yaitu berupa XElement.

Berikut adalah contoh penggunaan fungsi Single di Linq To XML :

1. Tambahkan button kedalam form yang sama, beri nama btnFind dan ketikkan kode berikut pada event click :

```
Private Sub btnFind_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnFindSingle.Click

    Dim fileName = "C:/ProgrammingBooks5.xml"
    If My.Computer.FileSystem.FileExists(fileName) Then

        Dim xDoc = XDocument.Load(fileName)

        Dim books = (From b In xDoc...<Book> _
            Where b.@ISBN = "1" _
            Select b).Single

        ListBox1.Items.Clear()
        ListBox1.Items.Add("Book title for ISBN=1 : " _
            & books.<Title>.Value)

    Else
        MessageBox.Show(fileName _
            & " file could not found.")
    End If

End Sub
```

Kode diatas digunakan untuk membaca data book yang memiliki nilai attribute sama dengan 1.

Membaca Multiple Data

Untuk membaca data yang mengembalikan lebih dari satu record anda dapat menggunakan filtering data dengan perintah Where yang digabungkan dengan perintah lainnya seperti Contains, StartsWith, dll. Nilai yang dihasilkan berupa collection dari IEnumerable(Of XElement) yang merupakan generic collection class.

Berikut adalah contoh kode program untuk mencari data Book yang mengandung kata “Visual” pada elemen Title :

1. Tambahkan button kedalam form yang sama, beri nama btnFinsRecords dan ketikkan kode berikut pada event click :

```
Private Sub btnFindRecords_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnFindRecords.Click

    Dim fileName = "C:/ProgrammingBooks5.xml"
    If My.Computer.FileSystem.FileExists(fileName) Then

        Dim xDoc = XDocument.Load(fileName)

        Dim books = From b In xDoc...<Book> _
            Where b.<Title>.Value.ToLower.Contains("visual")

        Select b

            ListBox1.Items.Clear()
            ListBox1.Items.Add( _
                & "Book title contains Visual :")

            For Each book In books
                ListBox1.Items.Add("ISBN : " & book.@ISBN _
                    & " Title : " & book.<Title>.Value)
            Next

        Else
            MessageBox.Show(fileName _
                & " file could not found.")
        End If

    End Sub
```

Join File XML

Linq To XML menyediakan teknik Join untuk menggabungkan beberapa file xml yang memiliki hubungan atau relasi diantara data-data yang terdapat didalamnya.

Berikut adalah contoh kode program bagaimana caranya untuk melakukan join pada file xml :

1. Tambahkan file xml baru kedalam project, beri nama Books.xml dan ketikkan data berikut :

```
<?xml version="1.0" encoding="utf-8" ?>
<Books>
  <Book CatID="1">
    <Title>Visual Basic 9.0</Title>
    <Publisher>Apress</Publisher>
  </Book>
  <Book CatID="1">
    <Title>Visual C# 3.0</Title>
    <Publisher>Adison Wesley</Publisher>
  </Book>
  <Book CatID="2">
    <Title>SQL Server 2008</Title>
    <Publisher>MS Press</Publisher>
  </Book>
</Books>
```

2. Tambahkan file xml baru kedalam project, beri nama Categories.xml dan ketikkan data berikut :

```
<?xml version="1.0" encoding="utf-8" ?>
<Categories>
  <Category ID="1">
    <Name>Programming</Name>
  </Category>
  <Category ID="2">
    <Name>Database</Name>
  </Category>
</Categories>
```

```
</Category>
<Category ID="3">
  <Name>Operating System</Name>
</Category>
</Categories>
```

3. Tambahkan button kedalam form yang sama, beri nama btnJoinXML dan ketikkan kode berikut pada event click :

```
Private Sub btnJoinXML_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles btnJoinXML.Click

    Dim xmlBook = XDocument.Load(".././Books.xml")
    Dim xmlCategory =
XDocument.Load(".././Categories.xml")

    Dim xmlData = From book In xmlBook...<Book> _
                  Join cat In xmlCategory...<Category> _
                  On book.@CatID Equals cat.@ID _
                  Select cat.<Name>, book.<Title>

    ListBox1.Items.Clear()

    For Each elem In xmlData
        Dim strCatName = CStr(elem.Name.Value)
        Dim strTitle = CStr(elem.Title.Value)

        Dim strBook = String.Format( _
            "Category:{0}, Title:{1}", strCatName,
strTitle)

        ListBox1.Items.Add(strBook)
    Next

End Sub
```

Kode diatas akan menampilkan data dari file Books dan Categories yang memiliki nilai CategoryID yang sama.

Membuat File XML dari Database

File xml yang dibuat pada contoh sebelumnya bersifat statis, lalu bagaimana caranya agar kita dapat membuat file xml dimana nilai elemen dan attributnya dinamis sesuai dengan data yang telah ada?

Di dalam XML Literal disediakan simbol atau ekspresi yang digunakan untuk menampung nilai dari sumber data secara dinamis yaitu dengan menggunakan `<%= ... %>`. Didalam simbol tersebut tinggal kita isi dengan sebuah variabel atau nilai tertentu.

Berikut adalah contoh penggunaan simbol tersebut untuk membuat file xml secara dinamis dimana datanya diambil dari tabel customers dalam database Northwind :

1. Tambahkan item baru kedalam project berupa Linq To SQL Classes dan beri nama Northwind.dbml.
2. Drag dan drop tabel Customers dan Orders dari jendela Server Explorer kedalam O/R designer.
3. Tambahkan button kedalam form yang sama, beri nama btnCustToXML dan ketikkan kode berikut pada event click :

```
Private Sub btnCustToXML_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnCustToXML.Click
```

```
Dim db As New NorthwindDataContext
```

```
Dim customersXML = <Customers>  
    <%= From cust In db.Customers _  
Select <Customer>  
    <ID><%= cust.CustomerID %></ID>  
    <Name><%= cust.CompanyName %></Name>  
    <City><%= cust.City %></City>
```

```

        <Country><%= cust.Country
%></Country>

    </Customer> %>
</Customers>

```

```

customersXML.Save("C:/customers.xml")
Process.Start("C:/customers.xml")

```

End Sub

Lalu bagaimana caranya untuk membuat file xml yang sifatnya nested? Misalnya saya ingin membuat file xml dari data customers dimana tiap customerID memiliki data Orders nya juga? Anda cukup membuat xml literal untuk menampilkan data Orders didalam elemen Customers. Berikut adalah contohnya :

1. Tambahkan button kedalam form yang sama, beri nama btnCustOrdersToXML dan ketikkan kode berikut pada event click :

```

Private Sub btnCustOrdersToXML_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnCustOrdersToXML.Click

```

```

    Dim db As New NorthwindDataContext

```

```

    Dim custOrders = <CustomersOrders>
        <%= From cust In db.Customers _
        Select <Customer>
            <ID><%= cust.CustomerID %></ID>
        <%= From ord In cust.Orders _
        Where ord.CustomerID = cust.CustomerID _
        Select <Orders>
            <OrderID><%= ord.OrderID %></OrderID>
            <Date><%= Format(ord.OrderDate, "MMM/dd/yyyy") %></Date>
            <Address><%= ord.ShipAddress %></Address>
        </Orders> %>

```

```
</Customer> %>  
</CustomersOrders>
```

```
custOrders.Save("C:/customerorders.xml")  
Process.Start("C:/customerorders.xml")
```

End Sub

Apabila salah satu elemen ingin dimapping menjadi atribut hal tersebut tentunya juga dapat dilakukan, anda cukup menempatkan atribut tersebut pada salah satu elemennya seperti pada contoh berikut ini :

1. Tambahkan button kedalam form yang sama, beri nama btnCustOrderAttr dan ketikkan kode berikut pada event click :

```
Private Sub btnCustOrderAttr_Click(ByVal sender As  
System.Object, ByVal e As System.EventArgs) Handles  
btnCustOrderAttr.Click
```

```
Dim db As New NorthwindDataContext
```

```
Dim custOrders = <CustomersOrders>  
    <%= From cust In db.Customers _  
    Select <Customer ID=<%= cust.CustomerID %>>  
    <%= From ord In cust.Orders _  
    Where ord.CustomerID = cust.CustomerID _  
    Select <Orders>  
<OrderID><%= ord.OrderID %></OrderID>  
<Date><%= Format(ord.OrderDate, "MMM/dd/yyyy") %></Date>  
<Address><%= ord.ShipAddress %></Address>  
</Orders> %>  
</Customer> %>  
</CustomersOrders>
```

```
custOrders.Save("C:/customerordersAtt.xml")  
Process.Start("C:/customerordersAtt.xml")
```

End Sub

Perbandingan antara XML DOM, XML LINQ API dengan XML Literal

XML DOM telah kita kenal sebelumnya pada .NET framework versi sebelumnya untuk membuat file xml. Berikut adalah contoh perbandingan ketiga teknik tersebut dalam kemudahannya didalam membuat file xml :

1. Buat dua buah class berikut di bawah class Form yang digunakan :

```
Public Class Book
    Private mISBN As String
    Public Property ISBN() As String
        Get
            Return mISBN
        End Get
        Set(ByVal value As String)
            mISBN = value
        End Set
    End Property

    Private mTitle As String
    Public Property Title() As String
        Get
            Return mTitle
        End Get
        Set(ByVal value As String)
            mTitle = value
        End Set
    End Property

    Private mPublisher As String
    Public Property Publisher() As String
        Get
            Return mPublisher
        End Get
        Set(ByVal value As String)
```

```

        mPublisher = value
    End Set
End Property
End Class

Class BookList
    Inherits List(Of Book)

    Public Sub New()
        Dim book1 = New Book With _
        { .ISBN = "1", .Title = "VB 9.0", .Publisher = _
        & "Apress"}
        Dim book2 = New Book With _
        { .ISBN = "2", .Title = "C# 3.0", .Publisher = "Wrox"}
        Dim book3 = New Book With _
        { .ISBN = "3", .Title = "LINQ", .Publisher = _
        & "MS Press"}

        Me.Add(book1)
        Me.Add(book2)
        Me.Add(book3)
    End Sub
End Class

```

Class tersebut digunakan untuk menyimpan data buku dan nantinya akan digunakan sebagai sumber data untuk membuat file xml dengan menggunakan ketiga teknik yang akan kita buat.

2. Buat prosedur berikut yang digunakan untuk membuat file xml dengan teknik XML DOM :

```

Private Function CreateElement(ByVal doc As
Xml.XmlDocument, ByVal elementName As String, _
ByVal value As String) As Xml.XmlElement

    Dim element As Xml.XmlElement = _
doc.CreateElement(elementName)

```

```

        element.InnerText = value
    Return element

End Function

Private Function XMLDOM(ByVal bookList As List(Of Book)) As
    Xml.XmlDocument

    Dim doc As New Xml.XmlDocument
    Dim root As Xml.XmlElement = doc.CreateElement("Books")

    For Each b As Book In bookList
        Dim bookElement As Xml.XmlElement = _
            doc.CreateElement("Book")

        bookElement.SetAttribute("ISBN", b.ISBN)

        Dim title As Xml.XmlElement = _
            CreateElement(doc, "Title", b.Title)

        Dim publisher As Xml.XmlElement = _
            CreateElement(doc, "Publisher", b.Publisher)

        With bookElement
            .AppendChild(title)
            .AppendChild(publisher)
        End With
        root.AppendChild(bookElement)
    Next
    doc.AppendChild(root)
    Return doc

End Function

```

Dapat kita lihat bahwa penggunaan XML DOM lebih kepada document centric tidak seperti halnya dengan XML LINQ API yang sudah dibahas sebelumnya.

3. Buat prosedur berikut untuk membuat file xml dengan menggunakan XML LINQ API yang terdapat didalam .NET framework 3.5 :

```
Public Function XLINQAPI(ByVal bookList As List(Of Book))  
As XElement
```

```
    Dim doc = New XElement("Books", _  
        From b In bookList _  
        Select New XElement("Book", _  
            New XAttribute("ISBN", b.ISBN), _  
            New XElement("Title", b.Title), _  
            New XElement("Publisher", b.Publisher)))  
    Return doc
```

```
End Function
```

4. Buat prosedur berikut untuk membuat file xml dengan menggunakan teknik XML Literal :

```
Public Function XMLLiterals(ByVal bookList As List(Of  
Book)) As XElement
```

```
    Dim doc = <Books>  
        <%= From b In bookList _  
        Select <Book ISBN=<%= b.ISBN %>>  
        <Title><%= b.Title %></Title>  
        <Publisher><%= b.Publisher %></Publisher>  
        </Book> %>  
    </Books>  
    Return doc
```

```
End Function
```

5. Tambahkan button kedalam form yang sama, beri nama btnComparison dan ketikkan kode berikut pada event click :

```
Private Sub btnComparison_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnComparison.Click

    Dim books As New BookList

    'XML DOM :
    Dim bookXMLDoc As Xml.XmlDocument = XMLDOM(books)

    My.Computer.FileSystem.WriteAllText( _
    "booksDOM.xml", bookXMLDoc.InnerXml, False)

    Process.Start("booksDOM.xml")

    'LINQ to XML API :
    Dim bookXMLAPI As XElement = XLINQAPI(books)

    bookXMLAPI.Save("booksXLINQ.xml")
    Process.Start("booksXLINQ.xml")

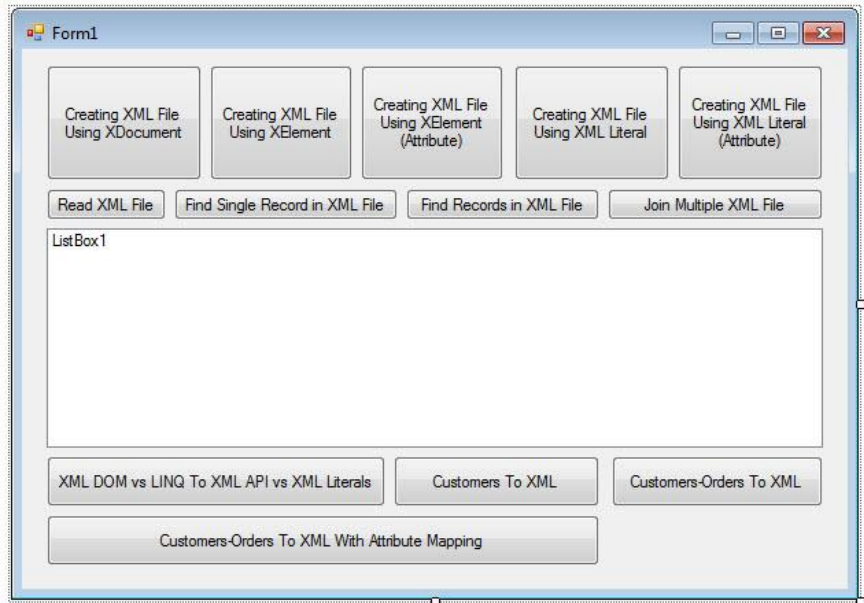
    'XML Literals :
    Dim bookXMLLiteral = XMLLiterals(books)

    bookXMLLiteral.Save("booksLiteral.xml")
    Process.Start("booksLiteral.xml")

End Sub
```

Dari ketiga perbandingan teknik diatas tentunya anda sudah dapat menyimpulkan metoda mana yang paling mudah untuk membuat file xml.

Berikut adalah design form yang digunakan pada lab tersebut :



Gambar 14.1 Design form

Manipulasi File XML

.NET framework 3.5 juga menyediakan teknik untuk melakukan manipulasi data file xml dengan Linq XML. Kita dapat menambahkan, memanipulasi, dan mengubah data. Kemudahan untuk memanipulasi data didukung juga dengan adanya fitur XML Literal yang memudahkan untuk menambahkan elemen baru kedalam file xml yang sudah ada.

Add Method

Method ini digunakan untuk menambahkan elemen baru pada posisi yang diinginkan. Berikut adalah contoh penggunaan method tersebut :

1. Tambahkan form baru kedalam project yang sama.

2. Tambahkan button kedalam form, beri nama btnAdd dan ketikkan kode berikut pada event click :

```
Private Sub btnAdd_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles btnAdd.Click  
  
    Dim xmlBook = XDocument.Load("../..Books.xml")  
  
    xmlBook.<Books>.First.Add( _  
        <Book CatID="1">  
            <Title>Visual C++ 2008</Title>  
            <Publisher>Sybex</Publisher>  
        </Book>)  
  
    'xmlBook.Save("../..Books.xml")  
    Console.WriteLine(xmlBook)  
    Console.ReadLine()  
  
End Sub
```

Kode diatas akan menambahkan element book baru pada baris paling bawah. Untuk menyimpan perubahan tersebut secara persistent anda dapat menghilangkan tanda komentar pada perintah Save.

AddAfterSelf

Perintah AddAfterSelf digunakan untuk menambahkan elemen baru pada posisi setelah elemen yang diinginkan. Tentunya untuk melakukan hal tersebut kita harus melakukan query xml file nya dengan Linq To XML.

Berikut adalah contoh penggunaannya :

1. Tambahkan button kedalam form yang sama, beri nama btnAddAfterSelf dan ketikkan kode berikut pada event click :

```

Private Sub btnAddAfterSelf_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnAddAfterSelf.Click

    Dim xmlBook = XDocument.Load("../..//Books.xml")

    Dim xmlData = From elem In xmlBook...<Book> _
                    Where elem.@CatID.ToString.Equals("1") _
                    Select elem

    xmlData.First.AddAfterSelf( _
        <Book CatID="1">
            <Title>Visual C++ 2008</Title>
            <Publisher>Sybex</Publisher>
        </Book>)

    'xmlBook.Save("../..//Books.xml")
    Console.WriteLine(xmlBook)
    Console.ReadLine()

End Sub

```

Hasil eksekusi kode diatas akan menambahkan elemen baru setelah elemen Book yang memiliki atribut CatID dengan nilai 1.

AddBeforeSelf

Kebalikan dari method AddAfterSelf, method ini digunakan untuk menambahkan elemen baru tepat sebelum elemen yang diinginkan. Berikut adalah contoh penggunaannya :

1. Tambahkan button kedalam form, beri nama btnAddBeforeSelf dan ketikkan kode berikut pada event click :

```

Private Sub btnAddBeforeSelf_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnAddBeforeSelf.Click

    Dim xmlBook = XDocument.Load("../..Books.xml")

    Dim xmlData = From elem In xmlBook...<Book> _
        Where elem.@CatID.ToString.Equals("1") _
        AndAlso elem.<Title>.Value.Contains("Visual Basic")
-
    Select elem

        xmlData.First.AddBeforeSelf( _
            <Book CatID="1">
                <Title>Visual C++ 2008</Title>
                <Publisher>Sybex</Publisher>
            </Book>)

        'xmlBook.Save("../..Books.xml")
        Console.WriteLine(xmlBook)
        Console.ReadLine()

End Sub

```

Kode diatas akan menambahkan elemen Book baru sebelum elemen Book yang mengandung Title “Visual Basic”.

SetElementValue

Perintah tersebut digunakan untuk merubah nilai pada elemen yang diinginkan. Berikut adalah contoh penggunaannya :

1. Tambahkan button kedalam form yang sama, beri nama btnSetElementValue dan ketikkan kode berikut pada event click :

```

Private Sub btnSetElementValue_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnSetElementValue.Click

    Dim xmlBook = XDocument.Load("../..//Books.xml")

    Dim xmlData = From elem In xmlBook...<Book> _
        Where elem.@CatID.ToString.Equals("1") _
        AndAlso elem.<Title>.Value.Contains("Visual Basic")
    -
    Select elem

    xmlData.First.SetElementValue("Title", _
        & "LINQ With VB 9.0")
        'atau
        'xmlData.Single.SetElementValue("Title", "LINQ With
VB 9.0")

        'xmlBook.Save("../..//Books.xml")
        Console.WriteLine(xmlBook)
        Console.ReadLine()

End Sub

```

SetValue

Perintah SetValue memiliki fungsi yang sama dengan perintah SetElementValue namun memiliki perbedaan dalam akses elemen yang ingin diubah nilainya. Berikut adalah contoh penggunaannya :

1. Tambahkan button kedalam form yang sama, beri nama btnSetValue dan ketikkan kode berikut pada event click :

```

Private Sub btnSetValue_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnSetValue.Click

    Dim xmlBook = XDocument.Load("../..//Books.xml")

```

```

Dim xmlData = From elem In xmlBook...<Book> _
    Where elem.@CatID.ToString.Equals("1") _
    AndAlso elem.<Title>.Value.Contains("Visual Basic")
-
    Select elem

xmlData...<Title>.First.SetValue("LINQ With VB
9.0")
'atau
'xmlData...<Title>.Single.SetValue("LINQ With VB
9.0")

'xmlBook.Save(".././Books.xml")
Console.WriteLine(xmlBook)
Console.ReadLine()

End Sub

```

Dari kode diatas anda dapat melihat perbedaan antara SetElementValue dan SetValue method.

SetAttributeValue

Perintah SetAttributeValue digunakan untuk mengubah nilai attribut pada elemen tertentu, berikut adalah contoh penggunaannya :

1. Tambahkan button kedalam form yang sama, beri nama btnSetAttributeValue dan tambahkan kode berikut pada event click :

```

Private Sub btnSetAttributeValue_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnSetAttributeValue.Click

    Dim xmlBook = XDocument.Load(".././Books.xml")

```

```

Dim xmlData = From elem In xmlBook...<Book> _
               Where elem.@CatID.ToString.Equals("1") _
               Select elem

xmlData.First.SetAttributeValue("CatID", "3")
'atau
$xmlData.Single.SetAttributeValue("CatID", "3")

$xmlBook.Save("../..//Books.xml")
Console.WriteLine(xmlBook)
Console.ReadLine()

```

End Sub

Kode diatas akan mengubah nilai atribut CatID yang memiliki nilai 1 menjadi 3.

Remove

Perintah Remove digunakan untuk menghapus elemen didalam file xml, berikut adalah contoh penggunaannya :

1. Tambahkan button kedalam form beri nama btnRemoveElement dan ketikkan kode berikut pada event click :

```

Private Sub btnRemoveElement_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
btnRemoveElement.Click

    Dim xmlBook = XDocument.Load("../..//Books.xml")

    Dim xmlData = From elem In xmlBook...<Book> _
                   Where elem.@CatID.ToString.Equals("2") _
                   Select elem

    xmlData.First.Remove()
'atau

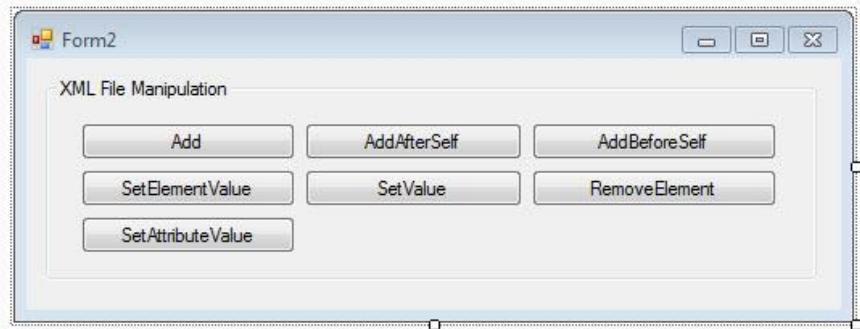
```

```
'xmlData.Single.Remove()

$xmlBook.Save("../..//Books.xml")
Console.WriteLine(xmlBook)
Console.ReadLine()
```

End Sub

Kode diatas digunakan untuk menghapus elemen yang memiliki nilai atribut CatID = 2.



Gambar 14.2 Design form

BAB 15

VB 6.0 To VB.NET Migration Tools Helper

.Net framework SDK menyediakan beberapa tools yang dapat digunakan untuk membantu proses upgrade aplikasi vb 6.0 ke vb.net. Tools tersebut juga dapat dikatakan sebagai COM interoperability tools.

Komponen atau library yang dibuat dengan menggunakan VB 6.0 dapat digunakan di aplikasi yang dibangun dengan VB.NET, begitu juga sebaliknya. Untuk itulah dibutuhkan interoperability tools antara COM dengan .NET.

Beberapa tools tersebut adalah Regasm.exe (Register Assembly), Tlbexp.exe (Type Library Exporter), Tlbimp.exe (Type Library Importer). Tools tersebut diakses lewat .NET Framework command prompt. Adapun cara yang lebih mudah untuk mengekspose .NET assembly ke COM type yaitu dengan mengaktifkan setting Register for COM Interop pada project properties, cara ini merupakan alternatif lain selain dengan menggunakan utility Tlbexp.exe

Selain itu didalam bab ini akan juga dibahas tools migrasi lainnya yaitu Interop Forms Toolkit 2.0 yang sangat membantu sekali dalam proses upgrading aplikasi VB 6.0 ke VB.NET. Toolkit ini nantinya diintegrasikan kedalam Visual Studio 2008 yang akan menyediakan beberapa project template baru.

Register for COM Interop

Perubahan setting Register for COM Interop pada project properties akan menghasilkan sebuah file .tlb dan sekaligus melakukan registrasi ke dalam database registry.

Agar type atau member yang terdapat didalam .NET assembly dapat diekspos ke COM maka harus ditambahkan atribut ComVisible yang berisi nilai True pada assembly, class, properti, method atau member lainnya. Nilai default untuk atribut tersebut yaitu False.

File .tlb menyimpan semua informasi yang dibutuhkan oleh COM client dalam memanggil object yang terdapat didalam .NET assembly. Semua .NET data type akan dikonversikan menjadi COM data type, misalnya System.Object di .NET akan di mapping menjadi tipe data variant di COM.

Berikut adalah contoh penggunaan setting Register for COM Interop untuk membuat COM library sebagai hasil konversi mapping dari .NET library :

1. Buat sebuah Class Library project template baru pada Visual Studio 2008 kedalam solution sebelumnya yang sudah dibuat. Berikan nama project tersebut MyNETLibrary.
2. Ubah nama file Class1.vb menjadi MyNETLibrary.vb.
3. Lakukan Imports pada namespace berikut ini :

Imports System.Runtime.InteropServices

4. Ketikkan kode berikut didalam class tersebut :

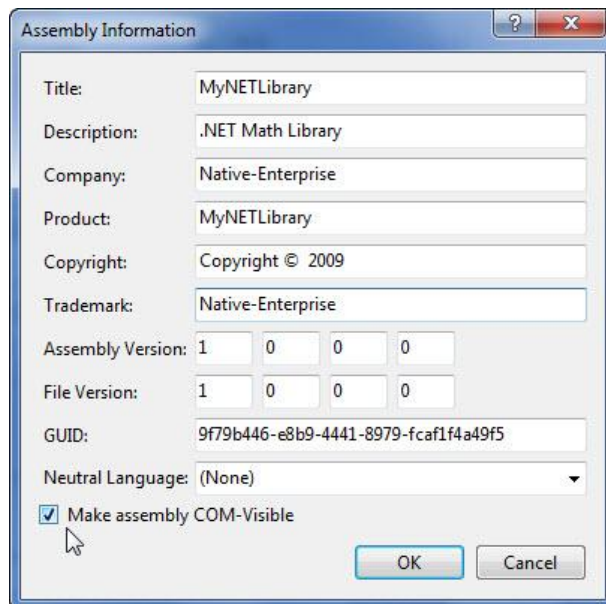
```
<ComVisible(True)> _  
Public Class MyNETLibrary  
  
    <ComVisible(True)> _  
    Public Function SumTwoNum(ByVal Num1 As Integer, _  
        ByVal Num2 As Integer) As Integer  
        Return Num1 + Num2  
    End Function  
End Class
```

End Function

```
<ComVisible(True)> _  
Public Function SubtractTwoNum( _  
    ByVal Num1 As Integer, ByVal Num2 As Integer) _  
    As Integer  
    Return Num1 + Num2  
End Function
```

End Class

5. Ubah setting COM-Visible ke True pada project properties di bagian Application tab dengan klik tombol Assembly Information dan lakukan perubahan yang diperlukan pada assembly informationnya.



Gambar 15.1 Setting COM Visible

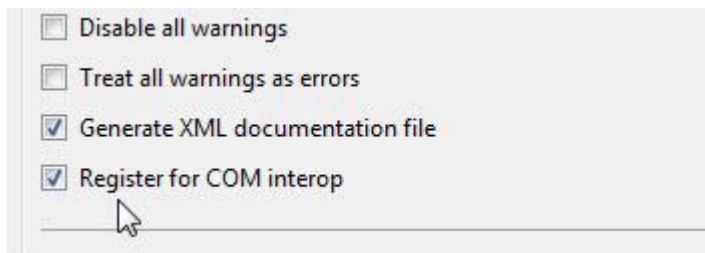
Perubahan setting COM Visible ke true diperlukan untuk mengekspose .NET type ke COM type. Perubahan setting tersebut berdampak pada setting assembly ComVisible attribute di file AssemblyInfo.vb yang dapat kita lihat apabila Show All Files di project diaktifkan. Assembly attribut tersebut memiliki nilai false secara default :

<Assembly: ComVisible(False)>

Setelah dilakukan perubahan setting COM Visible diatas menjadi true maka attribut ComVisible akan berubah menjadi:

<Assembly: ComVisible(True)>

6. Ubah setting Register for COM Interop menjadi aktif di bagian Compile project properties :



Gambar 10.2 Setting COM interop

7. Build project tersebut.
8. Hasil kompilasi akan menghasilkan file .tlb dengan nama yang sama dengan nama file .dll yang digunakan. File tersebut dapat dilihat pada direktori Debug atau Release dari project.

Name	Date modified	Type	Size
 MyNETLibrary.dll	9/27/2009 3:23 PM	Application extension	11 KB
 MyNETLibrary.pdb	9/27/2009 3:23 PM	PDB File	28 KB
 MyNETLibrary.tlb	9/27/2009 3:23 PM	TLB File	2 KB
 MyNETLibrary	9/27/2009 3:23 PM	XML Document	1 KB

Gambar 15.3 File .tlb

Untuk melihat output registry yang dihasilkan kita dapat menggunakan utility Regasm.exe dengan option /regfile seperti pada contoh berikut ini yang dijalankan dari Visual Studio 2008 command prompt :

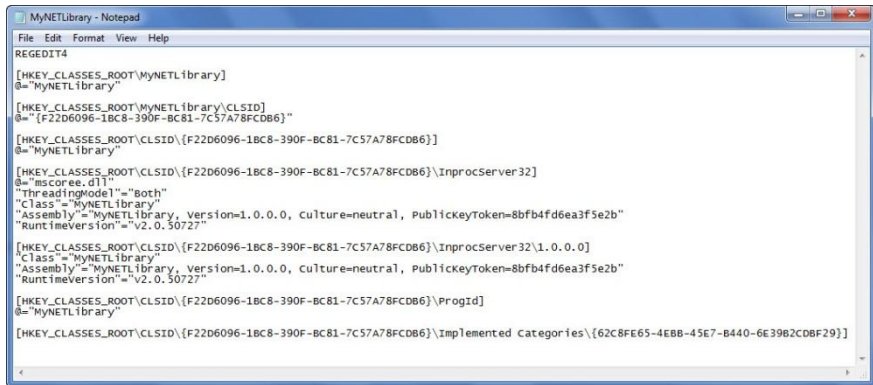
```
regasm.exe MyNETLibrary.dll /regfile:MyNETLibrary.reg
```

Berikut file registry yang dihasilkan :

Name	Date modified	Type	Size
 MyNETLibrary.dll	9/27/2009 3:23 PM	Application extension	11 KB
 MyNETLibrary.pdb	9/27/2009 3:23 PM	PDB File	28 KB
 MyNETLibrary	9/27/2009 3:28 PM	Registration Entries	1 KB
 MyNETLibrary.tlb	9/27/2009 3:23 PM	TLB File	2 KB
 MyNETLibrary	9/27/2009 3:23 PM	XML Document	1 KB

Gambar 15.4 File .reg yang dihasilkan dari perintah regasm.exe

Berikut isi dari file registry yang dihasilkan dan di register ke dalam database registry windows :



Gambar 15.5 Isi file registry

Untuk menghapus informasi registry gunakan option /u dari utility regasm.exe :

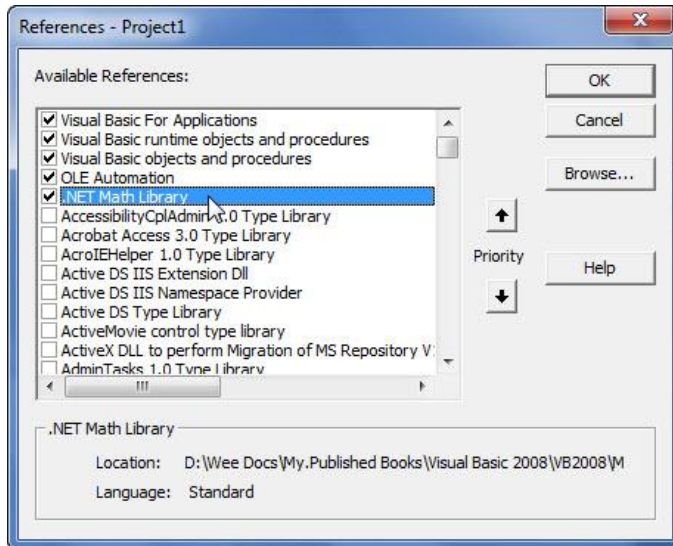
regasm.exe MyNETLibrary.dll /u

Mengakses .NET Assembly dari COM Client

Sekarang saatnya untuk mencoba akses COM type library yang telah dihasilkan dari hasil konversi .NET assembly pada lab sebelumnya dari VB 6.0 Application.

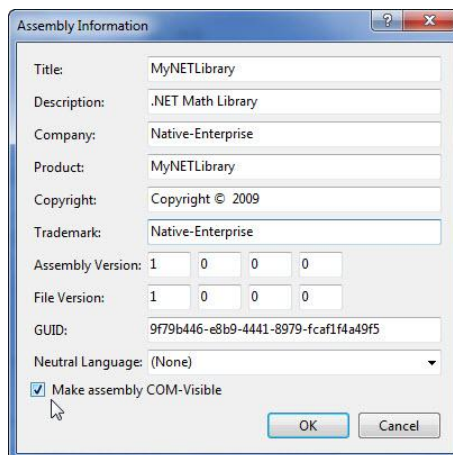
Berikut adalah langkah-langkah untuk akses .NET assembly yang telah di mapping menjadi COM type library :

1. Buka Visual Studio 6.0 dan buat sebuah Visual Basic 6.0 Standard Exe Application.
2. Tambahkan referensi ke MyNETLibrary assembly yang telah dibuat pada lab sebelumnya dengan klik menu Project – References.



Gambar 15.6 Reference ke file .tlb

Perlu diperhatikan bahwa nama assembly yang muncul di dalam kotak dialog references berdasarkan Description yang telah kita tambahkan pada Assembly Information saat pembuatan .NET assembly pada lab sebelumnya, sehingga yang muncul yaitu .NET Math Library.



Gambar 15.7 Description pada Assembly Information

3. Tambahkan sebuah button pada form tersebut kemudian lakukan double klik pada button untuk menghasilkan event handler click dan tambahkan kode berikut ini :

```
Private Sub Command1_Click()  
  
    Dim myMath As MyNETLibrary.MyNETLibrary  
    Set myMath = New MyNETLibrary.MyNETLibrary  
  
    Dim intSum As Integer  
    intSum = myMath.SumTwoNum(1, 2)  
    MsgBox intSum  
  
    Dim intSubs As Integer  
    intSubs = myMath.SubstractTwoNum(2, 1)  
    MsgBox intSubs  
  
End Sub
```

4. Jalankan aplikasi tersebut. Hasil eksekusi prosedur pertama akan menghasilkan nilai 3 sedangkan prosedur kedua akan menghasilkan nilai 1.

Interop Forms Toolkit 2.0

Toolkit ini merupakan add-in yang ditujukan untuk memudahkan phase proses upgrading aplikasi VB 6.0 ke VB.NET. Dengan toolkit ini forms yang telah dibuat di aplikasi VB.NET dapat digunakan di aplikasi VB 6.0.

Selain forms, .NET user controls juga dapat digunakan pada aplikasi VB 6.0 layaknya penggunaan ActiveX control biasa. Untuk menggunakan toolkit ini sebelumnya kita harus download terlebih dahulu di url berikut :

<http://msdn.microsoft.com/en-us/vbasic/bb419144.aspx>

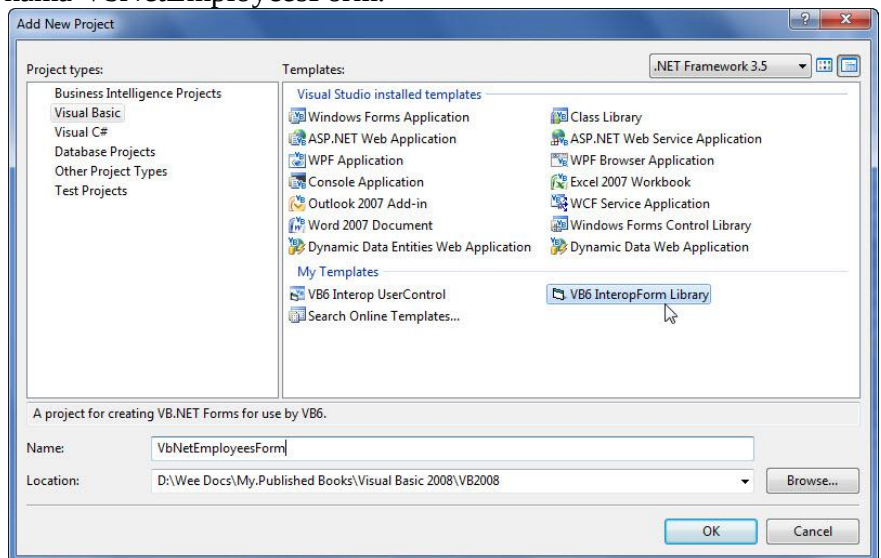
Setelah toolkit tersebut didownload tentunya anda dapat melakukan instalasi seperti biasa. Hasil instalasi akan menambahkan project template baru pada Visual Studio 2008. Project template tersebut yaitu berupa template untuk membuat forms interop dan user controls interops.

VB 6.0 InteropForm Library

Dengan project template tersebut kita dapat membuat .NET forms beserta control-control nya dan digunakan pada aplikasi VB 6.0.

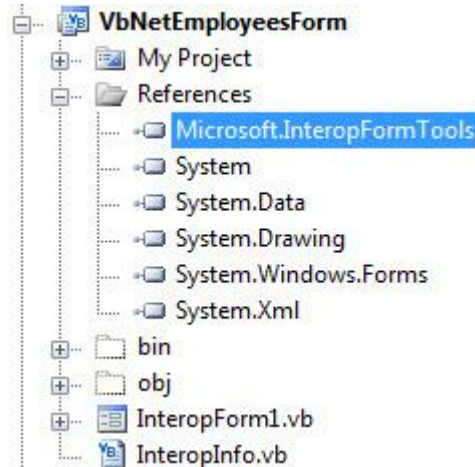
Berikut adalah contoh penggunaan project template tersebut :

1. Tambahkan project baru kedalam solution yang sama. Pilih VB 6.0 InteropForm Library project template dan beri nama VbNetEmployeesForm.



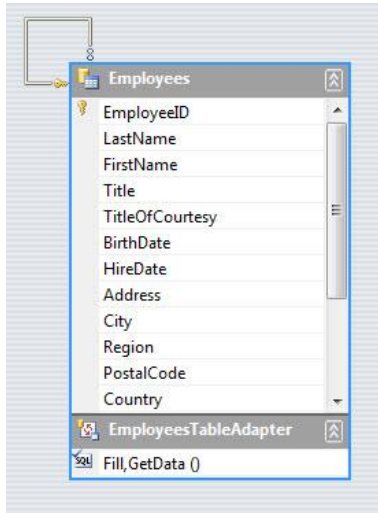
Gambar 15.8 VB6 InteropForm Library template

Struktur project yang dihasilkan dapat dilihat seperti berikut ini :



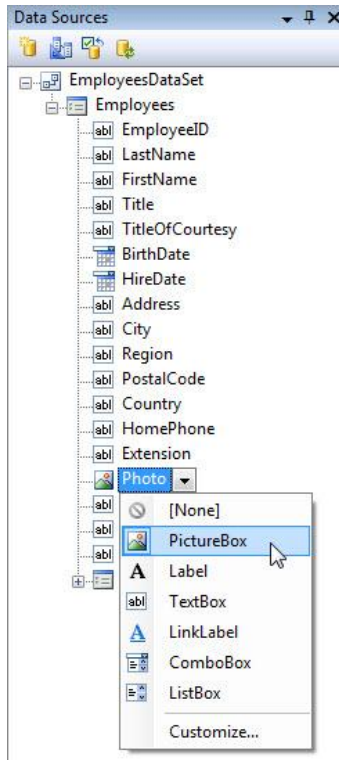
Gambar 15.9 Referensi terhadap Microsoft.InteropFormTools.dll

2. Tambahkan data source baru berupa Typed Dataset yang telah dibahas pada bab sebelumnya. Pilih database sebagai sumber data dan koneksikan ke database SQL Server untuk mengambil data dari database Northwind.
3. Pilih tabel Employees dan ubah nama dataset nya menjadi EmployeesDataSet kemudian klik button Finish.



Gambar 15.10 Employees.xsd file

4. Pilih mode details untuk Employees TypedDataSet di DataSource window. Ubah control field Photo menjadi PictureBox.



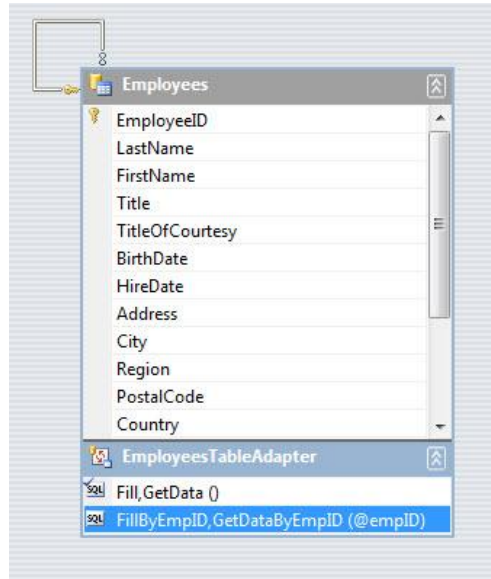
Gambar 15.11 Details mode dan PictureBox control

5. Drag dan drop Employees DataTable diatas ke dalam form dan lakukan perubahan design form seperti berikut :

The image shows a Windows application window titled "InteropForm1". Inside the window is a form with two columns of input fields. The left column contains: Employee ID, Last Name, First Name, Title, Title Of Courtesy, Birth Date (with a calendar icon), Hire Date (with a calendar icon), Address, City, Region, and a Photo placeholder. The right column contains: Postal Code, Country, Home Phone, Extension, Reports To, Photo Path, and a large Notes text area. The form is currently empty.

Gambar 15.12 Design Form

6. Tambahkan Query Adapter baru pada employees dataset untuk mengambil data employees berdasarkan EmployeeID. Teknik ini sudah dibahas pada bab sebelumnya. Hasilnya adalah seperti berikut ini :



Gambar 15.13 Penambahan Query Adapter baru.

7. Buka source code InteropForm1 kemudian imports namespace berikut untuk menampilkan MessageBox :

Imports System.Windows.Forms

8. Buat sebuah property berupa EmployeeID yang mengembalikan nilai integer. Agar property tersebut dapat diekspose ke COM maka kita harus menambahkan atribut <InteropFormProperty> begitu juga apabila kita ingin membuat Event atau method.

Private mID As Integer

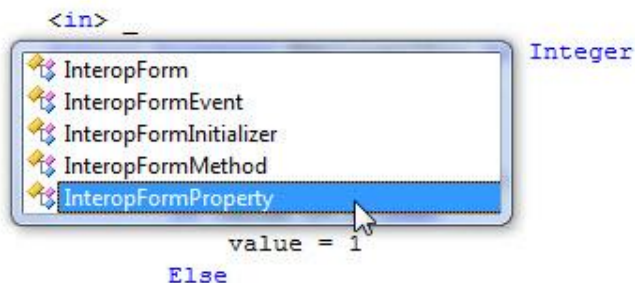
**<InteropFormProperty()> _
Public Property EmployeeID() As Integer
Get**

```

        Return mID
    End Get
    Set(ByVal value As Integer)
        If value <= 0 Then
            mID = 1
        Else
            mID = value
        End If
    End Set
End Property

```

Beberapa atribut yang tersedia selain untuk property yaitu:



Gambar 15.14 InteropForm attribute

9. Lakukan modifikasi kode berikut pada event save button :

```

Private Sub EmployeesBindingNavigatorSaveItem_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles EmployeesBindingNavigatorSaveItem.Click

```

```

    Try
        Me.Validate()
        Me.EmployeesBindingSource.EndEdit()
        Me.TableAdapterManager.UpdateAll( _
            Me.EmployeesDataSet)

```

```

        MessageBox.Show("Saved.")

```

```
Catch ex As Exception
    MessageBox.Show(ex.Message)
End Try
```

```
End Sub
```

10. Lakukan modifikasi kode berikut pada event Load dari Form :

```
Private Sub InteropForm1_Load(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
MyBase.Load
```

```
    Try
        Me.EmployeesTableAdapter.FillByEmpID( _
            EmployeesDataSet.Employees, EmployeeID)
    Catch ex As Exception
        MessageBox.Show(ex.Message)
    End Try
```

```
End Sub
```

Kode diatas digunakan untuk mengambil data employee berdasarkan employeeid yang dipilih.

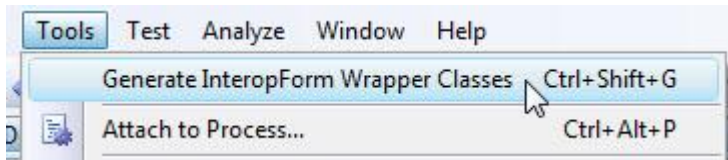
11. Buka file InteropInfo.vb dan lakukan perubahan setting attribute berikut menjadi True :

```
<Assembly: System.Runtime.InteropServices.ComVisible(True)>
```

Perubahan tersebut dilakukan agar .NET assembly dapat dikenali oleh COM client.

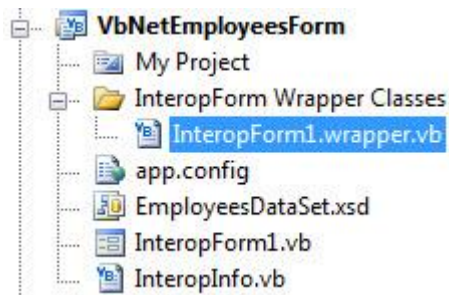
12. Agar method, event atau property yang dibuat dapat dikenali di COM client dan muncul di intellisense Visual Studio 6.0 editor maka terlebih dahulu harus dibuatkan

wrapper class. Untuk melakukan hal tersebut klik menu Tools – Generate InteropForm Wrapper Classes :



Gambar 15.15 InterorForm Wrapper Classes menu.

Struktur project baru yang dihasilkan oleh menu tersebut dapat dilihat seperti berikut ini :



Gambar 15.16 InteropForm wrapper classes.

Berikut adalah isi kode dari file InteropForm1.wrapper.vb :

```
Option Strict Off
Option Explicit On
```

```
Imports Microsoft.InteropFormTools
```

```
Namespace Interop
```

```
<System.Runtime.InteropServices.ClassInterface(Runtime.Inte
InteropServices.ClassInterfaceType.AutoDual), _
```

```

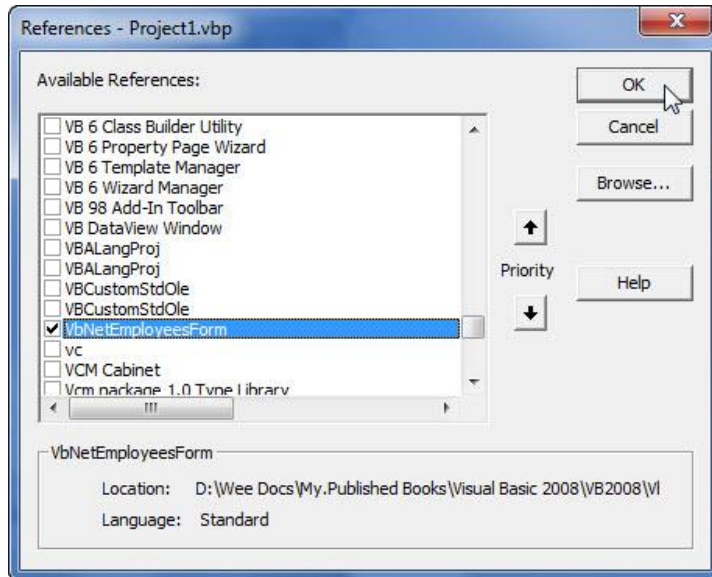
System.Runtime.InteropServices.ComVisible(true)> _
Partial Public Class InteropForm1
    Inherits InteropFormProxyBase

    Public Sub New()
        MyBase.New
        FormInstance = New
VbNetEmployeesForm.InteropForm1()
        RegisterFormInstance()
    End Sub

    Public Overridable Property EmployeeID() As Integer
    Get
        Dim castFormInstance As
VbNetEmployeesForm.InteropForm1 = FormInstance
        Return castFormInstance.EmployeeID
    End Get
    Set
        Dim castFormInstance As
VbNetEmployeesForm.InteropForm1 = FormInstance
        castFormInstance.EmployeeID = value
    End Set
    End Property
End Class
End Namespace

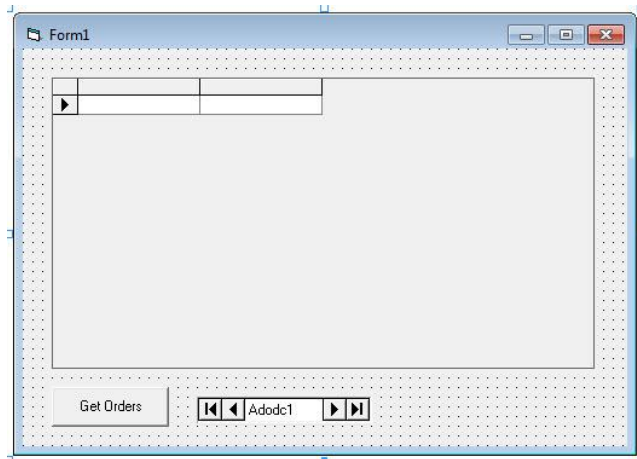
```

13. Lakukan proses Build. Proses Build ini akan sekaligus melakukan registrasi assembly kedalam registry sehingga dapat di referensi oleh COM client.
14. Buka Visual Studio 6.0 untuk membuat Visual Basic 6.0 Standard Application.
15. Tambahkan referensi ke assembly VbNetEmployeesForm.dll :

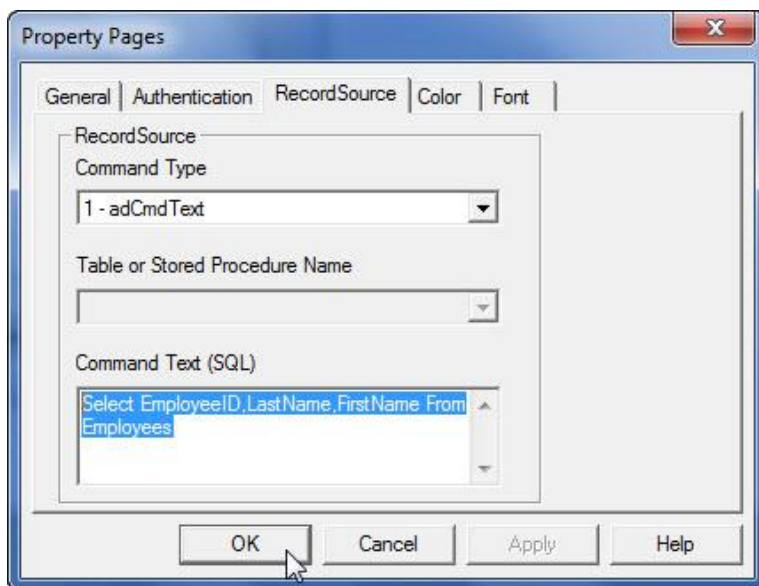


Gambar 15.17 Referensi ke VB.NET form assembly

16. Tambahkan DataGridView, button dan Adodc control ke dalam form.
17. Lakukan setting pada Adodc control sehingga terkoneksi ke database SQL Server untuk mengambil data dari database Northwind. Ketikkan query berikut pada Adodc control properties :



Gambar 15.18 Design VB 6.0 form



Gambar 15.19 Setting SQL Adodc control

18. Atur nilai properti DataSource dari DataGrid control ke Adodc1.
19. Double klik button pada form untuk menghasilkan event click dan tambahkan kode berikut ini :

```
Private Sub Command1_Click()
```

```
    Dim empID As Integer  
    empID = CInt(DataGrid1.Columns(0).Text)
```

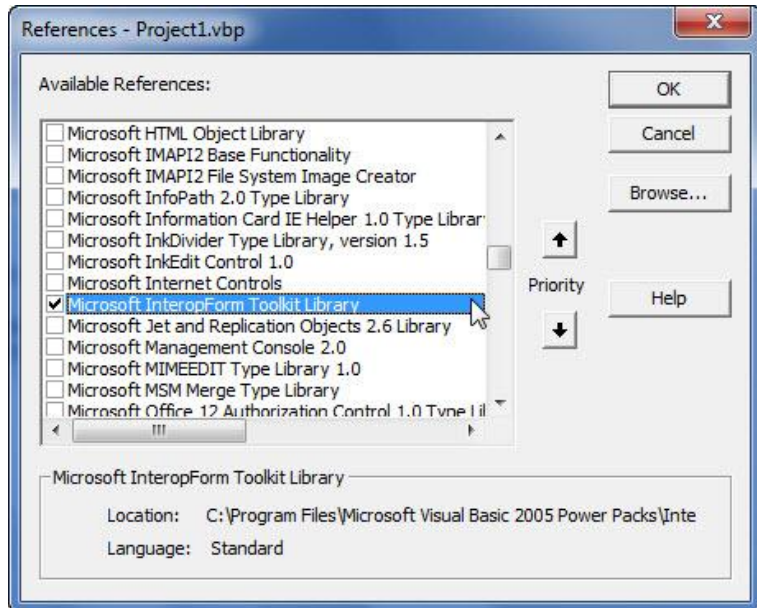
```
    Dim frm As New VbNetEmployeesForm_Interop_InteropForm1  
    frm.EmployeeID = empID  
    frm.Show
```

```
End Sub
```

Kode diatas digunakan untuk mengambil nilai EmployeeID dari datagrid row yang di pilih. Selain itu untuk memanggil form yang telah dibuat di VB.NET lakukan proses instantiasi terhadap InteropForm Wrapper Class yang telah dihasilkan.

20. Karena VB.NET form ini dipanggil oleh COM client maka interop form yang dipanggil tidak memiliki notifikasi secara default apabila aplikasi form VB 6.0 yang memanggil interop form tersebut di close.

Oleh karena itu kita harus menambahkan event notifikasi pada form VB 6.0 tersebut. Untuk melakukannya tambahkan referensi ke InteropForm Toolkit library :



Gambar 15.20 Referensi terhadap InteropForm Toolkit.

21. Deklarasikan Public class level variabel berikut terhadap InteropToolbox class :

```
Public itrToolbox As InteropToolbox
```

22. Tambahkan kode berikut pada event Load dan QueryUnload dari form untuk memberikan event notifikasi ketika form di load dan form di unload :

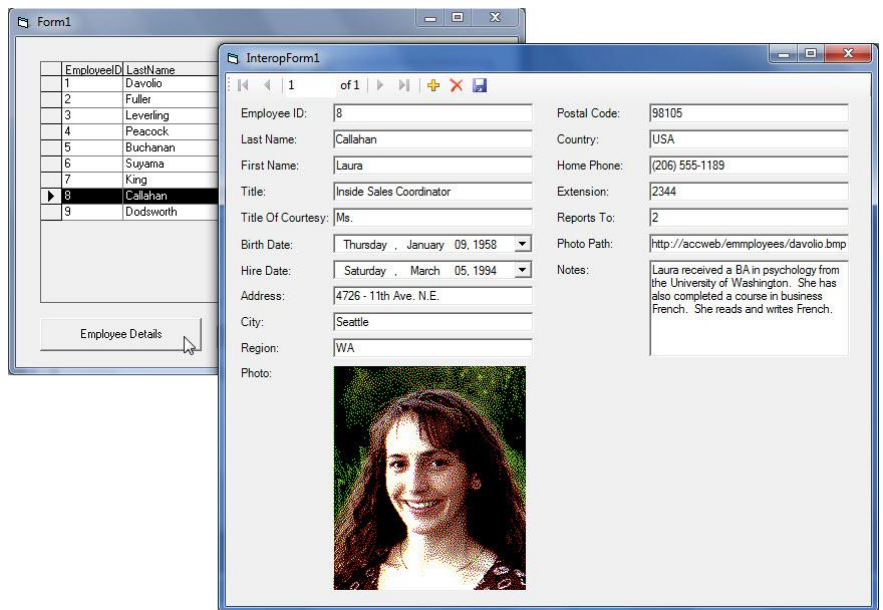
```
Private Sub Form_Load()  
    Set itrToolbox = New InteropToolbox  
    itrToolbox.Initialize  
  
    itrToolbox.EventMessenger.RaiseApplicationStartedupEvent  
End Sub
```

```

Private Sub Form_QueryUnload(Cancel As Integer, UnloadMode As Integer)
    itrToolbox.EventMessenger.RaiseApplicationShutdownEvent
End Sub

```

23. Berikut adalah hasilnya apabila aplikasi dijalankan :



Gambar 15.21 Hasil pemanggilan VB.NET form

VB6 Interop UserControl

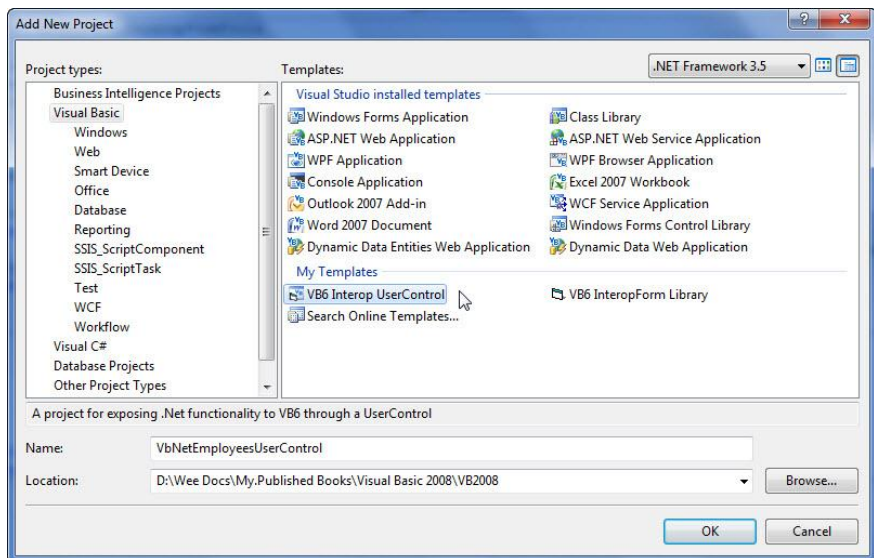
InteropFormsToolkit 2.0 menyediakan project template baru berupa Interop UserControl yang dapat diakses oleh COM client.

Project template tersebut akan menghasilkan wrapper class yang berguna untuk melakukan registrasi assembly ke dalam registry dengan memberikan informasi seperti ClassID, ProgID dan penanganan event-event user control tersebut.

Dengan usercontrol maka kita bisa tempatkan usercontrol tersebut pada VB 6 form.

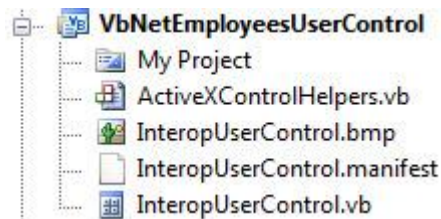
Berikut adalah contoh pembuatan Interop UserControl :

1. Tambahkan project baru kedalam solution yang sama. Pilih VB6 Interop UserControl project template dan beri nama VbNetEmployeesUserControl.



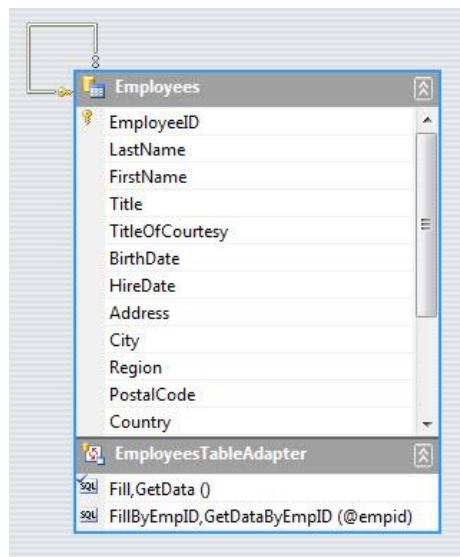
Gambar 15.22 VB6 Interop UserControl template

2. Struktur project yang dihasilkan :



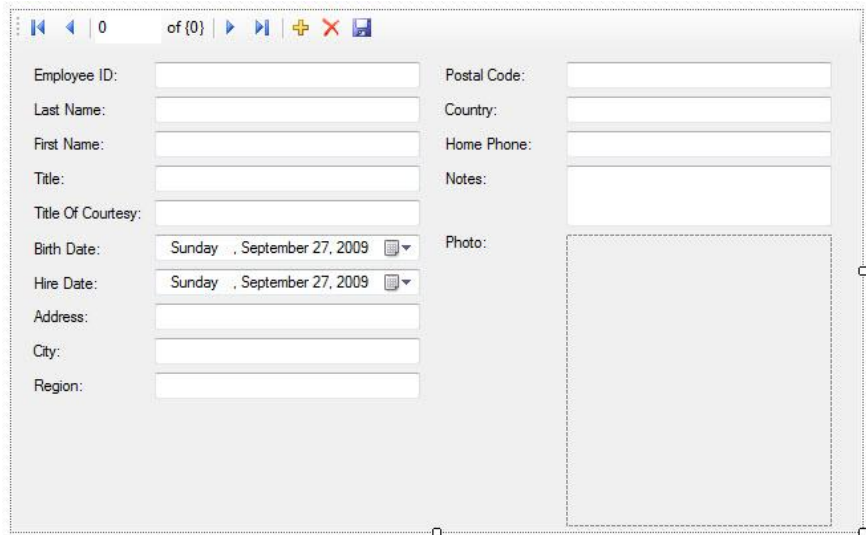
Gambar 15.23 Struktur project interop usercontrol

3. Tambahkan DataSource baru berupa data yang diambil dari database Northwind dimana koneksi database server terhubung ke SQL Server. Pilih tabel Employees. Ubah nama dataset nya menjadi EmployeesDataSet.
4. Setelah EmployeesDataSet.xsd terbentuk, tambahkan Query Adapter baru berupa query untuk mengambil data Employees berdasarkan EmployeeID seperti yang telah dilakukan pada lab sebelumnya.



Gambar 15.24 EmployeesDataSet.xsd

5. Tambahkan Employees datatable dari dataset yang terdapat di dalam DataSource window dengan mengubah mode Detail untuk Employee datatable dan PictureBox control untuk field Photo ke dalam UserControl designer. Lakukan design seperti berikut :



Gambar 15.25 Design UserControl

6. Tambahkan kode berikut pada source code usercontrol tersebut :

```

Private mID As Integer

Public Property EmployeeID() As Integer
    Get
        Return mID
    End Get
    Set(ByVal value As Integer)
        If value <= 0 Then
            mID = 1
        Else
            mID = value
        End If
        Try
            Me.EmployeesTableAdapter.FillByEmpID( _
EmployeesDataSet.Employees, EmployeeID)
        Catch ex As Exception
            MessageBox.Show(ex.Message)
        End Try
    End Set
End Property

```

End Set
End Property

Kode diatas merupakan properti untuk menentukan nilai EmployeeID yang digunakan dalam pengambilan data Employees dan sekaligus memanggil query TableAdapter berdasarkan parameter input EmployeeID pada prosedur Setter.

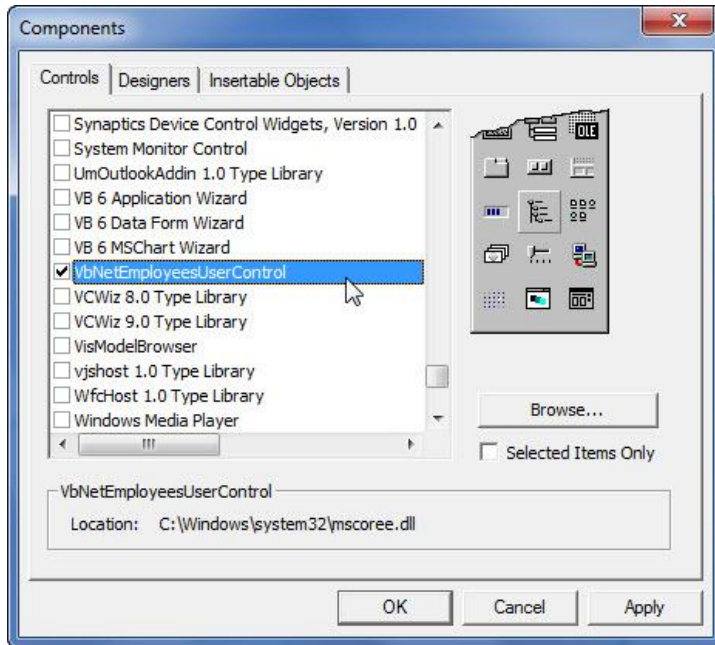
7. Modifikasi kode pada event click Save button toolstrip seperti berikut ini :

```
Private Sub EmployeesBindingNavigatorSaveItem_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)  
Handles EmployeesBindingNavigatorSaveItem.Click
```

```
    Try  
        Me.Validate()  
        Me.EmployeesBindingSource.EndEdit()  
        Me.TableAdapterManager.UpdateAll( _  
            Me.EmployeesDataSet)  
  
        MessageBox.Show("Saved.")  
    Catch ex As Exception  
        MessageBox.Show(ex.Message)  
    End Try
```

```
End Sub
```

8. Build Project untuk menghasilkan file .dll dan file .tlb dari project tersebut.
9. Buka kembali project Visual Basic 6.0 yang telah dibuat pada lab sebelumnya.
10. Tambahkan Components dengan klik menu Project – Components.
11. Pilih VbNetEmployeesUserControl :



Gambar 15.26 Menambahkan Component

Setelah menambahkan komponen tersebut maka nantinya akan terdapat control baru didalam toolbox.

12. Tambahkan button dan component yang baru saja ditambahkan ke dalam form dan lakukan design seperti berikut ini :

Gambar 15.27 Design Form

13. Tambahkan kode berikut pada button dengan caption “Show Details On UserControl” dengan melakukan double klik button tersebut :

```

Private Sub Command2_Click()
    Dim empID As Integer
    empID = CInt(DataGrid1.Columns(0).Text)

    Me.InteropUserControl1.EmployeeID = empID
End Sub

```

14. Jalankan aplikasi VB 6.0 tersebut. Pilih salah satu employee yang terdapat didalam DataGrid kemudian klik button “Show Details On UserControl” maka hasilnya dapat dilihat sebagai berikut :

The screenshot shows a Windows application window titled "Form1". At the top, there is a DataGrid with the following data:

EmployeeID	LastName	FirstName
1	Davolio	Nancy
2	Fuller	Andrew
3	Leverling	Janet
4	Peacock	Margaret
5	Buchanan	Steven
6	Suyama	Michael
7	King	Robert
8	Callahan	Laura
9	Dodsworth	Anne

Below the DataGrid, there are two buttons: "Employee Details" and "Show Details On UserControl". A mouse cursor is pointing at the "Show Details On UserControl" button.

Below the buttons, there is a section for displaying employee details. It includes a navigation bar showing "1 of 1" records. The details are organized into two columns:

- Left Column:**
 - Employee ID: 1
 - Last Name: Davolio
 - First Name: Nancy
 - Title: Sales Representative
 - Title Of Courtesy: Ms.
 - Birth Date: Wednesday, December 08, 1948
 - Hire Date: Friday, May 01, 1992
 - Address: 507 - 20th Ave. E Apt. 2A
 - City: Seattle
 - Region: WA
- Right Column:**
 - Postal Code: 98122
 - Country: USA
 - Home Phone: (206) 555-9857
 - Notes: Education includes a BA in psychology from Colorado State University in 1970. She also completed "The Art of the Cold"
 - Photo: A portrait photo of Nancy Davolio.

Gambar 15.28 Penggunaan Interop UserControl

BAB 16

Setup dan Deployment

Setup and Deployment merupakan salah satu project template yang tersedia di Visual Studio. Project template ini digunakan untuk membuat installer dari aplikasi yang telah selesai dibuat.

Project template ini dapat digunakan untuk berbagai jenis aplikasi seperti Desktop Application, Console Application atau Web Application.

Dengan project template ini kita tidak membutuhkan lagi third party installer. Visual studio sudah menyediakan fitur yang lengkap untuk memenuhi kebutuhan pembuatan installer untuk deployment.

Setup and Deployment project menyediakan fitur untuk membuat shortcut desktop, shortcut program files, menambahkan list di Add/Remove program, menulis registry, deploy shared assembly, install database file dengan custom action dan lain-lain.

Pembahasan pada bab ini akan difokuskan untuk pembuatan installer aplikasi desktop atau Windows Forms Application.

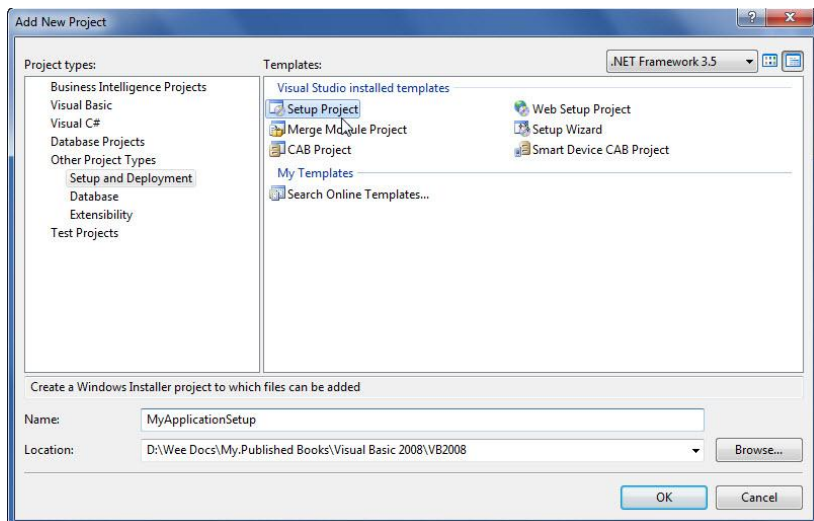
Setup Projects

Setup project merupakan template project yang digunakan untuk membuat installer aplikasi desktop. Terlebih dahulu kita harus mempersiapkan aplikasi desktop yang akan dibuatkan setup installernya.

Hal yang harus diperhatikan sebelum membuat setup installer yaitu setting konfigurasi Build. Terdapat dua setting yaitu Debug dan Release. Mode debug digunakan selama phase development untuk kepentingan debugging. Sedangkan mode Release dibutuhkan untuk kepentingan deployment. Mode release akan mengabaikan instruksi-instruksi debugging, sehingga dari sisi performansi lebih baik juga halnya dengan ukuran file yang lebih kecil.

Berikut adalah langkah-langkah pembuatan setup projects yang akan membuat setup installer untuk salah satu windows forms application di project yang telah kita buat sebelumnya pada bab terdahulu :

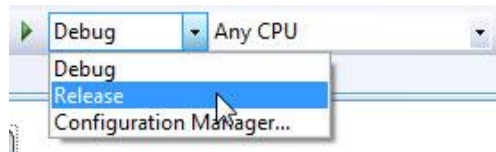
1. Tambahkan project baru pada solution yang telah dibuat sebelumnya pada bab terdahulu. Project template yang dipilih yaitu Setup Projects yang terdapat didalam Setup and Deployment Projects :



Gambar 16.1 Setup Project template

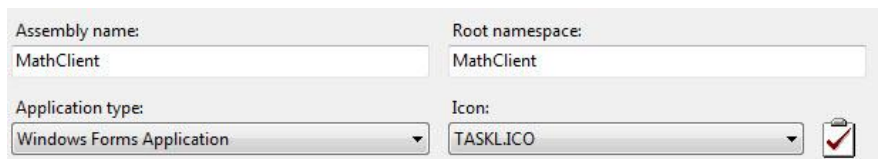
2. Buka salah satu form pada project yang akan dibuatkan installernya. Disini kita akan memilih MathClient project yang telah dibuat pada bab .NET Assemblies.

3. Ubah mode build configurationnya dari debug ke release :



Gambar 16.2 Release mode

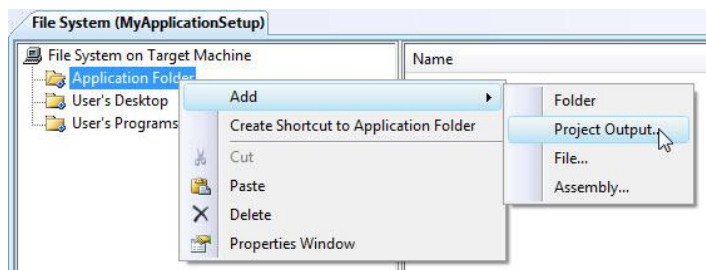
4. Selain itu kita dapat mengubah icon dari aplikasi dengan mengubahnya pada project properties seperti berikut :



Gambar 16.3 Mengubah icon project

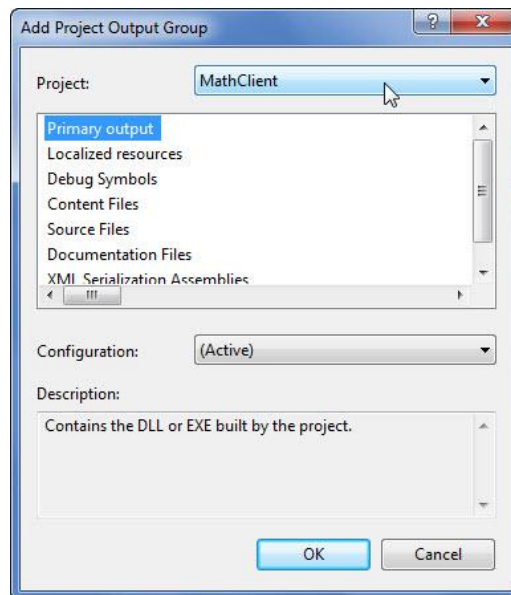
Perhatikan properti Icon yang telah diubah nilainya dari default menjadi icon yang kita inginkan.

5. Buka File System Editor pada toolbar setup project yang telah dibuat. Tambahkan output project pada Application folder.



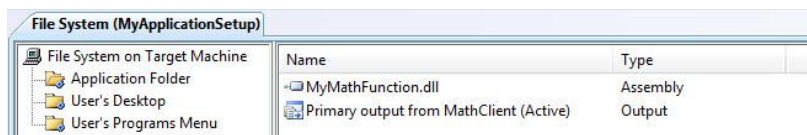
Gambar 16.4 Project Ouput

6. Pilih MathClient project yang terdapat didalam solution yang sama untuk Primary outputnya. Dan pilih configuration Active (dalam hal ini mode Release merupakan mode yang aktif di project tersebut).



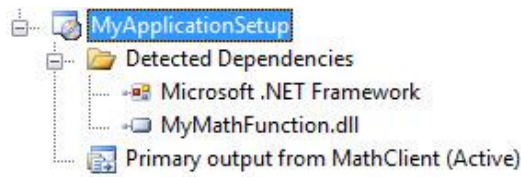
Gambar 16.5 Primary Output project

File-file yang akan di include kan pada Application Folder akan terlihat seperti pada gambar dibawah. Perlu diperhatikan bahwa semua library yang direferensi oleh project tersebut akan diikutkan kedalam project tersebut.



Gambar 16.6 File System window

Selain itu kita dapat melihat dependensi file-file yang dibutuhkan oleh aplikasi termasuk .net framework. .net framework ini harus sudah terinstal terlebih dahulu pada komputer yang akan diinstal.



Gambar 16.7 Dependensi file

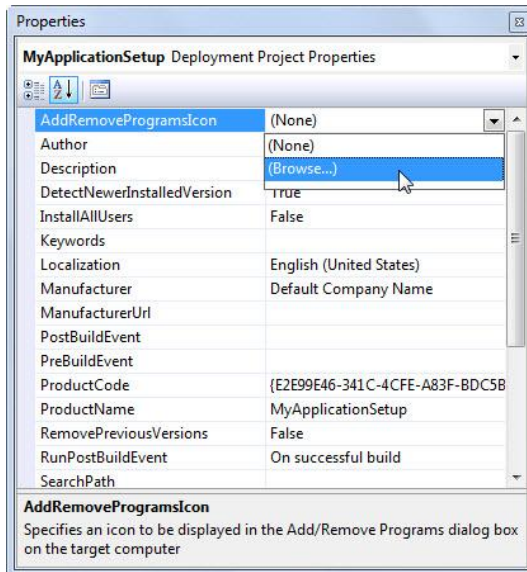
Merubah Icon Add/Remove Program dan Informasi Produk

Setiap aplikasi yang diinstal akan menambahkan list nya pada Add/Remove program agar dapat diubah dan di uninstall. Windows memiliki icon standar apabila aplikasi yang diinstal tidak memiliki icon khusus untuk aplikasi tersebut.

Informasi produk atau aplikasi dapat kita ubah sesuai dengan spesifikasi aplikasi tersebut. Misalnya kita ingin mengubah nama produk, deskripsi, nama perusahaan, url website, no telp dan lain-lain.

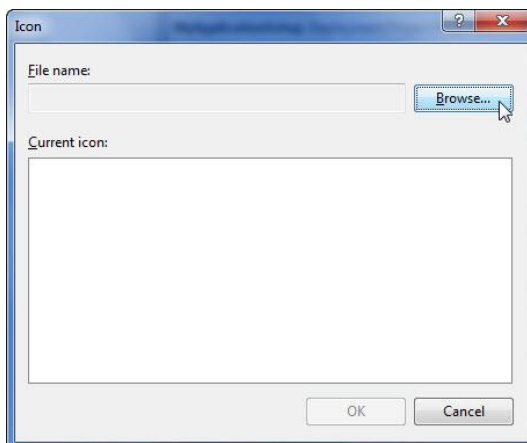
Berikut langkah-langkah untuk melakukan hal tersebut diatas :

1. Buka properties window dari setup project application tersebut.
2. Ubah icon Add/Remove program pada properties window.



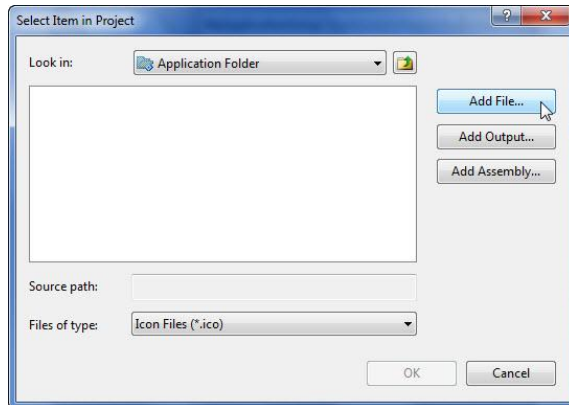
Gambar 16.8 Setup project properties window

3. Klik tombol browse

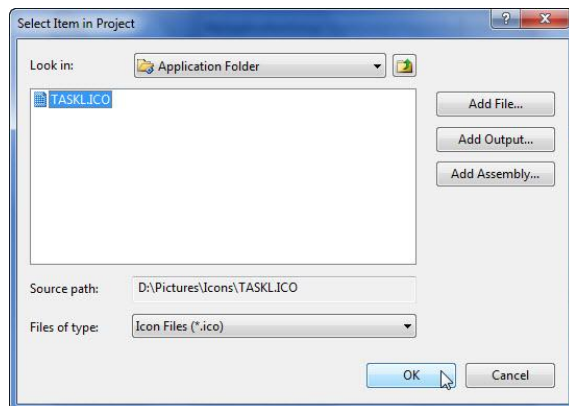


Gambar 16.9 Browse icon file

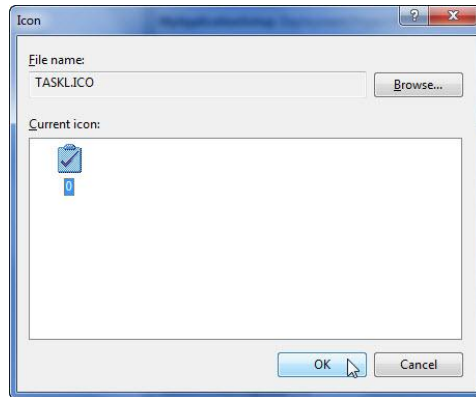
4. Klik tombol Add File untuk mencari icon file yang akan dimasukkan kedalam Application Folder.



Gambar 16.10 Add file icon ke Application Folder

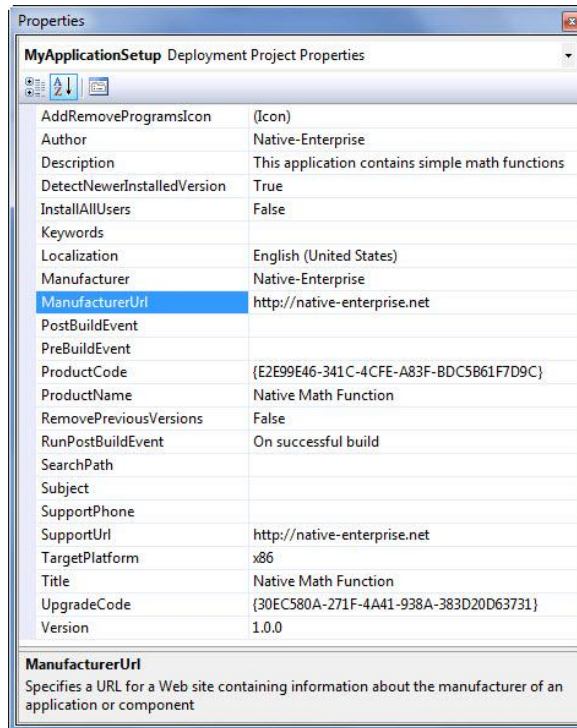


Gambar 16.11 File icon di Application Folder



Gambar 16.12 Icon yang dipilih

5. Modifikasi informasi yang dibutuhkan pada setup project properties window seperti author, description, manufacturer url, product name, support url, dan title.

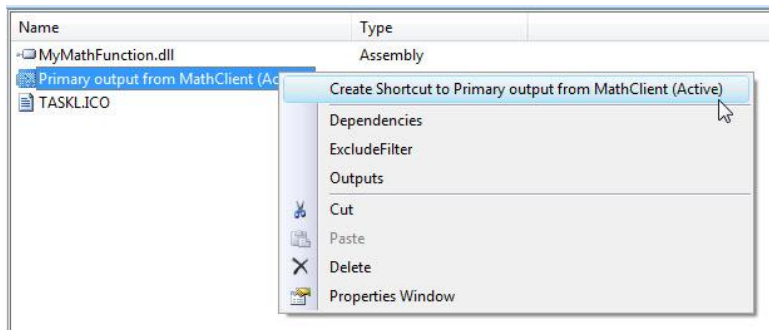


Gambar 16.13 Modifikasi informasi produk

Membuat shortcut

Shortcut sangat dibutuhkan untuk memudahkan akses terhadap aplikasi tersebut. Kita dapat membuat shortcut di desktop atau di menu program files. Icon dari shortcut juga dapat diubah agar tidak menggunakan default shortcut bawaan windows. Berikut langkah-langkahnya :

1. Klik kanan Primary Output project pada File System Editor, pilih menu Create Shortcut...

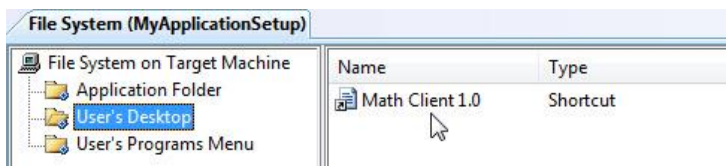


Gambar 16.14 Membuat shortcut

Name	Type
Math Client 1.0	Shortcut
MyMathFunction.dll	Assembly
Primary output from MathClient (Active)	Output
TASKLICO	File

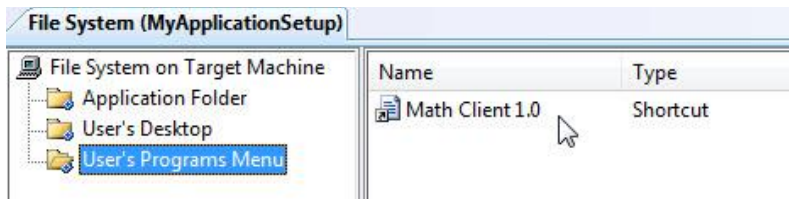
Gambar 16.15 Shortcut yang telah dibuat

2. Pindahkan shortcut tersebut ke User's Desktop shortcut dengan drag dan drop.



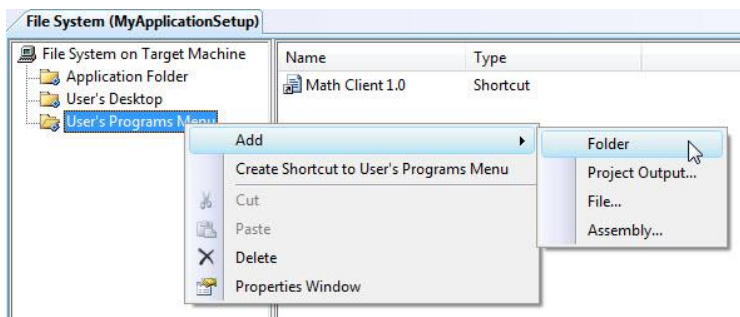
Gambar 16.16 Membuat shortcut di desktop

3. Lakukan langkah pertama kemudian pindahkan shortcut tersebut ke User's Programs Menu folder :



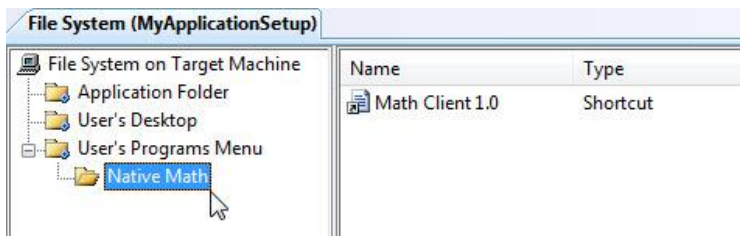
Gambar 16.17 Membuat shortcut di Program menu

4. Untuk membuat sub folder di User's Program Menu klik kanan folder tersebut pilih Add – Folder...



Gambar 16.18 Membuat sub folder di program menu

5. Beri nama folder tersebut Native Math dan pindahkan shortcutnya ke sub folder yang telah dibuat.



Gambar 16.19 Membuat shortcut di sub folder program menu

Launch Conditions

Editor ini digunakan untuk menentukan komponen yang harus diinstal terlebih dahulu atau komponen yang wajib ada dengan spesifikasi yang ditentukan sebelum aplikasi tersebut diinstal.

Salah satu contoh komponen tersebut yaitu .net framework. Komponen ini harus terinstal terlebih dahulu sebelum melakukan instalasi aplikasi yang berbasiskan .net framework.

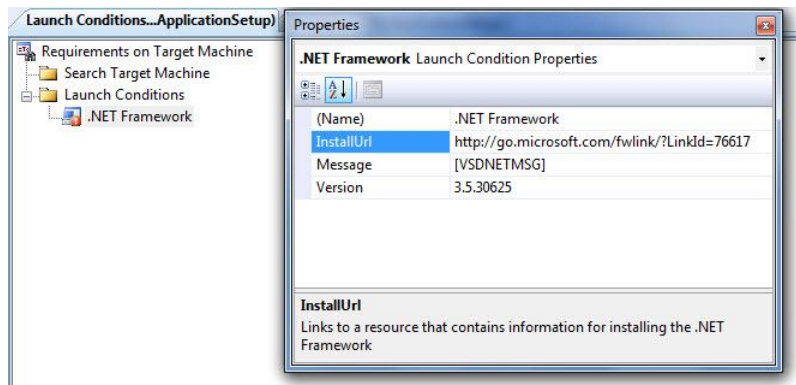
Terdapat beberapa properti yang dapat di modifikasi pada properti komponen .net framework ini, diantaranya yaitu InstallUrl dan Message. InstallUrl digunakan sebagai penunjuk ke lokasi dimana .net framework dapat dicari atau di download apabila didalam komputer tersebut belum terinstal .net framework. Sedangkan properti Message digunakan untuk menampilkan pesan apabila komponen tersebut belum ada di komputer yang akan digunakan.

Kedua properti diatas telah memiliki nilai default. Seperti misalnya properti InstallUrl yang merujuk ke lokasi url microsoft. Sehingga apabila .net framework tersebut belum ada di komputer yang digunakan maka installer akan merujuk ke lokasi url tersebut untuk mendownloadnya.

Properti InstallUrl dapat kita ubah apabila misalnya kita menyertakan file .net framework (dotnetfx.exe) dalam cd installer aplikasi. Sehingga nanti installer akan menjalankan .net framework tersebut langsung dari cd installer apabila komponen tersebut belum terinstal.

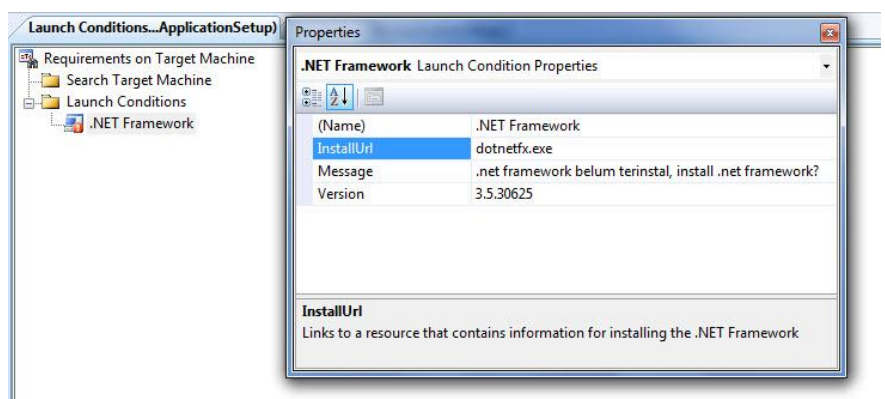
Berikut adalah langkah-langkah untuk mengubah properti komponen .net framework yang terdapat didalam Launch Conditions editor :

1. Buka launch conditions editor dengan melakukan klik pada button toolbar di solution explorer untuk setup project yang telah dibuat.



Gambar 16.20 Properties .net framework conditions

2. Lakukan perubahan pada properti .net framework conditions seperti berikut ini:



Gambar 16.21 modifikasi properti .net framework condition

Dengan perubahan diatas maka nantinya installer akan menjalankan file dotnetfx dari cd installer dan akan mengeluarkan pesan tersebut diatas apabila komponen .net framework belum terinstal.

User Interface Editor

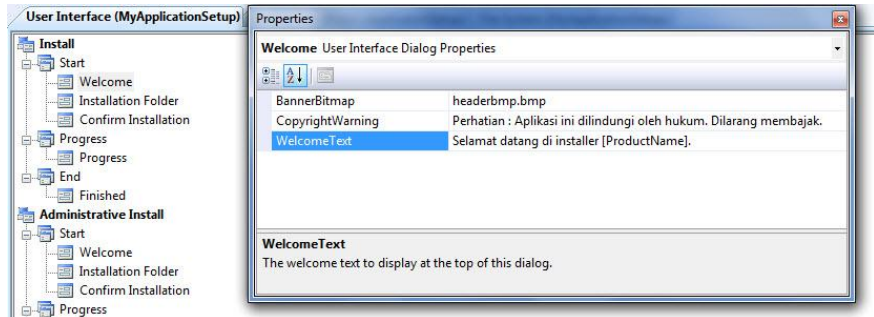
Editor ini dapat digunakan untuk mengubah banner bitmap form instalasi dan teks yang berhubungan dengan setiap proses wizard instalasi.

Terdapat dua bagian user interface yaitu Install dan Administrative Install. Install merupakan installer untuk user pada umumnya sedangkan Administrative Install digunakan untuk instalasi aplikasi yang dapat dijalankan dengan menggunakan perintah msiexec.exe yang mengeksekusi file .msi.

Contoh implementasi kedua kelompok instalasi tersebut yaitu misalnya apabila kita ingin agar user tidak dapat mengubah lokasi direktori dari aplikasi yang akan diinstal namun apabila kita menjalankan instalasi Administrative Install yang dijalankan dengan tools msiexec.exe maka kotak dialog wizard penentuan lokasi direktori aplikasi akan tetap muncul.

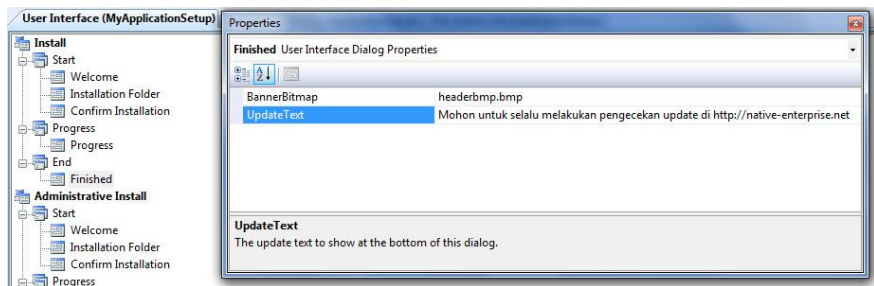
Berikut langkah-langkah untuk melakukan modifikasi User Interface dialog wizard :

1. Buka User Interface editor pada project setup.
2. Ubah bagian welcome editor seperti berikut :



Gambar 16.22 modifikasi BannerBitmap, CopyrightWarning dan WelcomeText properti.

3. Perubahan dapat anda lakukan pada form wizard lainnya misalnya modifikasi properties Installation Folder interface :



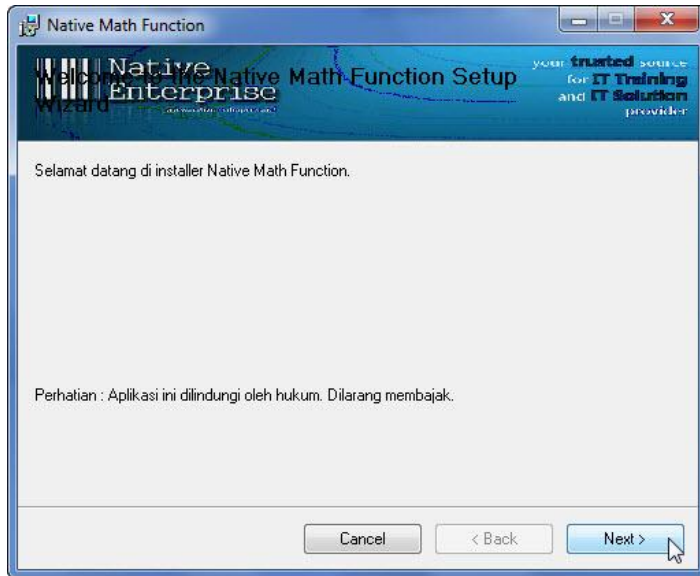
Gambar 16.23 modifikasi BannerBitmap, dan UpdateText properti.

Setelah aplikasi tersebut di build maka akan terdapat dua buah file seperti yang dapat anda lihat pada gambar berikut ini:

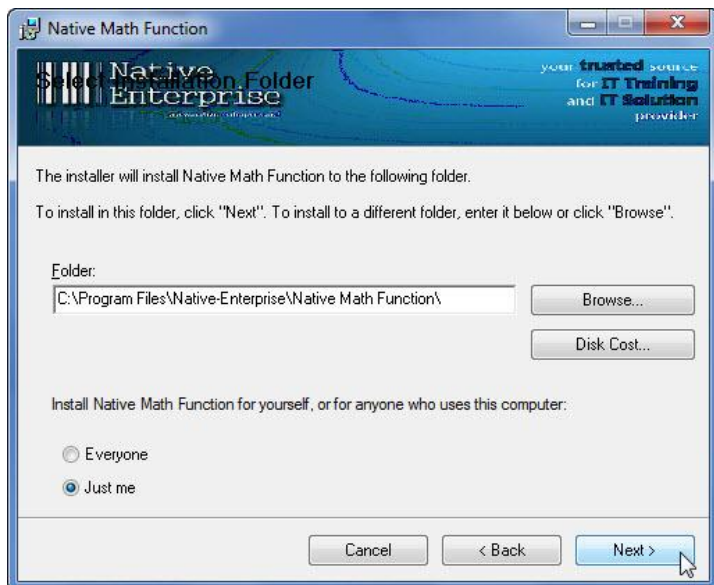
Name	Date modified	Type	Size
MyApplicationSetup	9/16/2009 8:37 PM	Windows Installer Package	1,010 KB
setup	9/16/2009 8:37 PM	Application	666 KB

Gambar 16.24 File setup.exe dan file windows installer.

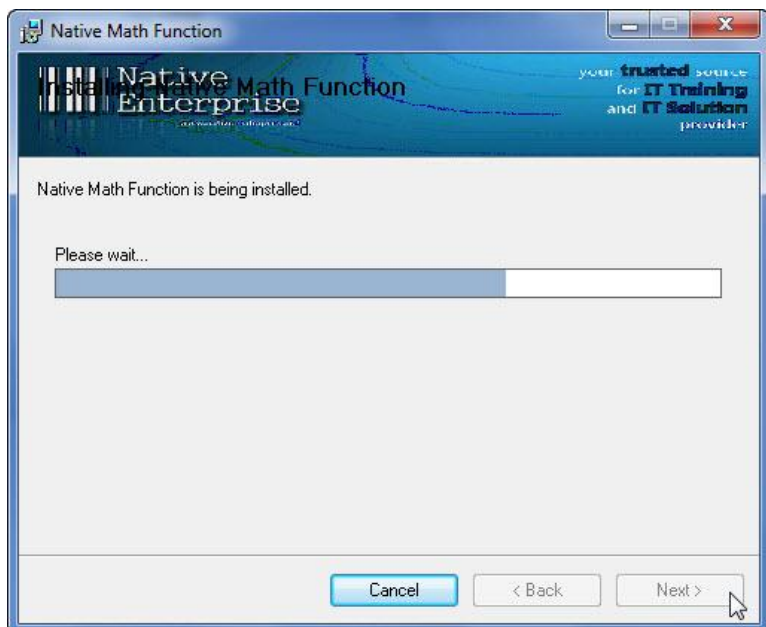
Apabila file setup.exe nya dijalankan akan terlihat perubahan pada form wizard seperti berikut ini :



Gambar 16.25 Welcome interface



Gambar 16.26 Installation folder interface



Gambar 16.27 Progress interface

Custom Action Editor

Editor ini digunakan untuk membuat sebuah program agar dapat dijalankan ketika proses instalasi berlangsung. Misalnya kita ingin database diinstal selama proses instalasi berjalan atau selesai.

Untuk melakukan instalasi file database dibutuhkan class installer yang berisi kode untuk melakukan eksekusi script database yang telah disiapkan.

Berikut adalah langkah-langkah untuk membuat custom action yang akan melakukan proses instalasi database pada saat installer dijalankan.

1. Tambahkan class library project baru kedalam solution yang sama, beri nama project tersebut MyCustomActions.
2. Hapus file class1.vb yang ada kemudian tambahkan Installer Class ke project dengan menambahkan item baru dan beri nama installer class tersebut dengan DBInstaller.vb
3. Ketikkan kode berikut pada file DBInstaller.vb

```
Public Sub New()  
    MyBase.New()  
  
    'This call is required by the Component Designer.  
    InitializeComponent()  
  
    'Add initialization code after the call to  
InitializeComponent  
  
End Sub  
  
Public Overrides Sub Install(ByVal stateSaver As  
System.Collections.IDictionary)  
    MyBase.Install(stateSaver)
```

```

        '//File Path ini didapatkan dari
        '//setting CustomActionData properti Install

        Dim strSqlFilePath As String =
Me.Context.Parameters.Item("args")

        '//gunakan sqlcmd.exe untuk execute script database
yang akan diinstal
        '//sesuaikan nama instance name server nya pada
opsi -S localhost

        Dim psi As ProcessStartInfo = New
ProcessStartInfo("sqlcmd.exe", "-Slocalhost -E -i " &
Chr(34) & strSqlFilePath & Chr(34))

        '//Chr(34) digunakan untuk
        '//menghasilkan apostrophe (single quote)

        psi.WindowStyle = ProcessWindowStyle.Normal
        psi.UseShellExecute = False

        Try
            Dim p As Process = Process.Start(psi)
        Catch e As Exception
            Throw New InstallException(e.Message &
strSqlFilePath)
        End Try
    End Sub

    Public Overrides Sub Commit(ByVal savedState As
System.Collections.IDictionary)
        MyBase.Commit(savedState)
    End Sub

    Public Overrides Sub Rollback(ByVal savedState As
System.Collections.IDictionary)
        MyBase.Rollback(savedState)
    End Sub

```

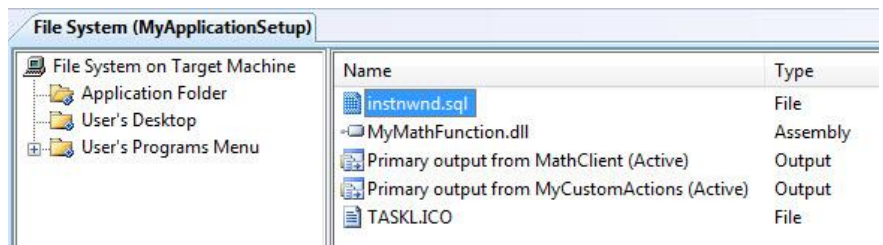
```

Public Overrides Sub Uninstall(ByVal savedState As
System.Collections.IDictionary)
    MyBase.Uninstall(savedState)
End Sub

```

Kode diatas digunakan untuk eksekusi script database dimana file sql nya akan didefinisikan nanti pada custom action editor. Untuk menjalankan script tersebut digunakan perintah sqlcmd.exe untuk database sql server 2005 dan versi diatasnya atau perintah osql.exe untuk database sql server 2000 dan versi dibawahnya.

4. Build class library project tersebut.
5. Buka File System Editor setup project lalu tambahkan file database dengan klik kanan Application Folder dan pilih menu Add - File. Pada contoh ini penulis menggunakan file instnwnd.sql yang akan melakukan instalasi database northwind. Anda dapat mencari file tersebut di internet atau yang sudah penulis sediakan pada project lab files.
6. Tambahkan kembali Output Project pada Application Folder tersebut dan pilih MyCustomAction class library yang telah dibuat sebelumnya.

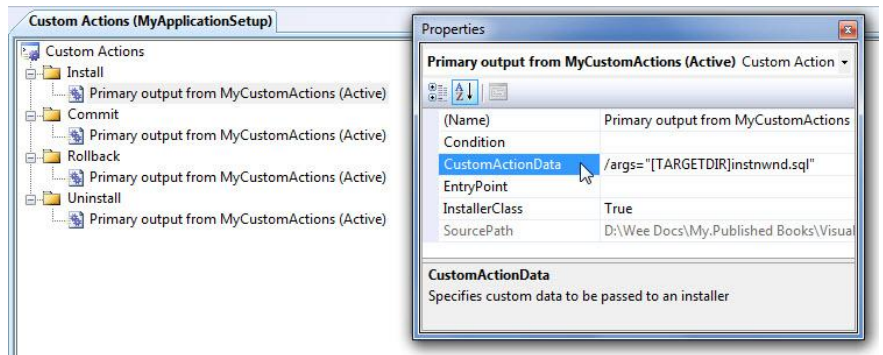


Gambar 16.28 File database .sql dan MyCustomAction class library telah ditambahkan pada Application Folder

7. Buka Custom Action Editor. Klik kanan Custom Action Node kemudian pilih menu Add Custom Action dan cari

project MyCustomActions yang terdapat didalam Application Folder.

8. Lakukan modifikasi properti CustomActionData untuk properti Primary Ouput yang terdapat dibawah node Install seperti pada gambar dibawah ini :



Gambar 16.29 Modifikasi CustomActionData properti pada Primary Output Install.

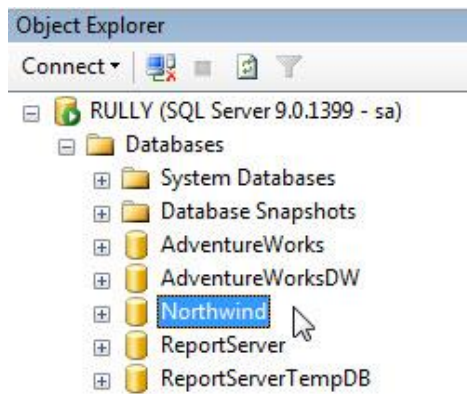
Properti CustomActionData tersebut diisi dengan variabel args yang memiliki nilai yang merujuk ke lokasi direktori instalasi aplikasi dimana file .sql tersebut disimpan. Parameter ini yang nantinya dibaca dari class library project MyCustomActions yang telah kita buat sebelumnya.

9. Build aplikasi dan lakukan instalasi dengan menjalankan file setup.exe hasil kompilasi.
10. Jendela dibawah ini akan muncul ketika proses instalasi database berlangsung :



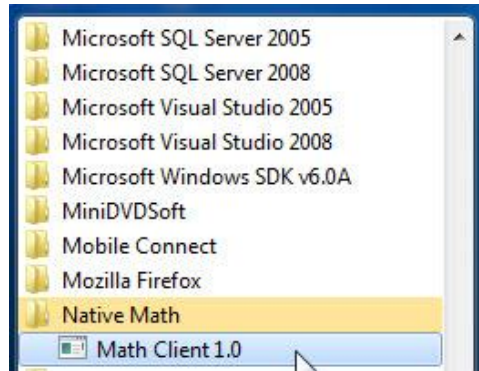
Gambar 16.30 Proses instalasi database file.

11. Buka SQL Server Management Studio 2005 atau 2008 dan pastikan database Northwind terinstal dengan sukses :



Gambar 16.31 Database Northwind sukses diinstal

12. Setelah proses instalasi selesai semuanya pastikan terdapat shortcut di desktop dan shortcut pada User's Program Menu.



Gambar 16.32 User's Program Menu